

Communication 接口文档

1.示教相关

1.1 寸动模式与步长

(SetMoveStepGrade\GetMoveStepSizePara\SetMoveStepSizePara)

1.1.1 说明

接口	功能	返回值	参数	参数说明
bool MotionInterface::SetMoveStepGrade(InoMoveStepGrade grade)	设置寸动模式	bool true 成功 false 失败	grade	InoMoveStepGrade: 寸动档位 (无 / G1 / G2 / G3 / U 等, 与 UI 一致)。 <pre>C++ enum InoMoveStepGrade { InoMoveStepGrade_None = 0, // 关闭寸动 (点动) InoMoveStepGrade_G1 = 1, // 等级 1 (关节步长 0.05°, 位置步长 0.05mm, 姿态步长 0.05°) InoMoveStepGrade_G2 = 2, // 等级 2 (关节步长 0.5°, 位置步长 0.5mm, 姿态步长 0.5°)</pre>

				<pre>InoMoveStepGrade_G3 = 3, // 等级 3（关节步 长 2.0°，位置步长 2.0mm，姿态步长 2.0°） InoMoveStepGrade_U = 4, // 自定义 };</pre>
bool MotionInterface::G etMoveStepSizePa ra(InoMoveStepSiz ePara ¶)	获取 寸动 模式 参数	bool true 成功 false 失败	para（出 参）	位置/姿态/关节步长等，由 _IMotion- >GetMoveStepSizePara 填 充。 <pre>C++ struct InoMoveStepSizePara { double JointSize; // 关节步长(°) double PositionSize; // 位置 步长(mm) double PoseSize; // 姿态步长(°) InoMoveStepSizePara() { JointSize = 0.01; PositionSize = 0.01; PoseSize = 0.01; } };</pre>
bool MotionInterface::S etMoveStepSizePa ra(InoMoveStepSiz ePara para)	设置 寸动 模式	bool true 成功 false 失败	para	InoMoveStepSizePara 同上

1.1.2 函数实现

```
C++
bool MotionInterface::SetMoveStepGrade(InoMoveStepGrade grade)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    _IMotion->SetMoveStepGrade(
        static_cast<InoRobBusiness::MoveStepGrade>(grade));
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool MotionInterface::GetMoveStepSizePara(InoMoveStepSizePara
&para)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoRobBusiness::MoveStepSizePara params;
    bool bRet = _IMotion->GetMoveStepSizePara(params);
    if (bRet) {
        memcpy(&para, &params, sizeof(params));
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return bRet;
}

bool MotionInterface::SetMoveStepSizePara(InoMoveStepSizePara
para)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (para.JointSize < 0.01) {
        para.JointSize = 0.01f;
    }

    if (para.PoseSize < 0.01) {
        para.PoseSize = 0.01f;
    }

    if (para.PositionSize < 0.01) {
        para.PositionSize = 0.01f;
    }

    if (para.JointSize > 10) {
```

```

        para.JointSize = 10;
    }

    if (para.PoseSize > 10) {
        para.PoseSize = 10;
    }

    if (para.PositionSize > 10) {
        para.PositionSize = 10;
    }

    InoRobBusiness::MoveStepSizePara params;
    memcpy(&params, &para, sizeof(params));
    LOG_INFO(QString("[SetMoveStepSizePara]jointsize = %1,
posesize = %2, positionsize = %3")
        .arg(QString::number(params.JointSize),
QString::number(params.PoseSize),
        QString::number(params.PositionSize)));
    FREQ_LOG_PRINT_TIMESTAMP
    return _IMotion->SetMoveStepSizePara(params);
}

```

1.1.3 调用示例

CommunicationThread::processTasks → m_commlInstance->SetMoveStepGrade /
GetMoveStepSizePara / SetMoveStepSizePara

```

C++
    CommunicationEngine::instance()->enqueueCmd_setData(
        this,
        AbstractCmd::CmdType::CmdType_MotionSetMoveStepGrade,
        grd);

    CommunicationEngine::instance()->enqueueCmd(
        this,
        AbstractCmd::CmdType::CmdType_MotionGetMoveStepSizePara);

    CommunicationEngine::instance()->enqueueCmd_setData(
        this,
        AbstractCmd::CmdType::CmdType_MotionSetMoveStepSizePara,
        static_cast<InoMoveStepSizePara>(params));

```

Plain Text

```

        case AbstractCmd::CmdType_MotionSetMoveStepGrade: {
            auto [grade] = ((CmdDatas<InoMoveStepGrade> *)absCmd)-
>m_data;
            bool bRet = m_commInstance->SetMoveStepGrade(grade);
            emit CommunicationEngine::instance()
                ->signal_setstepgrade_result(absCmd->m_object, bRet);
            break;
        }
        case AbstractCmd::CmdType_MotionGetMoveStepSizePara: {
            AbstractCmd *cmd = static_cast<AbstractCmd *>(absCmd);
            InoMoveStepSizePara params;
            bool bRet = m_commInstance->GetMoveStepSizePara(params);
            emit CommunicationEngine::instance()
                ->signal_getstepparams_result(cmd->m_object, bRet,
params);
            break;
        }
        case AbstractCmd::CmdType_MotionSetMoveStepSizePara: {
            auto [params] = ((CmdDatas<InoMoveStepSizePara> *)absCmd)-
>m_data;
            bool bRet = m_commInstance->SetMoveStepSizePara(params);
            emit CommunicationEngine::instance()
                ->signal_setstepparams_result(absCmd->m_object, bRet);
            break;
        }
    }

```

//调用后 接收返回值槽函数连接

```

connect(CommunicationEngine::instance(),
        &CommunicationEngine::signal_setstepparams_result,
        this,
        &StepModeForm::slot_setstepparams_result);

void StepModeForm::slot_setstepparams_result(QObject *object, bool
isSuccess)
{
    if (object != this) return;
    qDebug() << "[StepModeForm::slot_setstepparams_result]" <<
(object == this)
        << "|" << isSuccess;
    if (!isSuccess) {
        QMessageBox::warning(tr("Failed to set jog parameters."));
        return;
    }
}

```

1.2 关节点动 (axisMove)

1.2.1 说明

函数	返回值	参数
<code>bool RobotControllInterface::axisMove(int axisId, const bool &isPositive, const bool &isPressd)</code> {	<code>bool</code> true 成功 false 失败	<code>int axisId;</code> <code>const bool &isPositive;</code> <code>const bool &isPressd</code>

命令类型: `CmdType_AxisMove`

```
C++
class METATYPE_EXPORT CmdAxisMove : public AbstractCmd
{
public:
    int m_axisId = 0;
    bool m_isPositive = false;
    bool m_isPressd = false;
};
```

成员	说明
<code>m_axisId</code>	关节模式: 0-5 对应关节 J1-J6 ; 直角模式: 0-5 对应 X、Y、Z、RX、RY、RZ
<code>m_isPositive</code>	true 正方向, false 负方向。
<code>m_isPressd</code>	true 按下启动, false 松开停止。

1.2.2 调用

入队:

```
C++
void CommunicationEngine::enqueueCmd_axisMove(
    QObject *object,
    int axisId, const bool &isPositive, const bool &isPressd);
```

```
C++
case AbstractCmd::CmdType_AxisMove: {
    if (!Communication::instance()->IsEnable()) {
        PRINT_MSG(tr("Please enable the robot first."));
        break;
    }

    CmdAxisMove *cmd = static_cast<CmdAxisMove *>(absCmd);
    bool isSuccess = m_commInstance->axisMove(
        cmd->m_axisId, cmd->m_isPositive, cmd->m_isPressd);
    emit CommunicationEngine::instance()
        ->signal_axisMoveInterface_result(absCmd->m_object,
isSuccess);
    break;
}
```

1.2.3 函数实现

```
C++
bool RobotControlInterface::axisMove(
    int axisId, const bool &isPositive, const bool &isPressd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool ret = _IMotion->AxisMove(
        axisId,
        isPositive ?
            InoRobBusiness::RotateDir::POSITIVE :
            InoRobBusiness::RotateDir::REVERSE,
        isPressd ?
            InoRobBusiness::PressState::BTN_DOWN :
            InoRobBusiness::PressState::BTN_UP);
    LOG_INFO(QString("RobotControlInterface::axisMove
ret : %1").arg(QString::number(ret)));
}
```

```
FREQ_LOG_PRINT_TIMESTAMP
return ret;
}
```

--- 李昊 1860 60000996 2026年04月28日 17:44

1.3 寸动 / 笛卡尔连续点动 (robotMoveTeachStartOrStopInterface)

1.3.1 说明

函数	返回值	参数
bool RobotControllInterface::robotMoveTeachStartOrStopInterface(const TeachMode &teachMode, const bool &direction, CoordParam coordParam, bool isStart)	bool true 成功 false 失败	const TeachMode &teachMode, const bool &direction, CoordParam coordParam, bool isStart

命令类型： CmdType_RobotMoveTeachStartCmdType_RobotMoveTeachStop。

```
C++
class METATYPE_EXPORT CmdTeachMoveControl : public AbstractCmd
{
public:
    TeachMode m_teachMode = TeachMode_Invalid;
    bool m_direction = false;
    CoordParam m_coordParam;
};
```

```
C++
enum TeachMode {
    TeachMode_Invalid = 0,
```



```

TeachMode_Joint1,
TeachMode_Joint2,
TeachMode_Joint3,
TeachMode_Joint4,
TeachMode_Joint5,
TeachMode_Joint6,
TeachMode_Joint7,
TeachMode_MovX,
TeachMode_MovY,
TeachMode_MovZ,
TeachMode_RotX,
TeachMode_RotY,
TeachMode_RotZ
};

```

成员	说明
m_teachMode	关节：关节轴； 直线：X/Y/Z/RX/RX/RZ； 由 robotMoveTeachStartOrStopInterface 映射为 axisId 后调 _IMotion->AxisMove。
m_direction	运动方向正负。
m_coordParam	<pre> C++ Class CoordParam { QString m_coordType; QString m_coordName; QString m_toolName; } //m_coordType const char BaseCoordinate[] = "BaseCoord"; const char ToolCoordinate[] = "ToolCoord"; const char UserCoordinate[] = "UserCoord"; const char JoinCoordinate[] = "JoinCoord"; const char WorldCoordinate[] = "WorldCoord"; 当 m_coordType 为 UserCoord 时 m_coordName 为所选工件坐标系名称 </pre>

当 **m_coordType** 为 **ToolCoord** 时
m_toolName 为所选工具坐标系名称
其余情况时 **m_toolName**、**m_coordName** 都为空

1.3.2 调用

入队:

```
C++
void enqueueCmd_robotMoveTeachStart(
    QObject *object,
    const TeachMode &teachMode,
    const bool &direction,
    const CoordParam &coordParam);
void enqueueCmd_robotMoveTeachStop(
    QObject *object,
    const TeachMode &teachMode,
    const bool &direction,
    const CoordParam &coordParam);
```

```
C++
case AbstractCmd::CmdType_RobotMoveTeachStart: {
    if (!Communication::instance()->IsEnable()) {
        PRINT_MSG(tr("Please enable the robot first."));
        break;
    }

    CmdTeachMoveControl *cmd
        = static_cast<CmdTeachMoveControl *>(absCmd);
    bool isSuccess
        = m_commInstance->robotMoveTeachStartOrStopInterface(
            cmd->m_teachMode, cmd->m_direction, cmd-
>m_coordParam, true);
    emit CommunicationEngine::instance()
        -
>signal_robotMoveTeachStartInterface_result(isSuccess);

    break;
}
case AbstractCmd::CmdType_RobotMoveTeachStop: {
```

```

        CmdTeachMoveControl *cmd
            = static_cast<CmdTeachMoveControl *>(absCmd);
        bool isSuccess
            = m_commInstance->robotMoveTeachStartOrStopInterface(
                cmd->m_teachMode, cmd->m_direction, cmd->
                m_coordParam, false);
        break;
    }

```

1.3.3 函数实现

```

C++
bool RobotControlInterface::robotMoveTeachStartOrStopInterface(
    const TeachMode &teachMode,
    const bool &direction,
    CoordParam coordParam,
    bool isStart)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool ret = true;

    int axisId;
    if (teachMode >= TeachMode_Joint1 && teachMode <=
        TeachMode_Joint7) {
        axisId = teachMode;
    }
    else if (teachMode >= TeachMode_MovX && teachMode <=
        TeachMode_RotZ) {
        axisId = teachMode - TeachMode_MovX + 1;
    } else {
        return false;
    }

    InoRobUtil::CoordType coordType = _IMonitor-
    >GetCurrentCoordType();
    qDebug() << __FUNCTION__ << "current coordType:" << coordType
    << "axisId" << axisId;

    ret = _IMotion->AxisMove(
        axisId,
        direction ?
        InoRobBusiness::RotateDir::POSITIVE :

```

```

        InoRobBusiness::RotateDir::REVERSE,
        isStart ?
        InoRobBusiness::PressState::BTN_DOWN :
        InoRobBusiness::PressState::BTN_UP);
    qDebug() << "robotMoveTeachStartOrStopInterface::axisMove " <<
    (isStart ? "Start" : "Stop") << " ret : " << ret;

    if (ret != 0) {
        LOG_INFO(QString("robotMoveTeachStartOrStopInterface
ret : %1").arg(QString::number(ret)));
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
}

```

1.4 机械单元与位姿显示 (SetCurrentCoordType/ SetToolId/ SetWobjId/ SetLoadId/ SetPosFormat)

1.4.1 说明

命令类型:

CmdType_Control_SetCoordType CmdType_Control_SetToolId CmdType_Control_SetWObjId CmdType_Control_SetLoadId CmdType_Control_SetPosFormat。

接口	返回值	参数	说明
void MonitorInterface::SetCurrentCoordType(RobotCoordType coordType)	无	coordType	见 RobotCoordType。设置坐标系类型
bool MonitorInterface::SetToolId(uint16 toolId)	bool true 成功 false 失败	toolID	uint16; _ITool->SetCurrentId。 设置工具坐标系索引

	败		
bool MonitorInterface::SetWobjId(uint16 wobjID)	bool true 成功 false 失败	wobjID	quint16; _IWobj->SetCurrentId。 设置工件坐标系索引
bool MonitorInterface::SetLoadId(uint16 loadID)	bool true 成功 false 失败	loadID	quint16; _ILoad->SetCurrentId。 设置负载索引
void MonitorInterface::SetPosFormat(RobotCoordDisplayFormat posFormat)	无	posFormat	见 RobotCoordDisplayFormat, 映射到 InoRobUtil::PosFormat 后 _IPosition->SetPosFormat。 设置点位格式

```

C++
// coordparam.h
enum RobotCoordType {
    RobotCoordType_Base,
    RobotCoordType_Tool,
    RobotCoordType_User,
    RobotCoordType_Joint,
    RobotCoordType_World
};

enum RobotCoordDisplayFormat {
    RobotCoordDisplayFormat_Base,
    RobotCoordDisplayFormat_User,
    RobotCoordDisplayFormat_World,
    RobotCoordDisplayFormat_Flange,
    RobotCoordDisplayFormat_Joint,
};

```

1.4.2 调用

入队示例:

```
C++
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_Control_SetCoordType,
    static_cast<RobotCoordType>(index));
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_Control_SetToolId, index);
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_Control_SetWObjId, index);
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_Control_SetLoadId, index);
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_Control_SetPosFormat,
    static_cast<RobotCoordDisplayFormat>(index));
```

```
C++
    case AbstractCmd::CmdType_Control_SetToolId: {
        auto [toolId] = ((CmdDatas<int> *)absCmd)->m_data;
        bool bRet = Communication::instance()->SetToolId(toolId);
        emit CommunicationEngine::instance()
            ->signal_settool_result(absCmd->m_object, bRet,
toolId);
        break;
    }
    case AbstractCmd::CmdType_Control_SetWObjId: {
        auto [wobjId] = ((CmdDatas<int> *)absCmd)->m_data;
        bool bRet = Communication::instance()->SetWobjId(wobjId);
        emit CommunicationEngine::instance()
            ->signal_setwobj_result(absCmd->m_object, bRet,
wobjId);
        break;
    }
    case AbstractCmd::CmdType_Control_SetLoadId: {
        auto [loadId] = ((CmdDatas<int> *)absCmd)->m_data;
        bool bRet = Communication::instance()->SetLoadId(loadId);
        emit CommunicationEngine::instance()
            ->signal_setload_result(absCmd->m_object, bRet,
loadId);
        break;
    }
    case AbstractCmd::CmdType_Control_SetCoordType: {
```

```

        auto [coordType] = ((CmdDatas<RobotCoordType> *)absCmd)-
>m_data;
        Communication::instance()->SetCurrentCoordType(coordType);
        break;
    }
    case AbstractCmd::CmdType_Control_SetPosFormat: {
        auto [posFormat]
            = ((CmdDatas<RobotCoordDisplayFormat> *)absCmd)-
>m_data;
        Communication::instance()->SetPosFormat(
            static_cast<RobotCoordDisplayFormat>(posFormat));
        break;
    }
}

```

1.4.3 函数实现

```

C++
bool MonitorInterface::SetToolId(quint16 toolID)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    qDebug() << "MonitorInterface::SetToolId : " << toolID;
    return _ITool->SetCurrentId(toolID);
}

bool MonitorInterface::SetWobjId(quint16 wobjID)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    qDebug() << "MonitorInterface::SetWobjId : " << wobjID;
    return _IWobj->SetCurrentId(wobjID);
}

bool MonitorInterface::SetLoadId(quint16 loadID)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    qDebug() << "MonitorInterface::SetLoadId : " << loadID;
    return _ILoad->SetCurrentId(loadID);
}

void MonitorInterface::SetCurrentCoordType(RobotCoordType
coordType)
{

```

```

FREQ_LOG_PRINT_TIMESTAMP_THREAD

InoRobUtil::CoordType type;
switch(coordType) {
case RobotCoordType_Base:      type = COORD_BASE; break;
case RobotCoordType_Joint:     type = COORD_JOINT; break;
case RobotCoordType_Tool:      type = COORD_TOOL; break;
case RobotCoordType_User:      type = COORD_WOBJ; break;
case RobotCoordType_World:     type = COORD_WORLD; break;
default:                       type = COORD_UNKNOW; break;
}
_IPosition->SetCurrentCoordType(type);
FREQ_LOG_PRINT_TIMESTAMP
}

void MonitorInterface::SetPosFormat(RobotCoordDisplayFormat
posFormat)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoRobUtil::PosFormat format;
    switch(posFormat) {
    case RobotCoordDisplayFormat_Base:    format = FORMAT_BASE;
break;
    case RobotCoordDisplayFormat_User:    format = FORMAT_WOBJ;
break;
    case RobotCoordDisplayFormat_World:   format = FORMAT_WORLD;
break;
    case RobotCoordDisplayFormat_Flange:  format = FORMAT_FLANGLE;
break;
    case RobotCoordDisplayFormat_Joint:   format = FORMAT_JOINT;
break;
    default:                             format = FORMAT_BASE;
break;
    }
    _IPosition->SetPosFormat(format);
    FREQ_LOG_PRINT_TIMESTAMP
}

```

李昊 1860 60000990
2026年04月28日 17:44

李昊 1860 60000990
2026年04月28日 17:44

1.5 点文件列表、点列表、取当前点

1.5.1 说明

接口（经 Communication）	返回值	参数	说明
<code>std::vector<std::string> ProjectInterface::get RPointFileList()</code>	<code>std::vector<std ::string></code>	无	分别获取 P 点/JP 点文件 名列表
<code>QVector<InoRPointI nfo> RobotPointInterface Private::readRPoint s(std::string sFileName)</code>	<code>QVector<InoR PointInfo>vec RPointInfos</code>	<code>std::string("P.pts")</code>	按文件名读取点列表
<code>bool RobotPointInterface Private::changeRPoi nt(const InoRPointInfo &info, std::string sFileName)</code>	<code>Bool True 成功 False 失败</code>	<code>InoRPointInfo rp, std::string fileName;</code>	修改 P 点
<code>bool RobotPointInterface Private::saveRPoint s()</code>	<code>Bool True 成功 False 失败</code>	无	保存修改点位

接口（经 Communication）	返回值	参数	说明
<code>std::vector<std::strin g> ProjectInterface::get JPointFileList()</code>	<code>std::vector<std ::string></code>	无	分别获取 P 点/JP 点文件 名列表

QVector<InoJPointInfo> JointPointInterface::readJPoints(std::string sFileName)	QVector<InoJPointInfo> vecJPointInfos	std::string("JP.pts")	按文件名读取点列表
bool JointPointInterface::changeJPoint(const InoJPointInfo &info, std::string sFileName)	Bool True 成功 False 失败	InoJPointInfo jp, std::string fileName;	修改 JP 点
bool JointPointInterface::saveJPoints()	Bool True 成功 False 失败	无	保存修改点位

点数据结构:

```

C++
typedef struct InoRPointInfo {
    int pointNo = -1;    // 点位序号
    QString name;        // 名称
    QString label;       // 标签
    QString description;  // 描述（备注）
    InoRobPos pos;
} InoRPointInfo;

typedef struct InoRobPos {
    double RPosData[POSE_AXIS_COUNT]; // XYZABC
    int ArmParm[ARM_TYPE_COUNT];       // 臂参数
    double EPosData[EXT_AXIS_COUNT];   // E1-E6
} InoRobPos;

typedef struct InoJPointInfo {
    int pointNo = -1;    // 点位序号
    QString name;        // 名称
    QString label;       // 标签
    QString description;  // 描述（备注）
    InoJPos pos;
} InoJPointInfo;

```

```
typedef struct InoJPos {
    double JointData[JOINT_AXIS_COUNT]; // J1-J8
    double EPosData[EXT_AXIS_COUNT]; // E1-E6
} InoJPos;
```

命令类型:

CmdType_Control_Move2Point_GetFileList CmdType_Control_Move2Point_GetPtList CmdType_Control_Move2Point_SetCurrent。

1.5.2 调用

入队示例:

```
C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Control_Move2Point_GetFileList);

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_Control_Move2Point_GetPtList,
    filename);

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_Control_Move2Point_SetCurrent,
    type, rp, jp, filename);
```

```
C++
case AbstractCmd::CmdType_Control_Move2Point_GetFileList: {
    QStringList fileList;
    std::vector<std::string> vecFiles = m_commInstance-
>getRPointFileList();
    for (const std::string &file : vecFiles) {
        fileList << QString::fromStdString(file);
    }

    vecFiles = m_commInstance->getJPointFileList();
    for (const std::string &file : vecFiles) {
        fileList << QString::fromStdString(file);
    }

    emit CommunicationEngine::instance()-
>signal_getPtFileList_result(absCmd->m_object, fileList);
```

```

        break;
    }
    case AbstractCmd::CmdType_Control_Move2Point_GetPtList: {
        auto [filename] = ((CmdDatas<QString> *)absCmd)->m_data;
        QVector<InoRPointInfo> vecRPointInfos;
        QVector<InoJPointInfo> vecJPointInfos;
        if (filename.compare("JP.pts") == 0) {
            vecJPointInfos = m_commInstance-
>readJPoints(filename.toStdString());
        } else {
            vecRPointInfos = m_commInstance-
>readRPoints(filename.toStdString());
        }
        emit CommunicationEngine::instance()
            ->signal_control_getPtList_result(absCmd->m_object,
            vecRPointInfos, vecJPointInfos);
        break;
    }
    case AbstractCmd::CmdType_Control_Move2Point_SetCurrent: {
        auto [type, rp, jp, filename]
            = ((CmdDatas<int, InoRPointInfo, InoJPointInfo,
            std::string> *)absCmd)->m_data;
        bool bRet = false;
        if (type == 0) {
            bRet = m_commInstance->changeRPoint(rp, filename);
            if (bRet) bRet = m_commInstance->saveRPoints();
        } else {
            bRet = m_commInstance->changeJPoint(jp, filename);
            if (bRet) bRet = m_commInstance->saveJPoints();
        }
        emit CommunicationEngine::instance()
            ->signal_control_setcurrent_result(absCmd->m_object,
            bRet);
        break;
    }
}

```

1.5.3 函数实现

实现分布在 **projectinterface.cpp**（文件列表）、**robotpointinterface.cpp**（P 点读写/保存）、**jointpointinterface.cpp**（JP 点读写/保存）。

C++

```

std::vector<std::string> ProjectInterface::getRPointFileList()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    std::vector<std::string> fileNames = comm()->project()-
>GetRPointFileList();
    FREQ_LOG_PRINT_TIMESTAMP
    return fileNames;
}

std::vector<std::string> ProjectInterface::getJPointFileList()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    std::vector<std::string> fileNames = comm()->project()-
>GetJPointFileList();
    FREQ_LOG_PRINT_TIMESTAMP
    return fileNames;
}

QVector<InoJPointInfo>
JointPointInterface::readJPoints(std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::string errMsg;
    QVector<InoJPointInfo> infos;

    sFileName = sFileName.empty() ? m_sJPointFileName : sFileName;

    std::vector<MJPosItem> points;
    InoJPointInfo info;
    infos.clear();

    // 读取点位
    int ret = comm()->project()->GetJPoint()->ReadPoints(points,
sFileName);
    if (ret != ERR_OK) {
        return infos;
    }

    // 添加到缓存中
    comm()->project()->GetJPoint()->AddViewPoints(sFileName,
points);

    // 赋值

```

```

        size_t count = points.size();
        for (size_t i = 0; i < count; i++) {
            info.pointNo = points[i].PointNo; // 点位序号
            info.name =
QString("%1%2%3").arg("JP[").arg(points[i].PointNo, 4, 10,
QLatin1Char('0')).arg("]");
            info.label = QString::fromStdString(points[i].LabelName);
// 标签
            info.description =
QString::fromStdString(points[i].Description);
// 描述(备注)
            info.pos.JointData[0] =
QString::number(points[i].JData.JointData[0], 'f', 3).toDouble();
// J1-J6
            info.pos.JointData[1] =
QString::number(points[i].JData.JointData[1], 'f', 3).toDouble();
            info.pos.JointData[2] =
QString::number(points[i].JData.JointData[2], 'f', 3).toDouble();
            info.pos.JointData[3] =
QString::number(points[i].JData.JointData[3], 'f', 3).toDouble();
            info.pos.JointData[4] =
QString::number(points[i].JData.JointData[4], 'f', 3).toDouble();
            info.pos.JointData[5] =
QString::number(points[i].JData.JointData[5], 'f', 3).toDouble();
            info.pos.EPosData[0] =
QString::number(points[i].JData.EPosData[0], 'f', 3).toDouble();
// E1~E6
            info.pos.EPosData[1] =
QString::number(points[i].JData.EPosData[1], 'f', 3).toDouble();
            info.pos.EPosData[2] =
QString::number(points[i].JData.EPosData[2], 'f', 3).toDouble();
            info.pos.EPosData[3] =
QString::number(points[i].JData.EPosData[3], 'f', 3).toDouble();
            info.pos.EPosData[4] =
QString::number(points[i].JData.EPosData[4], 'f', 3).toDouble();
            info.pos.EPosData[5] =
QString::number(points[i].JData.EPosData[5], 'f', 3).toDouble();
            infos.append(info);
        }

        std::sort(infos.begin(), infos.end(),
            [](const InoJPointInfo &a, const InoJPointInfo &b) {
                return a.pointNo < b.pointNo;
            });

```

```

        FREQ_LOG_PRINT_TIMESTAMP
        return infos;
    }

bool JointPointInterface::saveJPoints()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int nRet = comm()->project()->GetJPoint()->SaveViewPoints(
        m_sJPointFileName, false);
    if (nRet != ERR_OK) {
        return false;
    }
    comm()->project()->updateLablesToTransfor();
    comm()->project()->sendSyncFlag(

static_cast<int>(InoRobBusiness::SyncProjcetInfoType::SYNC_GOBAL_J
POINT));
    return (nRet == ERR_OK);
}

QVector<InoRPointInfo>
RobotPointInterfacePrivate::readRPoints(std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    std::string errMsg;
    QVector<InoRPointInfo> infos;

    sFileName = sFileName.empty() ? m_sRPointFileName : sFileName;

    std::vector<MRobPosItem> points;
    InoRPointInfo info;
    infos.clear();

    // 读取点位
    int ret = q->comm()->project()->GetRPoint()-
>ReadPoints(points, sFileName);
    if (ret != ERR_OK) {
        return infos;
    }

    // 添加到缓存中
    q->comm()->project()->GetRPoint()->AddViewPoints(sFileName,
points);
}

```

```

// 赋值
size_t count = points.size();
for (size_t i = 0; i < count; i++) {
    info.pointNo = points[i].PointNo; // 点位序号
    info.name =
QString("%1%2%3").arg("P").arg(points[i].PointNo, 4, 10,
QLatin1Char('0')).arg("]");
    info.label = QString::fromStdString(points[i].LabelName);
// 标签
    info.description =
QString::fromStdString(points[i].Description);
// 描述(备注)
    info.pos.RPosData[0] =
QString::number(points[i].PData.RPosData[0], 'f', 3).toDouble();
// XYZABC
    info.pos.RPosData[1] =
QString::number(points[i].PData.RPosData[1], 'f', 3).toDouble();
    info.pos.RPosData[2] =
QString::number(points[i].PData.RPosData[2], 'f', 3).toDouble();
    info.pos.RPosData[3] =
QString::number(points[i].PData.RPosData[3], 'f', 3).toDouble();
    info.pos.RPosData[4] =
QString::number(points[i].PData.RPosData[4], 'f', 3).toDouble();
    info.pos.RPosData[5] =
QString::number(points[i].PData.RPosData[5], 'f', 3).toDouble();
    info.pos.ArmParm[0] =
QString::number(points[i].PData.ArmParm[0], 'f', 3).toDouble();
// 臂参数
    info.pos.ArmParm[1] =
QString::number(points[i].PData.ArmParm[1], 'f', 3).toDouble();
    info.pos.ArmParm[2] =
QString::number(points[i].PData.ArmParm[2], 'f', 3).toDouble();
    info.pos.ArmParm[3] =
QString::number(points[i].PData.ArmParm[3], 'f', 3).toDouble();
    info.pos.EPosData[0] =
QString::number(points[i].PData.EPosData[0], 'f', 3).toDouble();
// E1~E6
    info.pos.EPosData[1] =
QString::number(points[i].PData.EPosData[1], 'f', 3).toDouble();
    info.pos.EPosData[2] =
QString::number(points[i].PData.EPosData[2], 'f', 3).toDouble();
    info.pos.EPosData[3] =
QString::number(points[i].PData.EPosData[3], 'f', 3).toDouble();

```



```

        info.pos.EPosData[4] =
QString::number(points[i].PData.EPosData[4], 'f', 3).toDouble();
        info.pos.EPosData[5] =
QString::number(points[i].PData.EPosData[5], 'f', 3).toDouble();
        infos.append(info);
    }

    std::sort(infos.begin(), infos.end(),
        [](const InoRPointInfo &a, const InoRPointInfo &b) {
            return a.pointNo < b.pointNo;
        });

    FREQ_LOG_PRINT_TIMESTAMP
    return infos;
}

bool RobotPointInterfacePrivate::saveRPoints()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int nRet = q->comm()->project()->GetRPoint()->SaveViewPoints(
        m_sRPointFileName, false);
    if (nRet != ERR_OK) {
        return false;
    }

    q->comm()->project()->updateLablesToTransfor();

    q->comm()->project()->sendSyncFlag(
static_cast<int>(InoRobBusiness::SyncProjcetInfoType::SYNC_GOBAL_P
OINT));
    return (nRet == ERR_OK);
}

```

1.6 运动到点 — MovJ / MovL (robotMoveJointTeachInterface/ robotMoveLineInterface)

1.6.1 说明

函数	返回值	参数
bool RobotControllInterface::robotMoveJointTe achInterface(const RoadPoint &roadPoint)	bool true 成功 false 失败	const RoadPoint &roadPoint
bool RobotControllInterface::robotMoveLineInte rface(const RoadPoint &destPt)	bool true 成功 false 失败	const RoadPoint &destPt

参数说明

```
C++
class METATYPE_EXPORT RoadPoint
{
public:
    RoadPoint();
    RoadPoint(const RoadPoint &other);

    ...

    Pos m_position; // Unit: meters
    Ori m_orientation;
    double m_jointAngle[ROBOT_DOF];
    double m_jointRadian[ROBOT_DOF];
    int m_arm[ROBOT_ARM_NUM];
    bool m_isSingular = false;
};

class METATYPE_EXPORT Pos
{
public:
    Pos();
    ...
    double m_x; //Unit: millimeter
    double m_y;
    double m_z;
};
```

```

class METATYPE_EXPORT Ori
{
public:
    Ori();
    ...
    double m_w; /**< W = cos (0.5 × a), angle a*/
    double m_x; /**< x = X × sin (0.5 × a), axis (X Y Z) and angle
a*/
    double m_y; /**< y = Y × sin (0.5 × a), axis (X Y Z) and angle
a */
    double m_z; /**< z = Z × sin (0.5 × a), axis (X Y Z) and angle
a */

    double m_rx; //unit : degree
    double m_ry;
    double m_rz;
};

```

命令类型: `CmdType_RobotMoveJointTeach`、`CmdType_RobotMoveLineTeach`

1.6.2 调用

入队:

```

C++
void enqueueCmd_robotMoveJointTeach(QObject *object, const
RoadPoint &roadPoint);
void enqueueCmd_robotMoveLine(QObject *object, const RoadPoint
&destPt);

```

```

C++
case AbstractCmd::CmdType_RobotMoveJointTeach: {
    if (!Communication::instance()->IsEnable()) {
        PRINT_MSG(tr("Please enable the robot first.));
        break;
    }

    emit CommunicationEngine::instance()-
>signal_contimotion();

    CmdRobotMoveJoint *cmd = static_cast<CmdRobotMoveJoint
*>(absCmd);

```

```

        bool isSuccess = m_commInstance->robotMoveJointTeachInterface(cmd->m_roadPoint);
        emit CommunicationEngine::instance()
            ->signal_axisMoveInterface_result(absCmd->m_object,
            isSuccess);
        break;
    }
    case AbstractCmd::CmdType_RobotMoveLineTeach: {
        if (!Communication::instance()->IsEnable()) {
            PRINT_MSG(tr("Please enable the robot first."));
            break;
        }

        emit CommunicationEngine::instance()->signal_contimotion();

        CmdRobotMoveLine *cmd = static_cast<CmdRobotMoveLine
*>(absCmd);
        bool isSuccess = m_commInstance->robotMoveLineInterface(cmd->m_destPt);
        emit CommunicationEngine::instance()
            ->signal_axisMoveInterface_result(absCmd->m_object,
            isSuccess);
        break;
    }
}

```

1.6.3 函数实现

```

C++
bool RobotControlInterface::robotMoveJointTeachInterface(
    const RoadPoint &roadPoint)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool ret = true;

    ret = _IMotion->JointMoveToPointStart(
        MetaTypeConversion::tp2InoApi_roadPoint(roadPoint));

    if (ret != 0) {
        LOG_INFO(QString("robotMoveJointTeachInterface
ret : %1").arg(QString::number(ret)));
    }
}

```

```

    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
}

bool RobotControlInterface::robotMoveLineInterface(const RoadPoint
&destPt)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool ret = true;
    int16u toolId = comm()->GetCurToolId();
    ReferObj referObj = static_cast<ReferObj>(comm()-
>GetCurWObjId());

    RobPos robPos =
MetaTypeConversion::tp2InoApi_roadPoint2RobPos(destPt);
    ret = _IMotion->MovlToPointStart(
        robPos,
        toolId,
        referObj);
    qDebug() << __FUNCTION__ << "MoveLineTeach ret=" << ret
        << robPos.RPosData[0] << robPos.RPosData[1] <<
robPos.RPosData[2]
        << robPos.RPosData[3] << robPos.RPosData[4] <<
robPos.RPosData[5]
        << robPos.ArmParm[0] << "|" << toolId;
    if (ret != 0) {
        LOG_INFO(QString("robotMoveLineInterface
ret : %1").arg(QString::number(ret)));
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
}

```

1.7 笛卡尔逆解

(PositionInterface::KineInverseSolution)

1.7.1 说明

函数	返回值	参数
bool PositionInterface::KineInverseSolution(short toolId, short wobjId, short loadId, double pos[6], int armArgs[4],int &retCode, double joint[6])	bool true 成功 false 失败	short toolId, short wobjId, short loadId, double pos[6], int armArgs[4], int &retCode, double joint[6]

```
C++
class METATYPE_EXPORT CmdKineInverseSolution : public AbstractCmd
{
public:
    short toolId = 0;
    short wobjId = 0;
    short loadId = 0;
    double pos[6] = {};
    int armArgs[4] = {};
};
```

成员	说明
toolId / wobjId / loadId	工具、工件、负载编号。
pos[6]	位姿。
armArgs[4]	臂参数。

命令类型: **CmdType_KineInverseSolution**

1.7.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_KineInverseSolution(
    this, toolId, wobjId, Communication::instance()-
    >GetCurLoadId(), robPos.RPosData, robPos.ArmParm);
```

```

C++
void enqueueCmd_KineInverseSolution(
    QObject *object, short toolId, short wobjId, short loadId,
    double pts[6], int armArgs[])

    case AbstractCmd::CmdType_KineInverseSolution: {
        CmdKineInverseSolution *cmd =
static_cast<CmdKineInverseSolution *>(absCmd);
        int retCode = -1;
        double joint[6] = {0};
        bool bRet = m_commInstance->KineInverseSolution(
            cmd->toolId, cmd->wobjId, cmd->loadId, cmd->pos, cmd-
>armArgs,
            retCode, joint);
        QVector<double> vecJoint;
        for (int i = 0; i < 6; ++i) {
            vecJoint.push_back(joint[i]);
        }
        emit CommunicationEngine::instance()
            ->signal_KineInverseSolution_result(
                cmd->m_object, bRet, retCode, vecJoint);
        break;
    }

```

1.7.3 函数实现

```

C++
bool PositionInterface::KineInverseSolution(
    short toolId, short wobjId, short loadId, double pos[6], int
armArgs[4],
    int &retCode, double joint[6])
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    RobPos rPos;
    memcpy(rPos.RPosData, pos, sizeof(double) * 6);
    rPos.ArmParm[0] = armArgs[0];
    rPos.ArmParm[1] = armArgs[1];
    rPos.ArmParm[2] = armArgs[2];
    rPos.ArmParm[3] = armArgs[3];
    LOG_INFO(QString("KineInverseSolution Input : tid = %1, wid
= %2, lid = %3, armparams = %4|%5|%6|%7, pos

```

```

= %8|%9|%10|%11|%12|%13")
        .arg(QString::number(toolId),
            QString::number(wobjId),
            QString::number(loadId),
            QString::number(rPos.ArmParm[0]),
            QString::number(rPos.ArmParm[1]),
            QString::number(rPos.ArmParm[2]),
            QString::number(rPos.ArmParm[3]),
            QString::number(rPos.RPosData[0]),
            QString::number(rPos.RPosData[1]),
            QString::number(rPos.RPosData[2]),
            QString::number(rPos.RPosData[3]),
            QString::number(rPos.RPosData[4]),
            QString::number(rPos.RPosData[5])));

JPos Jpos;
bool bRet = _IPosition->KineInverseSolution(
    toolId, wobjId, loadId, rPos, retCode, Jpos);
LOG_INFO(QString("KineInverseSolution result : bRet = %1,
retCode = %2, JPos = %3|%4|%5|%6|%7|%8")
        .arg(QString::number(bRet),
            QString::number(retCode),
            QString::number(Jpos.JointData[0]),
            QString::number(Jpos.JointData[1]),
            QString::number(Jpos.JointData[2]),
            QString::number(Jpos.JointData[3]),
            QString::number(Jpos.JointData[4]),
            QString::number(Jpos.JointData[5])));
memcpy(joint, Jpos.JointData, sizeof(double) * 6);
FREQ_LOG_PRINT_TIMESTAMP
return bRet;
}

```

1.8 机械锁 (enableMechLock)

1.8.1 说明

命令类型: **CmdEnableMechLock**

函数	返回值	参数	说明
bool RobotControlInterface::enableMechLock(const bool &enable)	bool true 成功 false 失败	const bool &enable	true 上机械锁, false 解除 → _IControl->SetMechLock。

1.8.2 调用

C++

```
CommunicationEngine::instance()->enqueueCmd_enableMechLock(
    this, true);
```

C++

```
case AbstractCmd::CmdType_EnableMechLock: {
    bool isSuccess = m_commInstance->enableMechLock(
        static_cast<CmdEnableMechLock *>(absCmd)-
>m_enableMechLock);
    emit CommunicationEngine::instance()
        ->signal_enableMechLockInterface_result(absCmd-
>m_object, isSuccess);
    break;
}
```

1.8.3 函数实现

C++

```
bool RobotControlInterface::enableMechLock(const bool &enable)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool ret = _IControl->SetMechLock(enable);
    qDebug() << "RobotControlInterface::enableMechLock ret : " <<
ret;
    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
```

```
}
```

1.9 速度倍率 (MonitorInterface::SetSpeed)

1.9.1 说明

命令类型: **CmdType_Control_SetSpeed**

接口	返回值	参数	说明
bool MonitorInterface::SetSpeed (quint16 speed)	bool true 成功 false 失败	quint16 speed	设置机器人速度 speed 范围 1~100

1.9.2 调用

```
C++
// mainwidget.cpp 等
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_Control_SetSpeed, value);

//
case AbstractCmd::CmdType_Control_SetSpeed: {
    auto [speed] = ((CmdData<int> *)absCmd)->m_data;
    bool bRet = Communication::instance()->SetSpeed(speed);
    emit CommunicationEngine::instance()-
>signal_setspeed_result(bRet, speed);
    break;
}
```

1.9.3 函数实现

```
C++
bool MonitorInterface::SetSpeed(quint16 speed)
```

```
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IMotion->SetSpeed(speed);
}
```

1.10 机器人使能 (RobotControllInterface::enableRobot)

1.10.1 说明

命令类型: CmdType_EnableRobot

接口	返回值	参数	说明
bool RobotControllInterface::enableRobot(const bool &enable)	bool true 成功 false 失败	const bool &enable	true 上使能, false 下使能 → _IControl->SetEnable; 返回 0 == ret。

1.10.2 调用

```
C++
//调用
CommunicationEngine::instance()->enqueueCmd_enableRobot(this,
checked);

void enqueueCmd_enableRobot(QObject *object, const bool &enable);

case AbstractCmd::CmdType_EnableRobot: {
    bool isSuccess = m_commInstance->enableRobot(
        static_cast<CmdEnableRobot *>(absCmd)->m_enableRobot);
    emit CommunicationEngine::instance()
        ->signal_enableRobotInterface_result(absCmd->m_object,
isSuccess);
}
```

```
break;
}
```

1.10.3 函数实现

```
C++
bool RobotControlInterface::enableRobot(const bool &enable)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = _IControl->SetEnable(enable);
    qDebug() << "RobotControlInterface::enableRobot ret : " <<
ret;
    FREQ_LOG_PRINT_TIMESTAMP
    return 0 == ret;
}
```

1.11 零点位姿

(JointMoveToZeroStart/JointMoveToZeroStop)

1.11.1 说明

命令类型: **CmdType_Control_JointMoveZeroStart**

CmdType_Control_JointMoveZeroEnd

接口	返回值	参数	说明
bool MotionInterface:: JointMoveToZeroStart()	bool true 成功 false 失败	无	需先使能; MotionInterface 调 _IMotion->JointMoveToZeroStart()。
bool MotionInterface::	bool	无	JointMoveToZeroStop(), 无需使能检查

JointMoveToZeroStop()	true 成功	(与 processTasks 一致)。
	false 失败	

1.11.2 调用

```

C++
//调用
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Control_JointMoveZeroStart);
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Control_JointMoveZeroEnd);

case AbstractCmd::CmdType_Control_JointMoveZeroStart: {
    if (!Communication::instance()->IsEnable()) {
        PRINT_MSG(tr("Please enable the robot first."));
        break;
    }

    emit CommunicationEngine::instance()-
>signal_contimotion();

    bool bRet = Communication::instance()-
>JointMoveToZeroStart();
    emit CommunicationEngine::instance()
        ->signal_jointmovezerostart_result(absCmd->m_object,
bRet);
    break;
}
case AbstractCmd::CmdType_Control_JointMoveZeroEnd: {
    bool bRet = Communication::instance()-
>JointMoveToZeroStop();
    emit CommunicationEngine::instance()
        ->signal_jointmovezeroend_result(absCmd->m_object,
bRet);
    break;
}

```

1.11.3 函数实现

```
C++
bool MotionInterface::JointMoveToZeroStart()
{
    return _IMotion->JointMoveToZeroStart();
}

bool MotionInterface::JointMoveToZeroStop()
{
    return _IMotion->JointMoveToZeroStop();
}

---
```

1.12 初始位姿
(getWorikOriginPoint/robotMoveJointTeachInterface)

1.12.1 说明

命令类型: CmdType_RobotMoveJointToInitPosture

从控制器读取 工作原点 点位，组 `RoadPoint` 后调用与 1.6 (运动到点)相同的 **robotMoveJointTeachInterface**。失败则 emit **signal_needMainWidgetWarning**(tr("Failed to move to initial pose.")).

接口	返回值	参数	说明
bool RobotPointInterfacePrivate::getWorikOriginPoint (const int index, InoJPos &source, QList<double> &min, QList<double> &max)	bool true 成功 false 失败	const int index, InoJPos &source, QList<double> &min, QList<double> &max	index 索引 InoJPos (结构体内容同 1.5) Thrift typedef struct InoJPos { double JointData[JOI

			<pre> NT_AXIS_COUNT]; // J1-J8 double EPosData[EXT_ AXIS_COUNT]; // E1-E6 } InoJPos; min (传参用) max (传参用) </pre>
--	--	--	---

1.12.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_RobotMoveJointToInitPosture);

case AbstractCmd::CmdType_RobotMoveJointToInitPosture: {
    if (!Communication::instance()->IsEnable()) {
        PRINT_MSG(tr("Please enable the robot first.));
        break;
    }

    emit CommunicationEngine::instance()-
>signal_contimotion();

    InoJPos pos;
    QList<double> min, max;
    if (m_commInstance->getWorikOriginPoint(0, pos, min, max))
{
        RoadPoint point;
        point.setJointAngle(pos.JointData);
        bool isSuccess = m_commInstance-
>robotMoveJointTeachInterface(point);
        if (isSuccess)
            return;
    }
    emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
        tr("Failed to move to initial pose.));
    break;
}

```

```
}
```

1.12.3 函数实现

C++

//获取工作原点

```
bool RobotPointInterfacePrivate::getWorikOriginPoint(const int
index,
                                                    InoJPos &source,
                                                    QList<double> &min,
                                                    QList<double> &max)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    JPos pos;
    int ret = _IRobotArm->ReadWorkOriginPts(pos, index);
    source = MetaTypeConversion::tp2InoApi_jPos(pos);
    if (ret != ERROR_OK)
        return false;
    InoRobBusiness::IRobotParamRange *param = q->comm()-
>robotParam()->getRobotParamRange();
    std::string paraStructName = "WorkOriginParam" +
std::to_string(index);
    for (int j = 0; j < 6; ++j) {
        std::string paramName = "J" + std::to_string(j + 1) +
"(" + std::to_string(index) + ")";
        min.push_back(param->getDoubleMinValue(paraStructName,
paramName));
        max.push_back(param->getDoubleMaxValue(paraStructName,
paramName));
    }
    paraStructName = "WorkOriginParam" + std::to_string(index +
6);
    for (int j = 0; j < 6; ++j) {
        std::string paramName = "J" + std::to_string(j + 1) +
"(" + std::to_string(index + 6) + ")";
        min.push_back(param->getDoubleMinValue(paraStructName,
paramName));
        max.push_back(param->getDoubleMaxValue(paraStructName,
paramName));
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}
```


2.工程管理

2.1 刷新/获取工程列表（getProjectList / sortProjectList / readProjectList）

2.1.1 说明

接口	功能	返回值	参数	参数说明
QVector<InoProjectInfo> ProjectInterface::getProjectList(const ProjectSortType sortType)	获取控制器工程列表并排序	QVector<InoProjectInfo>	sortType	排序枚举，详见下方定义
QVector<InoProjectInfo> ProjectInterface::sortProjectList(const ProjectSortType sortType, std::vector<_ProjectFolderInfo> &infos)	对工程列表排序并将当前激活工程置顶	QVector<InoProjectInfo>	sortType, infos	infos 为底层工程目录结构数组
std::vector<_ProjectFolderInfo> ProjectInterface::readProjectList()	读取底层工程目录列表	std::vector<_ProjectFolderInfo>	无	comm()->project()->ReadProjectList(...)

参数说明

```
C++  
// projectdata.h  
enum ProjectSortType {  
    ProjectSortType_DascendingByTime = 0, // 时间降序排列  
    ProjectSortType_AscendingByWord      // 首字母升序排列  
};
```

```

C++
// projectdata.h
typedef struct InoProjectInfo {
    QString index = "0"; // 下标
    QString name = ""; // 工程名
    QString modifyTime; // 修改时间
    int memorySize; // 大小
} InoProjectInfo;

```

```

C++
// ProjectHelper.h
typedef struct _ProjectFolderInfo
{
    std::string projectName; // 工程名称
    std::string modifyTime; // 修改时间
    int memorySize; // 工程文件夹内存大小(kb)
} ProjectFolderInfo;

```

2.1.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_RefreshProjectDatas, m_sortType);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GetProjectList, m_sortType);

```

2.1.3 函数实现

```

C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_RefreshProjectDatas:
case AbstractCmd::CmdType_GetProjectList:
    getProjectList(absCmd);
    break;

```

```

C++
// CommunicationThread::getProjectList

```

```

sortType = cmd->m_value.value<ProjectSortType>();
vecProjectInfos = m_commInstance->getProjectList(sortType);
emit CommunicationEngine::instance()-
>signal_handleProject_result(...);

```

C++

```

// ProjectInterface
 QVector<InoProjectInfo> ProjectInterface::getProjectList(const
 ProjectSortType sortType)
 {
     std::vector<struct _ProjectFolderInfo> infos =
 readProjectList();
     return sortProjectList(sortType, infos);
 }

 QVector<InoProjectInfo> ProjectInterface::sortProjectList(
     const ProjectSortType sortType,
     std::vector<struct _ProjectFolderInfo> &infos)
 {
     FREQ_LOG_PRINT_TIMESTAMP_THREAD

     // 如果连接了控制器，则获取当前激活的工程名称，排序前将其剔除在外
     struct _ProjectFolderInfo activeInfo =
 extractActiveProject(infos);

     // 排序
     if (ProjectSortType_DescendingByTime == sortType) {
         comm()->project()->SortProjectsByTime(infos);
     } else if (ProjectSortType_AscendingByWord == sortType) {
         comm()->project()->SortProjectsByWord(infos);
     }

     // 如果当前激活的不为空，则插入到数组第一个
     if (!activeInfo.projectName.empty()) {
         infos.insert(infos.begin(), activeInfo);
     }

     FREQ_LOG_PRINT_TIMESTAMP
     return MetaTypeConversion::tp2InoApi_projectTypeInfos(infos);
 }

 std::vector<struct _ProjectFolderInfo>
 ProjectInterface::readProjectList()

```

```

{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<struct _ProjectFolderInfo> infos;
    infos.clear();

    const std::string ip = comm()->getIP().toStdString();
    if (ip.empty()) {
        return infos;
    }

    comm()->project()->ReadProjectList(infos, ip);

    FREQ_LOG_PRINT_TIMESTAMP
    return infos;
}

```

2.2 读取工程 (readProject / readLocalProject / readActiveProject / setActiveProject)

2.2.1 说明

接口	功能	返回值	参数	参数说明
int ProjectInterface::readProject(const QString &name)	读取控制器工程并处理激活切换	int	name	工程名
int ProjectInterface::readLocalProject(const QString	读取本地工程	int	name	工程名

&name)				
bool ProjectInterface::setActiveProject(const QString &projectName)	设置激活工程	int	projectName	工程名
int ProjectInterface::readActiveProject(const QString &name, bool isReadSameProject)	底层读取工程	int	name,isReadSameProject	工程名和是否同名重读

C++

// projectdata.h 返回值说明

```
const int Err_Success = 0;           //成功。
const int Err_Disconnect = 1;        //连接异常
const int Err_ReadProject = 2;       //读取失败
const int Err_Incomplete = 3;       //工程文件不完整
const int Err_ActiveProject = 4;     //激活设置失败
```

2.2.2 调用

C++

```
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReadProject, sProjectName);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReadLocalProject, sProjectName);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReloadActiveProject, sProjectName);
```

2.2.3 函数实现

```

C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_ReadProject:
    readProject(absCmd);
    break;
case AbstractCmd::CmdType_ReadLocalProject:
    readLocalProject(absCmd);
    break;
case AbstractCmd::CmdType_ReloadActiveProject:
    reloadActiveProject(absCmd);
    break;

```

```

C++
// CommunicationThread::readProject
int ret = m_commInstance->readProject(sProjectName);
emit CommunicationEngine::instance()-
>signal_handleProject_result(...);

```

```

C++
// ProjectInterface::readProject (片段)
int ProjectInterface::readProject(const QString &name)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (name.isEmpty()) {
        return Err_ReadProject;
    }

    const std::string ip = comm()->getIP().toStdString();
    if (ip.empty()) {
        return Err_Disconnect;
    }

    // 读取工程
    // 先记录切换前的工程名称
    QString originPrjName = getActiveProjectName();
    qDebug() << "current active project name = " << originPrjName;

    // 设置工程类型
    setProjectType(InoProjectType_Lua);

```

```

        // 加载工程
        int nRet = readActiveProject(name);
        qDebug() << "project load ret = " << nRet << ", name = " <<
name;
        if (nRet != ERR_OK) {
            qDebug() << "Err_ReadProject is readActiveProject" <<
Err_ReadProject;
            return Err_ReadProject;
        }

        QString sMainXmlFile = CONTROLLER_PROJECT_MAIN_XML_FILE(name);
        if (!FileUtils::isFileExist(sMainXmlFile)) {
            if (!originPrjName.isEmpty()) {
                nRet = readActiveProject(originPrjName);
                qDebug() << "project load origin ret = " << nRet << ",
name = " << originPrjName;
            }
            return Err_Incomplete;
        }

        // 设置当前激活的工程
        if (0 != name.compare(originPrjName)) {
            bool isOk = setActiveProject(name);
            qDebug() << "set project Active ret = " << isOk << ", name
= " << name;
            if (!isOk) {
                return Err_ActiveProject;
            }
        }

        // 工具 IO 默认描述语言不匹配, 重新加载工程
        if (!Communication::instance()->isToolIoItemDescLangCorrect())
        {
            // 加载工程
            nRet = readActiveProject(name, true);
            qDebug() << "project load ret = " << nRet << ", name = "
<< name;
            if (nRet != ERR_OK) {
                qDebug() << "Err_ReadProject is
isToolIoItemDescLangCorrect" << Err_ReadProject;
                return Err_ReadProject;
            }
        }
    }
}

```

```

        return Err_Success;
    }

int ProjectInterface::readLocalProject(const QString &name)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (name.isEmpty()) {
        return Err_ReadProject;
    }

    // 设置工程类型
    setProjectType(InoProjectType_Lua);

    QString sMainXmlFile = CONTROLLER_PROJECT_MAIN_XML_FILE(name);
    if (!FileUtils::isFileExist(sMainXmlFile)) {
        return Err_Incomplete;
    }

    // 读取工程
    QString path = CONTROLLER_PROJECT_FILE_PATH(name);
    if (path.isEmpty()) {
        return Err_ReadProject;
    }

    int nRet = comm()->project()-
>ReadLocalProjcet(path.toStdString());
    if (nRet != ERR_OK) {
        return Err_ReadProject;
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return Err_Success;
}

int ProjectInterface::readActiveProject(const QString &name, bool
isReadSameProject)
{
    int nReqRetryTimes = REQUEST_RETRY_TIMES;
    int ret = Err_Unknown;

    const std::string ip = comm()->getIP().toStdString();

```



```

        if (ip.empty()) {
            return Err_Disconnect;
        }

        if (name.isEmpty()) {
            return Err_Unknown;
        }

        ret = comm()->project()->ReadProject(ip, name.toStdString(),
isReadSameProject);
        if (ERR_OK == ret) {
            comm()->getTaskList();
        }

        return ret;
    }

bool ProjectInterface::setActiveProject(const QString
&projectName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool isOk = false;

    int nReqRetryTimes = REQUEST_RETRY_TIMES;
    while (nReqRetryTimes--) {
        int ret = comm()->project()-
>setActiveProject(projectName.toStdString());
        if (ret == ERR_OK) {
            isOk = true;
            nReqRetryTimes = 0;
            break;
        }

        QThread::msleep(300);
    }

    qDebug() << "ProjectInterface::setActiveProject ret = " <<
isOk
        << ", name = " << projectName;
    FREQ_LOG_PRINT_TIMESTAMP
    return isOk;
}

```

2.3 新建工程 (getDefaultProjectName / createProject / setProjectOperateMode)

2.3.1 说明

接口	功能	返回值	参数	参数说明
QString ProjectInterface::getDefaultProjectName(QString baseName)	生成默认名	QString	baseName	基础名前缀
bool ProjectInterface::createProject(const QString &name, InoProjectType type)	创建工程	bool	name,type	工程名+类型
bool ProjectInterface::setProjectOperateMode (const InoFileType &mode)	设置工程操作模式	bool	mode	本地/控制器

```
C++  
// projectdata.h  
enum InoFileType {  
    InoFileType_Local = 0,  
    InoFileType_Controller  
};  
  
enum InoProjectType {  
    InoProjectType_Pro = 0,  
    InoProjectType_Lua  
};
```

2.3.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GetDefaultProjectName, QString());

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CreateProject, sProjectName);

```

2.3.3 函数实现

```

C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_GetDefaultProjectName:
    getDefaultProjectName(absCmd);
    break;
case AbstractCmd::CmdType_CreateProject:
    createProject(absCmd);
    break;

```

```

C++
QString ProjectInterface::getDefaultProjectName(QString baseName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<struct _ProjectFolderInfo> infos =
readProjectList();
    return QString::fromStdString(
        comm()->project()->NewProjectDefaultName(
            infos, baseName.toStdString()));
    FREQ_LOG_PRINT_TIMESTAMP
}

bool ProjectInterface::createProject(const QString &name,
InoProjectType type)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    // 创建工程
    int nRet = comm()->project()->CreateProject(
        name.toStdString(),
        MetaTypeConversion::inoApi2tp_projectType(type));
}

```

```

        qDebug() << "ProjectInterface::createProject ret = " << nRet
                << " ,name = " << name;

        FREQ_LOG_PRINT_TIMESTAMP
        return nRet == ERR_OK;
    }

bool ProjectInterface::setProjectOperateMode(const InoFileType
&mode)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    comm()->project()->setProjectOperateMode(
MetaTypeConversion::inoApi2tp_projectFileOperateMode(mode));

    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

```

2.4 删除工程（deleteProject / getActiveProjectName）

2.4.1 说明

接口	功能	返回值	参数	参数说明
bool ProjectInterface::deleteProject(const QString &name)	删除工程	bool	name	工程名
QString ProjectInterface::getActiveProjectName()	删除前激活 校验	QString	无	激活工程不可删

2.4.2 调用

C++

```
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_DeleteProject, sProjectName);
```

2.4.3 函数实现

C++

```
// CommunicationThread::processTasks
case AbstractCmd::CmdType_DeleteProject:
    deleteProject(absCmd);
    break;
```

C++

```
// CommunicationThread::deleteProject
bool ProjectInterface::deleteProject(const QString &name)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int ret = comm()->project()->DelProject(name.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

QString ProjectInterface::getActiveProjectName()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int nReqRetryTimes = REQUEST_RETRY_TIMES;
    std::string sProjectName = "";

    while (nReqRetryTimes-->0) {
        int nRet = comm()->project()->getActiveProject(sProjectName);
        if (nRet == ERR_OK && !sProjectName.empty()) {
            nReqRetryTimes = 0;
            break;
        }

        QThread::msleep(300);
    }
}
```

```

    FREQ_LOG_PRINT_TIMESTAMP
    return QString::fromStdString(sProjectName);
}

```

2.5 复制/粘贴工程 (copyProject / getCopiedProjectName / pasteProject / getDefaultProjectName)

2.5.1 说明

接口	功能	返回值	参数	参数说明
bool ProjectInterface::copyProject(const QString &name)	复制工程	bool	name	源工程名
QString ProjectInterface::getCopiedProjectName()	读取复制源	QString	无	复制缓存名
bool ProjectInterface::pasteProject(const QString &name)	粘贴为新工程	bool	name	目标工程名

- CmdType_CopyProject: m_value = 源工程名。
- CmdType_PasteProject: m_value = 目标工程名。

- CmdType_GenPasteProjectName: 无额外入参。

2.5.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CopyProject, sProjectName);

CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_GenPasteProjectName);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_PasteProject, sName);
```

2.5.3 函数实现

```
C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_CopyProject:
    copyProject(absCmd);
    break;
case AbstractCmd::CmdType_GenPasteProjectName:
    genPasteProjectName(absCmd);
    break;
case AbstractCmd::CmdType_PasteProject:
    pasteProject(absCmd);
    break;
```

```
C++
// ProjectInterface
bool ProjectInterface::copyProject(const QString &name)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    comm()->project()->CopyProject(name.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

QString ProjectInterface::getCopiedProjectName()
{
}
```

```

        return QString::fromStdString(comm()->project()-
>GetCopiedProjectName());
    }

bool ProjectInterface::pasteProject(const QString &name)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = comm()->project()->PasteProject(name.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

QString ProjectInterface::getDefaultProjectName(QString baseName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<struct _ProjectFolderInfo> infos =
readProjectList();
    return QString::fromStdString(
        comm()->project()->NewProjectDefaultName(
            infos, baseName.toStdString()));
    FREQ_LOG_PRINT_TIMESTAMP
}

```

2.6 重命名工程 (renameProject / readProject)

2.6.1 说明

接口	功能	返回值	参数	参数说明
bool ProjectInterface::renameProject(QString oldName, QString newName)	重命名工程	bool	oldName,new Name	旧名+新名

int ProjectInterface::readProject(const QString &name)	激活工程重命名后重读	int	name	新工程名
---	------------	-----	------	------

2.6.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_RenameProject,
    QVector<QVariant>() << selectedProjectName() << sNewName);
```

2.6.3 函数实现

```
C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_RenameProject:
    renameProject(absCmd);
    break;
```

```
C++
// CommunicationThread::renameProject (核心)
bool ProjectInterface::renameProject(QString oldName, QString
newName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = comm()->project()->RenameProject(
        oldName.toStdString(), newName.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}
```

2.7 同步本地工程到控制器 (importProject)

2.7.1 说明

接口	功能	返回值	参数	参数说明
bool ProjectInterface::importProject(QString path)	本地 .prj 同步到控制器	bool	path	本地 .prj 路径

2.7.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ImportLocalProject, sProjectName);
```

2.7.3 函数实现

```
C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_ImportLocalProject:
    importLocalProject(absCmd);
    break;
```

```
Java
void CommunicationThread::importLocalProject(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QString sProjectName;
    QString sActiveProjectName;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to import the local project.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }
```

```

    }

    if (isOk) {
        sProjectName = cmd->m_value.toString();
        // 导入工程需要先删除控制柜中已有同名工程文件目录
        // 模型层不允许删除已经被激活的工程
        // 因此，不允许导入与激活工程同名的工程
        sActiveProjectName = m_commInstance-
>getActiveProjectName();
        if (!sActiveProjectName.isEmpty() &&
sProjectName.compare(sActiveProjectName) == 0) {
            sErrMsg = tr("Failed to import the project as its name
is the same as the name of the current active project.");
            isOk = false;
        }
    }

    if (isOk) {
        QString sMainXmlFile =
LOCAL_PROJECT_MAIN_XML_FILE(sProjectName);
        isOk = FileUtils::isFileExist(sMainXmlFile);
        if (!isOk) {
            sErrMsg = tr("Failed to sync the project to the
controller as \"main.xml\" file is missing in the project.");
        }
    }

    if (isOk) {
        QString sMainLuaFile =
LOCAL_PROJECT_MAIN_SCRIPT_FILE(sProjectName);
        isOk = FileUtils::isFileExist(sMainLuaFile);
        if (!isOk) {
            sErrMsg = tr("Failed to sync the project to the
controller as \"main.lua\" file is missing in the project.");
        }
    }

    if (isOk) {
        QString sPath = LOCAL_PROJECT_FILE_PATH(sProjectName);
        isOk = FileUtils::isFileExist(sPath);
        if (!isOk) {
            sErrMsg = tr("Project sync failed because project
file %1.prj does not exist.")
                .arg(sProjectName);
        }
    }

```

```

    }

    if (isOk) {
        isOk = m_commInstance->importProject(sPath);
        if (!isOk) {
            sErrMsg = tr("Failed to sync project.");
        }
    }
}

emit CommunicationEngine::instance()
    ->signal_handleProject_result(absCmd->m_object, absCmd-
>m_cmdType,
                                sProjectName, isOk,
sErrMsg);
}

bool ProjectInterface::importProject(QString path)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    const std::string ip = comm()->getIP().toStdString();
    if (ip.empty()) {
        return false;
    }

    path = QDir::fromNativeSeparators(path);
    int ret = comm()->project()->ImportProject(ip,
path.toStdString());
    qDebug() << "ProjectInterface::importProject ret = " << ret
        << ", path = " << path;
    if (ret != ERROR_OK) {
        return false;
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

```

李昊 1860 60000990
2026年04月28日 17:44

李昊 1860 60000990
2026年04月28日 17:44

2.8 导出控制器工程到本地 (exportProject)

2.8.1 说明

接口	功能	返回值	参数	参数说明
bool ProjectInterface::exportProject(const QString &name, QString folder)	导出工程到本地目录	bool	name,folder	工程名+目标目录

2.8.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ExportProject,
    QVector<QVariant>() << sProjectName <<
    LOCAL_PROJECT_DIR_PATH);
```

2.8.3 函数实现

```
C++
// CommunicationThread::processTasks
case AbstractCmd::CmdType_ExportProject:
    exportProject(absCmd);
    break;
```

```
C++
void CommunicationThread::exportProject(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    QString sProjectName;
    QString sFolderName;
    CmdCommonValue *cmd = nullptr;
    bool isOk = true;
```

```

cmd = dynamic_cast<CmdCommonValue *>(absCmd);
if (!cmd) {
    sErrMsg = tr("Failed to export project.")
               + SPACE_AND_CONVERT_PARAMETERS_FALID;
    isOk = false;
}

if (isOk) {
    vecValues = cmd->m_value.value<QVector<QVariant>>>();
    if (vecValues.count() < 2) {
        sErrMsg = tr("Failed to export project as the
parameters are invalid.");
        isOk = false;
    }
}

if (isOk) {
    sProjectName = vecValues.at(0).toString();
    sFolderName = vecValues.at(1).toString();
    isOk = m_commInstance->exportProject(sProjectName,
sFolderName);
    if (isOk) {
        sErrMsg = tr("Failed to export project.");
    }
}

emit CommunicationEngine::instance()
    ->signal_handleProject_result(absCmd->m_object, absCmd-
>m_cmdType,
                                sProjectName, isOk,
sErrMsg);
}

bool ProjectInterface::exportProject(const QString &name, QString
folder)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    const std::string ip = comm()->getIP().toStdString();
    if (ip.empty()) {
        return false;
    }
}

```

```
folder = QDir::fromNativeSeparators(folder);
int ret = comm()->project()->ExportProject(
    ip, name.toString(), folder.toString());

FREQ_LOG_PRINT_TIMESTAMP

return (ret == ERR_OK);
}
```

3.点文件相关

3.1 获取点文件列表 (getRPointFileList / getJPointFileList)

3.1.1 说明

接口	功能	返回值	参数	参数说明
std::vector<std::string> ProjectInterface::getRPointFileList()	获取机器人位置点文件列表 (P 点文件)	std::vector<std::string>	无	例如 P.pts
std::vector<std::string> ProjectInterface::getJPointFileList()	获取关节点文件列表 (JP 点文件)	std::vector<std::string>	无	例如 JP.pts

3.1.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_GetRobotPointFileList);
```

```
CommunicationEngine::instance()->enqueueCmd(  
    this, AbstractCmd::CmdType_GetJointPointFileList);
```

3.1.3 函数实现

```
C++  
void CommunicationThread::getRobotPointFileList(AbstractCmd  
*absCmd)  
{  
    QStringList listFiles;  
  
    std::vector<std::string> vecFiles = m_commInstance-  
>getRPointFileList();  
    for (const std::string &file : vecFiles) {  
        listFiles << QString::fromStdString(file);  
    }  
  
    emit CommunicationEngine::instance()-  
>signal_handleProject_result(  
        absCmd->m_object, absCmd->m_cmdType, QVariant(listFiles),  
        true, QString());  
}  
  
void CommunicationThread::getJointPointFileList(AbstractCmd  
*absCmd)  
{  
    QStringList listFiles;  
  
    std::vector<std::string> vecFiles = m_commInstance-  
>getJPointFileList();  
    for (const std::string &file : vecFiles) {  
        listFiles << QString::fromStdString(file);  
    }  
  
    if (!vecFiles.empty()) {  
        m_commInstance-  
>setCurJPointFile(QString::fromStdString(vecFiles.at(0)));  
    }  
  
    emit CommunicationEngine::instance()-  
>signal_handleProject_result(  
        absCmd->m_object, absCmd->m_cmdType, QVariant(listFiles),  
        true, QString());
```



```
}
```

```
C++
std::vector<std::string> ProjectInterface::getRPointFileList()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<std::string> fileNames = comm()->project()-
>GetRPointFileList();
    FREQ_LOG_PRINT_TIMESTAMP
    return fileNames;
}

std::vector<std::string> ProjectInterface::getJPointFileList()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<std::string> fileNames = comm()->project()-
>GetJPointFileList();

    FREQ_LOG_PRINT_TIMESTAMP

    return fileNames;
}
```

3.2 查看点文件详情 (setCurRPointFile / setCurJPointFile / readRPoints / readJPoints)

3.2.1 说明

接口	功能	返回值	参数	参数说明
void RobotPointInterface::setCur	设置当前 P 点文件	无	sName	当前选中的 P 点文件名

RPointFile(const QString &sName)				
void JointPointInterface::setCurJPointFile(const QString &sName)	设置当前 JP 点文件	无	sName	当前选中的 JP 点文件名
QVector<InoRPointInfo> RobotPointInterface::readRPoints(std::string sFileName)	读取当前 P 点文件点位明细	QVector<InoRPointInfo>	sFileName	文件名（空则用当前文件）
QVector<InoJPointInfo> JointPointInterface::readJPoints(std::string sFileName)	读取当前 JP 点文件点位明细	QVector<InoJPointInfo>	sFileName	文件名（空则用当前文件）

参数说明

```

C++
// pointdata.h
typedef struct InoRPointInfo {
    int pointNo = -1;
    QString name;
    QString label;
    QString description;
    InoRobPos pos;
} InoRPointInfo;

typedef struct InoJPointInfo {
    int pointNo = -1;
    QString name;
    QString label;
    QString description;
    InoJPos pos;

```

```
} InoJPointInfo;
```

```
C++
// pointdata.h
typedef struct InoRobPos {
    double RPosData[POSE_AXIS_COUNT]; // XYZABC
    int ArmParm[ARM_TYPE_COUNT]; // 臂参数
    double EPosData[EXT_AXIS_COUNT]; // E1-E6
} InoRobPos;

typedef struct InoJPos {
    double JointData[JOINT_AXIS_COUNT]; // J1-J8
    double EPosData[EXT_AXIS_COUNT]; // E1-E6
} InoJPos;
```

3.2.2 调用

```
C++
// RobotPointFileDetailForm
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_SetCurRobotPointFile, sName);
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReadRobotPoints,
    QString::number(nIndex));

// JointPointFileDetailForm
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_SetCurJointPointFile, sName);
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReadJointPoints,
    QString::number(nIndex));
```

3.2.3 函数实现

```
C++
void CommunicationThread::setCurRobotPointFile(AbstractCmd
*absCmd)
{
```

```

    QString sErrMsg;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to set current point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        m_commInstance->setCurRPointFile(cmd->m_value.toString());
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,
sErrMsg);
}

void CommunicationThread::setCurJointPointFile(AbstractCmd
*absCmd)
{
    QString sErrMsg;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to set current joint point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        m_commInstance->setCurJPointFile(cmd->m_value.toString());
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,
sErrMsg);
}

```

```

C++
void RobotPointInterface::setCurRPointFile(const QString &sName)
{
    d->setCurRPointFile(sName);
}

 QVector<InoRPointInfo> RobotPointInterface::readRPoints(string
 sFileName)
{
    return d->readRPoints(std::move(sFileName));
}

void JointPointInterface::setCurJPointFile(const QString &sName)
{
    if (!sName.isEmpty()) {
        m_sJPointFileName = sName.toStdString();
    }
}

 QVector<InoJPointInfo>
JointPointInterface::readJPoints(std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::string errMsg;
    QVector<InoJPointInfo> infos;

    sFileName = sFileName.empty() ? m_sJPointFileName : sFileName;

    std::vector<MJPosItem> points;
    InoJPointInfo info;
    infos.clear();

    int ret = comm()->project()->GetJPoint()->ReadPoints(points,
 sFileName);
    if (ret != ERR_OK) {
        return infos;
    }

    comm()->project()->GetJPoint()->AddViewPoints(sFileName,
 points);

    size_t count = points.size();
    for (size_t i = 0; i < count; i++) {

```

```

        info.pointNo = points[i].PointNo;
        info.name =
QString("%1%2%3").arg("JP[").arg(points[i].PointNo, 4, 10,
QLatin1Char('0')).arg("]");
        info.label = QString::fromStdString(points[i].LabelName);
        info.description =
QString::fromStdString(points[i].Description);
        info.pos.JointData[0] =
QString::number(points[i].JData.JointData[0], 'f', 3).toDouble();
        info.pos.JointData[1] =
QString::number(points[i].JData.JointData[1], 'f', 3).toDouble();
        info.pos.JointData[2] =
QString::number(points[i].JData.JointData[2], 'f', 3).toDouble();
        info.pos.JointData[3] =
QString::number(points[i].JData.JointData[3], 'f', 3).toDouble();
        info.pos.JointData[4] =
QString::number(points[i].JData.JointData[4], 'f', 3).toDouble();
        info.pos.JointData[5] =
QString::number(points[i].JData.JointData[5], 'f', 3).toDouble();
        info.pos.EPosData[0] =
QString::number(points[i].JData.EPosData[0], 'f', 3).toDouble();
        info.pos.EPosData[1] =
QString::number(points[i].JData.EPosData[1], 'f', 3).toDouble();
        info.pos.EPosData[2] =
QString::number(points[i].JData.EPosData[2], 'f', 3).toDouble();
        info.pos.EPosData[3] =
QString::number(points[i].JData.EPosData[3], 'f', 3).toDouble();
        info.pos.EPosData[4] =
QString::number(points[i].JData.EPosData[4], 'f', 3).toDouble();
        info.pos.EPosData[5] =
QString::number(points[i].JData.EPosData[5], 'f', 3).toDouble();
        infos.append(info);
    }

    std::sort(infos.begin(), infos.end(),
        [](const InoJPointInfo &a, const InoJPointInfo &b) {
            return a.pointNo < b.pointNo;
        });

    FREQ_LOG_PRINT_TIMESTAMP
    return infos;
}

```

3.3 新建点文件 (getDefaultRPFileName / createRPointFile)

3.3.1 说明

接口	功能	返回值	参数	参数说明
QString RobotPointInterface ::getDefaultRPFileName(QString baseName)	生成默认点 文件名	QString	baseName	可空，默认 自动命名
bool RobotPointInterface ::createRPointFile(const QString &sName)	创建 P 点文 件	bool	sName	点文件名 (不含后缀 或基础名)
bool ProjectInterface::saveProject(const InoSyncProjcetInfoType &type, const QString &sProjectName)	保存并同步 点文件变更	bool	type	本节点使用 InoSyncProj cetInfoType_ GlobalRPoin t

参数说明

```
C++  
// projectdata.h  
enum InoSyncProjcetInfoType  
{  
    InoSyncProjcetInfoType_ProjectInfo = 0,  
    InoSyncProjcetInfoType_ProgramFiles,  
    InoSyncProjcetInfoType_GlobalRPoint,  
    InoSyncProjcetInfoType_GlobalJPoint,  
    ...  
}
```

```
};
```

- InoSyncProjcetInfoType_GlobalRPoint: P 点文件变更保存同步类型。

3.3.2 调用

C++

```
CommunicationEngine::instance()->enqueueCmd(  
    this, AbstractCmd::CmdType_GetDefaultRPointFileName);  
  
CommunicationEngine::instance()->enqueueCmd_handleProject(  
    this, AbstractCmd::CmdType_CreateRobotPointFile, sFileName);
```

3.3.3 函数实现

C++

```
void CommunicationThread::getDefaultRPointFileName(AbstractCmd  
*absCmd)  
{  
    QString sErrMsg;  
    QString sFileName;  
    bool isOk = true;  
  
    bool isUpperLimit = m_commInstance->isRPointFileUpperLimit();  
    if (isUpperLimit) {  
        sErrMsg = tr("The number of point files reaches the  
limit.");  
        isOk = false;  
    }  
  
    if (isOk) {  
        sFileName = m_commInstance->getDefaultRPFileName();  
    }  
  
    emit CommunicationEngine::instance()-  
>signal_handleProject_result(  
        absCmd->m_object, absCmd->m_cmdType, sFileName, isOk,  
sErrMsg);  
}  
  
void CommunicationThread::createRobotPointFile(AbstractCmd
```



```

*absCmd)
{
    QString sErrMsg;
    QString sFileName;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to create point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        sFileName = cmd->m_value.toString();
        std::string msg;
        isOk = m_commInstance->isRPointFileNameValid(sFileName,
msg);
        if (!isOk) {
            sErrMsg = tr("Illegal point file name.");
        }
    }

    if (isOk) {
        bool isUpperLimit = m_commInstance-
>isRPointFileUpperLimit();
        if (isUpperLimit) {
            sErrMsg = tr("The number of point files reaches the
limit.");
            isOk = false;
        }
    }

    if (isOk) {
        bool isExist = m_commInstance-
>isRPointFileExist(sFileName);
        if (isExist) {
            sErrMsg = tr("Point file has already exists.");
            isOk = false;
        }
    }

    if (isOk) {
        isOk = m_commInstance->createRPointFile(sFileName);
    }
}

```

```

        if (!isOk) {
            sErrMsg = tr("Failed to create point file.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->saveProject(InoSyncProjcetInfoType_GlobalRPoint);
        if (!isOk) {
            FileUtils::delFile(CONTROLLER_TMP_DIR_PATH);
            m_commInstance->deleteRPointFile(sFileName);
            sErrMsg = tr("Failed to save point file.");
        }
    }

    if(isOk){
        emit CommunicationEngine::instance()->signal_pPorintFileChanged(QStringList()<<(sFileName + ".pts"),
        AddPointFile);
    }

    emit CommunicationEngine::instance()->signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant(sFileName) : QVariant(), isOk, sErrMsg);
}

```

3.4 删除点文件 (deleteRPointFile/saveProject)

3.4.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::deleteRPointFile (const QString	删除 P 点文件	bool	sName	点文件名

&sName)				
bool ProjectInterface::saveProject(InoSyncProjcetInfoType)	删除后保存 同步	bool	InoSyncProjcetInfoType_GlobalRPoint	同步点文件 变化

参数说明（含自定义结构体定义源码）

```
Thrift
enum InoSyncProjcetInfoType
{
    InoSyncProjcetInfoType_ProjectInfo = 0,      // 工程配置文件信息
    (包括静态任务)
    InoSyncProjcetInfoType_ProgramFiles,          // 程序文件
    InoSyncProjcetInfoType_GlobalRPoint,          // 全局 P 点位
    InoSyncProjcetInfoType_GlobalJPoint,          // 全局 JP 点位
    InoSyncProjcetInfoType_LabelInfo,             // 标签信息
    InoSyncProjcetInfoType_UserDefineWarning,     // 用户自定义报警
    SYNC_PROGRAM_COMPILE = 15                     //程序是否译码，0-
    不编译，1-非静态任务编译（示教器编程切调试/自动，全部保存），2-全部编译
};
```

```
C++
// pointdata.h
const QString DEFAULT_ROBOT_POINT_FILE = "P.pts";
```

- P.pts 为默认点文件，不允许删除。

3.4.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_DeleteRobotPointFile, sFileName);
```

3.4.3 函数实现

```
C++
void CommunicationThread::deleteRobotPointFile(AbstractCmd
```

```

*absCmd)
{
    QString sErrMsg;
    QString sFileName;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to delete point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        sFileName = cmd->m_value.toString();
        if (sFileName.compare(DEFAULT_ROBOT_POINT_FILE) == 0) {
            sErrMsg = tr("P.pts is not allowed to delete.");
            isOk = false;
        }
    }

    if (isOk) {
        std::string msg;
        isOk = m_commInstance->isRPointFileNameValid(sFileName,
msg);
        if (!isOk) {
            sErrMsg = tr("The file name is invalid.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->isRPointFileExist(sFileName);
        if (!isOk) {
            sErrMsg = tr("Point file [%1] does not
exist.").arg(sFileName);
        }
    }

    if (isOk) {
        isOk = m_commInstance->deleteRPointFile(sFileName);
        if (!isOk) {
            sErrMsg = tr("Failed to delete point file.");
        }
    }
}

```

```

        if (isOk) {
            isOk = m_commInstance-
>saveProject(InoSyncProjcetInfoType_GlobalRPoint);
            if (!isOk) {
                m_commInstance->addRobotPointConfig(sFileName);
                sErrMsg = tr("Failed to delete point file from the
project.");
            }
        }
        if(isOk){
            emit CommunicationEngine::instance()-
>signal_pPorintFileChanged(QStringList()<<sFileName,
DeletePointFile);
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            isOk ? QVariant(sFileName) : QVariant(), isOk, sErrMsg);
    }

bool RobotPointInterfacePrivate::deleteRPointFile(const QString
&sName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    if (sName.isEmpty()) {
        return false;
    }

    int nRet = q->comm()->project()->GetRPoint()-
>DelFile(sName.toStdString());
    if (ERR_OK != nRet) {
        return false;
    }

    q->comm()->project()->DelRPointConfig(sName.toStdString());

    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool ProjectInterface::saveProject(const InoSyncProjcetInfoType
&type,

```

```

const QString &sProjectName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int nRet = ERR_OK;
    if (InoSyncProjcetInfoType_ProgramFiles == type
        || InoSyncProjcetInfoType_ProjectInfo == type) {
        nRet = comm()->project()->SaveProjectConfig();
    } else {
        nRet = comm()->project()->saveProject();
    }

    if (nRet != ERR_OK) {
        return false;
    }

    if (!sProjectName.isEmpty()) {
        std::string sPrjCtrlPath =
ProjectHelper::ControllerActiveProjectFolder(sProjectName.toStdString());
        signed char flags[SYNC_TYPE_NUMBER] = {0};

        flags[(int)InoSyncProjcetInfoType::InoSyncProjcetInfoType_ProjectI
nfo] = 1;
        flags[(int)InoSyncProjcetInfoType::SYNC_PROGRAM_COMPILE] =
3;
        _ITransfor->syncProjectInfo(sPrjCtrlPath, flags);
    } else {
        comm()->project()->sendSyncFlag(static_cast<int>(type),
true);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

```

3.5 复制/粘贴点文件 (copyRPointFile / getCopiedRPointFileName / pasteRPointFile)

3.5.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::copyRobotPointFile(const QString &sName)	复制点文件	bool	sName	源文件名
QString RobotPointInterface::getCopiedRPointFileName()	获取复制缓存名	QString	无	用于生成粘贴默认名
bool RobotPointInterface::pasteRPointFile(const QString &sName)	粘贴为新点文件	bool	sName	目标文件名
QString RobotPointInterface::getDefaultRPFFileName(QString baseName)	生成粘贴名	QString	baseName	常用 copiedName 作为前缀

3.5.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CopyRobotPointFile, sFileName);

CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_GenPasterRPointFileName);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_PasteRobotPointFile, sFileName);
```

3.5.3 函数实现

```
C++
void CommunicationThread::copyRobotPointFile(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QString sFileName;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to copy point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        sFileName = cmd->m_value.toString();
        isOk = m_commInstance->isRPointFileExist(sFileName);
        if (!isOk) {
            sErrMsg = tr("Point file does not exist.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->copyRPointFile(sFileName);
        if (!isOk) {
            sErrMsg = tr("Failed to copy point file.");
        }
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(sFileName),
        isOk, sErrMsg);
}

void CommunicationThread::genPasteRobotFileName(AbstractCmd
*absCmd)
{
    QString sErrMsg;
    QString sFileName;
```



```

    bool isOk = true;

    bool isUpperLimit = m_commInstance->isRPointFileUpperLimit();
    if (isUpperLimit) {
        sErrMsg = tr("The number of point files reaches the
limit.");
        isOk = false;
    }

    if (isOk) {
        QString sRPointFileName = m_commInstance-
>getCopiedRPointFileName();
        if (!sRPointFileName.isEmpty()) {
            sFileName = m_commInstance-
>getDefaultRPFileName(sRPointFileName);
        }
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(sFileName),
isOk, sErrMsg);
}

void CommunicationThread::pasteRobotPointFile(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QString sFileName;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to paste point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        sFileName = cmd->m_value.toString();
        if (sFileName.compare(DEFAULT_ROBOT_POINT_FILE) == 0) {
            sErrMsg = tr("P.pts is not allowed to paste.");
            isOk = false;
        }
    }
}

```

```

        if (isOk) {
            std::string errMsg;
            isOk = m_commInstance->isRPointFileNameValid(sFileName,
errMsg);
            if (!isOk) {
                sErrMsg = tr("The point file name is invalid.");
                isOk = false;
            }
        }

        if (isOk) {
            bool isExist = m_commInstance-
>isRPointFileExist(sFileName);
            if (isExist) {
                sErrMsg = tr("The point file already exists.");
                isOk = false;
            }
        }

        if (isOk) {
            isOk = m_commInstance->pasteRPointFile(sFileName);
            if (!isOk) {
                m_commInstance->deleteRPointFile(sFileName);
                sErrMsg = tr("Failed to paste point file.");
            }
        }

        if (isOk) {
            isOk = m_commInstance-
>saveProject(InoSyncProjcetInfoType_GlobalRPoint);
            if (!isOk) {
                m_commInstance->deleteRPointFile(sFileName);
                sErrMsg = tr("Failed to save point file to the
project.");
            }
        }
        if(isOk){
            emit CommunicationEngine::instance()-
>signal_pPorintFileChanged(QStringList()<<(sFileName + ".pts"),
AddPointFile);
        }
        emit CommunicationEngine::instance()-
>signal_handleProject_result(

```

```

        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant(sFileName) : QVariant(), isOk, sErrMsg);
}

```

```

C++
bool RobotPointInterfacePrivate::copyRPointFile(const QString
&sName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    if (sName.isEmpty()) {
        return false;
    }

    q->comm()->project()->GetRPoint()-
>SetCopyFile(sName.toStdString());
    std::string copiedFileName = q->comm()->project()-
>GetRPoint()->GetCopiedName();
    // copied file name is without extension
    FREQ_LOG_PRINT_TIMESTAMP
    return
(sName.contains(QString::fromStdString(copiedFileName)));
}

QString RobotPointInterfacePrivate::getCopiedRPointFileName()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::string copiedFileName = q->comm()->project()-
>GetRPoint()->GetCopiedName();
    FREQ_LOG_PRINT_TIMESTAMP
    return QString::fromStdString(copiedFileName);
}

bool RobotPointInterfacePrivate::pasteRPointFile(const QString
&sName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    if (sName.isEmpty()) {
        return false;
    }

    std::string copiedFileName = q->comm()->project()-

```

```

>GetRPoint()->GetCopiedName();
    if (copiedFileName.empty()) {
        return false;
    }

    int nRet = q->comm()->project()->GetRPoint()-
>PasteFile(copiedFileName,

sName.toStdString());
    qDebug() << "pasteRPointFile, ret = " << nRet << ", file = "
<< sName;
    if (ERR_OK != nRet) {
        return false;
    }

    q->comm()->project()->AddRPointConfig(sName.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

QString RobotPointInterfacePrivate::getDefaultRPFileName(QString
baseName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<std::string> vecRPointFileList = q->comm()-
>project()->GetRPointFileList();

    std::string sFileName = "";
    if (baseName.isEmpty()) {
        sFileName = q->comm()->project()->GetRPoint()-
>NewDefaultFileName(vecRPointFileList);
    } else {
        sFileName = q->comm()->project()->GetRPoint()-
>NewDefaultFileName(
        vecRPointFileList, baseName.toStdString());
    }

    FREQ_LOG_PRINT_TIMESTAMP

    return QString::fromStdString(sFileName);
}

```

3.6 导入点文件 (importLocalRPointFile)

3.6.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::importLocalRPointFile(const QString &name, QString path)	导入本地点文件到工程	bool	name,path	点文件名 + 本地路径

参数说明

```
C++  
// CmdCommonValue.m_value 在导入时包装为 QVector<QVariant>  
// vec[0]: QVector<QString> fileNames  
// vec[1]: QVector<QString> filePaths
```

- 支持一次导入多个点文件。
- 每个文件都会先做名称合法性、上限、重名校验，再逐个导入。

3.6.2 调用

```
C++  
CommunicationEngine::instance()->enqueueCmd_handleProject(  
    this, AbstractCmd::CmdType_ImportRobotPointFile,  
    QVector<QVariant>() << QVariant::fromValue(fileNames)  
        << QVariant::fromValue(filePaths));
```

3.6.3 函数实现

```
C++
```

```

void CommunicationThread::importRobotPointFile(AbstractCmd
*absCmd)
{
    QString sErrMsg;
    QVector<QString> fileNames;
    QVector<QString> filePaths;
    QVector<QVariant> vecValues;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to import point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to import the point file as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        fileNames = vecValues.at(0).value<QVector<QString>>>();
        if (fileNames.isEmpty()) {
            sErrMsg = tr("Failed to parse point file name.");
            isOk = false;
        }
    }

    if (isOk) {
        filePaths = vecValues.at(1).value<QVector<QString>>>();
        if (filePaths.isEmpty()) {
            sErrMsg = tr("Failed to parse point file path.");
            isOk = false;
        }
    }

    if (isOk) {
        std::string msg;
        for (const QString &fileName : fileNames) {

```

```

        isOk = m_commInstance->isRPointFileNameValid(fileName,
msg);
        if (!isOk) {
            sErrMsg = tr("The point file name is invalid.");
            isOk = false;
            break;
        }
    }

    if (isOk) {
        bool isUpperLimit = m_commInstance-
>isRPointFileUpperLimit(fileNames.count());
        if (isUpperLimit) {
            sErrMsg = tr("The number of point files reaches the
limit.");
            isOk = false;
        }
    }

    if (isOk) {
        for (const QString &fileName : fileNames) {
            bool isExist = m_commInstance-
>isRPointFileExist(fileName);
            if (isExist) {
                sErrMsg = tr("The point file is exist.");
                isOk = false;
                break;
            }
        }
    }

    if (isOk) {
        for (int i = 0; i < fileNames.count(); i++) {
            isOk = m_commInstance-
>importLocalRPointFile(fileNames.at(i), filePaths.at(i));
            if (!isOk) {
                sErrMsg = tr("Failed to import point file.");
                break;
            }
        }

        emit CommunicationEngine::instance()-
>signal_pPorintFileChanged(QStringList()<<(fileNames.at(i) +
".pts"), AddPointFile);
    }

```


3.7 导出点文件 (exportRPointFile)

3.7.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInt erface::export RPointFile(QS tring name, QString folder)	导出点文件到 本地目录	bool	name,folder	点文件名 + 导 出目录

参数说明

```
C++
// CmdCommonValue.m_value 在导出时包装为 QVector<QVariant>
// vec[0]: QString sRPointFileName
// vec[1]: QString sExportFilePath
```

3.7.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ExportRobotPointFile,
    QVector<QVariant>() << sFileName << sExportPath);
```

3.7.3 函数实现

```
C++
void CommunicationThread::exportRobotPointFile(AbstractCmd
*absCmd)
{
    QString sErrMsg;
    QString sRPointFileName = QString();
    QString sExportFilePath = QString();
```

```

    QVector<QVariant> vecValues;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to export point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to export the point file as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        sRPointFileName = vecValues.at(0).toString();
        if (sRPointFileName.isEmpty()) {
            sErrMsg = tr("Failed to parse point file name.");
            isOk = false;
        }
    }

    if (isOk) {
        sExportFilePath = vecValues.at(1).toString();
        if (sExportFilePath.isEmpty()) {
            sErrMsg = tr("Failed to parse export file path.");
            isOk = false;
        }
    }

    if (isOk) {
        isOk = m_commInstance->exportRPointFile(sRPointFileName,
sExportFilePath);
        if (!isOk) {
            sErrMsg = tr("Failed to export point file.");
        }
    }

    emit CommunicationEngine::instance()-

```

```

>signal_handleProject_result(
    absCmd->m_object, absCmd->m_cmdType,
    isOk ? QVariant(sRPointFileName) : QVariant(), isOk,
    sErrMsg);
}

bool RobotPointInterfacePrivate::exportRPointFile(QString name,
QString folder)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<std::string> names;
    names.push_back(name.toStdString());

    folder = QDir::fromNativeSeparators(folder);
    int ret = q->comm()->project()->GetRPoint()->ExportFiles(
        names, folder.toStdString());

    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

```

4.点位操作相关

4.1 读取点位列表与定位检查（readRPoints / readJPoints / findRPoint / findJPoint）

4.1.1 说明

接口	功能	返回值	参数	参数说明
QVector<InoRPointInfo> > RobotPointInterface::readRPoints(string sFileName)	读取 P.pts 当前点位列表	QVector<InoRPointInfo> >	sFileName	文件名，空 则使用当前 文件

QVector<InoJPointInfo> JointPointInterface::readJPoints(std::string sFileName)	读取 JP.pts 当前点位列表	QVector<InoJPointInfo>	sFileName	文件名，空则使用当前文件
bool RobotPointInterface::findRPoint(int nIndex)	检查 P 点是否存在	bool	nIndex	点位索引
bool JointPointInterface::findJPoint(int nIndex)	检查 JP 点是否存在	bool	nIndex	点位索引

参数说明

```
C++
typedef struct InoRPointInfo {
    int pointNo = -1;
    QString name;
    QString label;
    QString description;
    InoRobPos pos;
} InoRPointInfo;

typedef struct InoJPointInfo {
    int pointNo = -1;
    QString name;
    QString label;
    QString description;
    InoJPos pos;
} InoJPointInfo;
```

4.1.2 调用

```
C++
// P 点列表读取
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReadRobotPoints,
    QString::number(nIndex));
```

```

// JP 点列表读取
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReadJointPoints,
    QString::number(nIndex));

// P 点定位检查
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CheckRobotPointIsExist,
    QVariant(nIndex));

// JP 点定位检查
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CheckJointPointIsExist,
    QVariant(nIndex));

```

4.1.3 函数实现

```

C++
void CommunicationThread::readRobotPoints(AbstractCmd *absCmd)
{
    QVector<InoRPointInfo> vecRPointInfos = m_commInstance-
>readRPoints();

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        QVariant::fromValue(vecRPointInfos), true,
        dynamic_cast<CmdCommonValue *>(absCmd)-
>m_value.toString());
}

void CommunicationThread::readJointPoints(AbstractCmd *absCmd)
{
    QVector<InoJPointInfo> vecJPointInfos = m_commInstance-
>readJPoints();

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        QVariant::fromValue(vecJPointInfos), true,
        dynamic_cast<CmdCommonValue *>(absCmd)-
>m_value.toString());
}

```

```
}
```

```
C++
QVector<InoRPointInfo>
RobotPointInterfacePrivate::readRPoints(std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    std::string errMsg;
    QVector<InoRPointInfo> infos;

    sFileName = sFileName.empty() ? m_sRPointFileName : sFileName;

    std::vector<MRobPosItem> points;
    InoRPointInfo info;
    infos.clear();

    // 读取点位
    int ret = q->comm()->project()->GetRPoint()-
>ReadPoints(points, sFileName);
    if (ret != ERR_OK) {
        return infos;
    }

    // 添加到缓存中
    q->comm()->project()->GetRPoint()->AddViewPoints(sFileName,
points);

    // 赋值
    size_t count = points.size();
    for (size_t i = 0; i < count; i++) {
        info.pointNo = points[i].PointNo; // 点位序号
        info.name =
QString("%1%2%3").arg("P").arg(points[i].PointNo, 4, 10,
QLatin1Char('0')).arg("]");
        info.label = QString::fromStdString(points[i].LabelName);
    // 标签
        info.description =
QString::fromStdString(points[i].Description);
    // 描述(备注)
        info.pos.RPosData[0] =
QString::number(points[i].PData.RPosData[0], 'f', 3).toDouble();
    // XYZABC
        info.pos.RPosData[1] =
```

```

QString::number(points[i].PData.RPosData[1], 'f', 3).toDouble();
    info.pos.RPosData[2] =
QString::number(points[i].PData.RPosData[2], 'f', 3).toDouble();
    info.pos.RPosData[3] =
QString::number(points[i].PData.RPosData[3], 'f', 3).toDouble();
    info.pos.RPosData[4] =
QString::number(points[i].PData.RPosData[4], 'f', 3).toDouble();
    info.pos.RPosData[5] =
QString::number(points[i].PData.RPosData[5], 'f', 3).toDouble();
    info.pos.ArmParm[0] =
QString::number(points[i].PData.ArmParm[0], 'f', 3).toDouble();
// 臂参数
    info.pos.ArmParm[1] =
QString::number(points[i].PData.ArmParm[1], 'f', 3).toDouble();
    info.pos.ArmParm[2] =
QString::number(points[i].PData.ArmParm[2], 'f', 3).toDouble();
    info.pos.ArmParm[3] =
QString::number(points[i].PData.ArmParm[3], 'f', 3).toDouble();
    info.pos.EPosData[0] =
QString::number(points[i].PData.EPosData[0], 'f', 3).toDouble();
// E1~E6
    info.pos.EPosData[1] =
QString::number(points[i].PData.EPosData[1], 'f', 3).toDouble();
    info.pos.EPosData[2] =
QString::number(points[i].PData.EPosData[2], 'f', 3).toDouble();
    info.pos.EPosData[3] =
QString::number(points[i].PData.EPosData[3], 'f', 3).toDouble();
    info.pos.EPosData[4] =
QString::number(points[i].PData.EPosData[4], 'f', 3).toDouble();
    info.pos.EPosData[5] =
QString::number(points[i].PData.EPosData[5], 'f', 3).toDouble();
    infos.append(info);
}

std::sort(infos.begin(), infos.end(),
    [](const InoRPointInfo &a, const InoRPointInfo &b) {
        return a.pointNo < b.pointNo;
    });

FREQ_LOG_PRINT_TIMESTAMP
return infos;
}

 QVector<InoJPointInfo>

```

```

JointPointInterface::readJPoints(std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::string errMsg;
    QVector<InoJPointInfo> infos;

    sFileName = sFileName.empty() ? m_sJPointFileName : sFileName;

    std::vector<MJPosItem> points;
    InoJPointInfo info;
    infos.clear();

    // 读取点位
    int ret = comm()->project()->GetJPoint()->ReadPoints(points,
sFileName);
    if (ret != ERR_OK) {
        return infos;
    }

    // 添加到缓存中
    comm()->project()->GetJPoint()->AddViewPoints(sFileName,
points);

    // 赋值
    size_t count = points.size();
    for (size_t i = 0; i < count; i++) {
        info.pointNo = points[i].PointNo; // 点位序号
        info.name =
QString("%1%2%3").arg("JP").arg(points[i].PointNo, 4, 10,
QLatin1Char('0')).arg("]");
        info.label = QString::fromStdString(points[i].LabelName);
    // 标签
        info.description =
QString::fromStdString(points[i].Description);
    // 描述(备注)
        info.pos.JointData[0] =
QString::number(points[i].JData.JointData[0], 'f', 3).toDouble();
    // J1-J6
        info.pos.JointData[1] =
QString::number(points[i].JData.JointData[1], 'f', 3).toDouble();
        info.pos.JointData[2] =
QString::number(points[i].JData.JointData[2], 'f', 3).toDouble();
        info.pos.JointData[3] =

```



```

QString::number(points[i].JData.JointData[3], 'f', 3).toDouble();
    info.pos.JointData[4] =
QString::number(points[i].JData.JointData[4], 'f', 3).toDouble();
    info.pos.JointData[5] =
QString::number(points[i].JData.JointData[5], 'f', 3).toDouble();
    info.pos.EPosData[0] =
QString::number(points[i].JData.EPosData[0], 'f', 3).toDouble();
// E1~E6
    info.pos.EPosData[1] =
QString::number(points[i].JData.EPosData[1], 'f', 3).toDouble();
    info.pos.EPosData[2] =
QString::number(points[i].JData.EPosData[2], 'f', 3).toDouble();
    info.pos.EPosData[3] =
QString::number(points[i].JData.EPosData[3], 'f', 3).toDouble();
    info.pos.EPosData[4] =
QString::number(points[i].JData.EPosData[4], 'f', 3).toDouble();
    info.pos.EPosData[5] =
QString::number(points[i].JData.EPosData[5], 'f', 3).toDouble();
    infos.append(info);
}

std::sort(infos.begin(), infos.end(),
    [](const InoJPointInfo &a, const InoJPointInfo &b) {
        return a.pointNo < b.pointNo;
    });

FREQ_LOG_PRINT_TIMESTAMP
return infos;
}

```

4.2 运动至点 (robotMoveJointTeachInterface / robotMoveLineInterface)

4.2.1 说明

接口	功能	返回值	参数	参数说明
----	----	-----	----	------

bool RobotControll nterface::robo tMoveJointTe achInterface(c onst RoadPoint &roadPoint)	关节运动到选 中点	bool	roadPoint	P.pts / JP.pts 都可转成关节 运动目标
bool RobotControll nterface::robo tMoveLineInte rface(const RoadPoint &destPt)	直线运动到选 中 P 点	bool	destPt	主要用于 P.pts

参数说明

```
C++
class METATYPE_EXPORT RoadPoint
{
public:
    Pos m_position;
    Ori m_orientation;
    double m_jointAngle[ROBOT_DOF];
    double m_jointRadian[ROBOT_DOF];
    int m_arm[ROBOT_ARM_NUM];
    bool m_isSingular = false;
};
```

4.2.2 调用

```
C++
// P 点 MoveJ / MoveL
CommunicationEngine::instance()-
>enqueueCmd_robotMoveJointTeach(this, point);
CommunicationEngine::instance()->enqueueCmd_robotMoveLine(this,
point);

// JP 点 MoveJ
```

```
CommunicationEngine::instance()-  
>enqueueCmd_robotMoveJointTeach(this, point);
```

4.2.3 函数实现

```
C++  
bool RobotControlInterface::robotMoveJointTeachInterface(const  
RoadPoint &roadPoint)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    bool ret = true;  
  
    ret = _IMotion->JointMoveToPointStart(  
        MetaTypeConversion::tp2InoApi_roadPoint(roadPoint));  
  
    if (ret != 0) {  
        LOG_INFO(QString("robotMoveJointTeachInterface  
ret : %1").arg(QString::number(ret)));  
    }  
  
    FREQ_LOG_PRINT_TIMESTAMP  
    return ret;  
}  
  
bool RobotControlInterface::robotMoveLineInterface(const RoadPoint  
&destPt)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    bool ret = true;  
    int16u toolId = comm()->GetCurToolId();  
    ReferObj referObj = static_cast<ReferObj>(comm()-  
>GetCurWObjId());  
  
    RobPos robPos =  
MetaTypeConversion::tp2InoApi_roadPoint2RobPos(destPt);  
    ret = _IMotion->MovlToPointStart(  
        robPos,  
        toolId,  
        referObj);  
    FREQ_LOG_PRINT_TIMESTAMP  
    return ret;  
}
```

4.3 添加点 (addRPoint / addJPoint)

4.3.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::addRPoint(int &nIndex, const std::string &sLabelName)	在 P.pts 添加 当前位姿为新 点	bool	nIndex,sLabel Name	返回新索引, 可带标签
bool JointPointInterface::addJPoi nt(int &nIndex, const std::string &sLableName)	在 JP.pts 添加 当前关节点	bool	nIndex,sLable Name	返回新索引, 可带标签

4.3.2 调用

```
C++  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType_AddRobotPoint, std::string(""));  
  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType_AddJointPoint, std::string(""));
```

4.3.3 函数实现

```
C++
void CommunicationThread::addRobotPoint(AbstractCmd *abcCmd)
{
    int nIndex = INVALID_INDEX;
    QString sErrMsg;
    auto [sLabelName] = ((CmdDatas<std::string> *)abcCmd)->m_data;

    bool isOk = m_commInstance->addRPoint(nIndex, sLabelName);
    if (!isOk || nIndex <= INVALID_INDEX) {
        isOk = false;
        sErrMsg = tr("Failed to add the point.");
    }

    if (isOk) {
        isOk = m_commInstance->saveRPoints();
        if (!isOk) {
            m_commInstance->deleteRPoint(nIndex);
            sErrMsg = tr("Failed to add the point due to save
error.");
        }
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        abcCmd->m_object, abcCmd->m_cmdType, QVariant(nIndex),
        isOk, sErrMsg);
}

void CommunicationThread::addJointPoint(AbstractCmd *abcCmd)
{
    int nIndex = INVALID_INDEX;
    QString sErrMsg;
    auto [sLabelName] = ((CmdDatas<std::string> *)abcCmd)->m_data;

    bool isOk = m_commInstance->addJPoint(nIndex, sLabelName);
    if (!isOk || nIndex <= INVALID_INDEX) {
        isOk = false;
        sErrMsg = tr("Failed to add joint point.");
    }
}
```

```

    if (isOk) {
        isOk = m_commInstance->saveJPoints();
        if (!isOk) {
            m_commInstance->deleteJPoint(nIndex);
            sErrMsg = tr("Failed to save joint point file.");
        }
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        abcCmd->m_object, abcCmd->m_cmdType, QVariant(nIndex),
        isOk, sErrMsg);
}

QVector<InoJPointInfo>
JointPointInterface::readJPoints(std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::string errMsg;
    QVector<InoJPointInfo> infos;

    sFileName = sFileName.empty() ? m_sJPointFileName : sFileName;

    std::vector<MJPosItem> points;
    InoJPointInfo info;
    infos.clear();

    // 读取点位
    int ret = comm()->project()->GetJPoint()->ReadPoints(points,
sFileName);
    if (ret != ERR_OK) {
        return infos;
    }

    // 添加到缓存中
    comm()->project()->GetJPoint()->AddViewPoints(sFileName,
points);

    // 赋值
    size_t count = points.size();
    for (size_t i = 0; i < count; i++) {
        info.pointNo = points[i].PointNo; // 点位序号
        info.name =

```

```

QString("%1%2%3").arg("JP[").arg(points[i].PointNo, 4, 10,
QLatin1Char('0')).arg("]");
    info.label = QString::fromStdString(points[i].LabelName);
// 标签
    info.description =
QString::fromStdString(points[i].Description);
// 描述(备注)
    info.pos.JointData[0] =
QString::number(points[i].JData.JointData[0], 'f', 3).toDouble();
// J1-J6
    info.pos.JointData[1] =
QString::number(points[i].JData.JointData[1], 'f', 3).toDouble();
    info.pos.JointData[2] =
QString::number(points[i].JData.JointData[2], 'f', 3).toDouble();
    info.pos.JointData[3] =
QString::number(points[i].JData.JointData[3], 'f', 3).toDouble();
    info.pos.JointData[4] =
QString::number(points[i].JData.JointData[4], 'f', 3).toDouble();
    info.pos.JointData[5] =
QString::number(points[i].JData.JointData[5], 'f', 3).toDouble();
    info.pos.EPosData[0] =
QString::number(points[i].JData.EPosData[0], 'f', 3).toDouble();
// E1~E6
    info.pos.EPosData[1] =
QString::number(points[i].JData.EPosData[1], 'f', 3).toDouble();
    info.pos.EPosData[2] =
QString::number(points[i].JData.EPosData[2], 'f', 3).toDouble();
    info.pos.EPosData[3] =
QString::number(points[i].JData.EPosData[3], 'f', 3).toDouble();
    info.pos.EPosData[4] =
QString::number(points[i].JData.EPosData[4], 'f', 3).toDouble();
    info.pos.EPosData[5] =
QString::number(points[i].JData.EPosData[5], 'f', 3).toDouble();
    infos.append(info);
}

std::sort(infos.begin(), infos.end(),
    [](const InoJPointInfo &a, const InoJPointInfo &b) {
        return a.pointNo < b.pointNo;
    });

FREQ_LOG_PRINT_TIMESTAMP
return infos;
}

```

```

bool JointPointInterface::addJPoint(int &nIndex, const string
&sLableName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoJPos pointValue;
    if (!readCurPoint(pointValue)) {
        return false;
    }

    MJPosItem point;
    point.PointNo = -1;
    point.LabelName = sLableName;
    point.Description = "";
    point.JData = MetaTypeConversion::inoApi2tp_jPos(pointValue);

    int nRet = comm()->project()->GetJPoint()->AddPoint(
        nIndex, m_sJPointFileName, point);

    if (nRet == ERROR_OK) {
        RoadPoint pt =
MetaTypeConversion::inoApi2tp_roadPoint(pointValue);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(
            PREFIX_JP, point.PointNo,
QString::fromStdString(point.LabelName), pt);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

```

4.4 复制/粘贴点 (copyRPoint / pasteRPoint / copyJPoint / pasteJPoint)

4.4.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::copyRobotPoint(const std::vector<int> &indexs)	复制 P 点	bool	indexs	可多选索引
int RobotPointInterface::pasteRPoint(std::vector<int> &indexs)	粘贴 P 点	int	indexs	返回新索引集合
bool JointPointInterface::copyJPoint(const std::vector<int> &indexs)	复制 JP 点	bool	indexs	可多选索引
int JointPointInterface::pasteJPPoint(std::vector<int> &indexs)	粘贴 JP 点	int	indexs	返回新索引集合

4.4.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CopyRobotPoint,
    QVariant::fromValue(m_selectPointIndexList));
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_PasteRobotPoint);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_CopyJointPoint,

```

```
QVariant::fromValue(m_selectPointIndexList));  
CommunicationEngine::instance()->enqueueCmd(  
    this, AbstractCmd::CmdType_PasteJointPoint);
```

4.4.3 函数实现

```
C++  
void CommunicationThread::copyRobotPoint(AbstractCmd *absCmd)  
{  
    QString sErrMsg;  
    bool isOk = true;  
  
    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);  
    if (!cmd) {  
        sErrMsg = tr("Failed to copy point.") +  
SPACE_AND_CONVERT_PARAMETERS_FALID;  
        isOk = false;  
    }  
  
    if (isOk) {  
        QList<int> copiedRPIndexs = cmd-  
>m_value.value<QList<int>>>();  
  
        std::vector<int> indexs;  
        for (const int &index : copiedRPIndexs) {  
            indexs.push_back(index);  
        }  
  
        isOk = m_commInstance->copyRPoint(indexs);  
        if (!isOk) {  
            sErrMsg = tr("Failed to copy point.");  
        }  
    }  
  
    emit CommunicationEngine::instance()-  
>signal_handleProject_result(  
        absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,  
sErrMsg);  
}  
  
void CommunicationThread::pasteRobotPointFile(AbstractCmd *absCmd)  
{
```

```

QString sErrMsg;
QString sFileName;
bool isOk = true;

CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
if (!cmd) {
    sErrMsg = tr("Failed to paste point file.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
    isOk = false;
}

if (isOk) {
    sFileName = cmd->m_value.toString();
    if (sFileName.compare(DEFAULT_ROBOT_POINT_FILE) == 0) {
        sErrMsg = tr("P.pts is not allowed to paste.");
        isOk = false;
    }
}

if (isOk) {
    std::string errMsg;
    isOk = m_commInstance->isRPointFileNameValid(sFileName,
errMsg);
    if (!isOk) {
        sErrMsg = tr("The point file name is invalid.");
        isOk = false;
    }
}

if (isOk) {
    bool isExist = m_commInstance-
>isRPointFileExist(sFileName);
    if (isExist) {
        sErrMsg = tr("The point file already exists.");
        isOk = false;
    }
}

if (isOk) {
    isOk = m_commInstance->pasteRPointFile(sFileName);
    if (!isOk) {
        m_commInstance->deleteRPointFile(sFileName);
        sErrMsg = tr("Failed to paste point file.");
    }
}

```

```

    }

    if (isOk) {
        isOk = m_commInstance-
>saveProject(InoSyncProjcetInfoType_GlobalRPoint);
        if (!isOk) {
            // 移除未保存成功的点文件
            m_commInstance->deleteRPointFile(sFileName);
            sErrMsg = tr("Failed to save point file to the
project.");
        }
    }
    if(isOk){
        emit CommunicationEngine::instance()-
>signal_pPorintFileChanged(QStringList()<<(sFileName + ".pts"),
AddPointFile);
    }
    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant(sFileName) : QVariant(), isOk, sErrMsg);
}

void CommunicationThread::copyJointPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to copy point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        QList<int> copiedJPIndexs = cmd-
>m_value.value<QList<int>>>();

        std::vector<int> indexs;
        for (const int &index : copiedJPIndexs) {
            indexs.push_back(index);
        }
    }
}

```

```

        isOk = m_commInstance->copyJPoint(indexs);
        if (!isOk) {
            sErrMsg = tr("Failed to copy joint point.");
        }
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,
sErrMsg);
}

void CommunicationThread::pasteJointPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    std::vector<int> indexs;
    bool isOk = true;

    if (isOk) {
        isOk = m_commInstance->pasteJPoint(indexs);
        if (!isOk) {
            sErrMsg = tr("Failed to paste joint point.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->saveJPoints();
        if (!isOk) {
            for (const int &index : indexs) {
                m_commInstance->deleteJPoint(index);
            }

            sErrMsg = tr("Failed to paste joint point due to save
error.");
        }
    }
    if(isOk){
        for (const int &index : indexs) {
            emit CommunicationEngine::instance()->
                signal_virtualKeyBoard_LabelChanged(
                    LabelType_GlobalVar_JP,"",
QString::number(index));
        }
    }
}

```

```

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            QVariant::fromValue(indexs), isOk, sErrMsg);
    }

```

```

C++
bool RobotPointInterfacePrivate::copyRPoint(const std::vector<int>
&indexs)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    q->comm()->project()->GetRPoint()-
>CopyPoints(m_sRPointFileName, indexs);

    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

int RobotPointInterfacePrivate::pasterPoint(std::vector<int>
&indexs)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int nRet = q->comm()->project()->GetRPoint()->PastePoints(
        indexs, m_sRPointFileName);
    if (nRet != ERR_OK) {
        return INVALID_INDEX;
    }

    if (indexs.size() == 0) {
        return INVALID_INDEX;
    }

    for (const int &index : indexs) {
        InoRPointInfo info = getRPoint(indexs[0]);
        if (info.pointNo == INVALID_INDEX) {
            continue;
        }

        RoadPoint pt =

```

```

MetaTypeConversion::inoApi2tp_roadPoint(info);
    ProjectRoadPointInfo::instance()->updateRoadPointInfo(
        PREFIX_P, info.pointNo, info.label, pt);
}

FREQ_LOG_PRINT_TIMESTAMP
return (nRet == ERR_OK);
}

bool JointPointInterface::copyJPoint(const std::vector<int>
&indexs)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    comm()->project()->GetJPoint()->CopyPoints(m_sJPointFileName,
indexs);

    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

int JointPointInterface::pasteJPoint(std::vector<int> &indexs)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int nRet = comm()->project()->GetJPoint()->PastePoints(
        indexs, m_sJPointFileName);
    if (nRet != ERR_OK) {
        return INVALID_INDEX;
    }

    if (indexs.size() == 0) {
        return INVALID_INDEX;
    }

    for (const int &index : indexs) {
        InoJPointInfo info = getJPoint(index);
        if (info.pointNo == INVALID_INDEX) {
            continue;
        }

        RoadPoint pt;
        pt.setJointAngle(info.pos.JointData);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(

```

```

        PREFIX_JP, info.pointNo, info.label, pt);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

```

4.5 替换点（replaceRPointCoordValue / replaceJPointCoordValue）

4.5.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::replaceRPointCoordValue(int index)	用当前位姿替换 P 点坐标	bool	index	索引
bool JointPointInterface::replaceJPointCoordValue(int index)	用当前关节值替换 JP 点	bool	index	索引

4.5.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ReplaceRobotPoint,
    m_selectPointIndexList.at(0));

CommunicationEngine::instance()->enqueueCmd_handleProject(

```



```
    this, AbstractCmd::CmdType_ReplaceJointPoint,  
    m_selectPointIndexList.at(0));
```

4.5.3 函数实现

```
C++  
void CommunicationThread::replaceRobotPoint(AbstractCmd *absCmd)  
{  
    QString sErrMsg;  
    int nIndex = INVALID_INDEX;  
    InoRPointInfo robotPointInfo;  
    bool isOk = true;  
  
    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);  
    if (!cmd) {  
        sErrMsg = tr("Failed to replace point.") +  
SPACE_AND_CONVERT_PARAMETERS_FALID;  
        isOk = false;  
    }  
  
    if (isOk) {  
        nIndex = cmd->m_value.toInt();  
        isOk = m_commInstance->replaceRPointCoordValue(nIndex);  
        if (!isOk) {  
            sErrMsg = tr("Failed to replace point.");  
        }  
    }  
  
    if (isOk) {  
        isOk = m_commInstance->saveRPoints();  
        if (!isOk) {  
            sErrMsg = tr("Failed to replace the point due to save  
error.");  
        }  
    }  
  
    if (isOk) {  
        robotPointInfo = m_commInstance->getRPoint(nIndex);  
    }  
  
    emit CommunicationEngine::instance()-  
>signal_handleProject_result(  

```

```

        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(robotPointInfo) : QVariant(),
        isOk, sErrMsg);
    }

void CommunicationThread::replaceJointPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    int nIndex = INVALID_INDEX;
    InoJPointInfo jointPointInfo;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to replace joint point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        nIndex = cmd->m_value.toInt();
        isOk = m_commInstance->replaceJPointCoordValue(nIndex);
        if (!isOk) {
            sErrMsg = tr("Failed to replace joint point.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->saveJPoints();
        if (!isOk) {
            sErrMsg = tr("Failed to replace joint point due to
save error.");
        }
    }

    if (isOk) {
        jointPointInfo = m_commInstance->getJPoint(nIndex);
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(jointPointInfo) : QVariant(),
        isOk, sErrMsg);
}

```

```
}
```

```
C++
```

```
bool JointPointInterface::replaceJPointCoordValue(int index)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoJPos pointValue;
    if (!readCurPoint(pointValue)) {
        return false;
    }

    MJPosItem point = comm()->project()->GetJPoint()->GetPoint(
        m_sJPointFileName, index);
    if (point.PointNo < 0) {
        return false;
    }

    point.JData = MetaTypeConversion::inoApi2tp_jPos(pointValue);
    int nRet = comm()->project()->GetJPoint()->ChangePoint(
        m_sJPointFileName, point, false);

    if (nRet == ERROR_OK) {
        RoadPoint pt =
MetaTypeConversion::inoApi2tp_roadPoint(pointValue);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(
            PREFIX_JP, point.PointNo,
QString::fromStdString(point.LabelName), pt);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

bool RobotPointInterfacePrivate::replaceRPointCoordValue(int
index)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoRobPos pointValue;
    if (!readCurPoint(pointValue)) {
        return false;
    }
}
```

```

        MRobPosItem point = q->comm()->project()->GetRPoint()-
>GetPoint(
            m_sRPointFileName, index);
        if (point.PointNo < 0) {
            return false;
        }

        point.PData =
MetaTypeConversion::inoApi2tp_robPos(pointValue);
        int nRet = q->comm()->project()->GetRPoint()->ChangePoint(
            m_sRPointFileName, point, false);

        if (nRet == ERROR_OK) {
            RoadPoint pt =
MetaTypeConversion::inoApi2tp_roadPoint(pointValue);
            ProjectRoadPointInfo::instance()->updateRoadPointInfo(
                PREFIX_P, point.PointNo,
QString::fromStdString(point.LabelName), pt);
        }

        FREQ_LOG_PRINT_TIMESTAMP
        return (nRet == ERR_OK);
    }
}

```

4.6 编辑点（changeRPoint / changeJPoint / changeRPointPos / changeJPointPos）

4.6.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::changeRPoint(const InoRPointInfo &info, string	编辑 P 点完整信息	bool	info,sFileName	点信息、文件名

sFileName)				
bool JointPointInterface::changeJointPoint(const InoJointPointInfo &info, std::string sFileName)	编辑 JP 点完整信息	bool	info,sFileName	点信息、文件名
bool RobotPointInterface::changeRPoint(const MRobPosItem &item)	仅改 P 点坐标	bool	item	底层点位对象
bool JointPointInterface::changeJointPoint(const MJPosItem &item)	仅改 JP 点坐标	bool	item	底层点位对象

4.6.2 调用

C++

```
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ChangeRobotPoint,
    QVector<QVariant>() << QVariant::fromValue(robotPointInfo) <<
    sOriginLabel);
```

```
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ChangeJointPoint,
    QVector<QVariant>() << QVariant::fromValue(jointPointInfo) <<
    sOriginLabel);
```

4.6.3 函数实现

```

C++
void CommunicationThread::changeRobotPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    InoRPointInfo robotPointInfo;
    QString sOriginLabel;
    QVector<QVariant> vecValues;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to change point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to modify the point as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        robotPointInfo = vecValues.at(0).value<InoRPointInfo>();
        QString curRPointFile = m_commInstance->curRPointFile();
        if (!curRPointFile.isEmpty() &&
curRPointFile.compare(DEFAULT_ROBOT_POINT_FILE)
&& !robotPointInfo.label.isEmpty()) {
            sErrMsg = tr("Labels are not supported for
modification in %1.").arg(curRPointFile);
            isOk = false;
        }
    }

    if (isOk) {
        sOriginLabel = vecValues.at(1).toString();
        if (!robotPointInfo.label.isEmpty() &&
sOriginLabel.compare(robotPointInfo.label)) {
            bool isExist = m_commInstance-
>isLabelExisted(robotPointInfo.label);
            if (isExist) {

```

```

        sErrMsg = tr("Failed to edit the label as it
already exists.");
        isOk = false;
    }
}

if (isOk) {
    isOk = m_commInstance->changeRPoint(robotPointInfo);
    if (!isOk) {
        sErrMsg = tr("Failed to change point.");
    }
}

if (isOk) {
    isOk = m_commInstance->saveRPoints();
    if (!isOk) {
        sErrMsg = tr("Failed to modify the point due to save
error.");
    }
}

emit CommunicationEngine::instance()-
>signal_handleProject_result(
    absCmd->m_object, absCmd->m_cmdType,
    isOk ? QVariant::fromValue(robotPointInfo) : QVariant(),
    isOk, sErrMsg);

if (isOk && sOriginLabel.compare(robotPointInfo.label)) {
    emit CommunicationEngine::instance()-
>signal_virtualKeyBoard_LabelChanged(
        LabelType_GlobalVar_P, sOriginLabel,
        robotPointInfo.label);
}

}

void CommunicationThread::changeRobotPointPos(AbstractCmd *absCmd)
{
    QString sErrMsg;
    RoadPoint roadPoint;
    int nIndex = INVALID_INDEX;
    QVector<QVariant> vecValues;
    MRobPosItem item;
    bool isOk = true;

```

```

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to change point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to modify the point as the
parameters are invalid.");
            isOk = false;
        }

        roadPoint = vecValues.at(1).value<RoadPoint>();
    }

    if (isOk) {
        nIndex = vecValues.at(0).toInt();
        if (nIndex <= INVALID_INDEX) {
            sErrMsg = tr("index is invalid.");
            isOk = false;
        }
    }

    QString curLoadedRPFileName = m_commInstance-
>curLoadedRPointFile();
    if (curLoadedRPFileName.isEmpty()) {
        curLoadedRPFileName = m_commInstance->curRPointFile();
    }

    if (isOk) {
        item = m_commInstance->getRobotMRobPosItem(nIndex);
        if (item.PointNo < 0) {
            sErrMsg = tr("P[%1] is not exists.").arg(nIndex, 4,
10, QLatin1Char('0'));
            isOk = false;
        }

        item.PData =
MetaTypeConversion::tp2InoApi_roadPoint2RobPos(roadPoint);
    }

```



```

        if (isOk) {
            if (m_commInstance->isProgramNotRunning()) {
                isOk = m_commInstance->changeRPoint(item);
                if (isOk) {
                    isOk = m_commInstance-
>saveProject(InoSyncProjcetInfoType_GlobalRPoint);
                }
            } else {
                isOk = m_commInstance-
>changeRPointInDebug(MAIN_PROGRAM_FILE,
MAIN_TASK_ID, item);
            }

            if (!isOk) {
                sErrMsg = tr("Failed to change point.");
            }
        }

#ifdef INOCOBOTTP_MSVC_QT5
        qRegisterMetaType<RoadPoint>("RoadPoint");
#endif
        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            isOk ? QVariant::fromValue(roadPoint) : QVariant(), isOk,
sErrMsg);
    }

void CommunicationThread::changeJointPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    InoJPointInfo jointPointInfo;
    QString sOriginLabel;
    QVector<QVariant> vecValues;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to modify joint point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }
}

```

```

        if (isOk) {
            vecValues = cmd->m_value.value<QVector<QVariant>>>();
            if (vecValues.count() < 2) {
                sErrMsg = tr("Failed to modify joint point as the
parameters are invalid.");
                isOk = false;
            }
        }

        if (isOk) {
            jointPointInfo = vecValues.at(0).value<InoJPointInfo>();
            sOriginLabel = vecValues.at(1).toString();
            if (!jointPointInfo.label.isEmpty() &&
sOriginLabel.compare(jointPointInfo.label)) {
                bool isExist = m_commInstance-
>isLabelExisted(jointPointInfo.label);
                if (isExist) {
                    sErrMsg = tr("Edit failed as the label already
exists. ");
                    isOk = false;
                }
            }
        }

        if (isOk) {
            isOk = m_commInstance->changeJPoint(jointPointInfo);
            if (!isOk) {
                sErrMsg = tr("Failed to change joint point.");
            }
        }

        if (isOk) {
            isOk = m_commInstance->saveJPoints();
            if (!isOk) {
                sErrMsg = tr("Failed to modify joint point due to save
error.");
            }
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            isOk ? QVariant::fromValue(jointPointInfo) : QVariant(),

```

```

isOk, sErrMsg);

    if (isOk && sOriginLabel.compare(jointPointInfo.label)) {
        emit CommunicationEngine::instance()->
            signal_virtualKeyboard_LabelChanged(
                LabelType_GlobalVar_JP, sOriginLabel,
jointPointInfo.label);
    }
}

void CommunicationThread::changeJointPointPos(AbstractCmd *absCmd)
{
    QString sErrMsg;
    RoadPoint roadPoint;
    int nIndex = INVALID_INDEX;
    QVector<QVariant> vecValues;
    MJPosItem item;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to modify joint point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to modify joint point as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        nIndex = vecValues.at(0).toInt();
        if (nIndex <= INVALID_INDEX) {
            sErrMsg = tr("index is invalid.");
            isOk = false;
        }

        roadPoint = vecValues.at(1).value<RoadPoint>();
    }
}

```

```

        if (isOk) {
            item = m_commInstance->getJointMJPosItem(nIndex);
            if (item.PointNo < 0) {
                sErrMsg = tr("JP[%1] is not exists.").arg(nIndex, 4,
10, QLatin1Char('0'));
                isOk = false;
            }

            item.JData =
MetaTypeConversion::tp2InoApi_roadPoint(roadPoint);
        }

        if (isOk) {
            if (m_commInstance->isProgramNotRunning()) {
                isOk = m_commInstance->changeJPoint(item);
                if (isOk) {
                    isOk = m_commInstance-
>saveProject(InoSyncProjcetInfoType_GlobalJPoint);
                }
            } else {
                isOk = m_commInstance-
>changeJPointInDebug(MAIN_PROGRAM_FILE,
MAIN_TASK_ID, item);
            }

            if (!isOk) {
                sErrMsg = tr("Failed to change joint point.");
            }
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            isOk ? QVariant::fromValue(roadPoint) : QVariant(), isOk,
sErrMsg);
    }

```

C++

```
bool RobotPointInterfacePrivate::changeRPoint(const InoRPointInfo
```

```

&info, std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    sFileName = sFileName.empty() ? m_sRPointFileName : sFileName;

    MRobPosItem point;
    point.PointNo = info.pointNo;
    point.LabelName = info.label.toStdString();
    point.Description = info.description.toStdString();
    point.PData = MetaTypeConversion::inoApi2tp_robPos(info.pos);

    int nRet = q->comm()->project()->GetRPoint()-
>ChangePoint(sFileName,
                                                    point,
false);

    if (nRet == ERROR_OK) {
        RoadPoint pt =
MetaTypeConversion::inoApi2tp_roadPoint(point);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(
            PREFIX_P, point.PointNo,
QString::fromStdString(point.LabelName), pt);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

bool JointPointInterface::changeJPoint(const InoJPointInfo &info,
std::string sFileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    sFileName = sFileName.empty() ? m_sJPointFileName : sFileName;

    MJPosItem point;
    point.PointNo = info.pointNo;
    point.LabelName = info.label.toStdString();
    point.Description = info.description.toStdString();
    point.JData = MetaTypeConversion::inoApi2tp_jPos(info.pos);

    int nRet = comm()->project()->GetJPoint()-
>ChangePoint(sFileName,

```

```

false);

    if (nRet == ERR_OK) {
        RoadPoint pt;
        pt.setJointAngle(point.JData.JointData);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(
            PREFIX_JP, point.PointNo,
            QString::fromStdString(point.LabelName), pt);
    }

    qDebug() << __FUNCTION__ << nRet;
    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

```

4.7 重命名点索引 (renameRPoint / renameJPoint)

4.7.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::renameRPoint(int nIndex, int nNewIndex)	修改 P 索引	bool	旧索引,新索引	点序号重命名
bool JointPointInterface::renameJPoint(int nIndex, int nNewIndex)	修改 JP 索引	bool	旧索引,新索引	点序号重命名

4.7.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_RenameRobotPoint,
    QVector<QVariant>() << oldIndex << newIndex);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_RenameJointPoint,
    QVector<QVariant>() << oldIndex << newIndex);
```

4.7.3 函数实现

```
C++
void CommunicationThread::renameRobotPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    int nIndex = INVALID_INDEX;
    int nNewIndex = INVALID_INDEX;
    QVector<QVariant> vecValues;
    InoRPointInfo robotPointInfo;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to rename point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to rename the point as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        nIndex = vecValues.at(0).toInt();
        nNewIndex = vecValues.at(1).toInt();
    }
}
```

```

        bool isExist = Communication::instance()-
>findRPoint(nNewIndex);
        if (isExist) {
            sErrMsg = tr("P[%1] already exists.").arg(nNewIndex,
4, 10, QLatin1Char('0'));
            isOk = false;
        }
    }

    if (isOk) {
        isOk = m_commInstance->renameRPoint(nIndex, nNewIndex);
        if (!isOk) {
            sErrMsg = tr("Failed to rename point.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->saveRPoints();
        if (!isOk) {
            m_commInstance->renameRPoint(nNewIndex, nIndex);
            sErrMsg = tr("Failed to rename the point due to save
error.");
        }
    }

    if (isOk) {
        robotPointInfo = m_commInstance->getRPoint(nNewIndex);
        emit CommunicationEngine::instance()->

signal_virtualKeyBoard_LabelChanged(LabelType_GlobalVar_P,

QString::number(nIndex),

QString::number(nNewIndex));
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(robotPointInfo) : QVariant(),
        isOk, sErrMsg);
}

void CommunicationThread::renameJointPoint(AbstractCmd *absCmd)

```



```

{
    QString sErrMsg;
    int nIndex = INVALID_INDEX;
    int nNewIndex = INVALID_INDEX;
    QVector<QVariant> vecValues;
    InoJPointInfo jointPointInfo;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to rename joint point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to rename joint point as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        nIndex = vecValues.at(0).toInt();
        nNewIndex = vecValues.at(1).toInt();
        bool isExist = Communication::instance()-
>findJPoint(nNewIndex);
        if (isExist) {
            sErrMsg = tr("JP[%1] already exists.").arg(nNewIndex,
4, 10, QLatin1Char('0'));
            isOk = false;
        }
    }

    if (isOk) {
        isOk = m_commInstance->renameJPoint(nIndex, nNewIndex);
        if (!isOk) {
            sErrMsg = tr("Failed to rename joint point.");
        }
    }

    if (isOk) {

```

```

        isOk = m_commInstance->saveJPoints();
        if (!isOk) {
            m_commInstance->renameJPoint(nNewIndex, nIndex);
            sErrMsg = tr("Failed to rename joint point due to save
error.");
        }
    }

    if (isOk) {
        jointPointInfo = m_commInstance->getJPoint(nNewIndex);
        emit CommunicationEngine::instance()->

signal_virtualKeyBoard_LabelChanged(LabelType_GlobalVar_JP,

QString::number(nIndex),

QString::number(nNewIndex));
    }

    emit CommunicationEngine::instance()->signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(jointPointInfo) : QVariant(),
        isOk, sErrMsg);
}

bool RobotPointInterfacePrivate::renameRPoint(int nIIndex, int
nNewIndex)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    if (nNewIndex < 0) {
        return false;
    }

    int nRet = q->comm()->project()->GetRPoint()->RenamePoint(
        m_sRPointFileName, nIIndex, nNewIndex);

    if (nRet == ERR_OK) {
        InoRPointInfo info = getRPoint(nNewIndex);
        RoadPoint pt =
MetaTypeConversion::inoApi2tp_roadPoint(info);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(
            PREFIX_P, info.pointNo, info.label, pt);
    }
}

```

```

        FREQ_LOG_PRINT_TIMESTAMP
        return (nRet == ERR_OK);
    }

bool JointPointInterface::renameJPoint(int nIndex, int nNewIndex)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (nNewIndex < 0) {
        return false;
    }

    int nRet = comm()->project()->GetJPoint()->RenamePoint(
        m_sJPointFileName, nIndex, nNewIndex);

    if (nRet == ERR_OK) {
        InoJPointInfo info = getJPoint(nNewIndex);
        RoadPoint pt;
        pt.setJointAngle(info.pos.JointData);
        ProjectRoadPointInfo::instance()->updateRoadPointInfo(
            PREFIX_JP, info.pointNo, info.label, pt);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

```

4.8 删除点 (deleteRPoint / deleteJPoint)

4.8.1 说明

接口	功能	返回值	参数	参数说明
bool RobotPointInterface::delete	删除 P 点	bool	nIndex	索引

RPoint(int nIndex)				
bool JointPointInter face::deleteJP oint(int nIndex)	删除 JP 点	bool	nIndex	索引

4.8.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_DeleteRobotPoint,
    QVector<QVariant>() << QVariant::fromValue(indexAndLabelMap));

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_DeleteJointPoint,
    QVector<QVariant>() << QVariant::fromValue(indexAndLabelMap));
```

4.8.3 函数实现

```
C++
void CommunicationThread::deleteRobotPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    int nIndex = INVALID_INDEX;
    QString sLabel;
    QVector<QVariant> vecValues;
    bool isOk = true;

    QMap<int,QString> indexAndLabelMap;
    QMap<int,QString> deletedIndexAndLabelMap;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to delete point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }
}
```

```

        if (isOk) {
            vecValues = cmd->m_value.value<QVector<QVariant>>>();
            if (vecValues.count() < 1) {
                sErrMsg = tr("Failed to delete point as the
parameters are invalid.");
                isOk = false;
            }
        }

        if(isOk){
            indexAndLabelMap =
vecValues.at(0).value<QMap<int,QString>>>();
            QMap<int,QString>::const_iterator it;
            for (it = indexAndLabelMap.constBegin(); it !=
indexAndLabelMap.constEnd(); ++it) {
                nIndex = it.key();
                isOk = m_commInstance->findRPoint(nIndex);
                if (!isOk) {
                    sErrMsg = tr("P[%1] is invalid.").arg(nIndex, 4,
10, QLatin1Char('0'));
                    break;
                }
            }
        }

        if (isOk) {
            QMap<int,QString>::const_iterator it;
            for (it = indexAndLabelMap.constBegin(); it !=
indexAndLabelMap.constEnd(); ++it) {
                nIndex = it.key();
                sLabel = it.value();
                isOk = m_commInstance->deleteRPoint(nIndex);
                if (!isOk) {
                    sErrMsg = tr("Failed to delete point.");
                    break;
                }
                else{
                    deletedIndexAndLabelMap.insert(nIndex,sLabel);
                }
            }
        }
    }
}

```

```

        if (isOk) {
            isOk = m_commInstance->saveRPoints();
            if (!isOk) {
                QMap<int,QString>::const_iterator it;
                for (it = deletedIndexAndLabelMap.constBegin(); it !=
deletedIndexAndLabelMap.constEnd(); ++it){
                    nIndex = it.key();
                    sLabel = it.value();
                    m_commInstance-
>addRPoint(nIndex,QString(sLabel).toStdString());
                }
                sErrMsg = tr("Failed to delete point due to save
error.");
            }
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,
sErrMsg);
        if(isOk){
            QMap<int,QString>::const_iterator it;
            for (it = deletedIndexAndLabelMap.constBegin(); it !=
deletedIndexAndLabelMap.constEnd(); ++it){
                sLabel = it.value();
                emit CommunicationEngine::instance()->

signal_virtualKeyBoard_LabelChanged(LabelType_GlobalVar_P,
                                     sLabel,
                                     "");
            }

        }
    }

void CommunicationThread::deleteJointPoint(AbstractCmd *absCmd)
{
    QString sErrMsg;
    int nIndex = INVALID_INDEX;
    QString sLabel;
    QVector<QVariant> vecValues;

    QMap<int,QString> indexAndLabelMap;

```

```

 QMap<int,QString> deletedIndexAndLabelMap;

 bool isOk = true;
 CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
 if (!cmd) {
     sErrMsg = tr("Failed to delete joint point.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
     isOk = false;
 }

 if (isOk) {
     vecValues = cmd->m_value.value<QVector<QVariant>>>();
     if (vecValues.count() < 1) {
         sErrMsg = tr("Failed to delete joint point as the
parameters are invalid.");
         isOk = false;
     }
 }

 if(isOk){
     indexAndLabelMap =
vecValues.at(0).value<QMap<int,QString>>>();
     QMap<int,QString>::const_iterator it;
     for (it = indexAndLabelMap.constBegin(); it !=
indexAndLabelMap.constEnd(); ++it) {
         nIndex = it.key();
         isOk = m_commInstance->findJPoint(nIndex);
         if (!isOk) {
             sErrMsg = tr("JP[%1] is illegal.").arg(nIndex, 4,
10, QLatin1Char('0'));
             break;
         }
     }
 }

 if (isOk) {
     QMap<int,QString>::const_iterator it;
     for (it = indexAndLabelMap.constBegin(); it !=
indexAndLabelMap.constEnd(); ++it) {
         nIndex = it.key();
         sLabel = it.value();
         isOk = m_commInstance->deleteJPoint(nIndex);
         if (!isOk) {

```



```

}

bool RobotPointInterfacePrivate::deleteRPoint(int nIndex)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int nRet = q->comm()->project()->GetRPoint()->DelPoint(
        m_sRPointFileName, nIndex);

    if (nRet == ERR_OK) {
        ProjectRoadPointInfo::instance()-
>deleteRoadPointInfo(PREFIX_P, nIndex);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

bool JointPointInterface::deleteJPoint(int nIndex)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int nRet = comm()->project()->GetJPoint()->DelPoint(
        m_sJPointFileName, nIndex);

    if (nRet == ERR_OK) {
        ProjectRoadPointInfo::instance()-
>deleteRoadPointInfo(PREFIX_JP, nIndex);
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return (nRet == ERR_OK);
}

```

5.标签、自定义报警

5.1 获取标签类型列表 (getLabelTypeList)

5.1.1 说明

接口	功能	返回值	参数	参数说明
----	----	-----	----	------

std::vector<std::string> ProjectInterface::getLabelTypeList()	获取工程支持的标签大类名称列表	std::vector<std::string>	无	
--	-----------------	--------------------------	---	--

5.1.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_GetLabelTypeList);
```

5.1.3 函数实现

```
C++
void CommunicationThread::getLabelTypeList(AbstractCmd *absCmd)
{
    QStringList listTypes;

    std::vector<std::string> types = m_commInstance-
>getLabelTypeList();
    for (const std::string &type : types) {
        // 暂不支持 IRLink 和 EtherCAT AD、DA 标签
        if (std::string::npos != type.find("AD")
            || std::string::npos != type.find("DA")) {
            continue;
        }

        listTypes << QString::fromStdString(type);
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(listTypes),
        true, QString());
}
```

```
C++
std::vector<std::string> ProjectInterface::getLabelTypeList()
```

```

{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<std::string> fileNames = comm()->project()-
>GetLabelTypeList();
    FREQ_LOG_PRINT_TIMESTAMP

    return fileNames;
}

```

5.2 IO 类标签初始化与读取（genInitIoItems / getIoItems）

5.2.1 说明

接口	功能	返回值	参数	参数说明
QVector<InoLabelItem> LabelInterface ::genInitIoItems(InoLabelType, InoLabelIoType, ShowType)	生成 IO 类初始标签行	QVector<InoLabelItem>	见枚举	标准 IO / 工具 IO / 总线 IO / 内存 IO 等
QVector<InoLabelItem> LabelInterface ::getIoItems(InoLabelType, InoLabelIoType, ShowType)	读取当前 IO 标签缓存	QVector<InoLabelItem>	同上	nNumberOfLabels<=0 时返回空

参数说明（含自定义结构体定义源码）

C++

代码块

// labeldata.h（节选）

```
typedef struct InoLabelItem {  
    int nLabelId;  
    int nIndex;  
    QString sLabel;  
    QString sDescription;  
    QString sOriginalName;  
    ...  
} InoLabelItem;
```

```
enum InoLabelType {  
    /// IO 输入 - bit  
    IO_INPUT_BIT = 0,  
    /// IO 输出 - bit  
    IO_OUTPUT_BIT,  
    /// IO 输入 - byte  
    IO_INPUT_BYTE,  
    /// IO 输出 - byte  
    IO_OUTPUT_BYTE,  
    /// IO 输入 - word  
    IO_INPUT_WORD,  
    /// IO 输出 - word  
    IO_OUTPUT_WORD,  
    /// DA 变量  
    IO_DA,  
    /// AD 变量  
    IO_AD,  
    /// B 变量  
    IO_B,  
    /// R 变量  
    IO_R,  
    /// D 变量  
    IO_D,  
    /// STR 变量  
    IO_STR,  
    /// 未知  
    IO_SIZE,  
};
```

```

enum InoLabelIOType {
    /// 标准 IO
    InoLabelIOType_StandardIO = 0,
    /// 从站现场 IO
    InoLabelIOType_SlaveFieldbusIO,
    /// 从站现场 IO
    InoLabelIOType_MasterFieldbusIO,
    /// 工具 IO
    InoLabelIOType_ToolIO,
    /// 内存 IO
    InoLabelIOType_MemoryIO,
};

enum ShowType {
    ShowType_Bit = 0,    // bit
    ShowType_Byte = 1,   // 字节
    ShowType_Word = 2,   // 字
    ShowType_Size = 3
};

```

5.2.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GenInitIoItems,
    QVector<QVariant>() << m_labelType << m_ioType << m_dataType);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GetIoItems,
    QVector<QVariant>() << m_labelType << m_ioType << m_dataType);

```

5.2.3 函数实现

```

C++
void CommunicationThread::genInitIoItems(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    InoLabelType labelType;

```

```

InoLabelIOType ioType;
ShowType dataType;
QVector<InoLabelItem> items;
bool isOk = true;

CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
if (!cmd) {
    sErrMsg = tr("Failed to init label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
    isOk = false;
}

if (isOk) {
    vecValues = cmd->m_value.value<QVector<QVariant>>();
    if (vecValues.count() < 3) {
        sErrMsg = tr("Failed to load the label as the
parameters are invalid.");
        isOk = false;
    }
}

if (isOk) {
    labelType = vecValues.at(0).value<InoLabelType>();
    ioType = vecValues.at(1).value<InoLabelIOType>();
#ifdef INOCOBOTTP_MSVC_QT5
    qRegisterMetaType<ShowType>("ShowType");
    qRegisterMetaType<ShowType>("ShowType&");
#endif
    dataType = vecValues.at(2).value<ShowType>();
    items = m_commInstance->genInitIoItems(labelType, ioType,
dataType);
}

emit CommunicationEngine::instance()-
>signal_handleProject_result(
    absCmd->m_object, absCmd->m_cmdType,
    isOk ? QVariant::fromValue(items) : QVariant(), isOk,
sErrMsg);
}

void CommunicationThread::getIoItems(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;

```

```

InoLabelType labelType;
InoLabelIOType ioType;
ShowType dataType;
QVector<InoLabelItem> items;
bool isOk = true;

CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
if (!cmd) {
    sErrMsg = tr("Failed to get label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
    isOk = false;
}

if (isOk) {
    vecValues = cmd->m_value.value<QVector<QVariant>>();
    if (vecValues.count() < 3) {
        sErrMsg = tr("Failed to modify the label as the
parameters are invalid.");
        isOk = false;
    }
}

if (isOk) {
    labelType = vecValues.at(0).value<InoLabelType>();
    ioType = vecValues.at(1).value<InoLabelIOType>();
    dataType = vecValues.at(2).value<ShowType>();
    items = m_commInstance->getIoItems(labelType, ioType,
dataType);
}

emit CommunicationEngine::instance()-
>signal_handleProject_result(
    absCmd->m_object, absCmd->m_cmdType,
    isOk ? QVariant::fromValue(items) : QVariant(), isOk,
sErrMsg);
}

```

```

C++
QVector<InoLabelItem> LabelInterface::genInitIoItems(
    const InoLabelType &labelType, const InoLabelIOType &ioType,
    const ShowType &dataType)
{

```

```

FREQ_LOG_PRINT_TIMESTAMP_THREAD

if (!isStatusOK()) {
    return QVector<InoLabelItem>();
}

std::vector<LabelItem> vItems = comm()->project()->GetLabel()-
>GenInitIoItems(
    MetaTypeConversion::inoApi2tp_labelType(labelType),
    MetaTypeConversion::inoApi2tp_ioType(ioType),
    MetaTypeConversion::inoApi2tp_dataType(dataType));

if (vItems.empty()) {
    return QVector<InoLabelItem>();
}

FREQ_LOG_PRINT_TIMESTAMP
return MetaTypeConversion::tp2InoApi_Items(vItems);
}

QVector<InoLabelItem> LabelInterface::getIoItems(const
InoLabelType &labelType,
                                                    const
InoLabelIoType &ioType,
                                                    const ShowType
&dataType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return QVector<InoLabelItem>();
    }

    IoLabelItems items = comm()->project()->GetLabel()-
>GetIoItems(
        MetaTypeConversion::inoApi2tp_labelType(labelType),
        MetaTypeConversion::inoApi2tp_ioType(ioType),
        MetaTypeConversion::inoApi2tp_dataType(dataType));
    if (items.nNumberOfLabels <= 0) {
        return QVector<InoLabelItem>();
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return MetaTypeConversion::tp2InoApi_Items(items.LabelsArray);
}

```



```
}

```

5.3 AD/DA 类标签初始化与读取 (genInitAaDaltems/getAdDaltems)

5.3.1 说明

接口	功能	返回值	参数	参数说明
QVector<InoLabelItem> LabelInterface ::genInitAaDaltems(InoLabelType, InoAdDaType)	初始化 AD/DA 标签表	QVector<InoLabelItem>	InoAdDaType_IRLink / InoAdDaType_EtherCAT	InoLabelType 见 5.2
QVector<InoLabelItem> LabelInterface ::getAdDaltems(InoLabelType, InoAdDaType)	读取 AD/DA 标签缓存	QVector<InoLabelItem>	同上	同上

参数说明 (含自定义结构体定义源码)

```
C++
// labeldata.h
enum InoAdDaType {
    InoAdDaType_IRLink = 0,
    InoAdDaType_EtherCAT,
};

```

5.3.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GenInitAdDaItems,
    QVector<QVariant>() << m_labelType << m_addaType);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GetAdDaItems,
    QVector<QVariant>() << m_labelType << m_addaType);
```

5.3.3 函数实现

```
C++
void CommunicationThread::genInitAdDaItems(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    InoLabelType labelType;
    InoAdDaType addaType;
    QVector<InoLabelItem> items;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to init label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to load the label as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        labelType = vecValues.at(0).value<InoLabelType>();
        addaType = vecValues.at(1).value<InoAdDaType>();
    }
}
```

```

        items = m_commInstance->genInitAaDaItems(labelType,
addaType);
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(items) : QVariant(), isOk,
sErrMsg);
}

void CommunicationThread::getAdDaItems(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    InoLabelType labelType;
    InoAdDaType addaType;
    QVector<InoLabelItem> items;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to get label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>>();
        if (vecValues.count() < 2) {
            sErrMsg = tr("Failed to modify the label as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        labelType = vecValues.at(0).value<InoLabelType>();
        addaType = vecValues.at(1).value<InoAdDaType>();
        items = m_commInstance->getAdDaItems(labelType, addaType);
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(

```

```

        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(items) : QVariant(), isOk,
sErrMsg);
}

```

```

C++
 QVector<InoLabelItem> LabelInterface::genInitAaDaItems(
    const InoLabelType &labelType, const InoAdDaType &addaType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return QVector<InoLabelItem>();
    }

    std::vector<LabelItem> vItems = comm()->project()->GetLabel()-
>GenInitAdDaItems(
        MetaTypeConversion::inoApi2tp_labelType(labelType),
        MetaTypeConversion::inoApi2tp_addaType(addaType));

    if (vItems.empty()) {
        return QVector<InoLabelItem>();
    }

    FREQ_LOG_PRINT_TIMESTAMP
        return MetaTypeConversion::tp2InoApi_Items(vItems);
}

 QVector<InoLabelItem> LabelInterface::getAdDaItems(
    const InoLabelType &labelType, const InoAdDaType &addaType )
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return QVector<InoLabelItem>();
    }

    IoLabelItems items = comm()->project()->GetLabel()-
>getAdDaItems(
        MetaTypeConversion::inoApi2tp_labelType(labelType),
        MetaTypeConversion::inoApi2tp_addaType(addaType));
    if (items.nNumberOfLabels <= 0) {

```

```
        return QVector<InoLabelItem>();
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return
    MetaTypeConversion::tp2InoApi_Items(items.LabelsArray);
}
```

5.4 B / R / D 类标签初始化与读取 (genInitOtherItems/getOtherItems)

5.4.1 说明

接口	功能	返回值	参数	参数说明
<code>QVector<InoLabelItem> LabelInterface ::genInitOtherItems(InoLabelType)</code>	生成 B/R/D 等「非 IO 分维」标签表	<code>QVector<InoLabelItem></code>	<code>IO_B / IO_R / IO_D</code>	<code>InoLabelType</code> 见 5.2
<code>QVector<InoLabelItem> LabelInterface ::getOtherItems(InoLabelType)</code>	读取对应类型标签缓存	<code>QVector<InoLabelItem></code>	单个 <code>InoLabelType</code>	同上

参数说明

```
C++
typedef struct InoLabelItem {
    ...
    int nLabelId;
```

```

    int nIndex;
    QString sLabel;
    QString sDescription;
    QString sOriginalName;

} InoLabelItem;

```

5.4.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GenInitOtherItems, m_labelType);

CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_GetOtherItems, m_labelType);

```

5.4.3 函数实现

```

C++
void CommunicationThread::genInitOtherItems(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<InoLabelItem> items;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to init label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        items = m_commInstance->genInitOtherItems(
            cmd->m_value.value<InoLabelType>());
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(items) : QVariant(), isOk,

```

```

sErrMsg);
}

void CommunicationThread::getOtherItems(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<InoLabelItem> items;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to modify label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        items = m_commInstance->getOtherItems(
            cmd->m_value.value<InoLabelType>());
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(items) : QVariant(), isOk,
sErrMsg);
}

```

```

C++
QVector<InoLabelItem> LabelInterface::genInitOtherItems(
    const InoLabelType &labelType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return QVector<InoLabelItem>();
    }

    std::vector<LabelItem> vItems = comm()->project()->GetLabel()-
>GenInitOtherItems(
        MetaTypeConversion::inoApi2tp_labelType(labelType));
    if (vItems.empty()) {

```

```

        return QVector<InoLabelItem>();
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return MetaTypeConversion::tp2InoApi_Items(vItems);
}

QVector<InoLabelItem> LabelInterface::getOtherItems(
    const InoLabelType &labelType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return QVector<InoLabelItem>();
    }

    IoLabelItems items = comm()->project()->GetLabel()->GetItems(
        MetaTypeConversion::inoApi2tp_labelType(labelType));
    if (items.nNumberOfLabels <= 0) {
        return QVector<InoLabelItem>();
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return MetaTypeConversion::tp2InoApi_Items(items.LabelsArray);
}

```

5.5 修改标签项 (modifyIoItem/modifyAdDaltem/modifyOtherItem)

5.5.1 说明

接口	功能	返回值	参数	参数说明
bool ProjectInterface::isLabelExisted(const QString &sLabel)	新标签名 是否已存在	bool	sLabel	空串返回 false; 修改前查

				重
<code>bool LabelInterface::modifyIoItem(const InoLabelType &labelType, const InoLabelIOType &ioType, const ShowType &dataType, const InoLabelItem &item))</code>	写 IO 行 标签+描述	<code>bool</code>	<code>InoLabelItem</code> (见 5.4) <code>InoLabelType</code> (见 5.2) <code>InoLabelIOType</code> (见 5.2) <code>ShowType</code> (见 5.2)	
<code>bool LabelInterface::modifyAdDaItem (const InoLabelType &labelType, const InoAdDaType &addaType, const InoLabelItem &item)</code>	写 AD/DA 行	<code>bool</code>	<code>InoLabelType</code> (见 5.2) <code>InoLabelItem</code> (见 5.4)	—

```
C++
enum InoAdDaType {
    InoAdDaType_IRLink = 0,
    InoAdDaType_EtherCAT,
};
```

```
C++
// labeldata.h (与描述列一并修改时使用)
typedef struct InoLabelAndDescriItem {
    int nIndex;
    QString sLabel;
    QString sDescription;
    ...
};
```

```
} InoLabelAndDescrItem;
```

5.5.2 调用

```
C++
// IN/OUT
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ModifyIoItem,
    QVector<QVariant>() << m_labelType << m_ioType << m_dataType
    << QVariant::fromValue(item) << oldLabel
    << oldDescr);

// AD/DA
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ModifyAdDaItem,
    QVector<QVariant>() << m_labelType << m_addaType
    << QVariant::fromValue(item) << oldLabel
    << oldDescr);

// B/R/D
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_ModifyOtherItem,
    QVector<QVariant>() << m_labelType
    << QVariant::fromValue(item) << oldLabel);
```

5.5.3 函数实现

```
C++
void CommunicationThread::modifyIoItem(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    InoLabelType labelType;
    InoLabelIoType ioType;
    ShowType dataType;
    InoLabelItem labelItem;
    QString sOldLabel;
    QString sOldDescr;
    InoLabelAndDescrItem oldLabelAndDescrItem;
```

```

bool isOk = true;

CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
if (!cmd) {
    sErrMsg = tr("Failed to modify label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
    isOk = false;
}

if (isOk) {
    vecValues = cmd->m_value.value<QVector<QVariant>>>();
    if (vecValues.count() < 5) {
        sErrMsg = tr("Failed to modify the label as parameters
are invalid.");
        isOk = false;
    }
}

if (isOk) {
    labelType = vecValues.at(0).value<InoLabelType>();
    ioType = vecValues.at(1).value<InoLabelIOType>();
    dataType = vecValues.at(2).value<ShowType>();
    labelItem = vecValues.at(3).value<InoLabelItem>();
    sOldLabel = vecValues.at(4).toString();
    sOldDescri = vecValues.at(5).toString();
    oldLabelAndDescriItem =
InoLabelAndDescriItem(labelItem.nIndex,sOldLabel,sOldDescri);

    if (!labelItem.sLabel.isEmpty() &&
labelItem.sLabel.compare(sOldLabel)) {
        bool isExist = m_commInstance-
>isLabelExisted(labelItem.sLabel);
        if (isExist) {
            sErrMsg = tr("The label already exists.");
            isOk = false;
        }
    }
}

if (isOk) {
    isOk = m_commInstance->modifyIoItem(labelType, ioType,
                                         dataType, labelItem);

    if (!isOk) {
        sErrMsg = tr("Failed to modify the label.");
    }
}

```

```

    }
}

if (isOk) {
    isOk = m_commInstance->saveAllLabels();
    if (!isOk) {
        sErrMsg = tr("Failed to save label.");
        // 撤销保存失败的修改
        InoLabelItem item = labelItem;
        item.sLabel = sOldLabel;
        m_commInstance->modifyIoItem(labelType, ioType,
dataType, item);
    }
}

emit CommunicationEngine::instance()-
>signal_handleProject_result(
    absCmd->m_object, absCmd->m_cmdType,
    isOk ? QVariant::fromValue(QVector<InoLabelItem>()) <<
labelItem) : QVariant(), isOk, sErrMsg);

if (isOk && labelItem.sLabel.compare(sOldLabel)) {
    AllLabelType enumOffset = LabelType_None;
    switch (labelType) {
    case IO_INPUT_BIT:
    case IO_INPUT_BYTE:
    case IO_INPUT_WORD:
        enumOffset = LabelType_Input_Start;
        break;
    case IO_OUTPUT_BIT:
    case IO_OUTPUT_BYTE:
    case IO_OUTPUT_WORD:
        enumOffset = LabelType_Output_Start;
        break;
    default:
        break;
    }

    if (enumOffset != LabelType_None){
        InoIOType type;
        if(ioType == InoLabelIOType_StandardIO )
            type = InoIOType_StandardIO;
        else if (ioType == InoLabelIOType_ToolIO)
            type = InoIOType_ToolIO;
    }
}

```

```

        else if(ioType == InoLabelIOType_SlaveFieldbusIO)
            type = InoIOType_FieldbusIO;
        else if(ioType == InoLabelIOType_MemoryIO)
            type = InoIOType_MemoryIO;
        else
            return;
        emit CommunicationEngine::instance()
            ->signal_virtualKeyBoard_LabelChanged(
                (AllLabelType)(enumOffset + type *
ShowType_Size + dataType),
                sOldLabel, labelItem.sLabel); // oldLabel
        emit CommunicationEngine::instance()
            ->signal_virtualKeyBoard_LabelChanged(
                (AllLabelType)(enumOffset + InoIOType_CommonIO
* ShowType_Size + dataType),
                sOldLabel, labelItem.sLabel); // oldLabel
    }

}

if (isOk&&
(labelItem.sLabel.compare(sOldLabel)||labelItem.sDescription.compa
re(sOldDescri))) {
    AllLabelType enumOffset = LabelType_None;
    switch (labelType) {
    case IO_INPUT_BIT:
    case IO_INPUT_BYTE:
    case IO_INPUT_WORD:
        enumOffset = LabelType_Input_Start;
        break;
    case IO_OUTPUT_BIT:
    case IO_OUTPUT_BYTE:
    case IO_OUTPUT_WORD:
        enumOffset = LabelType_Output_Start;
        break;
    default:
        break;
    }
    if (enumOffset != LabelType_None){
        InoIOType type;
        if(ioType == InoLabelIOType_StandardIO )
            type = InoIOType_StandardIO;
        else if (ioType == InoLabelIOType_ToolIO)
            type = InoIOType_ToolIO;
    }
}

```

```

        else if(ioType == InoLabelIOType_SlaveFieldbusIO)
            type = InoIOType_FieldbusIO;
        else if(ioType == InoLabelIOType_MemoryIO)
            type = InoIOType_MemoryIO;
        else
            return;
        InoLabelAndDescriItem
newLabelAndDescriItem(labelItem.nIndex,labelItem.sLabel,labelItem.
sDescription);
        emit CommunicationEngine::instance()
            ->signal_virtualKeyBoard_LabelOrDescriChanged(
                (AllLabelType)(enumOffset + type *
ShowType_Size + dataType),
                oldLabelAndDescriItem,newLabelAndDescriItem);
        emit CommunicationEngine::instance()
            ->signal_virtualKeyBoard_LabelOrDescriChanged(
                (AllLabelType)(enumOffset + InoIOType_CommonIO
* ShowType_Size + dataType),
                oldLabelAndDescriItem,newLabelAndDescriItem);
    }
}

void CommunicationThread::modifyAdDaItem(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    InoLabelType labelType;
    InoAdDaType addaType;
    InoLabelItem labelItem;
    QString sOldLabel;
    QString sOldDescri;
    InoLabelAndDescriItem oldLabelAndDescriItem;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to modify label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();

```

```

        if (vecValues.count() < 5) {
            sErrMsg = tr("Failed to modify the label as parameters
are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        labelType = vecValues.at(0).value<InoLabelType>();
        addaType = vecValues.at(1).value<InoAdDaType>();
        labelItem = vecValues.at(2).value<InoLabelItem>();
        sOldLabel = vecValues.at(3).toString();
        sOldDescri = vecValues.at(4).toString();
        oldLabelAndDescriItem =
InoLabelAndDescriItem(labelItem.nIndex,
                                sOldLabel,
                                sOldDescri);

        if (!labelItem.sLabel.isEmpty() &&
labelItem.sLabel.compare(sOldLabel)) {
            bool isExist = m_commInstance-
>isLabelExisted(labelItem.sLabel);
            if (isExist) {
                sErrMsg = tr("The label already exists.");
                isOk = false;
            }
        }
    }

    if (isOk) {
        isOk = m_commInstance->modifyAdDaItem(labelType, addaType,
labelItem);
        if (!isOk) {
            sErrMsg = tr("Failed to modify the label.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->saveAllLabels();
        if (!isOk) {
            sErrMsg = tr("Failed to save label.");
            // 撤销保存失败的修改
            InoLabelItem item = labelItem;
            item.sLabel = sOldLabel;

```

```

        m_commInstance->modifyAdDaItem(labelType, addaType,
item);
    }
}

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType,
        isOk ? QVariant::fromValue(QVector<InoLabelItem>()) <<
labelItem) : QVariant(), isOk, sErrMsg);

    if (isOk && labelItem.sLabel.compare(sOldLabel)) {
        emit CommunicationEngine::instance()
->signal_virtualKeyboard_LabelChanged(
            (AllLabelType)(LabelType_DA_Start + labelType),
            sOldLabel, labelItem.sLabel); // oldLabel1
    }
}

void CommunicationThread::modifyOtherItem(AbstractCmd *absCmd)
{
    QString sErrMsg;
    QVector<QVariant> vecValues;
    InoLabelType labelType;
    InoLabelItem labelItem;
    QString sOldLabel;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to modify label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>>();
        if (vecValues.count() < 3) {
            sErrMsg = tr("Failed to modify the label as parameters
are invalid.");
            isOk = false;
        }
    }
}

```



```

        if (isOk) {
            labelType = vecValues.at(0).value<InoLabelType>();
            labelItem = vecValues.at(1).value<InoLabelItem>();
            sOldLabel = vecValues.at(2).toString();

            if (!labelItem.sLabel.isEmpty() &&
labelItem.sLabel.compare(sOldLabel)) {
                bool isExist = m_commInstance-
>isLabelExisted(labelItem.sLabel);
                if (isExist) {
                    sErrMsg = tr("The label already exists.");
                    isOk = false;
                }
            }
        }

        if (isOk) {
            isOk = m_commInstance->modifyOtherItem(labelType,
labelItem);
            if (!isOk) {
                sErrMsg = tr("Failed to modify label.");
            }
        }

        if (isOk) {
            isOk = m_commInstance->saveAllLabels();
            if (!isOk) {
                sErrMsg = tr("Failed to save label.");
                // 撤销保存失败的修改
                InoLabelItem item = labelItem;
                item.sLabel = sOldLabel;
                m_commInstance->modifyOtherItem(labelType, item);
            }
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            isOk ? QVariant::fromValue(QVector<InoLabelItem>()) <<
labelItem) : QVariant(), isOk, sErrMsg);

        if (isOk && labelItem.sLabel.compare(sOldLabel)) {
            emit CommunicationEngine::instance()
                ->signal_virtualKeyBoard_LabelChanged(

```

```

        (AllLabelType)(LabelType_DA_Start + labelType),
        sOldLabel, labelItem.sLabel); // oldLabel1
    }
}

```

```

C++
bool ProjectInterface::isLabelExisted(const QString &sLabel)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (sLabel.isEmpty()) {
        return false;
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return comm()->project()-
>isLabelExisted(sLabel.toStdString());
}

bool LabelInterface::modifyIoItem(
    const InoLabelType &labelType, const InoLabelIOType &ioType,
    const ShowType &dataType, const InoLabelItem &item)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return false;
    }

    bool ret = modifyIoItemLabel(labelType, ioType, dataType,
item.nIndex,
                                item.sLabel);

    if (!ret) {
        return false;
    }

    ret = modifyIoItemDescription(labelType, ioType, dataType,
item.nIndex,
                                item.sDescription);

    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
}

```

```

bool LabelInterface::modifyAdDaItem(
    const InoLabelType &labelType, const InoAdDaType &addaType,
    const InoLabelItem &item)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return false;
    }

    bool ret = modifyAdDaItemLabel(labelType, addaType,
item.nIndex,
                                item.sLabel);

    if (!ret) {
        return false;
    }

    ret = modifyAdDaItemDescription(labelType, addaType,
item.nIndex,
                                item.sDescription);

    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
}

bool LabelInterface::modifyOtherItem(const InoLabelType
&labelType,
                                const InoLabelItem &item)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return false;
    }

    bool ret = modifyOtherItemLabel(labelType, item.nIndex,
item.sLabel);
    if (!ret) {
        return false;
    }

    ret = modifyOtherItemDescription(labelType, item.nIndex,
item.sDescription);
    FREQ_LOG_PRINT_TIMESTAMP

```

```

        return ret;
    }

    bool LabelInterface::saveAllLabels()
    {
        FREQ_LOG_PRINT_TIMESTAMP_THREAD

        if (!isStatusOK()) {
            return false;
        }

        int ret = comm()->project()->GetLabel()->SaveLabels();
        if (ret != ERR_OK) {
            return false;
        }

        comm()->project()->updateLablesToTransfor();

        comm()->project()->sendSyncFlag(

static_cast<int>(InoRobBusiness::SyncProjcetInfoType::SYNC_LABEL_I
NFO));
        FREQ_LOG_PRINT_TIMESTAMP
        return (ret == ERR_OK);
    }

```

5.6 更新工具 IO 描述 (updateToolItemDesc)

5.6.1 说明

接口	功能	返回值	参数	参数说明
bool LabelInterface ::updateItem Description(In oLabelType,	仅更新描述 (工具 IO 批 量场景)	bool	nIndex、描述 文本	不经由 modifyItem 双字段合一逻辑时调用

int, QString)				
InoLabelType LabelInterface ::getLabelType (const QString &namePrefix)	由 In/Out/InB... 前缀解析类型	InoLabelType (见 5.2)	名称前缀	updateToolIOItemDesc 内解析工具 IO 名
bool LabelInterface ::isToolItemDescLangCorrect()	工具 IO 描述语言是否满足约束	bool	无	不满足则处理流程直接失败

5.6.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::CmdType_UpdateToolIOItemDesc, content);
```

5.6.3 函数实现

```
C++
void CommunicationThread::updateToolIOItemDesc(AbstractCmd
*absCmd)
{
    QString content;
    QString sErrMsg;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to update label.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        content = cmd->m_value.toString();
        if (content.isEmpty()) {
            sErrMsg = tr("Failed to update label as the tool I/O
```

```

description is empty.");
        isOk = false;
    }

    if (Communication::instance()-
>isToolIoItemDescLangCorrect()) {
        isOk = false;
    }
}

if (isOk) {
    QStringList toolDescInfos = content.split("|");
    if (toolDescInfos.isEmpty()) {
        sErrMsg = tr("Failed to get tool I/O description.");
        isOk = false;
    }

    for (const QString &toolDescs : toolDescInfos) {
        QStringList toolDesc = toolDescs.split(":");
        if (toolDesc.count() != 2) {
            sErrMsg = QString("Tool IO description:%1 is
invalid.").arg(toolDescs);
            continue;
        }

        QString toolIName = toolDesc.at(0);
        int index = toolIName.mid(toolIName.indexOf("[") +
1, toolIName.indexOf("]")

- toolIName.indexOf("[") - 1).toInt();
        InoLabelType labelType = m_commInstance-
>getLabelType(toolIName.left(toolIName.indexOf("[")));

        m_commInstance->updateIoItemDescription(labelType,
index, toolDesc.at(1));
    }
}

if (isOk) {
    isOk = m_commInstance->saveAllLabels();
    if (!isOk) {
        sErrMsg = tr("Failed to save label.");
    }
}
}

```

```

        if (!isOk) {
            LOG_INFO(sErrMsg);
            sErrMsg.clear();
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,
            QString());
    }

```

```

C++
bool LabelInterface::updateIoItemDescription(const InoLabelType
&labelType,
                                           int nIndex, const
QString &sDescription)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!isStatusOK()) {
        return false;
    }

    int ret = comm()->project()->GetLabel()-
>ModifyItemDescription(
        MetaTypeConversion::inoApi2tp_labelType(labelType),
        nIndex,
        sDescription.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

InoLabelType LabelInterface::getLabelType(const QString
&namePrefix)
{
    InoLabelType labelType = IO_INPUT_BIT;
    if (0 == namePrefix.compare("In", Qt::CaseInsensitive)) {
        labelType = IO_INPUT_BIT;
    } else if (0 == namePrefix.compare("InB",
Qt::CaseInsensitive)) {
        labelType = IO_INPUT_BYTE;
    }
}

```

```

        } else if (0 == namePrefix.compare("InW",
Qt::CaseInsensitive)) {
            labelType = IO_INPUT_WORD;
        } else if (0 == namePrefix.compare("Out",
Qt::CaseInsensitive)) {
            labelType = IO_OUTPUT_BIT;
        } else if (0 == namePrefix.compare("OutB",
Qt::CaseInsensitive)) {
            labelType = IO_OUTPUT_BYTE;
        } else if (0 == namePrefix.compare("OutW",
Qt::CaseInsensitive)) {
            labelType = IO_OUTPUT_WORD;
        }

        return labelType;
    }
}

```

```

C++
bool LabelInterface::isToolIoItemDescLangCorrect()
{
    QString lang = Common::instance()->getConfigInfo().m_language;
    if (lang.isEmpty()) {
        lang = "en_US";
    }

    bool isContainsEmptyLabel = false;
    QList<bool> chineseDescList;
    for (int lType = IO_INPUT_BIT; lType <= IO_OUTPUT_WORD;
lType++) {
        InoLabelType labelType = static_cast<InoLabelType>(lType);
        for (int sType = ShowType_Bit; sType < ShowType_Size;
sType++) {
            ShowType showType = static_cast<ShowType>(sType);
            int nStartIndex = this-
>getIoStartIndex(InoLabelIOType_ToolIO, showType);
            int nEndIndex = this-
>getIoEndIndex(InoLabelIOType_ToolIO, showType);

            for (int nIndex = nStartIndex; nIndex <= nEndIndex;
nIndex++) {
                QString sDesc = this-
>getItemDescription(labelType, nIndex);

```



```
        if (sDesc.isEmpty()) {
            isContainsEmptyLabel = true;
            continue;
        }

        chineseDescList <<
StringUtils::isContainChinese(sDesc);
    }
}

if ((lang.compare("zh_CN") && chineseDescList.contains(true))
    || (!lang.compare("zh_CN")
&& !chineseDescList.contains(true))
    || !isContainsEmptyLabel) {
    return false;
}

return true;
}
```

5.7 获取自定义报警文件列表 (getDefineWarningFileList)

5.7.1 说明

接口	功能	返回值	参数	参数说明
std::vector<std::string> ProjectInterface::getDefineWarningFileList()	获取工程下用户自定义报警相关文件名称列表	std::vector<std::string>	无	IProject::GetDefineWarningFileList()

5.7.2 调用

```
C++
// DefineWarningListForm::refreshDatas
```

```
CommunicationEngine::instance()->enqueueCmd(  
    this, AbstractCmd::CmdType_GetUserDefineWarningFileList);
```

5.7.3 函数实现

```
C++  
void CommunicationThread::getUserDefineWarningFileList(AbstractCmd  
*absCmd)  
{  
    QStringList listFiles;  
  
    std::vector<std::string> files = m_commInstance->  
getDefineWarningFileList();  
    for (const std::string &file : files) {  
        listFiles << QString::fromStdString(file);  
    }  
  
    emit CommunicationEngine::instance()->  
signal_handleProject_result(  
        absCmd->m_object, absCmd->m_cmdType, QVariant(listFiles),  
        true, QString());  
}
```

```
C++  
std::vector<std::string>  
ProjectInterface::getDefineWarningFileList()  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
  
    std::vector<std::string> fileNames  
        = comm()->project()->GetDefineWarningFileList();  
  
    FREQ_LOG_PRINT_TIMESTAMP  
  
    return fileNames;  
}
```

5.8 读取自定义报警条目 (readWarnings/getWarnings)

5.8.1 说明

接口	功能	返回值	参数	参数说明
bool DefineWarningInterface::readWarnings()	从工程文件读取用户自定义报警到内存	bool	无	ERR_OK 为成功
std::vector<std::string> DefineWarningInterface::getWarnings()	获取已加载的报警文本序列	std::vector<std::string>	无	与 GetDefineWarning()->GetWarnings().Warnings 对应

5.8.2 调用

```
C++
// DefineWarningDetailForm::refreshDatas
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_GetUserDefineWarnings);
```

5.8.3 函数实现

```
C++
void CommunicationThread::getUserDefineWarnings(AbstractCmd
*absCmd)
{
    QString sErrMsg;
    QStringList listWarnings;

    bool isOk = m_commInstance->readWarnings();
    if (!isOk) {
        sErrMsg = tr("Failed to get user alarms data.");
    }
}
```

```

        if (isOk) {
            std::vector<std::string> vecWarnings = m_commInstance-
>getWarnings();
            for (const std::string &warning : vecWarnings) {
                listWarnings << QString::fromStdString(warning);
            }
        }

        emit CommunicationEngine::instance()-
>signal_handleProject_result(
            absCmd->m_object, absCmd->m_cmdType,
            isOk ? QVariant::fromValue(listWarnings) : QVariant(),
            isOk, sErrMsg);
    }

```

```

C++
bool DefineWarningInterface::readWarnings()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int ret = comm()->project()->GetDefineWarning()-
>ReadWarnings();
    LOG_TRACE(QString("[%1][%2][%3]")
                .arg(__FILE__, __FUNCTION__,
                    QString::number(QDateTime::currentMSecsSinceEpoch())));
    return (ret == ERR_OK);
}

std::vector<std::string> DefineWarningInterface::getWarnings()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<std::string> datas =
        comm()->project()->GetDefineWarning()-
>GetWarnings().Warnings;
    FREQ_LOG_PRINT_TIMESTAMP
    return datas;
}

```

5.9 修改自定义报警内容 (isWarningValid / modifyWarning / saveWarnings)

5.9.1 说明

接口	功能	返回值	参数	参数说明
bool DefineWarningInterface::isWarningValid(const QString &warning)	校验报警文本是否合法	bool	warning	内部 IsWarningValid(errMsg, ...) , 且 errMsg 需为空
bool DefineWarningInterface::modifyWarning(int index, const QString &warning)	修改指定索引处报警内容	bool	index、 warning	对应 ModifyWarning
bool DefineWarningInterface::saveWarnings()	保存到文件并同步	bool	无	成功后再 sendSyncFlag

5.9.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_handleProject(
    this, AbstractCmd::cmdType_ModifyUserDefineWarning,
    QVector<QVariant>() << ui->le_index->text().toInt()
    << m_sCurWarningContent << sWarningContent);
```

5.9.3 函数实现

```

C++
void CommunicationThread::modifyUserDefineWarnig(AbstractCmd
*absCmd)
{
    QString sErrMsg;
    int nIndex = INVALID_INDEX;
    QVector<QVariant> vecValues;
    QString sOriginWarnigContent;
    QString sWarnigContent;
    bool isOk = true;

    CmdCommonValue *cmd = dynamic_cast<CmdCommonValue *>(absCmd);
    if (!cmd) {
        sErrMsg = tr("Failed to modify user alarms.") +
SPACE_AND_CONVERT_PARAMETERS_FALID;
        isOk = false;
    }

    if (isOk) {
        vecValues = cmd->m_value.value<QVector<QVariant>>();
        if (vecValues.count() < 3) {
            sErrMsg = tr("Failed to modify user alarms as the
parameters are invalid.");
            isOk = false;
        }
    }

    if (isOk) {
        nIndex = vecValues.at(0).toInt();
        sOriginWarnigContent = vecValues.at(1).toString();
        sWarnigContent = vecValues.at(2).toString();

        isOk = m_commInstance->isWarningValid(sWarnigContent);
        if (!isOk) {
            sErrMsg = tr("The user alarm content is invalid.");
        }
    }

    if (isOk) {
        isOk = m_commInstance->modifyWarning(nIndex,
sWarnigContent);
        if (!isOk) {
            sErrMsg = tr("Failed to modify user alarms.");
        }
    }
}

```

```

    }

    if (isOk) {
        isOk = m_commInstance->saveWarnings();
        if (!isOk) {
            sErrMsg = tr("Failed to save user alarms.");
            // 撤销保存失败的修改
            m_commInstance->modifyWarning(nIndex,
sOriginWarnigContent);
        }
    }

    emit CommunicationEngine::instance()-
>signal_handleProject_result(
        absCmd->m_object, absCmd->m_cmdType, QVariant(), isOk,
sErrMsg);
}

```

```

C++
bool DefineWarningInterface::modifyWarning(int index, const
QString &warning)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int ret = comm()->project()->GetDefineWarning()-
>ModifyWarning(
        index, warning.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

bool DefineWarningInterface::isWarningValid(const QString
&warning)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::string errMsg;
    bool bRet = comm()->project()->GetDefineWarning()-
>IsWarningValid(errMsg, warning.toStdString());
    FREQ_LOG_PRINT_TIMESTAMP
    return (bRet && errMsg.empty());
}

```

```

bool DefineWarningInterface::saveWarnings()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int ret = comm()->project()->GetDefineWarning()-
>SaveWarnings();
    if (ret != ERR_OK) {
        return false;
    }

    comm()->project()->sendSyncFlag(static_cast<int>(
        InoRobBusiness::SyncProjcetInfoType::SYNC_LABEL_INFO));
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERR_OK);
}

```

6.机器人参数设置

6.1 读取机器人参数

(getRobotParams/getCoRotbotStructParameters/getReductionRatioParam/getRobotCompParam/GetRobotName/getRobotParamRange)

6.1.1 说明

接口	功能	返回值	参数	参数说明
void RobotParameters::get RobotParams(Abstract Cmd *absCmd)	从控制器读 结构/减速比 /补偿及机型 名、参数范 围	无（通过信 号回传）	absCmd	实现于 robotparam eters.cpp
int IRobotArm::getCoRotb otStructParameters(std ::vector<double> &)	读结构参数 (DH 等)	错误码	出参向量	ret1 == ERROR_O K 为成功

int IRobotArm::getReductionRatioParam(double reduRatio[AXIS_NUMB])	读减速比 J1-J6	错误码	数组	ret2
int IRobotArm::getRobotCompParam(double compParam[COM_PARAM_NUM])	读内部补偿	错误码	数组	ret3
bool IMonitor::GetRobotName(char robotStructName[128])	读机器人型号字符串	bool	缓冲区	用于界面「机器人型号」
InoRobBusiness::IRobotParamRange *IRobotParam::getRobotParamRange()	取各字段允许范围 (StructParam / ReductionRatio / InternalCompensation)	指针	—	与机型XML/配置一致

6.1.2 调用

```

C++
// 刷新按钮 pbn_upate（拼写为 upate）；页面显示且已连接时也会触发
void RobotParametersForm::on_pbn_upate_clicked()
{
    if (m_isConnected) {
        CommunicationEngine::instance()->enqueueCmd_getData(
            this, AbstractCmd::CmdType_GetRobotBodyParameters);
    }
}

// showEvent / 连接恢复且本页可见
void RobotParametersForm::UpdateUIData()
{

```

```

    if (!m_isConnected) {
        m_mangerStructuaral->clearEditText();
        m_mangerRadio->clearEditText();
        m_mangerCompensation->clearEditText();
    } else {
        on_pbn_upate_clicked();
    }
}

```

6.1.3 函数实现

```

C++
// communicationthread.cpp -- processTasks
case AbstractCmd::CmdType_GetRobotBodyParameters: {
    m_commInstance->getRobotParams(absCmd);
    break;
}

```

```

C++
void RobotParameters::getRobotParams(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<double> structParams;
    double reduRatio[AXIS_NUMB] = {};
    double compParam[COM_PARM_NUM] = {};
    double coupParam[AXIS_NUMB] = {}, coupParaS[AXIS_NUMB] = {};
    int ret1 = _IRobotArm-
>getCoRotbotStructParameters(structParams);
    int ret2 = _IRobotArm->getReductionRatioParam(reduRatio);
    int ret3 = _IRobotArm->getRobotCompParam(compParam);
    if (ret3 == ERROR_OK) {
        gLastCompensationParameters.clear();
        for (int i = 0; i < COM_PARM_NUM; ++i) {
            gLastCompensationParameters << compParam[i];
        }
    } else {
        for (int i = 0; i < COM_PARM_NUM; ++i) {
            gLastCompensationParameters << 0;
        }
    }
}

```

```

    // int ret4 = _IRobotParam->getCouplingParam(coupParaM,
coupParaS);
    InoRobBusiness::IRobotParamRange *param = comm()-
>robotParam()->getRobotParamRange();

    QList<double> structMax, structMin, reduRadioMax, reduRadioMin,
comMax, comMin, coupleMax, coupleMin;

    vector<std::string> sixStructName = {"a1(mm)", "a2(mm)",
"a3(mm)", "d3(mm)", "d4(mm)", "df(mm)", "d1(mm)", "d5(mm)",
"d6(mm)"};
    std::string keyName = "StructParam";
    for (const std::string &i : sixStructName) {
        structMin.push_back(param->getDoubleMinValue(keyName, i));
        structMax.push_back(param->getDoubleMaxValue(keyName, i));
    }

    vector<std::string> sixJointName = {"J1", "J2", "J3", "J4",
"J5", "J6"};
    vector<std::string> names;
    // 减速比
    std::string paraStructName = "ReductionRatio";
    for (const std::string &i : sixJointName) {
        reduRadioMin.push_back(param-
>getDoubleMinValue(paraStructName, i));
        reduRadioMax.push_back(param-
>getDoubleMaxValue(paraStructName, i));
    }
    // 补偿参数
    std::string unit = "(°)";
    paraStructName = "InternalCompensation";
    for (int i = 0; i < sixJointName.size(); ++i) {
        sixJointName[i] = sixJointName[i] + unit;
    }
    sixJointName.push_back("d3(mm)");
    sixJointName.push_back("d5(mm)");
    sixJointName.push_back("a4(mm)");
    sixJointName.push_back("a5(mm)");
    sixJointName.push_back("J7(°)");
    for (const std::string &i : sixJointName) {
        comMin.push_back(param->getDoubleMinValue(paraStructName,
i));
        comMax.push_back(param->getDoubleMaxValue(paraStructName,
i));
    }

```

```

    }
    // 耦合参数
    // 获取范围
    // paraStructName = "CouplingParam";
    // std::string paraName = "J5(分子)"; // 机型参数查询 必须写中文
    // coupleMin << param->getDoubleMinValue(paraStructName,
paraName);
    // coupleMax << param->getDoubleMaxValue(paraStructName,
paraName);

    // paraName = "J6(分母)"; // 机型参数查询 必须写中文
    // coupleMin << param->getDoubleMinValue(paraStructName,
paraName);
    // coupleMax << param->getDoubleMaxValue(paraStructName,
paraName);

    // 机器人型号 2
    char robotStructName[128] = {0};
    bool ret5 = _IMonitor->GetRobotName(robotStructName);
    QString name;
    if (ret5)
        name = robotStructName;
    else
        name = "Unknown";

    if (ret1 == ERROR_OK && ret2 == ERROR_OK
        && ret3 == ERROR_OK /*&& ret4 == ERROR_OK*/) {
        QList<double> compen = arrayToQList(compParam,
COM_PARM_NUM);
        double beta3 = compParam[13];
        compParam[13] = compen[10];
        compen[10] = beta3;
        emit CommunicationEngine::instance()-
>signal_getRobotParameters_result(
            absCmd->m_object,
            name,
            QList<double>(structParams.begin(),
structParams.end()),
            structMin,
            structMax,
            arrayToQList(reduRatio, AXIS_NUMB),
            reduRadioMin,
            reduRadioMax,
            compen,

```

```

        comMin,
        comMax,
        QList<double>() << coupParaM[4] << coupParaS[5],
        coupleMin,
        coupleMax);
    } else {
        emit CommunicationEngine::instance()-
>signal_getRobotParameters_isSuccess(
        absCmd->m_object, false);
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
        QObject::tr("Failed to get robot parameters."));
    }
    FREQ_LOG_PRINT_TIMESTAMP
    // int ret4 = _IRobotParam-
>getRotbotStructParameters(structParams);
}

```

```

C++
int
RobotArmDefault::getCoRotbotStructParameters(std::vector<double>
&data)
{
    UNREALTIME_CMD UnRealCmd;
    UnRealCmd.nId = ROBOT_PARA_ROBOT;
    UnRealCmd.nState = READ_DATA;
    int nRet = _pDataService->UnRealCmdIo(UnRealCmd);
    if (ERR_OK != nRet)
    {
        return nRet;
    }
    data = std::vector<double>(9);
    memcpy(&data[0], UnRealCmd.nContent, sizeof(double) * 9);

    return 0;
}

```

```

C++
// 读取减速比参数
int RobotArmDefault::getReductionRatioParam(double
reduRatio[AXIS_NUMB])
{

```

```

// 读取减速比参数
AXIS_DBPARAM reduParam;
int nRet = ReadSpeedDownRatio(reduParam);
if (nRet != ERR_OK)
{
    return -1;
}
memcpy(&reduRatio[0], &reduParam.dbValue[0], sizeof(double) *
AXIS_NUMB);

return 0;
}

```

```

C++
// 从控制器读取减速比参数
int RobotArmDefault::ReadSpeedDownRatio(AXIS_DBPARAM &AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    int ret = ReadAxisDbParam(DbParam, REDUCTION_RATION);
    if (ret == 0)
        memcpy(AxisDbParam.dbValue, DbParam, AXIS_NUM *
sizeof(double));
    return ret;
}

```

```

C++
// 获取内部补偿参数
int RobotArmDefault::getRobotCompParam(double
compParam[COM_PARM_NUM])
{
    int nRet = ReadRobotRotParam(compParam);
    if (nRet != ERR_OK)
    {
        return -1;
    }

    return 0;
}

```

```

C++
// 将内部补偿参数（旋转角度结构参数）写入控制器 — 读取侧

```

```

int RobotArmDefault::ReadRobotRotParam(double Param[16])
{
    UNREALTIME_CMD UnRealCmd;
    UnRealCmd.nId = ROTPARAM;
    UnRealCmd.nState = READ_DATA;

    int nRet = _pDataService->UnRealCmdIo(UnRealCmd);
    if (ERR_OK != nRet)
    {
        return nRet;
    }

    // 解析内容
    CDataConvert::MemInt8uToDouble(&UnRealCmd.nContent[0],
    &Param[0], 16);

    return ERR_OK;
}

```

6.2 保存机器人参数

(setRobotParams/saveCoRobotStructParam/saveRobotReductionRatioParam/saveFileCommond)

6.2.1 说明

接口	功能	返回值	参数	参数说明
void RobotParameters::set RobotParams(Abstract Cmd *absCmd)	校验状态后 依次下发结 构、减速 比、补偿， 必要时落盘 并刷新仿真	无	从 CmdDatas 解包	见实现内 StatusChec ks、补偿与 gLastComp ensationPar ameters 合 并逻辑

int IRobotArm::saveCoRobotStructParam(const std::vector<double> &data, bool autoSaveFileCommd)	写结构参数	错误码	第二参数为 false 时不弹 窗/不自动保 存文件	ret1
int IRobotArm::saveRobot ReductionRatioParam(double *reduRatio, bool autoSaveFileCommd)	写减速比	错误码	ret2	
int IRobotArm::saveRobot CompensationParam(d ouble *compParam, bool autoSaveFileCommd)	写内部补偿	错误码	ret3	
void IRobotArm::saveFileCo mmand(bool *ok)	向控制器发 保存到文件 类命令	无	任一子项成 功则尝试调 用	成功后提示 重新上电与 设零点等 (PRINT_ WARN)

前置条件：InoRobBusiness::StatusChecks::cobotCheck 需满足示教器权限、手动低速、去使能；否则只发警告，不下发。

6.2.2 调用

```
C++
void RobotParametersForm::on_pbn_save_clicked()
{
    CommunicationEngine::instance()->enqueueCmd_setData(
        this,
        AbstractCmd::CmdType_WriteRobotBodyParameters,
        m_mangerStructuaral->getEditDoubleValue(),
        m_mangerRadio->getEditDoubleValue(),
        m_mangerCompensation->getEditDoubleValue(),
        QList<double>() /* 耦合占位, 当前未用 */);
}
```



```
on_pbn_upate_clicked(); // 保存后再读一次刷新界面  
}
```

6.2.3 函数实现

```
C++  
// communicationthread.cpp – processTasks  
case AbstractCmd::CmdType_WriteRobotBodyParameters: {  
    m_commInstance->setRobotParams(absCmd);  
    break;  
}
```

```
C++  
void RobotParameters::setRobotParams(AbstractCmd *absCmd)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
  
    auto [structParams, reduRatioParam, compParam, coupling] =  
    ((CmdDatas<QList<double>, QList<double>, QList<double>,  
    QList<double>> *)absCmd)->m_data;  
    std::vector<double> vector;  
    std::copy(structParams.begin(), structParams.end(),  
    std::back_inserter(vector));  
    for (int i = reduRatioParam.size(); i < AXIS_NUMB; ++i)  
        reduRatioParam.push_back(0);  
  
    for (int i = compParam.size(); i < COM_PARM_NUM; ++i)  
        compParam.push_back(0);  
  
    double beta3 = compParam[10];  
    for (int i = 10; i < 16; ++i) {  
        compParam[i] = gLastCompensationParameters.at(i);  
    }  
    compParam[13] = beta3;  
    // qDebug()<<__FUNCTION__<<compParam;  
  
    bool check =  
    InoRobBusiness::StatusChecks::getInstance().cobotCheck(  
        InoRobBusiness::StatusItemGroup::CTRL_AUTHORITY_TEACHPAD  
        | InoRobBusiness::StatusItemGroup::DEVICE_MODE_MANUAL_LOW  
        | InoRobBusiness::StatusItemGroup::CTRL_STATUS_DISENABLE);
```

```

        if (!check) {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                RobotParametersTr::tr("Save failed. Please switch to
manual low mode, disable the robot, and change control authority
to the teach pendant, and then try again.));
            return;
        }

        int ret1 = _IRobotArm->saveCoRobotStructParam(vector, false);
        int ret2 = _IRobotArm-
>saveRobotReductionRatioParam(&reduRatioParam[0], false);
        int ret3 = _IRobotArm-
>saveRobotCompensationParam(&compParam[0], false);
        // int ret4 = _IRobotParam->getCouplingParam(coupParaM,
coupParaS);
        // if(ret4 == ERROR_OK){
        //     QList<double> temp;
        //     for(int i=0;i<AXIS_NUMB;++i){
        //         temp.push_back(coupParaM[i]);
        //     }
        //     for(int i=0;i<AXIS_NUMB;++i){
        //         temp.push_back(coupParaS[i]);
        //     }
        //     temp[4] = coupling[0];
        //     temp[AXIS_NUMB + 5] = coupling[1];
        //     ret4 = _IRobotParam->saveRobotCouplingParam(&temp[0],
false);
        // }
        // coupling
        QString error;
        if (ret1 != ERROR_OK) {
            error += QObject::tr("Failed to save robot structal
parameters. ") + "\n";
        }
        if (ret2 != ERROR_OK) {
            error += QObject::tr("Failed to save robot reduction
parameters.") + "\n";
        }
        if (ret3 != ERROR_OK) {
            error += QObject::tr("Failed to save robot internal
compensation parameters.");
        }
        // if(ret4 != ERROR_OK){

```

```

        //      error += QObject::tr("Save robot coupling compensation
parameters falid!");
        // }
        if (!error.isEmpty()) {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(error);
        }
        if (ret1 == ERROR_OK && ret2 == ERROR_OK && ret3 == ERROR_OK){
            PRINT_MSG(QObject::tr("Robot parameters sent
successfully."));
        }

        if (ret1 == ERROR_OK || ret2 == ERROR_OK || ret3 == ERROR_OK
/*&& ret4 != ERROR_OK*/) {
            bool ok = true;

            _IRobotArm->saveFileCommond(&ok);
            if (ok) {
                PRINT_WARN(RobotParametersTr::tr("After modifying the
structure and kinematic parameters of the robot, please power it
on again and set the zero point!"));
                setRobotParamsForSimulation(absCmd);
            }
        }
        FREQ_LOG_PRINT_TIMESTAMP
    }
}

```

C++

```

// 保存机器人结构参数
int RobotArmDefault::saveCoRobotStructParam(const
std::vector<double> &data, bool autoSaveFileCommd)
{
    int size = data.size() * sizeof(double);
    if (size > CMD_CONTENT_LEN)
        return -1;
    UNREALTIME_CMD UnRealCmd;
    UnRealCmd.nId = ROBOT_PARA_ROBOT;
    UnRealCmd.nState = WRITE_DATA;
    memcpy(UnRealCmd.nContent, &data[0], size);
    int nRet = _pDataService->UnRealCmdIo(UnRealCmd);
    if (nRet != ERR_OK)
    {
        if (autoSaveFileCommd)

```

```

        outputError(MsgCond::PRINT_POPUP, MTR("下发设置数据失败!"));
        return -1;
    }
    // 设置值确认时间延迟
    Sleep(100);

    // 向 Arm 发送保存命令
    if (autoSaveFileCommd)
    {
        saveFileCommd();
        outputSuccess(MsgCond::PRINT_POPUP, MTR("机器人结构参数已修改, 请重新上电, 并设定零点!"));
    }
    return 0;
}

```

C++

```

// 保存机器人的减速比
int RobotArmDefault::saveRobotReductionRatioParam(double
reduRatio[AXIS_NUMB], bool autoSaveFileCommd)
{
    // 保存前权限检查
    if (checkPermissionBeforeSaveRobotParam() == false)
    {
        return -1;
    }

    AXIS_DBPARAM param;
    memcpy(&param.dbValue[0], &reduRatio[0], sizeof(double) *
AXIS_NUMB);
    int nRet = WriteSpeedDownRatio(param);
    if (nRet != ERR_OK)
    {
        if (autoSaveFileCommd)
            outputError(MsgCond::PRINT_POPUP, MTR("下发设置数据失
败!"));
        return -1;
    }

    // 设置值确认时间延迟
    Sleep(100);
}

```

```

// 向 Arm 发送保存命令
if (autoSaveFileCommd)
{
    saveFileCommond();
    outputSuccess(MsgCond::PRINT_POPUP, MTR("机器人减速比参数已
下发, 请重新上电, 并设定零点!"));
}

return 0;
}

```

```

C++
// 保存机器人内部补偿参数
int RobotArmDefault::saveRobotCompensationParam(double
compParam[COM_PARM_NUM], bool autoSaveFileCommd)
{
    // 保存前权限检查
    if (checkPermissionBeforeSaveRobotParam() == false)
    {
        return -1;
    }

    RBTTYPE_E robotType = getRobotType();
    if (robotType == RBTYPE_SIXR_A)
    {
        if ((compParam[0] < 80 || compParam[0] > 100)
            || (compParam[1] < -10 || compParam[1] > 10)
            || (compParam[2] < 80 || compParam[2] > 100)
            || (compParam[3] < -100 || compParam[3] > -80)
            || (compParam[4] < 80 || compParam[4] > 100)
            || (compParam[5] < -10 || compParam[5] > 10)
            || (compParam[6] < -10 || compParam[6] > 10)
            || (compParam[7] < -10 || compParam[7] > 10)
            || (compParam[8] < -10 || compParam[8] > 10)
            || (compParam[9] < -10 || compParam[9] > 10))
        {
            outputError(MsgCond::PRINT_POPUP, MTR("参数超出范围, 保
存失败!"));
            return -1;
        }
    }
}

```

```

// 下发补偿参数
int nRet = WriteRobotRotParam(compParam);
if (nRet != ERR_OK)
{
    if (autoSaveFileCommd)
        outputError(MsgCond::PRINT_POPUP, MTR("下发设置数据失
败!"));
    return -1;
}

// 设置值确认时间延迟
Sleep(100);

// 向 Arm 发送保存命令
if (autoSaveFileCommd)
{
    saveFileCommond();
    outputSuccess(MsgCond::PRINT_POPUP, MTR("机器人内部补偿参数
已下发, 请重新上电, 并设定零点!"));
}
return 0;
}

```

```

C++
// 从控制器读取内部补偿参数（旋转角度结构参数）- 写入侧
int RobotArmDefault::WriteRobotRotParam(double Param[16])
{
    UNREALTIME_CMD UnRealCmd;

    UnRealCmd.nId = ROTPARAM;
    UnRealCmd.nState = WRITE_DATA;

    // 设置内容
    CDataConvert::MemDoubleToInt8u(&Param[0],
    &UnRealCmd.nContent[0], 16);

    int nRet = _pDataService->UnRealCmdIo(UnRealCmd);
    if (ERR_OK != nRet)
    {
        return nRet;
    }
}

```

```

    return ERR_OK;
}

```

```

C++
// 将减速比参数写入控制器
int RobotArmDefault::WriteSpeedDownRatio(AXIS_DBPARAM
&AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    memcpy(DbParam, AxisDbParam.dbValue, AXIS_NUM *
sizeof(double));
    return WriteAxisDbParam(DbParam, REDUCTION_RATION);
}

```

setRobotParams 末尾调用的 **saveFileCommnd(bool *ok)**

```

C++
void RobotArmDefault::saveFileCommnd(bool *ok)
{
    int saveFlag = 1;
    if (writeFileSaveFlag(saveFlag) != ERR_OK)
    {
        if (ok != 0)
            *ok = false;
        outputError(MsgCond::PRINT_POPUP, MTR("固化设置失败!"));
        return;
    }

    // 等待控制完成保存
    Sleep(200);

    int64u dwStartTime1 =
InoRobUtil::DateTimeTool::TimeSpentMsSinceEpoch();
    while (1)
    {
        if
(InoRobUtil::DateTimeTool::SystemTimeSpanMilliseconds(dwStartTime1
) >= 10000)
        {
            if (ok != 0)
                *ok = false;
            outputError(MsgCond::PRINT_POPUP, MTR("保存命令响应超
时!"));
        }
    }
}

```

```
        break;
    }

    int nMark = 0;
    int iRet = readFileSaveFlag(nMark);
    if (iRet != ERR_OK || (nMark >= 1 && nMark <= 10))
    {
        Sleep(200);
    }
    else
    {
        break;
    }
}
if (ok != 0)
    *ok = true;
return;
}
```

6.3 读取运行参数

(readCoorMaxSpeed/readMaxGuiseSpeed/getAxisMaxSpeed/readCoorMaxAcc/readMaxGuiseAcc/getAxisMaxAcc/readCoorMaxDec/readTransJogDistan)

6.3.1 说明

接口	功能	返回值	参数	说明
int RobotArmDefault::readCoorMaxSpeed(double &dbSpeed)	从控制器读取最大允许位置速度（笛卡尔线速度上限）	int：底层 UnrealReadData 的返回码；成功一般为 ERR_OK	dbSpeed：出参，单位 mm/s	通过 IDataService::UnrealReadData(..., CPOSITION_MAX_VEI) 读单值

int RobotArmDefault::readMaxGuiseSpeed(double &dbGuiseSpeed)	读取最大允许姿态速度（姿态角速度上限）	同上	dbGuiseSpeed: 出参, 单位 °/s	与位置速度同属运行速度界面中的笛卡尔「姿态」项
int RobotArmDefault::getAxisMaxSpeed(double AxisDbParam[AXIS_NUM])	读取各轴最大允许关节速度（运动用）	0 表示成功; readAxisMaxSpeed 非 ERR_OK 时返回 -1	AxisDbParam: 长度 AXIS_NUM 的出参数组, 单位 °/s	内部调用 readAxisMaxSpeed → ReadAxisDbParam(..., MAX_VEL)
int RobotArmDefault::readCoordMaxAcc(double &dbAcc)	读取最大允许位置加速度	int: UnRealReadData 返回码	dbAcc: 出参, 单位 mm/s ²	对应运行加速度里「位置加速度」
int RobotArmDefault::readMaxGuiseAcc(double &dbGuiseAcc)	读取最大允许姿态加速度	同上	dbGuiseAcc: 出参, 单位 °/s ²	对应运行加速度里「姿态加速度」
int RobotArmDefault::getAxisMaxAcc(double AxisDbParam[AXIS_NUMB])	读取各轴最大允许关节加速度（运动用）	0 成功; readAxisMaxAcc 失败返回 -1	AxisDbParam: 长度 AXIS_NUMB 的出参, 单位 °/s ²	内部 readAxisMaxAcc → ReadAxisDbParam(..., MAX_ACC)
int RobotArmDefault::readCoordMaxDec(double &dbStopDec)	读取最大允许位置减速度（停止减速度类参数）	int: UnRealReadData 返回码	dbStopDec: 出参	与「运行减速度」中笛卡尔位置项对应
int	读取过渡特性	ERR_OK 或中	dbJoint、	先读

RobotArmDefault::readTransJogDistance(double &dbJoint, double &dbPos, double &dbAngle)	相关距离/精度 (关节区、位置区、姿态角)	途失败的错误码	dbPos、dbAngle: 三个出参; 当前实现里姿态角仍读GES_ZONEP ARA (与关节共用注释「暂时无姿态」)	GES_ZONEP ARA→ZONE PARA→再读GES_ZONEP ARA 到 dbAngle; 用于过渡特性编辑框
--	-----------------------	---------	---	--

6.3.2 调用

```
C++
// 刷新 pbn_update
void MotionParametersForm::on_pbn_update_clicked()
{
    if (m_isConnected) {
        CommunicationEngine::instance()->enqueueCmd_getData(
            this, AbstractCmd::CmdType_GetMotionParameters);
    }
}
```

6.3.3 函数实现

```
C++
// communicationthread.cpp – processTasks
case AbstractCmd::CmdType_GetMotionParameters: {
    m_commInstance->getMotionParams(absCmd);
    break;
}
```

```
C++
void RobotParameters::getMotionParams(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
```

```
QList<double> posVelRangeMin;
QList<double> posVelRangeMax;

QList<double> jointVelRangeMin;
QList<double> jointVelRangeMax;

QList<double> posAccRangeMin;
QList<double> posAccRangeMax;

QList<double> jointAccRangeMin;
QList<double> jointAccRangeMax;

QList<double> posDecRangeMin;
QList<double> posDecRangeMax;

QList<double> jointDecRangeMin;
QList<double> jointDecRangeMax;

QList<double> transRangeMin;
QList<double> transRangeMax;

// 最大允许位置速度（示教）
double posSpeed = 0.0;
int retPSpeed = _IRobotArm->readCoorMaxSpeed(posSpeed);

// 最大允许姿态速度（示教）
double guiseSpeed = 0.0;
int retGSpeed = _IRobotArm->readMaxGuiseSpeed(guiseSpeed);

// 最大允许关节速度（运动）
double jSpeedParam[AXIS_NUM] = {0.0};
int retJSpeed = _IRobotArm->getAxisMaxSpeed(jSpeedParam);

// 最大允许位置加速度（运动）
double posAcc = 0.0;
int retPAcc = _IRobotArm->readCoorMaxAcc(posAcc);

// 最大允许姿态加速度（运动）
double guiseAcc = 0.0;
int retGAcc = _IRobotArm->readMaxGuiseAcc(guiseAcc);

// 最大允许关节加速度（运动）
double jAccParam[AXIS_NUMB] = {0.0};
int retJAcc = _IRobotArm->getAxisMaxAcc(jAccParam);
```

```

// 最大允许位置减速度（运动）
double posDec = 0.0;
int retPDec = _IRobotArm->readCoorMaxDec(posDec);

// 最大允许姿态减速度（运动）
double guiseDec = 0.0;
int retGDec = _IRobotArm->readGuiseMaxDec(guiseDec);

// 最大允许关节减速度（运动）
double jDecParam[AXIS_NUMB] = {0.0};
int retJDec = _IRobotArm->getAxisMaxDec(jDecParam);

// 过渡特性（运动）
double jointTransUnit = 0.0;
double posTransUnit = 0.0;
double angleTransUnit = 0.0;
int retTrans = _IRobotArm->readTransJogDistan(jointTransUnit,
posTransUnit, angleTransUnit);

InoRobBusiness::IRobotParamRange *param = comm()-
>robotParam()->getRobotParamRange();

getParamRangeDoubleValue(param, "RunVelocityParam",
"coorMaxSpeed(mm/s)",
posVelRangeMin, posVelRangeMax);
getParamRangeDoubleValue(param, "RunVelocityParam",
"guiseMaxSpeed(°/s)",
posVelRangeMin, posVelRangeMax);

vector<std::string> paraVelNames = {"J1(°/s)", "J2(°/s)",
"J3(°/s)",
"J4(°/s)", "J5(°/s)",
"J6(°/s)"};
getParamRangeDoubleValue(param, "RunVelocityParam",
paraVelNames,
jointVelRangeMin, jointVelRangeMax);

getParamRangeDoubleValue(param, "RunAccParam",
"coorMaxAcc(mm/s²)",
posAccRangeMin, posAccRangeMax);
getParamRangeDoubleValue(param, "RunAccParam",
"guiseMaxAcc(°/s²)",
posAccRangeMin, posAccRangeMax);

```

```

paraVelNames.clear();
paraVelNames = {"J1(°/s²)", "J2(°/s²)", "J3(°/s²)",
                "J4(°/s²)", "J5(°/s²)", "J6(°/s²)"};
getParamRangeDoubleValue(param, "RunAccParam", paraVelNames,
                          jointAccRangeMin, jointAccRangeMax);

getParamRangeDoubleValue(param, "RunDecreaseAccParam",
"coordMaxDec(mm/s²)",
                          posDecRangeMin, posDecRangeMax);
getParamRangeDoubleValue(param, "RunDecreaseAccParam",
"guiseMaxDec(°/s²)",
                          posDecRangeMin, posDecRangeMax);
getParamRangeDoubleValue(param, "RunDecreaseAccParam",
paraVelNames,
                          jointDecRangeMin, jointDecRangeMax);

getParamRangeDoubleValue(param, "RunTransitionParam",
"jointTrans",
                          transRangeMin, transRangeMax);
getParamRangeDoubleValue(param, "RunTransitionParam",
"posTrans",
                          transRangeMin, transRangeMax);

if (retPSpeed == ERROR_OK && retGSpeed == ERROR_OK &&
retJSpeed == ERROR_OK
    && retPAcc == ERROR_OK && retGAcc == ERROR_OK && retJAcc
== ERROR_OK
    && retPDec == ERROR_OK && retGDec == ERROR_OK && retJDec
== ERROR_OK
    && retTrans == ERROR_OK) {
    emit CommunicationEngine::instance()-
>signal_getMotionParameters_result(
        absCmd->m_object,
        QList<double>() << posSpeed << guiseSpeed,
        posVelRangeMin,
        posVelRangeMax,
        arrayToQList(jSpeedParam, AXIS_NUM),
        jointVelRangeMin,
        jointVelRangeMax,
        QList<double>() << posAcc << guiseAcc,
        posAccRangeMin,
        posAccRangeMax,
        arrayToQList(jAccParam, AXIS_NUMB),
        jointAccRangeMin,

```

```

        jointAccRangeMax,
        QList<double>() << posDec << guiseDec,
        posDecRangeMin,
        posDecRangeMax,
        arrayToQList(jDecParam, AXIS_NUMB),
        jointDecRangeMin,
        jointDecRangeMax,
        QList<double>() << jointTransUnit << posTransUnit,
        transRangeMin,
        transRangeMax);
    } else {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
        QObject::tr("Failed to get motion parameters."));
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

```

C++

// 获取最大允许关节速度（运动）

```

int RobotArmDefault::getAxisMaxSpeed(double AxisDbParam[AXIS_NUM])
{
    AXIS_DBPARAM param;
    int nRet = readAxisMaxSpeed(param);
    if (nRet != ERR_OK)
    {
        return -1;
    }

    memcpy(&AxisDbParam[0], &param.dbValue[0], sizeof(double) *
AXIS_NUM);

    return 0;
}

```

// 读取最大允许关节速度（运动）

```

int RobotArmDefault::readAxisMaxSpeed(AXIS_DBPARAM &AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    int ret = ReadAxisDbParam(DbParam, MAX_VEL);
    if (ret == 0)
        memcpy(AxisDbParam.dbValue, DbParam, AXIS_NUM *
sizeof(double));
}

```

```

        return ret;
    }

    // 读取最大允许位置速度（运动）
    int RobotArmDefault::readCoorMaxSpeed(double &dbSpeed)
    {
        return _pDataService->UnRealReadData(dbSpeed,
        CPOSITION_MAX_VE1);
    }

    // 读取最大允许姿态速度（运动）
    int RobotArmDefault::readMaxGuiseSpeed(double &dbGuiseSpeed)
    {
        return _pDataService->UnRealReadData(dbGuiseSpeed,
        CPOSITION_MAX_GVE1);
    }

    // 获取最大允许关节加速度（运动）
    int RobotArmDefault::getAxisMaxAcc(double AxisDbParam[AXIS_NUMB])
    {
        AXIS_DBPARAM param;
        int nRet = readAxisMaxAcc(param);
        if (nRet != ERR_OK)
        {
            return -1;
        }

        memcpy(&AxisDbParam[0], &param.dbValue[0], sizeof(double) *
        AXIS_NUMB);

        return 0;
    }

    // 读取最大允许关节加速度（运动）
    int RobotArmDefault::readAxisMaxAcc(AXIS_DBPARAM &AxisDbParam)
    {
        double DbParam[AXIS_NUM] = { 0 };
        int ret = ReadAxisDbParam(DbParam, MAX_ACC);
        if (ret == 0)
            memcpy(AxisDbParam.dbValue, DbParam, AXIS_NUM *
        sizeof(double));
        return ret;
    }

```

```

// 读取最大允许位置加速度（运动）
int RobotArmDefault::readCoorMaxAcc(double &dbAcc)
{
    return _pDataService->UnRealReadData(dbAcc, CPOSITION_ACC);
}

// 读取最大允许姿态加速度（运动）
int RobotArmDefault::readMaxGuiseAcc(double &dbGuiseAcc)
{
    return _pDataService->UnRealReadData(dbGuiseAcc,
CPOSITION_GES_ACC);
}

// 获取最大允许关节减速度
int RobotArmDefault::getAxisMaxDec(double AxisDbParam[AXIS_NUMB])
{
    AXIS_DBPARAM param;
    int nRet = ReadAxisMaxDec(param);
    if (nRet != ERR_OK)
    {
        return -1;
    }

    memcpy(&AxisDbParam[0], &param.dbValue[0], sizeof(double) *
AXIS_NUMB);

    return 0;
}

// 读取最大允许关节减速度
int RobotArmDefault::ReadAxisMaxDec(AXIS_DBPARAM &AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    int ret = ReadAxisDbParam(DbParam, STOP_DEC);
    if (ret == 0)
        memcpy(AxisDbParam.dbValue, DbParam, AXIS_NUM *
sizeof(double));
    return ret;
}

// 读取最大允许位置减速度
int RobotArmDefault::readCoorMaxDec(double &dbStopDec)
{
    return _pDataService->UnRealReadData(dbStopDec,

```



```

CPOSITION_STOP_DEC);
}

// 读取最大允许姿态减速度
int RobotArmDefault::readGuiseMaxDec(double &dbGuiseStopDec)
{
    return _pDataService->UnRealReadData(dbGuiseStopDec,
CPOSITION_GES_STOP_DEC);
}

// 读取过渡精度数据
int RobotArmDefault::readTransJogDistan(double &dbJoint, double
&dbPos, double &dbAngle)
{
    int nRet;

    nRet = _pDataService->UnRealReadData(dbJoint, GES_ZONEPARA);
    if (nRet != 0)
    {
        return nRet;
    }

    nRet = _pDataService->UnRealReadData(dbPos, ZONEPARA);
    if (nRet != 0)
    {
        return nRet;
    }

    nRet = _pDataService->UnRealReadData(dbAngle, GES_ZONEPARA);
    if (nRet != 0)
    {
        return nRet;
    }

    // 暂时无姿态，关节、姿态共用

    return ERR_OK;
}

```

6.4 保存运行参数

(saveAxisMaxAcc/writeAxisMaxSpeed/writeCoorMaxSpeed/writeMaxGuiseSpeed/saveAxisMaxAcc/writeAxisMaxAcc/writeCoorMaxAcc/writeMaxGuiseAcc/saveAxisMaxDec/writeAxisMaxDec/writeCoorMaxDec/writeGuiseMaxDec/writeTransJogDistan)

6.4.1 说明

接口	功能	返回值	参数	参数说明
int RobotArmDefault::saveAxisMaxAcc(double AxisDbParam[AXIS_NUMB])	保存各轴最大允许关节加速度（运动）	0 成功；底层失败返回 -1	AxisDbParam: 长度 AXIS_NUMB 数组，单位 $^{\circ}/s^2$	封装层：组包后调用 writeAxisMaxAcc
int RobotArmDefault::writeAxisMaxSpeed(AXIS_DBPARAM &AxisDbParam)	写入各轴最大允许关节速度（运动）	WriteAxisDbParam 返回值（成功 ERR_OK）	AxisDbParam: 关节速度结构体	先拷贝到 double DbParam[AXIS_NUM], 写入
int RobotArmDefault::writeCoorMaxSpeed(double dbSpeed)	写入最大允许位置速度	UnRealWriteData 返回值	dbSpeed: mm/s	写入
int RobotArmDefault::writeMaxGuiseSpeed(double dbGuiseSpeed)	写入最大允许姿态速度	UnRealWriteData 返回值	dbGuiseSpeed: $^{\circ}/s$	写入
int RobotArmDefault::sa	保存各轴最大允许关节	0 成功；失	AxisDbParam: 长度	你给的列表里该函数重

veAxisMaxAcc(double AxisDbParam[AXIS_NUMB])	加速度（运动）	败 -1	AXIS_NUMB 数组	复一次，这里保持一次说明即可
int RobotArmDefault::writeAxisMaxAcc(AXIS_DBPARAM &AxisDbParam)	写入各轴最大允许关节加速度（运动）	WriteAxisDbParam 返回值	AxisDbParam: 关节加速度结构体	写入
int RobotArmDefault::writeCoorMaxAcc(double dbAcc)	写入最大允许位置加速度	UnRealWriteData 返回值	dbAcc: mm/s ²	写入
int RobotArmDefault::writeMaxGuiseAcc(double dbGuiseAcc)	写入最大允许姿态加速度	UnRealWriteData 返回值	dbGuiseAcc: °/s ²	写入
int RobotArmDefault::saveAxisMaxDec(double AxisDbParam[AXIS_NUMB])	保存各轴最大允许关节减速度	0 成功; 失败 -1	AxisDbParam: 长度 AXIS_NUMB 数组	封装层: 组包后调用 writeAxisMaxDec
int RobotArmDefault::writeAxisMaxDec(AXIS_DBPARAM &AxisDbParam)	写入各轴最大允许关节减速度	WriteAxisDbParam 返回值	AxisDbParam: 关节减速度结构体	写入
int RobotArmDefault::writeCoorMaxDec(double dbStopDec)	写入最大允许位置减速度	UnRealWriteData 返回值	dbStopDec: 位置减速度值	写入
int RobotArmDefault::writeGuiseMaxDec(double	写入最大允许姿态减速度	UnRealWriteData 返回值	dbGuiseStopDec: 姿态减速度值	

dbGuiseStopDec)				
int RobotArmDefault::writeTransJogDistance(double dbJoint, double dbPos, double dbAngle)	写入过渡特性（关节/位置过渡）	ERR_OK 或中途错误码	dbJoint、dbPos、dbAngle	依次写dbJoint dbPos dbAngle 暂未使用

成功：全部 `ERROR\_OK` 后 调用 `IRobotArm->saveFileCommand(&isOK)`

6.4.2 调用

```
C++
void MotionParametersForm::on_pbn_save_clicked()
{
    CommunicationEngine::instance()->enqueueCmd_setData(
        this,
        AbstractCmd::CmdType_WriteMotionParameters,
        m_posVelocity->getEditDoubleValue(),
        m_jointVelocity->getEditDoubleValue(),
        m_posAcceleration->getEditDoubleValue(),
        m_jointAcceleration->getEditDoubleValue(),
        m_posDeceleration->getEditDoubleValue(),
        m_jointDeceleration->getEditDoubleValue(),
        m_transChara->getEditDoubleValue());
    on_pbn_update_clicked();
}
```

6.4.3 函数实现

```
C++
// communicationthread.cpp - processTasks
case AbstractCmd::CmdType_WriteMotionParameters: {
    m_commInstance->setMotionParams(absCmd);
    break;
}
```

```
}
```

```
C++
void RobotParameters::setMotionParams(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    bool check =
InoRobBusiness::StatusChecks::getInstance().cobotCheck(
    InoRobBusiness::StatusItemGroup::CTRL_AUTHORITY_TEACHPAD
    | InoRobBusiness::StatusItemGroup::DEVICE_MODE_MANUAL_LOW
    | InoRobBusiness::StatusItemGroup::CTRL_STATUS_DISENABLE);
    if (!check) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            RobotParametersTr::tr("Save failed. Please switch to
manual low mode, disable the robot, and change control authority
to the teach pendant, and then try again.));
        return;
    }

    if (!_IRobotArm->checkFileSaveFlag()) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            RobotParametersTr::tr("System busy, please try again
later.));
        return;
    }

    auto [posVelParams, jointVelParams, posAccParams,
jointAccParams, posDecParams, jointDecParams, transParams] =
((CmdDatas<QList<double>, QList<double>, QList<double>,
QList<double>, QList<double>, QList<double>, QList<double>>
*)absCmd)->m_data;

    int retPSpeed = _IRobotArm-
>writeCoorMaxSpeed(posVelParams.at(0));
    int retGSpeed = _IRobotArm-
>writeMaxGuiseSpeed(posVelParams.at(1));

    double jSpeedParam[AXIS_NUM] = {0.0};
    for (int i = 0; i < jointVelParams.count(); i++) {
        jSpeedParam[i] = jointVelParams.at(i);
    }
}
```

```

    }

    int retJSpeed = _IRobotArm->saveAxisMaxSpeed(jSpeedParam);

    int retPAcc = _IRobotArm->writeCoorMaxAcc(posAccParams.at(0));
    int retGAcc = _IRobotArm-
>writeMaxGuiseAcc(posAccParams.at(1));

    double jAccParam[AXIS_NUMB] = {0.0};
    for (int i = 0; i < jointAccParams.count(); i++) {
        jAccParam[i] = jointAccParams.at(i);
    }

    int retJAcc = _IRobotArm->saveAxisMaxAcc(jAccParam);

    int retPDec = _IRobotArm->writeCoorMaxDec(posDecParams.at(0));
    int retGDec = _IRobotArm-
>writeGuiseMaxDec(posDecParams.at(1));

    double jDecParam[AXIS_NUMB] = {0.0};
    for (int i = 0; i < jointDecParams.count(); i++) {
        jDecParam[i] = jointDecParams.at(i);
    }

    int retJDec = _IRobotArm->saveAxisMaxDec(jDecParam);

    double angleTransUnit = 0.0;
    int retTrans = _IRobotArm-
>writeTransJogDistan(transParams.at(0), transParams.at(1),
angleTransUnit);

    QString error;
    if (retPSpeed != ERROR_OK) {
        error += QObject::tr("Failed to save Maximum allowable
position velocity.") + "\n";
    }
    if (retGSpeed != ERROR_OK) {
        error += QObject::tr("Failed to save Maximum allowable
orientation velocity.") + "\n";
    }
    if (retJSpeed != ERROR_OK) {
        error += QObject::tr("Failed to save Maximum allowable
joint velocity.") + "\n";
    }

```

```

        if (retPAcc != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
position acceleration.") + "\n";
        }
        if (retGAcc != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
orientation acceleration.") + "\n";
        }
        if (retJAcc != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
joint acceleration.") + "\n";
        }
        if (retPDec != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
position deleration.") + "\n";
        }
        if (retGDec != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
orientation deleration.") + "\n";
        }
        if (retJDec != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
joint deleration.") + "\n";
        }
        if (retTrans != ERROR_OK) {
            error += QObject::tr("Failed to save transition
characteristic settings.");
        }
        if (!error.isEmpty()) {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(error);
        }
        if (retPSpeed == ERROR_OK && retGSpeed == ERROR_OK &&
retJSpeed == ERROR_OK
            && retPAcc == ERROR_OK && retGAcc == ERROR_OK && retJAcc
== ERROR_OK
            && retPDec == ERROR_OK && retGDec == ERROR_OK && retJDec
== ERROR_OK
            && retTrans == ERROR_OK) {
            bool isOK = true;
            _IRobotArm->saveFileCommond(&isOK);
            if (isOK) {
                PRINT_MSG(QObject::tr("Motion parameters sent
successfully.));
            }
        }
    }
}

```

```

    } else {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            QObject::tr("Failed to save motion parameters."));
    }
}
FREQ_LOG_PRINT_TIMESTAMP
}

```

```
C++
// 保存最大允许关节速度（运动）
int RobotArmDefault::saveAxisMaxSpeed(double
AxisDbParam[AXIS_NUM])
{
    // 保存数据
    AXIS_DBPARAM jointParam;
    memcpy(&jointParam.dbValue[0], &AxisDbParam[0], sizeof(double)
* AXIS_NUMB);
    int nRet = writeAxisMaxSpeed(jointParam);
    if (nRet != ERR_OK)
    {
        return -1;
    }

    return 0;
}

// 写入最大允许关节速度（运动）
int RobotArmDefault::writeAxisMaxSpeed(AXIS_DBPARAM &AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    memcpy(DbParam, AxisDbParam.dbValue, AXIS_NUM *
sizeof(double));
    return WriteAxisDbParam(DbParam, MAX_VEL);
}

// 写入最大允许位置速度（运动）
int RobotArmDefault::writeCoorMaxSpeed(double dbSpeed)
{
    return _pDataService->UnRealWriteData(dbSpeed,
CPOSITION_MAX_VEL);
}
```



```

// 写入最大允许姿态速度（运动）
int RobotArmDefault::writeMaxGuiseSpeed(double dbGuiseSpeed)
{
    return _pDataService->UnRealWriteData(dbGuiseSpeed,
CPOSITION_MAX_GVE1);
}

// 保存最大允许关节加速度（运动）
int RobotArmDefault::saveAxisMaxAcc(double AxisDbParam[AXIS_NUMB])
{
    AXIS_DBPARAM jointParam;
    memcpy(&jointParam.dbValue[0], &AxisDbParam[0], sizeof(double)
* AXIS_NUMB);
    int nRet = writeAxisMaxAcc(jointParam);
    if (nRet != ERR_OK)
    {
        return -1;
    }

    return 0;
}

// 写入最大允许关节加速度（运动）
int RobotArmDefault::writeAxisMaxAcc(AXIS_DBPARAM &AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    memcpy(DbParam, AxisDbParam.dbValue, AXIS_NUM *
sizeof(double));
    return WriteAxisDbParam(DbParam, MAX_ACC);
}

// 写入最大允许位置加速度（运动）
int RobotArmDefault::writeCoorMaxAcc(double dbAcc)
{
    return _pDataService->UnRealWriteData(dbAcc, CPOSITION_ACC);
}

// 写入最大允许姿态加速度（运动）
int RobotArmDefault::writeMaxGuiseAcc(double dbGuiseAcc)
{
    return _pDataService->UnRealWriteData(dbGuiseAcc,
CPOSITION_GES_ACC);
}

```

```

// 保存最大允许关节减速度
int RobotArmDefault::saveAxisMaxDec(double AxisDbParam[AXIS_NUMB])
{
    AXIS_DBPARAM jointParam;
    memcpy(&jointParam.dbValue[0], &AxisDbParam[0], sizeof(double)
* AXIS_NUMB);
    int nRet = writeAxisMaxDec(jointParam);
    if (nRet != ERR_OK)
    {
        return -1;
    }

    return 0;
}

// 写入最大允许关节减速度
int RobotArmDefault::writeAxisMaxDec(AXIS_DBPARAM &AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    memcpy(DbParam, AxisDbParam.dbValue, AXIS_NUM *
sizeof(double));
    return WriteAxisDbParam(DbParam, STOP_DEC);
}

// 写入最大允许位置减速度
int RobotArmDefault::writeCoorMaxDec(double dbStopDec)
{
    return _pDataService->UnRealWriteData(dbStopDec,
CPOSITION_STOP_DEC);
}

// 写入最大允许姿态减速度
int RobotArmDefault::writeGuiseMaxDec(double dbGuiseStopDec)
{
    return _pDataService->UnRealWriteData(dbGuiseStopDec,
CPOSITION_GES_STOP_DEC);
}

// 写入过渡精度数据
int RobotArmDefault::writeTransJogDistan(double dbJoint, double
dbPos, double dbAngle)
{
    int nRet;

```

```

nRet = _pDataService->UnRealWriteData(dbJoint, GES_ZONEPARA);
if (nRet != 0)
{
    return nRet;
}

nRet = _pDataService->UnRealWriteData(dbPos, ZONEPARA);
if (nRet != 0)
{
    return nRet;
}

// 暂时无姿态，关节、姿态共用

return ERR_OK;
}

```

(saveFileCommond 与 6.2 机器人参数相同，见
RobotArmDefault::saveFileCommond(bool *)。)

6.5 读取示教参数

(getTeachParams/readTeachCoorMaxSpeed/readTeachMaxGuiseSpeed/getTeachAxisMaxSpeed/readTeachCoorMaxAcc/readTeachMaxGuiseAcc/getTeachAxisMaxAcc)

6.5.1 说明

接口	功能	返回值	参数	参数说明
void RobotParameters::getTeachParams(AbstractCmd *)	汇总读取示教速度/加速度及范围并发信号	无	absCmd	成功发 signal_getTeachParameters_result

int RobotArmDefault::readTeachCoordMaxSpeed(double &)	读示教最大位置速度	int	dbSpeed	
int RobotArmDefault::readTeachMaxGuiseSpeed(double &)	读示教最大姿态速度	int	dbGuiseSpeed	
int RobotArmDefault::getTeachAxisMaxSpeed(double[AXIS_NUM])	读示教最大关节速度	int	数组出参	
int RobotArmDefault::readTeachCoordMaxAcc(double &)	读示教最大位置加速度	int	dbAcc	
int RobotArmDefault::readTeachMaxGuiseAcc(double &)	读示教最大姿态加速度	int	dbGuiseAcc	
int RobotArmDefault::getTeachAxisMaxAcc(double[AXIS_NUM])	读示教最大关节加速度	int	数组出参	

6.5.2 调用

```
C++
void TeachParametersForm::on_pbn_update_clicked()
```

```

{
    if (m_isConnected) {
        CommunicationEngine::instance()->enqueueCmd_getData(this,
AbstractCmd::CmdType_GetTeachParameters);
    }
}

```

6.5.3 函数实现

```

C++
// communicationthread.cpp – processTasks
case AbstractCmd::CmdType_GetTeachParameters: {
    m_commInstance->getTeachParams(absCmd);
    break;
}

```

```

C++
// robotparameters.cpp – 全文
void RobotParameters::getTeachParams(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    QList<double> posVelRangeMin;
    QList<double> posVelRangeMax;

    QList<double> jointVelRangeMin;
    QList<double> jointVelRangeMax;

    QList<double> posAccRangeMin;
    QList<double> posAccRangeMax;

    QList<double> jointAccRangeMin;
    QList<double> jointAccRangeMax;

    // 最大允许位置速度（示教）
    double posSpeed = 0.0;
    int retPSpeed = _IRobotArm->readTeachCoorMaxSpeed(posSpeed);
}

```

```

// 最大允许姿态速度（示教）
double guiseSpeed = 0.0;
int retGSpeed = _IRobotArm->readTeachMaxGuiseSpeed(guiseSpeed);

// 最大允许关节速度（示教）
double jSpeedParam[AXIS_NUM] = {0.0};
int retJSpeed = _IRobotArm->getTeachAxisMaxSpeed(jSpeedParam);

// 最大允许位置加速度（示教）
double posAcc = 0.0;
int retPAcc = _IRobotArm->readTeachCoorMaxAcc(posAcc);

// 最大允许姿态加速度（示教）
double guiseAcc = 0.0;
int retGAcc = _IRobotArm->readTeachMaxGuiseAcc(guiseAcc);

// 最大允许关节加速度（示教）
double jAccParam[AXIS_NUMB] = {0.0};
int retJAcc = _IRobotArm->getTeachAxisMaxAcc(jAccParam);

InoRobBusiness::IRobotParamRange *param = comm()->robotParam()->getRobotParamRange();

getParamRangeDoubleValue(param, "TeachVelocityParam",
"coorMaxSpeed(mm/s)",
                                posVelRangeMin, posVelRangeMax);
getParamRangeDoubleValue(param, "TeachVelocityParam",
"guiseMaxSpeed(°/s)",
                                posVelRangeMin, posVelRangeMax);

vector<std::string> paraVelNames = {"J1(°/s)", "J2(°/s)",
"J3(°/s)",
                                "J4(°/s)", "J5(°/s)",
"J6(°/s)"};
getParamRangeDoubleValue(param, "TeachVelocityParam",
paraVelNames,
                                jointVelRangeMin, jointVelRangeMax);

getParamRangeDoubleValue(param, "TeachAccParam",
"coorMaxAcc(mm/s²)",
                                posAccRangeMin, posAccRangeMax);
getParamRangeDoubleValue(param, "TeachAccParam",

```

```

"guiseMaxAcc(°/s²)",
                                posAccRangeMin, posAccRangeMax);

    paraVelNames.clear();
    paraVelNames = {"J1(°/s²)", "J2(°/s²)", "J3(°/s²)",
                    "J4(°/s²)", "J5(°/s²)", "J6(°/s²)"};
    getParamRangeDoubleValue(param, "TeachAccParam", paraVelNames,
                                jointAccRangeMin, jointAccRangeMax);

    if (retPSpeed == ERROR_OK && retGSpeed == ERROR_OK &&
retJSpeed == ERROR_OK
        && retPAcc == ERROR_OK && retGAcc == ERROR_OK && retJAcc
== ERROR_OK) {
        emit CommunicationEngine::instance()-
>signal_getTeachParameters_result(
            absCmd->m_object,
            QList<double>() << posSpeed << guiseSpeed,
            posVelRangeMin,
            posVelRangeMax,
            arrayToQList(jSpeedParam, AXIS_NUM),
            jointVelRangeMin,
            jointVelRangeMax,
            QList<double>() << posAcc << guiseAcc,
            posAccRangeMin,
            posAccRangeMax,
            arrayToQList(jAccParam, AXIS_NUMB),
            jointAccRangeMin,
            jointAccRangeMax);
    } else {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            QObject::tr("Failed to get teach parameters."));
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

```

```

C++
int RobotArmDefault::getTeachAxisMaxSpeed(double
AxisDbParam[AXIS_NUM])
{
    AXIS_DBPARAM param;
    int nRet = readTeachAxisMaxSpeed(param);
    if (nRet != ERR_OK)

```

```

    {
        return -1;
    }

    memcpy(&AxisDbParam[0], &param.dbValue[0], sizeof(double) *
    AXIS_NUM);

    return 0;
}

```

```

C++
int RobotArmDefault::readTeachAxisMaxSpeed(AXIS_DBPARAM
&AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    int ret = ReadAxisDbParam(DbParam, T_MAX_VEL);
    if (ret == 0)
        memcpy(AxisDbParam.dbValue, DbParam, AXIS_NUM *
sizeof(double));
    return ret;
}

```

```

C++
int RobotArmDefault::readTeachCoorMaxSpeed(double &dbSpeed)
{
    return _pDataService->UnRealReadData(dbSpeed,
CPOSITION_TMAX_VEL);
}

```

```

C++
int RobotArmDefault::readTeachMaxGuiseSpeed(double &dbGuiseSpeed)
{
    return _pDataService->UnRealReadData(dbGuiseSpeed,
CPOSITION_TMAX_GVEL);
}

```

```

C++
int RobotArmDefault::getTeachAxisMaxAcc(double
AxisDbParam[AXIS_NUMB])
{
    AXIS_DBPARAM param;

```



```

int nRet = readTeachAxisMaxAcc(param);
if (nRet != ERR_OK)
{
    return -1;
}

memcpy(&AxisDbParam[0], &param.dbValue[0], sizeof(double) *
AXIS_NUMB);

return 0;
}

```

```

C++
int RobotArmDefault::readTeachAxisMaxAcc(AXIS_DBPARAM
&AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    int ret = ReadAxisDbParam(DbParam, T_MAX_ACC);
    if (ret == 0)
        memcpy(AxisDbParam.dbValue, DbParam, AXIS_NUM *
sizeof(double));
    return ret;
}

```

```

C++
int RobotArmDefault::readTeachCoorMaxAcc(double &dbAcc)
{
    return _pDataService->UnRealReadData(dbAcc, CPOSITION_TACC);
}

```

```

C++
int RobotArmDefault::readTeachMaxGuiseAcc(double &dbGuiseAcc)
{
    return _pDataService->UnRealReadData(dbGuiseAcc,
CPOSITION_TGES_ACC);
}

```

李昊 1860 60000
2026年04月28日 17:44

李昊 1860 60000
2026年04月28日 17:44

6.6 保存示教参数

(setTeachParams/writeTeachCoorMaxSpeed/writeTeachMaxGuiseSpeed/saveTeachAxisMaxSpeed/writeTeachCoorMaxAcc/writeTeachMaxGuiseAcc/saveTeachAxisMaxAcc/checkFileSaveFlag/saveFileCommond)

6.6.1 说明

接口	功能	返回值	参数	参数说明
void RobotParameters::setTeachParams(AbstractCmd *)	权限检查 + 系统忙检查 + 分项写入	无	absCmd	
int RobotArmDefault::writeTeachCoorMaxSpeed(double)	写示教位置速度	int	dbSpeed	
int RobotArmDefault::writeTeachMaxGuiseSpeed(double)	写示教姿态速度	int	dbGuiseSpeed	
int RobotArmDefault::saveTeachAxisMaxSpeed(double[A_XIS_NUMB])	写示教关节速度	int	关节数组	封装 writeTeachAxisMaxSpeed
int RobotArmDefault::writeTea	写示教位置加速度	int	dbAcc	

chCoorMaxAcc(double)				
int RobotArmDefault::writeTeachMaxGuiseAcc(double)	写示教姿态加速度	int	dbGuiseAcc	
int RobotArmDefault::saveTeachAxisMaxAcc(double[AXIS_NUMB])	写示教关节加速度	int	关节数组	封装 writeTeachAxisMaxAcc
bool RobotArmDefault::checkFileSaveFlag()	保存前系统忙检查	bool	—	忙则返回 false
void RobotArmDefault::saveFileCommand(bool *ok)	固化参数到文件	无	ok 出参	轮询 readFileSaveFlag，最多约 10 秒

6.6.2 调用

```

C++
void TeachParametersForm::on_pbn_save_clicked()
{
    CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_WriteTeachParameters,
    m_posVelocity->getEditDoubleValue(),
    m_jointVelocity->getEditDoubleValue(),
    m_posAcceleration->getEditDoubleValue(),

```

```

m_jointAcceleration->getEditDoubleValue());
    on_pbn_update_clicked();
}

```

6.6.3 函数实现

```

C++
// communicationthread.cpp – processTasks
case AbstractCmd::CmdType_WriteTeachParameters: {
    m_commInstance->setTeachParams(absCmd);
    break;
}

```

```

C++
// robotparameters.cpp – 全文
void RobotParameters::setTeachParams(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    bool check =
InoRobBusiness::StatusChecks::getInstance().cobotCheck(
    InoRobBusiness::StatusItemGroup::CTRL_AUTHORITY_TEACHPAD
    | InoRobBusiness::StatusItemGroup::DEVICE_MODE_MANUAL_LOW
    | InoRobBusiness::StatusItemGroup::CTRL_STATUS_DISENABLE);
    if (!check) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            RobotParametersTr::tr("Save failed. Please switch to
manual low mode, disable the robot, and change control authority
to the teach pendant, and then try again.));
        return;
    }

    if (!_IRobotArm->checkFileSaveFlag()) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            RobotParametersTr::tr("System busy, please try again
later.));
        return;
    }
}

```

```

        auto [posVelParams, jointVelParams, posAccParams,
jointAccParams] = ((CmdDatas<QList<double>, QList<double>,
QList<double>, QList<double>> *)absCmd)->m_data;

        int retPSpeed = _IRobotArm-
>writeTeachCoorMaxSpeed(posVelParams.at(0));
        int retGSpeed = _IRobotArm-
>writeTeachMaxGuiseSpeed(posVelParams.at(1));

        double jSpeedParam[AXIS_NUM] = {0.0};
        for (int i = 0; i < jointVelParams.count(); i++) {
            jSpeedParam[i] = jointVelParams.at(i);
        }

        int retJSpeed = _IRobotArm-
>saveTeachAxisMaxSpeed(jSpeedParam);

        int retPAcc = _IRobotArm-
>writeTeachCoorMaxAcc(posAccParams.at(0));
        int retGAcc = _IRobotArm-
>writeTeachMaxGuiseAcc(posAccParams.at(1));

        double jAccParam[AXIS_NUMB] = {0.0};
        for (int i = 0; i < jointAccParams.count(); i++) {
            jAccParam[i] = jointAccParams.at(i);
        }

        int retJAcc = _IRobotArm->saveTeachAxisMaxAcc(jAccParam);

        QString error;
        if (retPSpeed != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
position velocity.") + "\n";
        }
        if (retGSpeed != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
orientation velocity.") + "\n";
        }
        if (retJSpeed != ERROR_OK) {
            error += QObject::tr("Failed to save Maximum allowable
joint velocity.") + "\n";
        }
        if (retPAcc != ERROR_OK) {

```



```

    {
        return -1;
    }

    return 0;
}

```

```

C++
int RobotArmDefault::writeTeachAxisMaxSpeed(AxisDbParam
&AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    memcpy(DbParam, AxisDbParam.dbValue, AXIS_NUM *
sizeof(double));
    return WriteAxisDbParam(DbParam, T_MAX_VEL);
}

```

```

C++
int RobotArmDefault::writeTeachCoorMaxSpeed(double dbSpeed)
{
    return _pDataService->UnRealWriteData(dbSpeed,
CPOSITION_TMAX_VEL);
}

```

```

C++
int RobotArmDefault::writeTeachMaxGuiseSpeed(double dbGuiseSpeed)
{
    return _pDataService->UnRealWriteData(dbGuiseSpeed,
CPOSITION_TMAX_GVEL);
}

```

```

C++
int RobotArmDefault::saveTeachAxisMaxAcc(double
jointAcc[AXIS_NUMB])
{
    AxisDbParam jointParam;
    memcpy(&jointParam.dbValue[0], &jointAcc[0], sizeof(double) *
AXIS_NUMB);
    int nRet = writeTeachAxisMaxAcc(jointParam);
    if (nRet != ERR_OK)
    {

```

```

        return -1;
    }

    return 0;
}

```

```

C++
int RobotArmDefault::writeTeachAxisMaxAcc(Axis_DbParam
&AxisDbParam)
{
    double DbParam[AXIS_NUM] = { 0 };
    memcpy(DbParam, AxisDbParam.dbValue, AXIS_NUM *
sizeof(double));
    return WriteAxisDbParam(DbParam, T_MAX_ACC);
}

```

```

C++
int RobotArmDefault::writeTeachCoorMaxAcc(double dbAcc)
{
    return _pDataService->UnRealWriteData(dbAcc, CPOSITION_TACC);
}

```

```

C++
int RobotArmDefault::writeTeachMaxGuiseAcc(double dbGuiseAcc)
{
    return _pDataService->UnRealWriteData(dbGuiseAcc,
CPOSITION_TGES_ACC);
}

```

```

C++
bool RobotArmDefault::checkFileSaveFlag()
{
    int nMark = 0;
    int nRet = readFileSaveFlag(nMark);
    if (nRet != ERR_OK || (nMark >= 1 && nMark <= 10))
    {
        outputError(MsgCond::PRINT_POPUP, MTR("系统繁忙，请稍候再
试!"));
        return false;
    }
}

```



```
    return true;
}
```

```
C++
void RobotArmDefault::saveFileCommond(bool *ok)
{
    int saveFlag = 1;
    if (writeFileSaveFlag(saveFlag) != ERR_OK)
    {
        if (ok != 0)
            *ok = false;
        outputError(MsgCond::PRINT_POPUP, MTR("固化设置失败!"));
        return;
    }

    // 等待控制完成保存
    Sleep(200);

    int64u dwStartTime1 =
InoRobUtil::DateTimeTool::TimeSpentMsSinceEpoch();
    while (1)
    {
        if
(InoRobUtil::DateTimeTool::SystemTimeSpanMilliseconds(dwStartTime1
) >= 10000)
        {
            if (ok != 0)
                *ok = false;
            outputError(MsgCond::PRINT_POPUP, MTR("保存命令响应超
时!"));
            break;
        }

        int nMark = 0;
        int iRet = readFileSaveFlag(nMark);
        if (iRet != ERR_OK || (nMark >= 1 && nMark <= 10))
        {
            Sleep(200);
        }
        else
        {

```

```
        break;
    }
}
if (ok != 0)
    *ok = true;
return;
}
```

6.7 工具 (Refresh / Save/ Clear)

6.7.1 说明

接口	功能	返回值	参数	参数说明
bool ToolCalibrateInterface::Refresh(int toolNo, ToolParams *toolParam)	从控制器刷新当前工具参数	bool	工具号、出参	
bool ToolCalibrateInterface::Save(int toolNo, ToolParams *toolParam, bool bSaveNeed)	保存工具参数到控制器	bool	工具号、参数、固化标记	
bool ToolCalibrateInterface::Clear(int toolNo)	清空指定工具参数	bool	工具号	

参数说明（含自定义结构体定义源码）

```

C++
// commandinfo.h
class METATYPE_EXPORT CmdToolSave : public AbstractCmd
{
public:
    int m_toolId = 0;
    ToolParams m_params;
    bool m_bSaveNeed = false;
};

typedef struct _ToolParams_
{
    ...
    bool robHold;      /// > whether tool is holded
    Pos pos;           /// > tool pos, x y z
    Ori ori;           /// > tool orient, A B C
    LoadParams tLoad;  /// > tool load params
} ToolParams;

class METATYPE_EXPORT Pos
{
    ...

    double m_x;  //Unit: millimeter
    double m_y;
    double m_z;
};

class METATYPE_EXPORT Pos
{
    ...

    double m_x;  //Unit: millimeter
    double m_y;
    double m_z;
};

```

6.7.2 调用

李昊 1860 60000996
2026年04月28日 17:44

李昊 1860 60000996
2026年04月28日 17:44

C++

```
ToolParams toolParams;  
toolParams.robHold = ui->cbox_robhold->isChecked();  
ui->coordsetForm->GetPosOri(toolParams.pos, toolParams.ori);  
ui->loadsetForm->GetParams(toolParams.tLoad);  
CommunicationEngine::instance()->enqueueCmd_ToolSave(  
    this, ui->combox_tool_select->currentIndex(), toolParams);
```

C++

```
// ToolForm::UpdateUIData  
CommunicationEngine::instance()->enqueueCmd_handleToolCalibrate(  
    this, AbstractCmd::CmdType_Tool_Refresh, curToolId);
```

C++

```
// ToolForm::slot_clear_pbn_clicked  
CommunicationEngine::instance()->enqueueCmd_handleToolCalibrate(  
    this, AbstractCmd::CmdType_Tool_Clear, ui->combox_tool_select->  
    currentIndex());
```

6.7.3 函数实现

C++

```
case AbstractCmd::CmdType_Tool_Save: {  
    CmdToolSave *cmd = static_cast<CmdToolSave *>(absCmd);  
    ToolParams oldToolParam;  
    Communication::instance()->GetCurToolParams(cmd->m_toolId,  
    oldToolParam);  
  
    bool bRet = Communication::instance()  
        ->Save(cmd->m_toolId, &cmd->m_params, cmd->  
    m_bSaveNeed);  
    if (bRet == true) {  
        m_commInstance->comm()->SetCurToolParams(  
            cmd->m_toolId, cmd->m_params);  
    }  
    emit CommunicationEngine::instance()->signal_tool_Save_result(  
        cmd->m_object, bRet);  
    if (m_commInstance->GetCurToolId() == cmd->m_toolId) {  
        emit CommunicationEngine::instance()->signal_ToolChanged(  
            cmd->m_toolId, cmd->m_params);  
    }  
}
```

```

        cmd->m_toolId);
    }

    if(!(oldToolParam == cmd->m_params)){
        emit CommunicationEngine::instance()-
>signal_toolParamChanged(cmd->m_toolId);
    }
    break;
}
case AbstractCmd::CmdType_Tool_Refresh: {
    CmdCommonToolCalibrate *cmd
        = static_cast<CmdCommonToolCalibrate *>(absCmd);
    ToolParams toolParams;
    bool bRet = Communication::instance()
        ->Refresh(cmd->m_value.toInt(), &toolParams);
    emit CommunicationEngine::instance()
        ->signal_tool_Refresh_result(cmd->m_object, bRet,
toolParams);
    break;
}
case AbstractCmd::CmdType_Tool_Clear: {
    CmdCommonToolCalibrate *cmd
        = static_cast<CmdCommonToolCalibrate *>(absCmd);
    bool bRet = Communication::instance()->Clear(cmd-
>m_value.toInt());
    emit CommunicationEngine::instance()-
>signal_tool_clear_result(bRet);
    break;
}
}

```

C++

```

// toolcalibrateinterface.cpp
bool ToolCalibrateInterface::Save(int toolNo, ToolParams
*toolParam, bool bSaveNeed)
{
    ToolData toolData;
    toolData.RobHold = toolParam->robHold;
    toolData.TFrame.Data[0] = toolParam->pos.m_x;
    toolData.TFrame.Data[1] = toolParam->pos.m_y;
    toolData.TFrame.Data[2] = toolParam->pos.m_z;
    toolData.TFrame.Data[5] = toolParam->ori.m_rx;
    toolData.TFrame.Data[4] = toolParam->ori.m_ry;
}

```

```

        toolData.TFrame.Data[3] = toolParam->ori.m_rz;
        memcpy(&toolData.TLoad, &toolParam->tLoad, sizeof(toolParam-
>tLoad));
        return _ITool->Save(toolNo, &toolData, bSaveNeed);
    }

bool ToolCalibrateInterface::Refresh(int toolNo, ToolParams
*toolParam)
{
    ToolData toolData;
    if (!_ITool->Refresh(toolNo, &toolData)) {
        return false;
    }
    Pos pos(toolData.TFrame.Data[0], toolData.TFrame.Data[1],
        toolData.TFrame.Data[2]);
    Ori ori(toolData.TFrame.Data[5], toolData.TFrame.Data[4],
        toolData.TFrame.Data[3]);
    LoadParams lParams;
    memcpy(&lParams, &toolData.TLoad, sizeof(toolData.TLoad));
    toolParam->SetParams(toolData.RobHold, pos, ori, lParams);
    return true;
}

bool ToolCalibrateInterface::Clear(int toolNo)
{
    return _ITool->Clear(toolNo);
}

```

6.8 工具标定（Calibrate / SaveCalibratePoints / RefreshCalibratePoints）

6.8.1 说明

接口	功能	返回值	参数	参数说明
----	----	-----	----	------

bool ToolCalibrateInterface::Calibrate(const CalibratePoints &, Pos &, Ori &)	调用控制器执行标定计算	bool	输入点位，输出位姿	
bool ToolCalibrateInterface::SaveCalibratePoints(quint16, const CalibratePoints &)	保存标定点到控制器	bool	工具号、点位集	
bool ToolCalibrateInterface::RefreshCalibratePoints(quint16, CalibratePoints &)	刷新已存标定点	bool	工具号、出参	

参数说明（含自定义结构体定义源码）

```

C++
// commandinfo.h
class METATYPE_EXPORT CmdToolCalibrate : public AbstractCmd
{
public:
    QString m_sCalibrateName;
    CalibratePoints m_calpts;
};

class METATYPE_EXPORT CmdToolSaveCalibratePoints : public AbstractCmd
{
public:
    int m_toolId = 0;
    CalibratePoints m_calpts;
};

```

```

typedef struct _CalibratePoints_
{
    ...
    int MethodType;          /// >
    calibrate method
    RobCalPos Points[MAX_POINTS_NUM_CALIBRATE];    /// >
    calibrate points
} CalibratePoints;

typedef struct _RobCalPos_
{
    ...
    Pos pos; /// > calibrate point pos, x y z
    Ori ori; /// > calibrate point orient, A B C
    int ArmParm[NUM_ROB_CAL_ARM_ARGS]; /// > calibrate point arm
    params
    double EPosData[NUM_ROB_CAL_EXTERN_AXLE_ARGS]; /// > calibrate
    external axle params
} RobCalPos;

class METATYPE_EXPORT Pos
{
    ...

    double m_x; //Unit: millimeter
    double m_y;
    double m_z;
};

class METATYPE_EXPORT Pos
{
    ...

    double m_x; //Unit: millimeter
    double m_y;
    double m_z;
};

```

6.8.2 调用


```

C++
// ToolCalForm::on_pbn_calculate_clicked
CalibratePoints points;
ui->calibratePointsForm->GetCalibratePoints(points);
CommunicationEngine::instance()->enqueueCmd_ToolCalibrate(
    this, ui->combox_method->currentText(), points);

```

```

C++
// ToolCalForm::on_pbn_apply_clicked
CalibratePoints points;
ui->calibratePointsForm->GetCalibratePoints(points);
CommunicationEngine::instance()
    ->enqueueCmd_ToolCalibratePoints(this, m_toolId, points);

```

6.8.3 函数实现

```

C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_Tool_Calibrate: {
    Pos pos;
    Ori ori;
    CmdToolCalibrate *cmd = static_cast<CmdToolCalibrate
*>(absCmd);
    cmd->m_calpts.MethodType
        = static_cast<int>(
            m_commInstance->CalibMethodByName(cmd-
>m_sCalibrateName));
    bool bRet = m_commInstance->Calibrate(cmd->m_calpts, pos,
ori);
    emit CommunicationEngine::instance()
        ->signal_tool_Calibrate_result(bRet, pos, ori);
    break;
}
case AbstractCmd::CmdType_Tool_SaveCalibratePoints: {
    CmdToolSaveCalibratePoints *cmd
        = static_cast<CmdToolSaveCalibratePoints *>(absCmd);
    bool bRet = m_commInstance->SaveCalibratePoints(
        cmd->m_toolId, cmd->m_calpts);
}

```

```

        emit CommunicationEngine::instance()
            ->signal_tool_SaveCalibratePoints_result(absCmd->m_object,
bRet);
        break;
    }
case AbstractCmd::CmdType_Tool_RefreshCalibratePoints: {
    CalibratePoints calPoints;
    CmdCommonToolCalibrate *cmd
        = static_cast<CmdCommonToolCalibrate *>(absCmd);
    bool bRet = m_commInstance->RefreshCalibratePoints(
        cmd->m_value.toInt(), calPoints);
    emit CommunicationEngine::instance()
        ->signal_tool_RefreshCalibratePoints_result(bRet,
calPoints);
    break;
}

```

```

C++
// toolcalibrateinterface.cpp (节选)
bool ToolCalibrateInterface::Calibrate(const CalibratePoints
&midPos,
                                     Pos &calResultPos, Ori
&calResultOri)
{
    CsCalibPoints csCaliPoints;
    csCaliPoints.MethodType = static_cast<int>(midPos.MethodType);
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        csCaliPoints.Points[i].RPosData[0] =
midPos.Points[i].pos.m_x;
        csCaliPoints.Points[i].RPosData[1] =
midPos.Points[i].pos.m_y;
        csCaliPoints.Points[i].RPosData[2] =
midPos.Points[i].pos.m_z;
        csCaliPoints.Points[i].RPosData[5] =
midPos.Points[i].ori.m_rx;
        csCaliPoints.Points[i].RPosData[4] =
midPos.Points[i].ori.m_ry;
        csCaliPoints.Points[i].RPosData[3] =
midPos.Points[i].ori.m_rz;
        memcpy(csCaliPoints.Points[i].ArmParm,
midPos.Points[i].ArmParm,
            sizeof(midPos.Points[i].ArmParm));
    }
}

```

```

        memcpy(csCaliPoints.Points[i].EPosData,
midPos.Points[i].EPosData,
                sizeof(midPos.Points[i].ArmParm));
    }
    Pose calResultPose;
    if (!_ITool->Calibrate(csCaliPoints, calResultPose)) {
        return false;
    }
    calResultPos.m_x = calResultPose.Data[0];
    calResultPos.m_y = calResultPose.Data[1];
    calResultPos.m_z = calResultPose.Data[2];
    calResultOri.m_rx = calResultPose.Data[5];
    calResultOri.m_ry = calResultPose.Data[4];
    calResultOri.m_rz = calResultPose.Data[3];
    return true;
}

bool ToolCalibrateInterface::SaveCalibratePoints(quint16 toolNo,
                                                const
CalibratePoints &point)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    CsCalibPoints csCaliPoints;
    csCaliPoints.MethodType = static_cast<int>(point.MethodType);

    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        csCaliPoints.Points[i].RPosData[0] =
point.Points[i].pos.m_x;
        csCaliPoints.Points[i].RPosData[1] =
point.Points[i].pos.m_y;
        csCaliPoints.Points[i].RPosData[2] =
point.Points[i].pos.m_z;
        csCaliPoints.Points[i].RPosData[5] =
point.Points[i].ori.m_rx;
        csCaliPoints.Points[i].RPosData[4] =
point.Points[i].ori.m_ry;
        csCaliPoints.Points[i].RPosData[3] =
point.Points[i].ori.m_rz;
        memcpy(csCaliPoints.Points[i].ArmParm,
point.Points[i].ArmParm,
                sizeof(point.Points[i].ArmParm));
        memcpy(csCaliPoints.Points[i].EPosData,
point.Points[i].EPosData,
                sizeof(point.Points[i].ArmParm));
    }
}

```

```

    }
    FREQ_LOG_PRINT_TIMESTAMP
    return _ITool->SaveCalibratePoints(toolNo, csCaliPoints);
}

bool ToolCalibrateInterface::RefreshCalibratePoints(quint16
toolNo,

CalibratePoints &points)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    ToolCalibData toolCalData;
    if (!_ITool->RefreshCalibratePoints(toolCalData)) {
        return false;
    }

    if (toolCalData.MethodDatas.size() <= toolNo) {
        qDebug() << "Controller RefreshCalibratePoints Func
Failed, toolNo : "
            << toolNo
            << ", ArrSize : " <<
toolCalData.MethodDatas.size();
        return false;
    }
    CsCalibPoints csCaliPoints =
std::move(toolCalData.MethodDatas[toolNo]);

    points.MethodType = static_cast<Robot_ToolCalibrateType>(
        csCaliPoints.MethodType);
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        points.Points[i].pos.m_x =
csCaliPoints.Points[i].RPosData[0];
        points.Points[i].pos.m_y =
csCaliPoints.Points[i].RPosData[1];
        points.Points[i].pos.m_z =
csCaliPoints.Points[i].RPosData[2];
        points.Points[i].ori.m_rx =
csCaliPoints.Points[i].RPosData[5];
        points.Points[i].ori.m_ry =
csCaliPoints.Points[i].RPosData[4];
        points.Points[i].ori.m_rz =
csCaliPoints.Points[i].RPosData[3];
        memcpy(points.Points[i].ArmParm,
csCaliPoints.Points[i].ArmParm,

```

```

        sizeof(csCaliPoints.Points[i].ArmParm));
        memcpy(points.Points[i].EPosData,
csCaliPoints.Points[i].EPosData,
        sizeof(csCaliPoints.Points[i].ArmParm));
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

```

6.9 工件 (RefreshWObjParams/ SaveWObjParams / ClearWobjParams)

6.9.1 说明

接口	功能	返回值	参数	参数说明
bool WObjCalibrateInterface::RefreshWObjParams(int wobjNo, WobjParams *data)	刷新工件参数	bool	工件号、出参	
bool WObjCalibrateInterface::SaveWObjParams(int wobjNo, WobjParams *data)	保存工件参数	bool	工件号、入参	
bool WObjCalibrateInterface::Cl	重置工件参数	bool	工件号	

earWobjParams(int wobjNo)				
---------------------------	--	--	--	--

参数说明

```

C++
代码块
// commandinfo.h
class METATYPE_EXPORT CmdWObjSave : public AbstractCmd
{
public:
    int m_wobjId = 0;
    WobjParams m_params;
};

class METATYPE_EXPORT CmdCommonWObjCalibrate : public AbstractCmd
{
public:
    QVariant m_value;
};

typedef struct _WobjParams_
{
    ...

    bool RobHold;          /// > 工件是否夹持  true 夹
持;  FALSE 非夹持
    bool UFFix;            /// > 工件是否固定（是否相对
于大地坐标系固定、是否相对于法兰盘固定）
    char UFMec[MAX_NAME_LEN_MECHUNIT];  /// > 关联的机械单元名称
    Pos UFramePos;         /// > 用户坐标位置
    Ori UFrameOri;         /// > 用户坐标姿态
    Pos OFramePos;         /// > 工件坐标位置
    Ori OFrameOri;         /// > 工件坐标姿态
}

Pos Ori 同 6.7

```

6.9.2 调用

```

C++
// WorkPieceForm::UpdateUIData
CommunicationEngine::instance()->enqueueCmd_handleWObjCalibrate(
    this, AbstractCmd::CmdType_WObj_RefreshWObjParams,
    ui->combox_curworkpiece->currentIndex());

```

```

C++
// WorkPieceForm::on_pbn_apply_clicked
WobjParams params;
params.RobHold = ui->cbox_isrobhold->isChecked();
params.UFFix = ui->cbox_uffix->isChecked();
// ... UFMec / UFrame / OFrame 赋值 ...
CommunicationEngine::instance()->enqueueCmd_WObjSave(
    this, ui->combox_curworkpiece->currentIndex(), params);

```

```

C++
// WorkPieceForm::on_pbn_reset_clicked
CommunicationEngine::instance()->enqueueCmd_handleWObjCalibrate(
    this, AbstractCmd::CmdType_WObj_ClearWObjParams,
    ui->combox_curworkpiece->currentIndex());

```

6.9.3 函数实现

```

C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_WObj_SaveWObjParams: {
    CmdWObjSave *cmd = static_cast<CmdWObjSave *>(absCmd);

    WobjParams oldWobjParam;
    Communication::instance()->GetCurWobjParams(cmd->m_wobjId,
oldWobjParam);

    bool bRet = Communication::instance()->SaveWobjParams(
        cmd->m_wobjId, &cmd->m_params);
    if (bRet == true) {
        m_commInstance->comm()->SetCurWobjParams(

```

```

        cmd->m_wobjId, cmd->m_params);
    }
    emit CommunicationEngine::instance()-
>signal_wobj_Save_result(bRet);
    if (m_commInstance->GetCurWObjId() == cmd->m_wobjId) {
        emit CommunicationEngine::instance()->signal_WobjChanged(
            cmd->m_wobjId);
    }
    if(!(oldWobjParam == cmd->m_params)){
        emit CommunicationEngine::instance()-
>signal_wobjParamChanged(cmd->m_wobjId);
    }
    break;
}
case AbstractCmd::CmdType_WObj_RefreshWObjParams: {
    CmdCommonWObjCalibrate *cmd
        = static_cast<CmdCommonWObjCalibrate *>(absCmd);
    WobjParams wobjParams;
    bool bRet = Communication::instance()
        ->RefreshWObjParams(cmd->m_value.toInt(),
        &wobjParams);
    emit CommunicationEngine::instance()
        ->signal_wobj_Refresh_result(cmd->m_object, bRet,
        wobjParams);
    break;
}
case AbstractCmd::CmdType_WObj_ClearWobjParams: {
    CmdCommonWObjCalibrate *cmd
        = static_cast<CmdCommonWObjCalibrate *>(absCmd);
    bool bRet = Communication::instance()
        ->ClearWobjParams(cmd->m_value.toInt());
    emit CommunicationEngine::instance()-
>signal_wobj_clear_result(bRet);
    break;
}
}

```

```

C++
// wobjcalibrateinterface.cpp (节选)
bool WObjCalibrateInterface::SaveWObjParams(int wobjNo, WobjParams
*data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

```



```

WobjData wobjData;

wobjData.RobHold = data->RobHold;
wobjData.UFFix = data->UFFix;
memcpy(wobjData.UFMec, data->UFMec, MECHUNIT_NAME_LENGTH);

Pose uFrame;
uFrame.Data[0] = data->UFramePos.m_x;
uFrame.Data[1] = data->UFramePos.m_y;
uFrame.Data[2] = data->UFramePos.m_z;
uFrame.Data[5] = data->UFrameOri.m_rx;
uFrame.Data[4] = data->UFrameOri.m_ry;
uFrame.Data[3] = data->UFrameOri.m_rz;
wobjData.UFrame = uFrame;

Pose oFrame;
oFrame.Data[0] = data->OFramePos.m_x;
oFrame.Data[1] = data->OFramePos.m_y;
oFrame.Data[2] = data->OFramePos.m_z;
oFrame.Data[5] = data->OFrameOri.m_rx;
oFrame.Data[4] = data->OFrameOri.m_ry;
oFrame.Data[3] = data->OFrameOri.m_rz;
wobjData.OFrame = oFrame;

qDebug() << "wobj save ufix = " << wobjData.UFFix;
if (!_IWobj->Save(wobjNo, &wobjData)) {
    qDebug() << "Call Wobj Func Save Failed.";
    return false;
}
LOG_TRACE(QString("[%1][%2][%3]")
            .arg(__FILE__, __FUNCTION__,
                QString::number(QDateTime::currentMSecsSinceEpoch())));
return true;
}

bool WObjCalibrateInterface::RefreshWObjParams(int wobjNo,
WobjParams *data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    WobjData wobjData;
    if (!_IWobj->Refresh(wobjNo, &wobjData)) {
        return false;
    }
}

```

```

qDebug() << "wobj refresh uffix = " << wobjData.UFFix;
data->RobHold = wobjData.RobHold;
data->UFFix = wobjData.UFFix;
memcpy(data->UFMec, wobjData.UFMec, MECHUNIT_NAME_LENGTH);

data->UFramePos.m_x = wobjData.UFrame.Data[0];
data->UFramePos.m_y = wobjData.UFrame.Data[1];
data->UFramePos.m_z = wobjData.UFrame.Data[2];
data->UFrameOri.m_rx = wobjData.UFrame.Data[5];
data->UFrameOri.m_ry = wobjData.UFrame.Data[4];
data->UFrameOri.m_rz = wobjData.UFrame.Data[3];

data->OFramePos.m_x = wobjData.OFrame.Data[0];
data->OFramePos.m_y = wobjData.OFrame.Data[1];
data->OFramePos.m_z = wobjData.OFrame.Data[2];
data->OFrameOri.m_rx = wobjData.OFrame.Data[5];
data->OFrameOri.m_ry = wobjData.OFrame.Data[4];
data->OFrameOri.m_rz = wobjData.OFrame.Data[3];
LOG_TRACE(QString("[%1][%2][%3]")
            .arg(__FILE__, __FUNCTION__,
                QString::number(QDateTime::currentMSecsSinceEpoch())));
return true;
}

bool WObjCalibrateInterface::ClearWobjParams(int wobjNo)
{
    return _IWobj->Clear(wobjNo);
}

```

6.10 工件标定 (WObjCalibrate / SaveWObjCalibratePoints/ RefreshWObjCalibratePoints)

6.10.1 说明

接口	功能	返回值	参数	参数说明
bool WObjCalibrateInterface:: WObjCalibrate(const QString &mechUnitName, Robot_WObjType type, bool isUFFix, const CalibratePoints &midUFramePos, const CalibratePoints &midOFramePos, const Pos &oldUFramePos, const Ori &oldUFrameOri, Pos &uFrameCalibResultPos , Ori &uFrameCalibResultOri, Pos &oFrameCalibResultPos , Ori &oFrameCalibResultOri)	调用控制器 执行工件标 定	bool	机组名、工 件类型、 UFFix、中 间点、旧 UFrame	
Bool WObjCalibrateInterface:: SaveWObjCalibratePoint s(quint16 wobjNo, const CalibratePoints &uFramePoint, const CalibratePoints &oFramePoint)	保存工件标 定中间点	bool	工件号、 U/O 点集	
bool WObjCalibrateInterface:: RefreshWObjCalibrateP oints(quint16 wobjNo, CalibratePoints &uFramePoint, CalibratePoints &oFramePoint)	刷新已保存 中间点	bool	工件号、 U/O 出参	

参数说明（含自定义结构体定义源码）

C++

代码块

// commandinfo.h

```
class METATYPE_EXPORT CmdWObjCalibrate : public AbstractCmd
{
public:
    int m_wobjId = 0;
    Robot_WObjType m_wobjType = Robot_WObjType_EXTERN_WOBJ;
    CalibratePoints m_midUFramePos;
    CalibratePoints m_midOFramePos;
    Pos m_oldUFramePos; //同 6.7
    Ori m_oldUFrameOri; //同 6.7
};
```

```
class METATYPE_EXPORT CmdWObjSaveCalibratePoints : public
AbstractCmd
{
public:
    int m_wobjId = 0;
    CalibratePoints m_uFramePoint;
    CalibratePoints m_oFramePoint;
};
```

```
enum Robot_WObjType
{
    Robot_WObjType_EXTERN_WOBJ = 0,
    Robot_WObjType_HOLD_WOBJ   = 1
};
```

```
typedef struct _CalibratePoints_
{
    ...

    int MethodType;                /// >
calibrate method
    RobCalPos Points[MAX_POINTS_NUM_CALIBRATE];    /// >
calibrate points
} CalibratePoints;
```

```
typedef struct _RobCalPos_
{
    ...

    Pos pos; /// > calibrate point pos, x y z
```

```

Ori ori; ///  

int ArmParm[NUM_ROB_CAL_ARM_ARGS]; ///  

params  

double EPosData[NUM_ROB_CAL_EXTERN_AXLE_ARGS]; ///  

external axle params  

} RobCalPos;  
  

#define MAX_POINTS_NUM_CALIBRATE (6)  

#define NUM_ROB_CAL_ARM_ARGS (4)  

#define NUM_ROB_CAL_EXTERN_AXLE_ARGS (6)

```

6.10.2 调用

```

C++  

// WorkPieceCalForm::on_pbn_calculate_clicked  

CommunicationEngine::instance()->enqueueCmd_WObjCalibrate(  

    this, m_wobjId, wObjType, uFramePoints, oFramePoints,  

    oldUFramePos, oldUFrameOri);

```

```

C++  

// WorkPieceCalForm::on_pbn_apply_clicked  

CommunicationEngine::instance()->enqueueCmd_WObjCalibratePoints(  

    this, m_wobjId, uFramePoints, oFramePoints);

```

6.10.3 函数实现

```

C++  

// communicationthread.cpp (节选)  

case AbstractCmd::CmdType_WObj_WObjCalibrate: {  

    CmdWObjCalibrate *cmd = static_cast<CmdWObjCalibrate  

*>(absCmd);  

    WobjParams params;  

    Communication::instance()->GetWobjParam(cmd->m_wobjId,  

params);

```

```

    Pos uFramepos, oFramepos;
    Ori uFrameori, oFrameori;
    bool bRet = m_commInstance->WObjCalibrate(
        params.UFMec, cmd->m_wobjType,
        params.UFFix,
        cmd->m_midUFramePos, cmd->m_midOFramePos,
        cmd->m_oldUFramePos, cmd->m_oldUFrameOri,
        uFramepos, uFrameori,
        oFramepos, oFrameori);
    emit CommunicationEngine::instance()
        ->signal_wobj_WObjCalibrate_result(bRet, uFramepos,
        uFrameori,
        oFramepos, oFrameori);

    break;
}
case AbstractCmd::CmdType_WObj_SaveWObjCalibratePoints: {
    CmdWObjSaveCalibratePoints *cmd
        = static_cast<CmdWObjSaveCalibratePoints *>(absCmd);
    bool bRet = m_commInstance->SaveWObjCalibratePoints(
        cmd->m_wobjId, cmd->m_uFramePoint, cmd->m_oFramePoint);
    emit CommunicationEngine::instance()
        ->signal_wobj_SaveWObjCalibratePoints_result(
        absCmd->m_object, bRet);
    break;
}
case AbstractCmd::CmdType_WObj_RefreshWObjCalibratePoints: {
    CalibratePoints uFramePoints, oFramePoints;
    CmdCommonWObjCalibrate *cmd
        = static_cast<CmdCommonWObjCalibrate *>(absCmd);
    bool bRet = m_commInstance->RefreshWObjCalibratePoints(
        cmd->m_value.toInt(), uFramePoints, oFramePoints);
    emit CommunicationEngine::instance()
        ->signal_wobj_RefreshWObjCalibratePoints_result(
        bRet, uFramePoints, oFramePoints);
    break;
}
}

```

C++

// wobjcalibrateinterface.cpp (节选)

```

bool WObjCalibrateInterface::WObjCalibrate(const QString
&mechUnitName,
        Robot_WObjType type,

```

```

bool isUFFix,
const CalibratePoints
&midUFramePos,
const CalibratePoints
&midOFramePos,
const Pos
&oldUFramePos,
const Ori
&oldUFrameOri,
Pos
&uFrameCalibResultPos,
Ori
&uFrameCalibResultOri,
Pos
&oFrameCalibResultPos,
Ori
&oFrameCalibResultOri)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    CsCalibPoints csUFrameCaliPoints;
    csUFrameCaliPoints.MethodType =
static_cast<int>(midUFramePos.MethodType);
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        csUFrameCaliPoints.Points[i].RPosData[0] =
            midUFramePos.Points[i].pos.m_x;
        csUFrameCaliPoints.Points[i].RPosData[1] =
            midUFramePos.Points[i].pos.m_y;
        csUFrameCaliPoints.Points[i].RPosData[2] =
            midUFramePos.Points[i].pos.m_z;
        csUFrameCaliPoints.Points[i].RPosData[5] =
            midUFramePos.Points[i].ori.m_rx;
        csUFrameCaliPoints.Points[i].RPosData[4] =
            midUFramePos.Points[i].ori.m_ry;
        csUFrameCaliPoints.Points[i].RPosData[3] =
            midUFramePos.Points[i].ori.m_rz;
        memcpy(csUFrameCaliPoints.Points[i].ArmParm,
            midUFramePos.Points[i].ArmParm,
            sizeof(midUFramePos.Points[i].ArmParm));
        memcpy(csUFrameCaliPoints.Points[i].EPosData,
            midUFramePos.Points[i].EPosData,
            sizeof(midUFramePos.Points[i].ArmParm));
    }

    CsCalibPoints csOFrameCaliPoints;

```

```

        csOFrameCaliPoints.MethodType =
static_cast<int>(midOFramePos.MethodType);
        for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
            csOFrameCaliPoints.Points[i].RPosData[0] =
                midOFramePos.Points[i].pos.m_x;
            csOFrameCaliPoints.Points[i].RPosData[1] =
                midOFramePos.Points[i].pos.m_y;
            csOFrameCaliPoints.Points[i].RPosData[2] =
                midOFramePos.Points[i].pos.m_z;
            csOFrameCaliPoints.Points[i].RPosData[5] =
                midOFramePos.Points[i].ori.m_rx;
            csOFrameCaliPoints.Points[i].RPosData[4] =
                midOFramePos.Points[i].ori.m_ry;
            csOFrameCaliPoints.Points[i].RPosData[3] =
                midOFramePos.Points[i].ori.m_rz;
            memcpy(csOFrameCaliPoints.Points[i].ArmParm,
                midOFramePos.Points[i].ArmParm,
                sizeof(midOFramePos.Points[i].ArmParm));
            memcpy(csOFrameCaliPoints.Points[i].EPosData,
                midOFramePos.Points[i].EPosData,
                sizeof(midOFramePos.Points[i].ArmParm));
        }

        Pose oldUFrame;
        oldUFrame.Data[0] = oldUFramePos.m_x;
        oldUFrame.Data[1] = oldUFramePos.m_y;
        oldUFrame.Data[2] = oldUFramePos.m_z;
        oldUFrame.Data[5] = oldUFrameOri.m_rx;
        oldUFrame.Data[4] = oldUFrameOri.m_ry;
        oldUFrame.Data[3] = oldUFrameOri.m_rz;

        Pose uFrameCalibResult;
        Pose oFrameCalibResult;
        if (!_IWobj->
            Calibrate(mechUnitName.toLocal8Bit().data(),
                static_cast<InoRobBusiness::RobotHoldWobjType>(type),
                isUFFix, csUFrameCaliPoints, csOFrameCaliPoints,
oldUFrame,
                uFrameCalibResult, oFrameCalibResult)) {
            qDebug() << "WObj Calibrate Failed.";
            return false;
        }

        uFrameCalibResultPos.m_x = uFrameCalibResult.Data[0];

```



```

uFrameCalibResultPos.m_y = uFrameCalibResult.Data[1];
uFrameCalibResultPos.m_z = uFrameCalibResult.Data[2];

uFrameCalibResultOri.m_rx = uFrameCalibResult.Data[5];
uFrameCalibResultOri.m_ry = uFrameCalibResult.Data[4];
uFrameCalibResultOri.m_rz = uFrameCalibResult.Data[3];

oFrameCalibResultPos.m_x = oFrameCalibResult.Data[0];
oFrameCalibResultPos.m_y = oFrameCalibResult.Data[1];
oFrameCalibResultPos.m_z = oFrameCalibResult.Data[2];

oFrameCalibResultOri.m_rx = oFrameCalibResult.Data[5];
oFrameCalibResultOri.m_ry = oFrameCalibResult.Data[4];
oFrameCalibResultOri.m_rz = oFrameCalibResult.Data[3];
LOG_TRACE(QString("[%1][%2][%3]")
           .arg(__FILE__, __FUNCTION__,
               QString::number(QDateTime::currentMSecsSinceEpoch())));
return true;
}

bool WObjCalibrateInterface::SaveWObjCalibratePoints(
    quint16 wobjNo, const CalibratePoints &uFramePoint,
    const CalibratePoints &oFramePoint)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    CsCalibPoints csUFrameCaliPoints;
    csUFrameCaliPoints.MethodType = uFramePoint.MethodType;
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        csUFrameCaliPoints.Points[i].RPosData[0] =
            uFramePoint.Points[i].pos.m_x;
        csUFrameCaliPoints.Points[i].RPosData[1] =
            uFramePoint.Points[i].pos.m_y;
        csUFrameCaliPoints.Points[i].RPosData[2] =
            uFramePoint.Points[i].pos.m_z;
        csUFrameCaliPoints.Points[i].RPosData[5] =
            uFramePoint.Points[i].ori.m_rx;
        csUFrameCaliPoints.Points[i].RPosData[4] =
            uFramePoint.Points[i].ori.m_ry;
        csUFrameCaliPoints.Points[i].RPosData[3] =
            uFramePoint.Points[i].ori.m_rz;
        memcpy(csUFrameCaliPoints.Points[i].ArmParm,
            uFramePoint.Points[i].ArmParm,
            sizeof(uFramePoint.Points[i].ArmParm));
        memcpy(csUFrameCaliPoints.Points[i].EPosData,

```

```

        uFramePoint.Points[i].EPosData,
        sizeof(uFramePoint.Points[i].ArmParm));
    }

    CsCalibPoints csOFrameCaliPoints;
    csOFrameCaliPoints.MethodType = oFramePoint.MethodType;
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        csOFrameCaliPoints.Points[i].RPosData[0] =
            oFramePoint.Points[i].pos.m_x;
        csOFrameCaliPoints.Points[i].RPosData[1] =
            oFramePoint.Points[i].pos.m_y;
        csOFrameCaliPoints.Points[i].RPosData[2] =
            oFramePoint.Points[i].pos.m_z;
        csOFrameCaliPoints.Points[i].RPosData[5] =
            oFramePoint.Points[i].ori.m_rx;
        csOFrameCaliPoints.Points[i].RPosData[4] =
            oFramePoint.Points[i].ori.m_ry;
        csOFrameCaliPoints.Points[i].RPosData[3] =
            oFramePoint.Points[i].ori.m_rz;
        memcpy(csOFrameCaliPoints.Points[i].ArmParm,
            oFramePoint.Points[i].ArmParm,
            sizeof(oFramePoint.Points[i].ArmParm));
        memcpy(csOFrameCaliPoints.Points[i].EPosData,
            oFramePoint.Points[i].EPosData,
            sizeof(oFramePoint.Points[i].ArmParm));
    }
    if (!_IWobj->SaveCalibratePoints(
        wobjNo, csUFrameCaliPoints, csOFrameCaliPoints)) {
        qDebug() << "Wobj Save Mid Points Failed.";
        return false;
    }
    LOG_TRACE(QString("[%1][%2][%3]")
        .arg(__FILE__, __FUNCTION__,
            QString::number(QDateTime::currentMSecsSinceEpoch())));
    return true;
}

bool WObjCalibrateInterface::RefreshWObjCalibratePoints(
    quint16 wobjNo, CalibratePoints &uFramePoint, CalibratePoints
    &oFramePoint)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    WobjCalibData wobjCalibData;
    if (!_IWobj->RefreshCalibratePoints(wobjCalibData)) {

```

```

        qDebug() << "Refresh WObj Mid Points Failed.";
        return false;
    }

    uFramePoint.MethodType =
wobjCalibData.UMethodDatas[wobjNo].MethodType;
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        uFramePoint.Points[i].pos.m_x =

wobjCalibData.UMethodDatas[wobjNo].Points[i].RPosData[0];
        uFramePoint.Points[i].pos.m_y =

wobjCalibData.UMethodDatas[wobjNo].Points[i].RPosData[1];
        uFramePoint.Points[i].pos.m_z =

wobjCalibData.UMethodDatas[wobjNo].Points[i].RPosData[2];
        uFramePoint.Points[i].ori.m_rx =

wobjCalibData.UMethodDatas[wobjNo].Points[i].RPosData[5];
        uFramePoint.Points[i].ori.m_ry =

wobjCalibData.UMethodDatas[wobjNo].Points[i].RPosData[4];
        uFramePoint.Points[i].ori.m_rz =

wobjCalibData.UMethodDatas[wobjNo].Points[i].RPosData[3];
        memcpy(uFramePoint.Points[i].ArmParm,

wobjCalibData.UMethodDatas[wobjNo].Points[i].ArmParm,

sizeof(wobjCalibData.UMethodDatas[wobjNo].Points[i].ArmParm));
        memcpy(uFramePoint.Points[i].EPosData,

wobjCalibData.UMethodDatas[wobjNo].Points[i].EPosData,

sizeof(wobjCalibData.UMethodDatas[wobjNo].Points[i].ArmParm));
    }

    oFramePoint.MethodType =
wobjCalibData.OMethodDatas[wobjNo].MethodType;
    for (int i = 0; i < CALIB_POINTS_COUNT; ++i) {
        oFramePoint.Points[i].pos.m_x =

wobjCalibData.OMethodDatas[wobjNo].Points[i].RPosData[0];
        oFramePoint.Points[i].pos.m_y =

```

```

wobjCalibData.OMethodDatas[wobjNo].Points[i].RPosData[1];
    oFramePoint.Points[i].pos.m_z =

wobjCalibData.OMethodDatas[wobjNo].Points[i].RPosData[2];
    oFramePoint.Points[i].ori.m_rx =

wobjCalibData.OMethodDatas[wobjNo].Points[i].RPosData[5];
    oFramePoint.Points[i].ori.m_ry =

wobjCalibData.OMethodDatas[wobjNo].Points[i].RPosData[4];
    oFramePoint.Points[i].ori.m_rz =

wobjCalibData.OMethodDatas[wobjNo].Points[i].RPosData[3];
    memcpy(oFramePoint.Points[i].ArmParm,

wobjCalibData.OMethodDatas[wobjNo].Points[i].ArmParm,

sizeof(wobjCalibData.OMethodDatas[wobjNo].Points[i].ArmParm));
    memcpy(oFramePoint.Points[i].EPosData,

wobjCalibData.OMethodDatas[wobjNo].Points[i].EPosData,

sizeof(wobjCalibData.OMethodDatas[wobjNo].Points[i].ArmParm));
}
LOG_TRACE(QString("[%1][%2][%3]")
            .arg(__FILE__, __FUNCTION__,
                QString::number(QDateTime::currentMSecsSinceEpoch())));
return true;
}

```

6.11 负载 (LoadRefresh/ LoadSave/ LoadClear/CheckCurLoadValid)

6.11.1 说明

接口	功能	返回值	参数	参数说明
bool LoadInterface: :LoadRefresh(quint16 loadId, LoadParams ¶ms)	从控制器读取 负载参数	bool	负载号、出参 结构	
bool LoadInterface: :LoadSave(qu int16 loadId, const LoadParams ¶ms, bool bSaveNeed)	下发保存负载 参数	bool	负载号、参 数、保存标志	
bool LoadInterface: :LoadClear(qu int16 loadId)	清空/重置负载 参数	bool	负载号	
bool LoadInterface: :CheckCurLoa dValid(const LoadParams ¶ms)	校验质量/质心 /惯量合法性	bool	LoadParams	不合法会直接 返回保存失败

参数说明

```

C++
// commandinfo.h
class METATYPE_EXPORT CmdLoadSave : public AbstractCmd
{
public:
    quint16 m_loadId = 0;
    LoadParams m_params;
    bool m_bIsSaveNeed = true;
};

```

```

class METATYPE_EXPORT CmdLoadClear : public AbstractCmd
{
public:
    quint16 m_loadId = 0;
};

class METATYPE_EXPORT CmdLoadRefresh : public AbstractCmd
{
public:
    quint16 m_loadId = 0;
};

#define LOADPARAMS_POSIT_DIMENT      (3)
#define LOADPARAMS_ORIENT_DIMENT    (3)
#define LOADPARAMS_INERTIA_DIMENT    (3)
/// @brief Load Params Desc
typedef struct _LoadParams_
{
    ...

    double Mass;                               /// > mass
    double Cog[LOADPARAMS_POSIT_DIMENT];      /// > cogX cogY
cogZ
    double Orient[LOADPARAMS_ORIENT_DIMENT];  /// > orienta
orientB orientC
    double Inertia[LOADPARAMS_INERTIA_DIMENT]; /// > inertiaIX
inertiaIY inertiaIZ
} LoadParams;

```

6.11.2 调用

```

C++
// LoaderForm::UpdateUIData
CommunicationEngine::instance()->enqueueCmd_LoadRefresh(
    this, ui->combox_curLoad->currentIndex());

```

```

C++
// LoaderForm::on_pbn_save_clicked
LoadParams params;

```

```
ui->loadsetForm->GetParams(params);
CommunicationEngine::instance()->enqueueCmd_LoadSave(
    this, ui->combox_curLoad->currentIndex(), params);
```

```
C++
// LoaderForm::on_pbn_reset_clicked
CommunicationEngine::instance()->enqueueCmd_LoadClear(
    this, ui->combox_curLoad->currentIndex());
```

6.11.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_Load_Refresh: {
    CmdLoadRefresh *cmd = static_cast<CmdLoadRefresh *>(absCmd);

    LoadParams params;
    bool bRet = m_commInstance->LoadRefresh(cmd->m_loadId,
params);
    emit CommunicationEngine::instance()
        ->signal_loadRefresh_result(absCmd->m_object, bRet,
params);
    break;
}
case AbstractCmd::CmdType_Load_Clear: {
    CmdLoadClear *cmd = static_cast<CmdLoadClear *>(absCmd);

    bool bRet = m_commInstance->LoadClear(cmd->m_loadId);
    emit CommunicationEngine::instance()-
>signal_loadClear_result(bRet);
    break;
}
case AbstractCmd::CmdType_Load_Save: {
    CmdLoadSave *cmd = static_cast<CmdLoadSave *>(absCmd);

    bool bRet = m_commInstance->CheckCurLoadValid(cmd->m_params);
    if (!bRet) {
        emit CommunicationEngine::instance()
            ->signal_loadSave_result(cmd->m_object, false);
```

```

        break;
    }

    bRet = m_commInstance->LoadSave(cmd->m_loadId, cmd->m_params,
cmd->m_bIsSaveNeed);
    emit CommunicationEngine::instance()-
>signal_loadSave_result(cmd->m_object, bRet);
    break;
}

```

C++

```

// toolcalibrateinterface.cpp (LoadInterface, 完整函数)
bool LoadInterface::LoadSave(quint16 loadId, const LoadParams
&params, bool bSaveNeed)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    LoadData loadData;
    memcpy(&loadData, &params, sizeof(params));
    FREQ_LOG_PRINT_TIMESTAMP
    return _ILoad->Save(static_cast<int>(loadId), loadData,
bSaveNeed);
}

bool LoadInterface::LoadRefresh(quint16 loadId, LoadParams
&params)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    LoadData loadData;
    if (!_ILoad->Refresh(static_cast<int>(loadId), loadData)) {
        return false;
    }

    memcpy(&params, &loadData, sizeof(loadData));
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool LoadInterface::LoadClear(quint16 loadId)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _ILoad->Clear(loadId);
}

```



```
bool LoadInterface::CheckCurLoadValid(const LoadParams &params)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    LoadData loadData;
    memcpy(&loadData, &params, sizeof(params));
    FREQ_LOG_PRINT_TIMESTAMP
    return _ILoad->CheckCurLoadValid(loadData);
}
```

6.12 负载名称与初始化 (GetLoadCount/LoadIDNameSearch/getLoadNames)

6.12.1 说明

负载页初始化会先请求名称列表，再按当前项刷新参数。

接口	功能	返回值	参数	参数说明
quint16 LoadInterface: :GetLoadCou nt()	获取负载数量	quint16	—	用于循环取名
QString LoadInterface: :LoadIDName Search(quint1 6 loadId)	根据负载号取 名称	QString	负载号	例如 Load0/Load1.. .
QStringList DataCollectio nInterface::get LoadNames() const	读取本地缓存 的名称	QStringList	—	

6.12.2 调用

```
C++
// LoaderForm::InitUIData
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Load_GetNames);

ui->combox_curLoad->addItem(Communication::instance()-
>getLoadNames());
```

6.12.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_Load_GetNames: {
    QStringList sNames;
    quint16 nNumLoad = m_commInstance->GetLoadCount();
    for (int i = 0; i < nNumLoad; ++i) {
#ifdef INOCOBOTTP_MSVC_QT5
        sNames.push_back(m_commInstance->LoadIDNameSearch(i));
    #else
        sNames.emplace_back(m_commInstance->LoadIDNameSearch(i));
    #endif
    }
    emit CommunicationEngine::instance()
        ->signal_loadGetNames_result(sNames);
    break;
}
```

```
C++
// toolcalibrateinterface.cpp (LoadInterface, 完整函数)
quint16 LoadInterface::GetLoadCount()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _ILoad->GetCount();
}
```

```

QString LoadInterface::LoadIDNameSearch(quint16 loadId)
{
    #ifdef FREQ_LOG_PRINT_TIMESTAMP_THREAD
    std::string sName = _ILoad->LoadIDNameSearch(
        static_cast<InoRobBusiness::LoadID>(loadId));
    #endif
    return QString::fromStdString(sName);
}

```

```

C++
// datacollectioninterface.cpp / datacollectioninterface_p.h
QStringList DataCollectionInterface::getLoadNames() const
{
    return d->getLoadNames();
}

QStringList getLoadNames() const
{
    return m_vLoadNames;
}

```

6.13 轴参数初始化与读取

(ReadTorqueLimit/ReadAvrLoadLimit/ReadOverHeatLimit/ReadOverLoadLimit)

6.13.1 说明

接口	功能	返回值	参数	参数说明
int RobotArmInterface::ReadTorqueLimit(bool &bSwitch, int IntValue[ROB	读取电流限制开关和各轴百分比	int	开关出参、数组出参	0 成功, 非 0 失败

OT_AXIS_NUM])				
int RobotArmInterface::ReadAvrLoadLimit(bool &bSwitch, int IntValue[ROBOT_AXIS_NUM])	读取平均负载率限制开关和各轴百分比	int	开关出参、数组出参	对应“平均负载率限制”
int RobotArmInterface::ReadOverHeatLimit(bool &bSwitch)	读取过热限制开关	int	开关出参	对应 switch_overheat
int RobotArmInterface::ReadOverLoadLimit(bool &bSwitch)	读取过载限制开关	int	开关出参	对应 switch_overrate

6.13.2 调用

```

C++
// AxisParamsForm::UpdateUIData
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_AxisParams_ReadTorqueLimit);
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_AxisParams_ReadAvrLoadLimit);
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_AxisParams_ReadOverHeatLimit);
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_AxisParams_ReadOverLoadLimit);

```

6.13.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_AxisParams_ReadTorqueLimit: {
    bool bCurrentSwitch = false;
    int retCurrentValue[ROBOT_AXIS_NUM] = {100, 100, 100, 100,
100, 100};
    int iRet = m_commInstance->ReadTorqueLimit(bCurrentSwitch,
retCurrentValue);
    QVector<int> vecRet;
    for (int i = 0; i < ROBOT_AXIS_NUM; ++i) {
        vecRet.push_back(retCurrentValue[i]);
    }
    emit CommunicationEngine::instance()
        ->signal_readTorqueLimit_result(iRet, bCurrentSwitch,
vecRet);
    break;
}
case AbstractCmd::CmdType_AxisParams_ReadAvrLoadLimit: {
    bool bLoadRateSwitch = false;
    int retLoadRatValue[ROBOT_AXIS_NUM] = {100, 100, 100, 100,
100, 100};
    int iRet = m_commInstance->ReadAvrLoadLimit(bLoadRateSwitch,
retLoadRatValue);
    QVector<int> vecRet;
    for (int i = 0; i < ROBOT_AXIS_NUM; ++i) {
        vecRet.push_back(retLoadRatValue[i]);
    }
    emit CommunicationEngine::instance()
        ->signal_readAvrLoadLimit_result(iRet, bLoadRateSwitch,
vecRet);
    break;
}
case AbstractCmd::CmdType_AxisParams_ReadOverHeatLimit: {
    bool bOverHeatSwitch = false;
    int iRet = m_commInstance->ReadOverHeatLimit(bOverHeatSwitch);
    emit CommunicationEngine::instance()
        ->signal_readOverHeatLimit_result(iRet, bOverHeatSwitch);
    break;
}
case AbstractCmd::CmdType_AxisParams_ReadOverLoadLimit: {
```

```

    bool bOverLoadSwitch = false;
    int iRet = m_commInstance->ReadOverLoadLimit(bOverLoadSwitch);
    emit CommunicationEngine::instance()
        ->signal_readOverLoadLimit_result(iRet, bOverLoadSwitch);
    break;
}

```

C++

```

int RobotArmInterface::ReadTorqueLimit(bool &bSwitch, int
IntValue[ROBOT_AXIS_NUM])
{
    quint16 uSwitch = 0;
    int value[8] = {0};
    int ret = _IRobotArm->readTorqueLimit(uSwitch, value);
    memcpy(IntValue, value, ROBOT_AXIS_NUM * sizeof(int));
    bSwitch = static_cast<bool>(uSwitch);
    return ret;
}

int RobotArmInterface::ReadAvrLoadLimit(bool &bSwitch, int
IntValue[ROBOT_AXIS_NUM])
{
    quint16 uSwitch = 0;
    int value[8] = {0};
    int ret = _IRobotArm->readAvrLoadLimit(uSwitch, value);
    memcpy(IntValue, value, ROBOT_AXIS_NUM * sizeof(int));
    bSwitch = static_cast<bool>(uSwitch);
    return ret;
}

int RobotArmInterface::ReadOverHeatLimit(bool &bSwitch)
{
    quint16 uSwitch = 0;
    int ret = _IRobotArm->readOverHeatLimit(uSwitch);
    bSwitch = static_cast<bool>(uSwitch);
    return ret;
}

int RobotArmInterface::ReadOverLoadLimit(bool &bSwitch)
{
    quint16 uSwitch = 0;
    int ret = _IRobotArm->readOverLoadLimit(uSwitch);
}

```

```
bSwitch = static_cast<bool>(uSwitch);  
return ret;  
}
```

6.14 轴参数应用

(saveTorqueLimit/saveAvrLoadLimit/saveOverHeatAndLoadLimit)

6.14.1 说明

接口	功能	返回值	参数	参数说明
int RobotArmInterface::saveTorqueLimit(bool bSwitch, int IntValue[ROBOT_AXIS_NUM])	保存电流限制参数	int	开关、各轴值	写控制器
int RobotArmInterface::saveAvrLoadLimit(bool bSwitch, int IntValue[ROBOT_AXIS_NUM])	保存平均负载率参数	int	开关、各轴值	写控制器
int RobotArmInterface::saveOverHeatAndLoadLimit(bool bSwitch, int IntValue[ROBOT_AXIS_NUM])	保存过热+过载开关	int	过热开关、过载开关	先写过载，再写过热

adLimit(bool heatSwitch, bool loadSwitch)				
--	--	--	--	--

参数说明

```
C++
// commandinfo.h
class METATYPE_EXPORT CmdSaveTorqueLimit : public AbstractCmd
{
public:
    bool bSwitch = false;
    int IntValue[ROBOT_AXIS_NUM] = {0};
};

class METATYPE_EXPORT CmdSaveAvrLoadLimit : public AbstractCmd
{
public:
    bool bSwitch = false;
    int IntValue[ROBOT_AXIS_NUM] = {0};
};

class METATYPE_EXPORT CmdSaveOverHeatAndLoadLimit : public
AbstractCmd
{
public:
    bool bOverHeatSwitch = false;
    bool bOverLoadSwitch = false;
};
```

6.14.2 调用

```
C++
// AxisParamsForm::on_pbn_current_apply_clicked
int values[ROBOT_AXIS_NUM] = {0};
// ... values[0..5/6] 从 edit_current_j* 读取 ...
bool checked = ui->switch_current->isChecked();
CommunicationEngine::instance()->enqueueCmd_saveTorqueLimit(
    this, AbstractCmd::CmdType_AxisParams_SaveTorqueLimit,
```



```
checked, values);
```

```
C++
// AxisParamsForm::on_pbn_loadrate_apply_clicked
int values[ROBOT_AXIS_NUM] = {0};
// ... values[0..5/6] 从 edit_loadrate_j* 读取 ...
bool checked = ui->switch_loadrate->isChecked();
CommunicationEngine::instance()->enqueueCmd_saveAvrLoadLimit(
    this, AbstractCmd::CmdType_AxisParams_SaveAvrLoadLimit,
    checked, values);
```

```
C++
// AxisParamsForm::on_switch_overheat_clicked /
// on_switch_overnote_clicked
CommunicationEngine::instance()-
>enqueueCmd_saveOverHeatAndLoadLimit(
    this,
    AbstractCmd::CmdType_AxisParams_SaveOverHeatAndLoadLimit,
    bOverHeat, bOverLoad);
```

6.14.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_AxisParams_SaveTorqueLimit: {
    CmdSaveTorqueLimit *cmd = static_cast<CmdSaveTorqueLimit
*>(absCmd);
    int iRet = m_commInstance->saveTorqueLimit(cmd->bSwitch, cmd-
>IntValue);
    emit CommunicationEngine::instance()-
>signal_saveTorqueLimit_result(iRet);
    break;
}
case AbstractCmd::CmdType_AxisParams_SaveAvrLoadLimit: {
    CmdSaveAvrLoadLimit *cmd = static_cast<CmdSaveAvrLoadLimit
*>(absCmd);
    int iRet = m_commInstance->saveAvrLoadLimit(cmd->bSwitch, cmd-
```

```

>IntValue);
    emit CommunicationEngine::instance()-
>signal_saveAvrLoadLimit_result(iRet);
    break;
}
case AbstractCmd::CmdType_AxisParams_SaveOverHeatAndLoadLimit: {
    CmdSaveOverHeatAndLoadLimit *cmd
        = static_cast<CmdSaveOverHeatAndLoadLimit *>(absCmd);
    int iRet = m_commInstance->saveOverHeatAndLoadLimit(
        cmd->bOverHeatSwitch, cmd->bOverLoadSwitch);
    emit CommunicationEngine::instance()
        ->signal_saveOverHeatAndLoadLimit_result(iRet);
    break;
}

```

```

C++
// robotarminterface.cpp (完整函数)
int RobotArmInterface::saveTorqueLimit(bool bSwitch, int
IntValue[ROBOT_AXIS_NUM])
{
    int value[8] = {0};
    memcpy(value, IntValue, ROBOT_AXIS_NUM * sizeof(int));
    return _IRobotArm-
>saveTorqueLimit(static_cast<quint16>(bSwitch), value);
}

int RobotArmInterface::saveAvrLoadLimit(bool bSwitch, int
IntValue[ROBOT_AXIS_NUM])
{
    int value[8] = {0};
    memcpy(value, IntValue, ROBOT_AXIS_NUM * sizeof(int));
    return _IRobotArm-
>saveAvrLoadLimit(static_cast<quint16>(bSwitch), value);
}

int RobotArmInterface::saveOverHeatAndLoadLimit(bool heatSwitch,
bool loadSwitch)
{
    int iRet = _IRobotArm-
>writeOverLoadLimit(static_cast<quint16>(loadSwitch));
    if (iRet != 0) {
        return iRet;
    }
}

```

```

return _IRobotArm-
>writeOverHeatLimit(static_cast<quint16>(heatSwitch));
}

```

6.15 安装姿态 (getRobotCompParam/ saveRobotCompensationParam)

6.15.1 说明

接口	功能	返回值	参数	参数说明
int RobotArmInterface::getRobotCompParam(double compParam[16])	读取机器人补偿参数	int	16 元素数组 (出参)	其中 compParam[10]=alpha、 compParam[11]=beta
int RobotArmInterface::saveRobotCompensationParam(double compParam[16], bool autoSaveFile Commd=true)	保存机器人补偿参数	int	16 元素数组 (入参)	线程层仅改写 alpha/beta 后 保存

6.15.2 调用

```

C++
// InstallationPostureForm::on_pbn_save_clicked
m_compParam[10] = m_alpha;
m_compParam[11] = m_beta;
CommunicationEngine::instance()->enqueueCmd_setData(

```

```
    this, AbstractCmd::CmdType_InstallationPosture_Write,  
    m_compParam);
```

6.15.3 函数实现

```
C++  
// communicationthread.cpp (节选)  
case AbstractCmd::CmdType_InstallationPosture_Read: {  
    double params[16] = {0.0f};  
    int iRet = m_commInstance->getRobotCompParam(params);  
    if (iRet != 0) {  
        break;  
    }  
    emit CommunicationEngine::instance()  
        ->signal_installationposture_read_result(absCmd->m_object,  
iRet == 0, params);  
    break;  
}  
case AbstractCmd::CmdType_InstallationPosture_Write: {  
    auto [params] = ((CmdDatas<double *> *)absCmd)->m_data;  
  
    double retReadArgs[16] = {0.0f};  
    int iRet = m_commInstance->getRobotCompParam(retReadArgs);  
    if (iRet != 0) {  
        break;  
    }  
  
    retReadArgs[10] = params[10];  
    retReadArgs[11] = params[11];  
  
    iRet = m_commInstance-  
>saveRobotCompensationParam(retReadArgs);  
    if (iRet == 0) {  
        emit CommunicationEngine::instance()  
            ->signal_intallposture_changed(params[10],  
params[11]);  
        m_commInstance->setPostureAlphaBeta(params[10],  
params[11]);  
    }  
    emit CommunicationEngine::instance()  
        ->signal_installationposture_write_result(absCmd-  
>m_object, iRet == 0);
```

```
break;
}
```

```
C++
// robotarminterface.cpp (完整函数)
int RobotArmInterface::getRobotCompParam(double compParam[])
{
    int ret = _IRobotArm->getRobotCompParam(compParam);
    return ret;
}

int RobotArmInterface::saveRobotCompensationParam(double
compParam[], bool autoSaveFileCommd)
{
    int ret = _IRobotArm->writeRobotInstallParam(compParam);
    return ret;
}
```

6.16 松抱闸 (readBrakeState / releaseBrake / closeBrake)

6.16.1 说明

接口	功能	返回值	参数	参数说明
bool RobotControll nterface::read BrakeState(ch ar stateArray[6])	读取 6 轴抱闸 状态	bool	6 轴状态数组 出参	0 表示已松 闸，1 表示合 闸
void RobotControll	指定轴执行松	无	轴号 1~6	

nterface::relea seBrake(int axisNo)	抱闸			
void RobotControll nterface::clos eBrake(int axisNo)	指定轴执行合 抱闸	无	轴号 1~6	

```

C++
// commandinfo.h
class METATYPE_EXPORT CmdReleaseBrake : public AbstractCmd
{
public:
    int m_axisNo = 0;
};

```

6.16.2 调用

```

C++
// ReleaseBrakeForm::slot_refreshAxisBrakeData
CommunicationEngine::instance()->enqueueCmd_handleReleaseBrake(
    this, AbstractCmd::CmdType_ReadBrakeState);

```

```

C++
// ReleaseBrakeForm::on_pbn_releaseBrake_clicked
CommunicationEngine::instance()->enqueueCmd_handleReleaseBrake(
    this, AbstractCmd::CmdType_ReleaseBrake, m_axisNo);

```

```

C++
// ReleaseBrakeForm::on_pbn_closeBrake_clicked
CommunicationEngine::instance()->enqueueCmd_handleReleaseBrake(
    this, AbstractCmd::CmdType_CloseBrake, m_axisNo);

```

6.16.3 函数实现

```

C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_ReadBrakeState: {
    QVector<char> vecStates;
    char stateArray[6] = {1, 1, 1, 1, 1, 1};
    bool isOk = m_commInstance->readBrakeState(stateArray);
    if (isOk) {
        for (int i = 0; i < 6; i++) {
            vecStates.append(stateArray[i]);
        }
    }
    emit CommunicationEngine::instance()
        ->signal_handleReleaseBrake(absCmd->m_object, absCmd-
>m_cmdType,

    QVariant::fromValue(vecStates), isOk, QString());
    break;
}
case AbstractCmd::CmdType_ReleaseBrake: {
    // ... 连接态/电柜类型/使能/急停/上电状态校验 ...
    CmdReleaseBrake *cmd = static_cast<CmdReleaseBrake *>(absCmd);
    m_commInstance->releaseBrake(cmd->m_axisNo);
    // ... 轮询 readBrakeState, 确认目标轴状态变为 0 ...
    emit CommunicationEngine::instance()
        ->signal_handleReleaseBrake(absCmd->m_object, absCmd-
>m_cmdType,

                                QVariant(), isOk, sErrMsg);

    break;
}
case AbstractCmd::CmdType_CloseBrake: {
    CmdReleaseBrake *cmd = static_cast<CmdReleaseBrake *>(absCmd);
    m_commInstance->closeBrake(cmd->m_axisNo);
    emit CommunicationEngine::instance()
        ->signal_handleReleaseBrake(absCmd->m_object, absCmd-
>m_cmdType,

                                QVariant(), isOk, sErrMsg);

    break;
}
}

```

```

C++
// robotcontrolinterface.cpp (完整函数)
bool RobotControlInterface::readBrakeState(char stateArray[6])
{

```

```

    int ret = _IControl->readBrakeState(stateArray);
    return (ret == ERROR_OK);
}

void RobotControlInterface::releaseBrake(int axisNo)
{
    if (axisNo < 1 || axisNo > 6) {
        return;
    }
    _IControl->setBrakeCurAxisNo(axisNo);
    _IControl->cleanBrakeCount();
    _IControl->setBrakeFirstCmd(true);
    _IControl->setBrakeThreadState(true);
}

void RobotControlInterface::closeBrake(int axisNo)
{
    _IControl->setBrakeThreadState(false);
    _IControl->cleanBrakeCount();
    _IControl->writeBrakeState(static_cast<char>(1), 0, axisNo,
true);
}

```

6.17 轨迹恢复参数 (ReadTrajRecoveryParam / WriteTrajRecoveryParam)

6.17.1 说明

接口	功能	返回值	参数	参数说明
int TrajRecoveryInterface::ReadTrajRecoveryParam(CobotTrajRecoveryParam ¶m)	读取轨迹恢复 阈值参数	int	结构体出参	包含 i32Mode、手 动阈值、自动 阈值

int TrajRecoveryInterface::WriteTrajRecoveryParam(CobotTrajRecoveryParam ¶m)	写入轨迹恢复 阈值参数	int	结构体入参	控制器持久化
---	----------------	-----	-------	--------

参数说明（结构体）

```
C++
// trajrecoverydefines.h（核心字段）
typedef struct stCobotTrajRecoveryParam
{
    int i32Mode; // 0:关节模式 1:位置模式
    CobotTrajRecoveryThreshold stManualParam; // 位置模式-手动阈值
    CobotTrajRecoveryThreshold stAutoParam; // 位置模式-自动阈值
} CobotTrajRecoveryParam;
```

6.17.2 调用

```
C++
// TrajRecoveryForm::updateUI
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_TrajRecv_ReadParams);
```

```
C++
// TrajRecoveryForm::on_pbn_save_clicked
CobotTrajRecoveryParam params;
params.i32Mode = ui->rbtn_joint->isChecked() ? 0 : 1;
params.stManualParam.f64TCPDistance = ui->edit_manual_tcppos->text().toDouble();
params.stManualParam.f64TCPRotation = ui->edit_manual_tcpori->text().toDouble();
params.stManualParam.f64ExternalDistance = ui->edit_manual_armpos->text().toDouble();
params.stManualParam.f64ExternalRotation = ui->edit_manual_armori-
```

```

>text().toDouble();
params.stAutoParam.f64TCPDistance = ui->edit_auto_tcppos-
>text().toDouble();
params.stAutoParam.f64TCPRotation = ui->edit_auto_tcpori-
>text().toDouble();
params.stAutoParam.f64ExternalDistance = ui->edit_auto_armpos-
>text().toDouble();
params.stAutoParam.f64ExternalRotation = ui->edit_auto_armori-
>text().toDouble();

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_TrajRecv_WriteParams, params);

```

6.17.3 函数实现

```

C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_TrajRecv_ReadParams: {
    CobotTrajRecoveryParam params;
    int iRet = Communication::instance()-
>ReadTrajRecoveryParam(params);
    emit CommunicationEngine::instance()
        ->signal_trajrecovery_readparams(absCmd->m_object, iRet ==
0, params);
    break;
}
case AbstractCmd::CmdType_TrajRecv_WriteParams: {
    auto [params]
        = ((CmdDatas<CobotTrajRecoveryParam> *)absCmd)->m_data;
    int iRet = Communication::instance()-
>WriteTrajRecoveryParam(params);
    emit CommunicationEngine::instance()
        ->signal_trajrecovery_writeparams(absCmd->m_object, iRet
== 0);
    break;
}

```

```

C++
// trajrecoveryinterface.cpp (完整函数)

```

```

int
TrajRecoveryInterface::WriteTrajRecoveryParam(CobotTrajRecoveryParam &param)
{
    TrajRecoveryParam innerParams;
    memcpy(&innerParams, &param, sizeof(param));
    int iRet = _ITrajectoryRecovery->WriteTrajRecoveryParam(innerParams);
    return iRet;
}

int
TrajRecoveryInterface::ReadTrajRecoveryParam(CobotTrajRecoveryParam &param)
{
    TrajRecoveryParam innerParams;
    int iRet = _ITrajectoryRecovery->ReadTrajRecoveryParam(innerParams);
    memcpy(&param, &innerParams, sizeof(param));
    return iRet;
}

```

7.系统

7.1 控制器时间 (isNeedTimeSynToController / setControllerTime)

7.1.1 说明

接口	功能	返回值	参数	参数说明
void ControllerSystemConfigPrivate::isNeedTi	读取控制器时间并判断是否需同步	无	—	本地与控制器相差 $\geq 180s$ 触发提示

meSynToController()				
void ControllerSystemConfigPrivate::setControllerTime(AbstractCmd *cmd)	写控制器系统时间	无	命令中 QDateTime	通过 _IConnection->writeDevSysTime

7.1.2 调用

```
C++
// ControllerTimeForm::operationWhenShowOrHide
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Get_ConrtollerTime);
```

```
C++
// ControllerTimeForm::on_btn_save_clicked (同步本地时间)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_Set_ConrtollerTime,
    QDateTime::currentDateTime());
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Get_ConrtollerTime);
```

```
C++
// ControllerTimeForm::on_btn_saveByCustom_clicked (同步自定义时间)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_Set_ConrtollerTime, ui->dateTimeEdit_year->dateTime());
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_Get_ConrtollerTime);
```

7.1.3 函数实现

C++

// communicationthread.cpp (节选)

```
case AbstractCmd::CmdType_Get_ConrtollerTime: {
    m_commInstance->isNeedTimeSynToController();
    break;
}
case AbstractCmd::CmdType_Set_ConrtollerTime: {
    m_commInstance->setControllerTime(absCmd);
    break;
}
```

C++

// controllersystemconfig.cpp (完整函数)

```
void ControllerSystemConfigPrivate::isNeedTimeSynToController()
{
    GSYSTEMTIME systemTime;
    int ret = _IConnection->readDevSysTime(systemTime);
    if (ret != ERROR_OK)
        return;

    QDateTime currentDate = QDateTime::currentDateTime();
    QDateTime controllerTime;
    controllerTime.setDate(QDate(systemTime.wYear,
systemTime.wMonth, systemTime.wDay));
    controllerTime.setTime(QTime(systemTime.wHour,
systemTime.wMinute, systemTime.wSecond));

    if (qAbs(currentDate.secsTo(controllerTime)) >= 180) {
        emit CommunicationEngine::instance()-
>signal_controllerTimeIsNeedSysnFromLocal(true, controllerTime);
    } else {
        emit CommunicationEngine::instance()-
>signal_controllerTimeIsNeedSysnFromLocal(false, controllerTime);
    }
}

void ControllerSystemConfigPrivate::setControllerTime(AbstractCmd
*cmd)
{
    auto [currentDate] = ((CmdDdatas<QDateTime> *)cmd)->m_data;
    GSYSTEMTIME systemTime;
    systemTime.wYear = currentDate.date().year();
```

```

systemTime.wMonth = currentDateTime.date().month();
systemTime.wDay = currentDateTime.date().day();
systemTime.wHour = currentDateTime.time().hour();
systemTime.wMinute = currentDateTime.time().minute();
systemTime.wSecond = currentDateTime.time().second();

if (_IConnection->writeDevSysTime(systemTime, 1) != ERROR_OK)
{
    QString str = QObject::tr("Failed to synchronize time to
the controller.");
    emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(str);
    PRINT_MSG(str);
} else {
    PRINT_MSG(QObject::tr("Controller time synchronized
successfully."));
}
}

```

7.2 机器人控制权（getCurrentAuthority / setCurrentAuthority）

7.2.1 说明

接口	功能	返回值	参数	参数说明
void ControlAuthority::getCurrentAuthority(AbstractCmd *absCmd)	读取当前控制权和默认速度	无	命令对象	读取 authority + 子模式 + 默认速度
void ControlAuthority::setCurrentAuthority(AbstractCmd *absCmd)	设置控制权	无	命令对象（含	调 changeAuthor

ty::setCurrent Authority(Abst ractCmd *absCmd)			type/speed)	ity
---	--	--	-------------	-----

7.2.2 调用

```
C++
// ControlAuthorityForm::updateUI
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_GetControlAuthority);
```

```
C++
// ControlAuthorityForm::on_pbn_save_clicked
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType_SetControlAuthority,
    (InoCtrlAuthority)m_currentEditType,
    ui->edit_speed->getQVariant().toInt());
```

7.2.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_GetControlAuthority: {
    m_commInstance->getCurrentAuthority(absCmd);
    break;
}
case AbstractCmd::CmdType_SetControlAuthority: {
    m_commInstance->setCurrentAuthority(absCmd);
    break;
}
case AbstractCmd::CmdType_SetCtlAuthority:
    setCtlAuthority(absCmd);
```

```
break;
```

```
C++
// controlauthority.cpp (完整函数)
void ControlAuthority::setCurrentAuthority(AbstractCmd *absCmd)
{
    auto [type, speed] = ((CmdDatas<InoCtrlAuthority, int>
*)absCmd)->m_data;
    InoRobBusiness::CtrlAuthority auth =
InoRobBusiness::CtrlAuthority::NOCTRL;
    switch (type) {
        case InoCtrlAuthority_Unknown: auth =
InoRobBusiness::CtrlAuthority::TEACHPAD; break;
        case InoCtrlAuthority_TeachPad: auth =
InoRobBusiness::CtrlAuthority::TEACHPAD; break;
        case InoCtrlAuthority_InorobShop: auth =
InoRobBusiness::CtrlAuthority::INOROBSHOP; break;
        case InoCtrlAuthority_Ethernet: auth =
InoRobBusiness::CtrlAuthority::RMT_ETHERNET; break;
        case InoCtrlAuthority_IO: auth =
InoRobBusiness::CtrlAuthority::RMT_IO; break;
        case InoCtrlAuthority_Modbus: auth =
InoRobBusiness::CtrlAuthority::RMT_MODBUS; break;
        case InoCtrlAuthority_IO_AUTO: auth =
InoRobBusiness::CtrlAuthority::RMT_IO_AUTO; break;
        case InoCtrlAuthority_NoCtrl: auth =
InoRobBusiness::CtrlAuthority::NOCTRL; break;
    }

    int ret = _IControlAuthority->changeAuthority(auth, speed);
    if (ret != ERROR_OK) {
        getCurrentAuthority(absCmd);
    }
    emit CommunicationEngine::instance()-
>signal_setControlAuthorityRes(
        absCmd->m_object, ret == ERROR_OK);
}

void ControlAuthority::getCurrentAuthority(AbstractCmd *absCmd)
{
    InoRobBusiness::CtrlAuthority type =
InoRobBusiness::CtrlAuthority::NOCTRL;
```



```

        InoRobBusiness::RmtIoSubMode rmtIoSubMode =
InoRobBusiness::RmtIoSubMode::UNKNOWN;
        int speed = 0;
        int ret1 = _IControlAuthority->readAuthority(type);
        int ret2 = _IControlAuthority->readRmtIoSubMode(rmtIoSubMode);
        int ret3 = _IControlAuthority->readRmtDefaultSpeed(speed);
        if (ret1 != ERROR_OK || ret2 != ERROR_OK || ret3 != ERROR_OK)
        {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to get
controller permission status.));
            return;
        }
        if (type == InoRobBusiness::CtrlAuthority::RMT_IO &&
rmtIoSubMode == InoRobBusiness::RmtIoSubMode::ONLYAUTO) {
            type = InoRobBusiness::CtrlAuthority::RMT_IO_AUTO;
        }

        InoCtrlAuthority ans;
        switch (type) {
            case InoRobBusiness::CtrlAuthority::TEACHPAD: ans =
InoCtrlAuthority_TeachPad; break;
            case InoRobBusiness::CtrlAuthority::INOROBSHOP: ans =
InoCtrlAuthority_InorobShop; break;
            case InoRobBusiness::CtrlAuthority::RMT_ETHERNET: ans =
InoCtrlAuthority_Ethernet; break;
            case InoRobBusiness::CtrlAuthority::RMT_IO: ans =
InoCtrlAuthority_IO; break;
            case InoRobBusiness::CtrlAuthority::RMT_MODBUS: ans =
InoCtrlAuthority_Modbus; break;
            case InoRobBusiness::CtrlAuthority::RMT_IO_AUTO: ans =
InoCtrlAuthority_IO_AUTO; break;
            case InoRobBusiness::CtrlAuthority::NOCTRL: ans =
InoCtrlAuthority_NoCtrl; break;
        }
        comm()->setCurCtrlAuthority(ans);
        emit CommunicationEngine::instance()-
>signal_getControlAuthorityRes(absCmd->m_object, ans, speed);
    }

```

--- 李昊 1860 6000
2026年04月28日 17:44

李昊 1860 6000
2026年04月28日 17:44

7.3 RS485 配置读取 (readRs485Config)

7.3.1 说明

接口	功能	返回值	参数	参数说明
void ControllerSystemConfigPrivate::readRs485Config(AbstractCmd *cmd)	读取 RS485 配置、复用模 式、控制器 485 开关	无	命令对象	汇总多个读取 结果后统一回 传

7.3.2 调用

```
C++
// RS485ConfigForm::updateUI
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_ReadRS485Config, ALL);
```

7.3.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_ReadRS485Config: {
    m_commInstance->readRs485Config(absCmd);
    break;
}
```

```
C++
// controllersystemconfig.cpp (节选)
void ControllerSystemConfigPrivate::readRs485Config(AbstractCmd
```

```

*cmd)
{
    bool readToolIsSuccess = false, readRcIsSuccess = false,
    read485ConfigIsSuccess = false;

    std::string ans;
    int ret = _IRCCConfig->readCobotToolRs485Config(ans);
    QString errorStr = "";
    ToolIO_RS485_Config controllerType;
    ToolIO_RS485_Config tool;
    if (ret != ERROR_OK) {
        errorStr = CommTr::tr("Failed to read RS485
configuration");
    } else {
        QJsonObject object =
QJsonDocument::fromJson(ans.c_str()).object();
        QJsonObject cObj = object["RC485"].toObject();
        QJsonObject tObj = object["Tool485"].toObject();
        controllerType.type = Controller_RS485;
        controllerType.baudrate = cObj["Baudrate"].toInt();
        controllerType.checkWay =
cObj["CheckWay"].toVariant().value<DataVerificationMethod>();
        controllerType.dataBit = cObj["DataBit"].toInt();
        controllerType.stopBit = cObj["StopBit"].toInt();
        tool.type = TOOLIO_RS485;
        tool.baudrate = tObj["Baudrate"].toInt();
        tool.checkWay =
tObj["CheckWay"].toVariant().value<DataVerificationMethod>();
        tool.dataBit = tObj["DataBit"].toInt();
        tool.stopBit = tObj["StopBit"].toInt();
        read485ConfigIsSuccess = true;
    }

    ToolIO_RS485OrAD_ReuseMode model = ReuseMode_Unknown;
    ans.clear();
    ret = _IRCCConfig->readCobotToolRs485ReuseMode(ans);
    if (ret == ERROR_OK) {
        QJsonObject object =
QJsonDocument::fromJson(ans.c_str()).object();
        model =
object["GPIOMode"].toVariant().value<ToolIO_RS485OrAD_ReuseMode>()
;
        readToolIsSuccess = true;
    }
}

```

```

bool state = false;
ret = _IRCCConfig->readCobotRcRs485State(state);
if (ret == ERROR_OK) {
    readRcIsSuccess = true;
}

emit CommunicationEngine::instance()->signal_getRS485Result(
    cmd->m_object, read485ConfigIsSuccess, controllerType,
    tool,
    readToolIsSuccess, model, readRcIsSuccess, state);
}

```

7.4 RS485 配置保存 (writeRs485Config /writeToolReuseModel)

7.4.1 说明

接口	功能	返回值	参数	参数说明
void ControllerSystemConfig::writeRs485Config (AbstractCmd *cmd)	保存 RS485 配置	无	命令对象	内部包含：控制器 485 开关、GPIO 复用模式、Tool485 配置（可选）、RC485 配置
void ControllerSystemConfig::writeToolReuseModel (Abstract	单独保存复用模式 + 控制器 485 开关	无	命令对象	

Cmd *cmd)				
-----------	--	--	--	--

7.4.2 调用

C++

```
// RS485ConfigForm::on_pbn_save_clicked
ToolIO_RS485OrAD_ReuseMode model = ui->cmb_reuseType->currentData().value<ToolIO_RS485OrAD_ReuseMode>();

ToolIO_RS485_Config toolConfig;
toolConfig.type = TOOLIO_RS485;
toolConfig.baudrate = ui->cmb_baudrate->currentData().toInt();
toolConfig.dataBit = ui->cmb_dataBitTool->currentData().toInt();
toolConfig.stopBit = ui->cmb_stopBitTool->currentData().toInt();
toolConfig.checkWay = ui->cmb_checkway->currentData().value<DataVerificationMethod>();

ToolIO_RS485_Config rcConfig;
rcConfig.type = Controller_RS485;
rcConfig.baudrate = ui->cmb_baudrate_2->currentData().toInt();
rcConfig.dataBit = ui->cmb_dataBitRc->currentData().toInt();
rcConfig.stopBit = ui->cmb_stopBitRc->currentData().toInt();
rcConfig.checkWay = ui->cmb_checkway_2->currentData().value<DataVerificationMethod>();

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SetRS485Config, model,
    ui->pbn_rc485Active->isChecked(), model == ReuseMode_RS485,
    toolConfig, rcConfig);
```

7.4.3 函数实现

C++

```
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_SetRS485OrADModel: {
    m_commInstance->writeToolReuseModel(absCmd);
    break;
}
case AbstractCmd::CmdType_SetRS485Config: {
```

```

m_commInstance->writeRs485Config(absCmd);
break;
}

```

```

C++
// controllersystemconfig.cpp (节选)
void ControllerSystemConfigPrivate::writeRs485Config(AbstractCmd
*cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [type, activeRc485, isWriteToolRS485, toolConfig, config]
= BIND(cmd, ToolIO_RS485OrAD_ReuseMode, bool, bool,
ToolIO_RS485_Config, ToolIO_RS485_Config);
    QString errorStr;
    int ret = _IRCCConfig->writeCobotRcRs485State(activeRc485);
    if (ret != ERROR_OK) {
        errorStr = CommTr::tr("Failed to set controller RS485
status.");
    }

    QJsonObject object;
    object["GPIOMode"] = type;
    ret = _IRCCConfig-
>writeCobotToolRs485ReuseMode(QJsonDocument(object).toJson(QJsonDo
cument::Compact).toString());
    if (ret != ERROR_OK) {
        combinErrorStr(errorStr, CommTr::tr("Failed to set GPIO
reuse model.));
    }

    bool ans = true;
    if (isWriteToolRS485) {
        QJsonObject object;
        object["Type"] = config.type;
        object["Baudrate"] = config.baudrate;
        object["DataBit"] = config.dataBit;
        object["StopBit"] = config.stopBit;
        object["CheckWay"] = config.checkWay;
        qDebug() <<
QJsonDocument(object).toJson(QJsonDocument::Compact);
        int ret = _IRCCConfig-
>writeCobotToolRs485Config(QJsonDocument(object).toJson(QJsonDocum

```

```

ent::Compact).toStdString());
    if (ret != ERROR_OK) {
        combinErrorStr(errorStr, CommTr::tr("Failed to set
tool RS485 config.));
    }
}
QJsonObject object2;
object2["Type"] = config.type;
object2["Baudrate"] = config.baudrate;
object2["DataBit"] = config.dataBit;
object2["StopBit"] = config.stopBit;
object2["CheckWay"] = config.checkWay;
qDebug() <<
QJsonDocument(object2).toJson(QJsonDocument::Compact);
ret = _IRCCConfig-
>writeCobotToolRs485Config(QJsonDocument(object2).toJson(QJsonDocu
ment::Compact).toStdString());
    if (ret != ERROR_OK) {
        combinErrorStr(errorStr, CommTr::tr("Failed to set
controller RS485 config.));
    }
    if (!errorStr.isEmpty())
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(errorStr);
    else
        PRINT_MSG(ControllerSystemConfigTr::tr("RS485 config saved
successfully.));
        LOG_TRACE(QString("[%1][%2][%3]")
                    .arg(__FILE__, __FUNCTION__,
QString::number(QDateTime::currentMSecsSinceEpoch())));
}

void
ControllerSystemConfigPrivate::writeToolReuseModel(AbstractCmd
*cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [type, activeRc485] =
((CmdDatas<ToolIO_RS485OrAD_ReuseMode, bool> *)cmd)->m_data;
    QString errorStr;
    QJsonObject object;
    object["GPIOMode"] = type;
    int ret = _IRCCConfig-
>writeCobotToolRs485ReuseMode(QJsonDocument(object).toJson(QJsonDo

```

```

cument::Compact).toStdString());
    if (ret != ERROR_OK) {
        errorStr = CommTr::tr("Failed to set GPIO reuse model.");
    }

    ret = _IRCCConfig->writeCobotRcRs485State(activeRc485);
    if (ret != ERROR_OK) {
        combinErrorStr(errorStr, CommTr::tr("Failed to set
controller RS485 status."));
    }

    if (!errorStr.isEmpty())
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(errorStr);
    else
        PRINT_MSG(ControllerSystemConfigTr::tr("RS485 enabled
status saved successfully.));
        LOG_TRACE(QString("[%1][%2][%3]")
                    .arg(__FILE__, __FUNCTION__,
QString::number(QDateTime::currentMSecsSinceEpoch())));
    }

```

7.5 RS485 调试 (writeRs485Debugging)

7.5.1 说明

接口	功能	返回值	参数	参数说明
void ControllerSystemConfig::writeRs485Debugging(AbstractCmd *cmd)	执行 RS485 调试发送并读取响应	无	命令对象	内部做 CRC16(Modbus)拼包与返回 码解析

7.5.2 调用

```
C++
// RS485ConfigForm::on_pbn_debugging_clicked
ToolIO_RS485_Debugging temp;
temp.type = ui->cmb_channel->currentData().value<RS485_Channel>();
temp.checkType = ui->cmb_debuggingCheckType->currentData().value<ToolIO_RS485_VerCheckType>();
temp.content = ui->edit_sendData->toPlainText();

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SetRS485Debugging, temp);
```

7.5.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_SetRS485Debugging: {
    m_commInstance->writeRs485Debugging(absCmd);
    break;
}
```

```
C++
// controllersystemconfig.cpp (节选)
void
ControllerSystemConfigPrivate::writeRs485Debugging(AbstractCmd
*cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    auto [config] = ((CmdDatas<ToolIO_RS485_Debugging> *)cmd)->m_data;
    QList<char> datain[20];
    QStringList contentList = config.content.split(' ');

    int size = contentList.size();
    if (size > 65535)
```

```

        return;
    QList<unsigned char> inputData;
    inputData.push_back(config.type);
    inputData.push_back(0);
    inputData.push_back(0);
    int count = 0;
    for (int i = 0; i < size; ++i) {
        if (contentList[i].size() > 2) {
            return;
        }
        if (contentList[i].size() == 0)
            continue;
        if (contentList[i].size() == 1) {
            contentList[i] = "0" + contentList[i];
        }
        inputData.push_back(contentList[i].toUShort(0, 16));
        ++count;
    }
    quint16 realSize = count + sizeof(quint16);
    memcpy(&inputData[1], &realSize, sizeof(quint16));
    InoRobUtil::Crc16 caluator;
    quint16 verificaton = caluator.CRC16_MODBUS(&inputData[3],
count);

    inputData.push_back(0);
    inputData.push_back(0);
    memcpy(&inputData[inputData.size() - sizeof(quint16)],
&verificaton, sizeof(quint16));
    qDebug() << inputData;
    std::vector<quint8> ans;
    int ret = _IRCCConfig-
>writeCobotToolRs485Debugging(&inputData[0], inputData.size(),
ans);
    if (ret != ERROR_OK || ans.size() < sizeof(quint16)) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(CommTr::tr("RS485 debugging
failed."));
        return;
    }
    QList<quint8> ansList(ans.begin(), ans.end());
    qDebug() << ansList;
    quint16 len = 0;
    quint16 errorCode = 0;
    memcpy(&errorCode, &ansList[0], sizeof(quint16));

```

```

memcpy(&len, &ansList[sizeof(quint16)], sizeof(quint16));
QString errorStr;
switch (errorCode) {
case 0: {
    QString printStr;
    if (ansList.size() != len + sizeof(quint16) * 2) {
        errorStr = CommTr::tr("RS485 debugging attempt failed
because the data length does not match the actual cache length.");
    } else {
        if (len == 0)
            printStr = CommTr::tr("The data was sent
successfully, but the data received is empty.");
        else {
            for (int i = sizeof(quint16) * 2; i <
ansList.size(); ++i) {
                QString temp = QString::number(ansList[i],
16).toUpper();
                printStr += ((temp.size() == 1 ? "0" : "") +
temp + " ");
            }
        }
        QString time =
QDateTime::currentDateTime().toString("hh:mm:ss:");
        emit CommunicationEngine::instance()-
>signal_tryRs485DebuggingResult(cmd->m_object, time + "\r\n" +
printStr);
        return;
    }
    break;
}
case 1:
    errorStr = CommTr::tr("RS485 debugging attempt failed
because the length of data sent exceeds the maximum limit.");
    break;
case 2:
    errorStr = CommTr::tr("RS485 debugging attempt failed
because the target device did not respond.");
    break;
default:
    errorStr = CommTr::tr("RS485 debugging attempt failed due
to unknown error: %1.").arg(QString::number(errorCode));
    break;
}
PRINT_ERROR(errorStr);

```

```

emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(errorStr);
LOG_TRACE(QString("[%1][%2][%3]")
            .arg(__FILE__, __FUNCTION__,
                QString::number(QDateTime::currentMSecsSinceEpoch())));
}

```

7.6 配置文件备份/加载

(startConfigFilesStateToUsb/startLoadConfigFilesFromUsb/readConfigFilesBackupStateToUsb/readLoadConfigFilesStateFromUsb)

7.6.1 说明

接口	功能	返回值	参数	参数说明
void ControllerSystemConfigPrivate::startConfigFilesStateToUsb(AbstractCmd *cmd)	启动“备份配置文件到 USB”	无	命令对象	funCode=6
void ControllerSystemConfigPrivate::startLoadConfigFilesFromUsb(AbstractCmd *cmd)	启动“从 USB 加载配置文件”	无	命令对象	funCode=7
void ControllerSystemConfigPrivate::queryBackupState()	轮询备份状态	无	命令对象	走统一状态解析

ate::readConfigFilesBackupStateToUsb(AbstractCmd *cmd)				
void ControllerSystemConfigPrivate::readLoadConfigFilesStateFromUsb(AbstractCmd *cmd)	轮询加载状态	无	命令对象	走统一状态解析

7.6.2 调用

```
C++
// SystemBackupLoadForm::on_pbn_backupConfigFile_clicked
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_startConfigFilesBackupToUsb);
```

```
C++
// SystemBackupLoadForm::on_pbn_loadConfigFile_clicked
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_startLoadConfigFilesFromUsb);
```

7.6.3 函数实现

```
C++
// communicationthread.cpp (节选)
case AbstractCmd::CmdType_startConfigFilesBackupToUsb: {
    m_commInstance->startConfigFilesStateToUsb(absCmd);
    break;
}
case AbstractCmd::CmdType_readConfigFilesBackupStateToUsb: {
```

```

        m_commInstance->readConfigFilesBackupStateToUsb(absCmd);
        break;
    }
    case AbstractCmd::CmdType_startLoadConfigFilesFromUsb: {
        m_commInstance->startLoadConfigFilesFromUsb(absCmd);
        break;
    }
    case AbstractCmd::CmdType_readLoadConfigFilesSateFromUsb:{
        m_commInstance->readLoadConfigFilesStateFromUsb(absCmd);
        break;
    }
}

```

```

C++
// controllersystemconfig.cpp (节选)
void ControllerSystemConfigPrivate::startConfigFileOperation(int
funCode, AbstractCmd *cmd)
{
    if (_IMaintenance->checkControllerUSB() == false) {
        emit MIANWARNING(ControllerSystemConfigTr::tr("The
controller USB is not ready yet and the operation is invalid.));
        return;
    }
    int nMark = 0;
    int nRet = _IRCCConfig->readFileSaveFlag(nMark);
    if (nRet != ERR_OK || nMark == 2 || (nMark >= 1 && nMark <=
10)) {
        emit MIANWARNING(ControllerSystemConfigTr::tr("System
busy, please try again later.));
        return;
    }
    nRet = _IRCCConfig->writeFileSaveFlag(funCode);
    if (nRet != ERR_OK) {
        emit
MIANWARNING(ControllerSystemConfigTr::tr("Communictaion with
controller failed.));
        return;
    }
    m_lastControllerBackUpFileWithUsbHasFinish = false;
    emit CommunicationEngine::instance()-
>signal_handleSystemBackupAndLoad(cmd->m_object, cmd->m_cmdType,
QVariant(), true, "");
}

```

```

void
ControllerSystemConfigPrivate::readConfigFileToUsbOperationState(A
bstractCmd *cmd)
{
    if(m_lastControllerBackUpFileWithUsbHasFinish)
        return;
    int nMark = 0;
    int nRet = _IRCCConfig->readFileSaveFlag(nMark);
    QString tips = getErrorStr(nMark);
    InoTaskState state = Task_Falid;
    if (nRet == ERR_OK) {
        switch (nMark) {
            case 2:
                state = Task_InProcess;
                break;
            case 61:
                state = Task_FinishedSuccess;
                m_lastControllerBackUpFileWithUsbHasFinish = true;
                break;
            default:
                m_lastControllerBackUpFileWithUsbHasFinish = true;
                break;
        }
    }else{
        m_lastControllerBackUpFileWithUsbHasFinish = true;
    }
    qDebug() << "state is" << state << cmd->m_cmdType << tips;
    emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_currentState(
        cmd->m_object, cmd->m_cmdType, 0, state, tips);
}

```

7.7 导出到本地 / 从本地导入

(startExportConfigFileToLocal/startImportConfigFileFromLocal/readExportConfigFileOperateStatusToLocal/readImportConfigFileOperateStatusFromLocal)

7.7.1 说明

接口	功能	返回值	参数	参数说明
void ControllerSystemConfigPrivate::startExportConfigFileToLocal(AbstractCmd *cmd)	启动导出到本地	无	命令对象	检查系统状态后导出
void ControllerSystemConfigPrivate::startImportConfigFileFromLocal(AbstractCmd *cmd)	启动从本地导入	无	命令对象（含路径）	先 UDF 上传后触发导入
void ControllerSystemConfigPrivate::readExportConfigFileOperateStatusToLocal(AbstractCmd *cmd)	轮询导出状态	无	命令对象	超时/成功/失败处理
void ControllerSystemConfigPrivate::readImportConfigFileOperateStatusFromLocal(AbstractCmd *cmd)	轮询导入状态	无	命令对象	完成后提示重启等

7.7.2 调用


```

C++
// SystemBackupLoadForm::on_pbn_backupConfigFileToLocal_clicked
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_ExportConfigFileToLocal);

ProcessBarFormWithComm::instance()-
>setCommunicationType(tr("Import config file from local to
controller")),

AbstractCmd::CmdType_ReadImportConfigFileStatusFromLocal);

```

```

C++
// SystemBackupLoadForm::on_pbn_loadConfigFileFromLocal_clicked
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_ImportConfigFileFromLocal,
    m_importConfigFileToLocalFilePath);

```

7.7.3 函数实现

```

C++
// controllersystemconfig.cpp (节选)
void
ControllerSystemConfigPrivate::startExportConfigFileToLocal(Abstra
ctCmd *cmd)
{
    bool ans = _IRCConfig-
>checkSystemIsReadyForBackupConfigFile();
    if (!ans) return;
    int ret = _IRCConfig->exportConfigFileToLocal();
    if (ret != ERROR_OK) return;
    m_hasDownloadConfigFile = false;
    timeForExportFile = QDateTime::currentDateTime();
    emit CommunicationEngine::instance()-
>signal_handleSystemBackupAndLoad(cmd->m_object, cmd->m_cmdType,
    "", true, "");
}

void

```

```

ControllerSystemConfigPrivate::startImportConfigFileFromLocal(Abst
ractCmd *cmd)
{
    bool ans = _IRCCConfig-
>checkSystemIsReadyForBackupConfigFile();
    if (!ans) return;
    auto [path] = BIND(cmd, QString);
    UDF udf;
    ans = udf.uploadFile("./Exchange/robotcfg.cfg.bk",
path.toStdString());
    if (!ans) {
        emit MIANWARNING(ControllerSystemConfigTr::tr("Failed to
upload config file to the controller.));
        return;
    }
    int ret = _IRCCConfig->importConfigFileToController();
    if (ret != ERROR_OK) return;
    m_hasfinishLastImport = false;
    timeForImportFile = QDateTime::currentDateTime();
    emit CommunicationEngine::instance()-
>signal_handleSystemBackupAndLoad(cmd->m_object, cmd->m_cmdType,
"", true, "");
}

void
ControllerSystemConfigPrivate::readExportConfigFileOperateStatusTo
Local(AbstractCmd *cmd)
{
    if (m_hasDownloadConfigFile)
        return;
    InoTaskState state = Task_Falid;
    QString errorStr;
    if (timeForExportFile.secsTo(QDateTime::currentDateTime()) <
180) {
        int flag = 0;
        int ret = _IRCCConfig->checkConfigFileStatus(flag);
        if (ret != ERROR_OK) {
            state = Task_Falid;
            m_hasDownloadConfigFile = true;
        } else {
            if (flag == 1)
                state = Task_FinishedSuccess;
            else if (flag == 100)
                state = Task_InProcess;
        }
    }
}

```

```

        else
            m_hasDownloadConfigFile = true;
        }
        if (!m_hasDownloadConfigFile && state ==
Task_FinishedSuccess) {
            UDF udf;
            auto [path] = BIND(cmd, QString);
            bool ans =
            udf.downloadFile("./Exchange/robotcfg.cfg.bk",
            path.toString());
            if (!ans)
                state = Task_Failed;
            m_hasDownloadConfigFile = true;
        }
    } else {
        errorStr = ControllerSystemConfigTr::tr("Operation
timeout.");
        m_hasDownloadConfigFile = true;
    }
    emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_currentState(
        cmd->m_object, cmd->m_cmdType, 0, state, errorStr);
}

void
ControllerSystemConfigPrivate::readImportConfigFileOperateStatusFr
omLocal(AbstractCmd *cmd)
{
    if (m_hasfinishLastImport)
        return;
    InoTaskState state = Task_Failed;
    QString tips;
    if (timeForImportFile.secsTo(QDateTime::currentDateTime()) <
180) {
        int flag = 0;
        int ret = _IRCCConfig->checkConfigFileStatus(flag);
        if (ret != ERROR_OK) {
            state = Task_Failed;
            m_hasfinishLastImport = true;
        } else {
            if (flag == 1) {
                tips = ControllerSystemConfigTr::tr("Successfully
loaded configuration file, please restart the controller.");
                state = Task_FinishedSuccess;
            }
        }
    }
}

```

```

        m_hasfinishLastImport = true;
    } else if (flag == 100) {
        state = Task_InProcess;
    } else {
        m_hasfinishLastImport = true;
    }
}
} else {
    m_hasfinishLastImport = true;
    tips = ControllerSystemConfigTr::tr("Operation timeout.");
}
emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_currentState(
    cmd->m_object, cmd->m_cmdType, 0, state, tips);
}

```

7.8 存储卡格式化 (checkControllerUSB/startRestoreFactory/readRestoreFactoryState)

7.8.1 说明

接口	功能	返回值	参数	参数说明
bool RobotControll ninterface::che ckControllerUS B()	检查控制器 USB 就绪	bool	—	格式化前置检 查
void ControllerSyst emConfigPriv ate::startRest oreFactory(Ab	发起恢复出厂	无	命令对象	校验忙状态后 写入标记

structCmd *cmd)				
void ControllerSystemConfigPrivate::readRestoreFactoryState(AbstractCmd *cmd)	轮询恢复出厂 状态	无	命令对象	回传进度状态

7.8.2 调用

```
C++
// SystemBackupLoadForm::on_pbn_formatStorageCard_clicked
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_FormatMemoryCard);
```

```
C++
// SystemBackupLoadForm::on_pbn_factoryDefault_clicked
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_startRestoreFactory);
```

```
C++
// SystemBackupLoadForm::slot_loadPointFile
CommunicationEngine::instance()-
>enqueueCmd_handleSystemBackupAndLoad(
    this, AbstractCmd::CmdType_CheckPointFileIsExist,
    sFileAbsolutePath);
```

7.8.3 函数实现

```
C++
// communicationthread.cpp (节选)
```

```

case AbstractCmd::CmdType_FormatMemoryCard: { ... }
case AbstractCmd::CmdType_ReadMemoryCardFormatState: { ... }
case AbstractCmd::CmdType_startRestoreFactory: {
    m_commInstance->startRestoreFactory(absCmd);
    break;
}
case AbstractCmd::CmdType_readRestoreFactoryState: {
    m_commInstance->readRestoreFactoryState(absCmd);
    break;
}
case AbstractCmd::CmdType_CheckPointFileIsExist: { ... }
case AbstractCmd::CmdType_LoadPointFile: { ... }
case AbstractCmd::CmdType_DecideReplacePointFile: { ... }

```

C++

// controllersystemconfig.cpp (节选)

```

void
ControllerSystemConfigPrivate::startRestoreFactory(AbstractCmd
*cmd)
{
    int nMark1 = 0, nMark2 = 0;
    int nRet = _IRCCConfig->readFileSaveFlag(nMark1);
    if (nRet != ERR_OK || (nMark1 >= 1 && nMark1 <= 10)) {
        emit MIANWARNING(ControllerSystemConfigTr::tr("System
busy, please try again later.));
        return;
    }
    nRet = _IRCCConfig->readDefValueMark(nMark2);
    if (nRet != ERR_OK || (nMark2 == 1 || nMark2 == 2)) {
        emit MIANWARNING(ControllerSystemConfigTr::tr("System
busy, please try again later.));
        return;
    }
    nRet = _IRCCConfig->writeDefValueMark(1);
    if (nRet != ERR_OK) {
        emit MIANWARNING(ControllerSystemConfigTr::tr("Failed to
send command.));
        return;
    }
    emit CommunicationEngine::instance()-
>signal_handleSystemBackupAndLoad(
    cmd->m_object, cmd->m_cmdType, QVariant(), true, "");
}

```

```

}

bool RobotControlInterface::checkControllerUSB()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _IMaintenance->checkControllerUSB();
}

void
ControllerSystemConfigPrivate::readRestoreFactoryState(AbstractCmd
*cmd)
{
    int nMark = 0;
    int nRet = _IRCCConfig->readDefValueMark(nMark);
    QString tips = ControllerSystemConfigTr::tr("Communicatoin
with controller failed.");
    InoTaskState state = Task_Falid;
    if (nRet == ERR_OK) {
        switch (nMark) {
            case 1:
            case 2:
                state = Task_InProcess;
                tips = ControllerSystemConfigTr::tr("In progress,
please do not turn off the power.");
                break;
            case 0:
                state = Task_FinishedSuccess;
                tips = ControllerSystemConfigTr::tr("Factory reset
successful, please restart the controller.");
                break;
            default:
                break;
        }
    }
    qDebug()<<__FUNCTION__<<nMark<<nRet;
    emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_currentState(
        cmd->m_object, cmd->m_cmdType, 0, state, tips);
}

```

--- 李昊 1860 6000
 2026年04月28日 17:44

8.外设配置

8.1 总线开关（getSwitchEnableStatus）

8.1.1 说明

接口	功能	返回值	参数	参数说明
bool PeripheralConfigInterface::getSwitchEnableStatus(bool ðernetipEnable, bool ðercatEnable)	读取 Modbus 连接状态中的地址类型及总线安装标志, 判定 EtherNet/IP、EtherCAT 页是否应对用户开放	bool	出参：是否允许 EIP、是否允许 EtherCAT	内部调用 _IFieldBus->getModbusSwitch()->readModbusConnStatus 与 _IMonitor->GetBusInstall; NEW_MODBUS_VISION 与硬件 EtherCAT 安装位共同决定结果

8.1.2 调用

C++

```
// BusSwitchForm::showEvent（经线程转发后调用  
Communication::getSwitchEnableStatus →  
PeripheralConfigInterface::getSwitchEnableStatus）  
CommunicationEngine::instance()->enqueueCmd(this,  
AbstractCmd::CmdType_GetBusSwitchEnableStatus);
```

C++

```
BusSwitchForm::instance()  
    ->AddSlaveWidget(BusSwitch_TabIdx_ModBusForm, this,  
tr("Modbus"));
```


C++

```
BusSwitchForm::instance()  
    ->AddSlaveWidget(BusSwitch_TabIdx_EthernetIP, this,  
tr("EtherNet/IP"));
```

C++

```
BusSwitchForm::instance()  
    ->AddSlaveWidget(BusSwitch_TabIdx_EthernetCat, this,  
"EtherCAT");
```

C++

```
BusSwitchForm::instance()  
    ->AddMasterWidget(BusSwitch_TabIdx_EthernetCat, this,  
"ConfigImport");
```

8.1.3 函数实现

C++

```
// peripheralconfiginterface.cpp  
bool PeripheralConfigInterface::getSwitchEnableStatus(  
    bool &ethernetipEnable, bool &ethercatEnable)  
{  
    // Modbus 状态  
    ModbusConnectSts modbusConnectSts;  
    int nRet = _IFieldBus->getModbusSwitch()-  
>readModbusConnStatus(modbusConnectSts);  
    if (nRet != ERR_OK) {  
        return false;  
    }  
    int modbusAddrType = modbusConnectSts.ModbusAddrType; // 总线  
地址类型  
  
    // 总线安装标志  
    char filedBusInstallType[4];  
    _IMonitor->GetBusInstall(filedBusInstallType);
```

```

// EtherCAT 第一个字节的第 0 位
bool isHasEthercat = (filedBusInstallType[0] >> 0) & 0x01;
if (isHasEthercat) {
    if (modbusAddrType == NEW_MODBUS_VISION) {
        // 新版
        ethercatEnable = true;
    } else {
        // 旧版
        ethercatEnable = false;
    }
} else {
    ethercatEnable = false;
}

// 判断 EthernetIP 和 Profinet
if (modbusAddrType == NEW_MODBUS_VISION) {
    // 新版
    ethernetipEnable = true;
} else {
    // 旧版
    ethernetipEnable = false;
}
return true;
}

```

8.2 Modbus

**(writeModbusConfig/readModbusConnStatus/check
ModbusOperatePermission/checkSaveModbusPermis
sion/checkOtherFieldBusIsActive/setModbusConnect
Scheduler)**

8.2.1 说明

接口	功能	返回值	参数	参数说明
int ModbusInterface::writeModbusConfig(CobotModbusParaConfig modbusConfig)	将 TP 侧 Modbus 参数写入控制器并保存工程	int (iRet==0 视为成功)	modbusConfig	经 memcpy 为 ModbusParaConfig 后下发 getModbusSwitch()->writeModbusConfig
int ModbusInterface::readModbusConnStatus(CobotModbusConnectSts &modbusPara)	读取 RTU/TCP 连接与参数状态	int	modbusPara 出参	从 readModbusConnStatus 拷回 UI 用结构
bool ModbusInterface::checkModbusOperatePermission()	是否允许操作 Modbus 开关/参数	bool	—	委托 getModbusSwitch()
bool ModbusInterface::checkSaveModbusPermission()	是否允许保存 Modbus 配置	bool	—	保存前权限校验
bool FieldBusInterface::checkOtherFieldBusIsActive(int currFieldBus)	检查除当前类型外是否仍有其它现场总线处于活动（互斥）	bool	currFieldBus : 0 Modbus / 1 EtherNet/IP / 2 EtherCAT	_IFieldBus->checkOtherFieldBusIsActive
void ResourceInterface::setModbusConnectScheduler(const	开启或关闭 Modbus 连接状态调度（页面驻留时轮	无	true/false	_IResource->setModbusConnectScheduler

bool scheduler)	询)			
--------------------	----	--	--	--

8.2.2 调用

```
C++
// ModBusForm::updateUI → 线程中调用
ModbusInterface::readModbusConnStatus
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_ReadModbusConnStatus);
```

```
C++
// ModBusForm::on_pbn_save_clicked → 线程中先
checkSaveModbusPermission 再 writeModbusConfig
CommunicationEngine::instance()-
>enqueueCmd_writeModbusConfig(this, config);
```

```
C++
// ModBusForm 开关点击 → 线程中
FieldBusInterface::checkOtherFieldBusIsActive(0)
CommunicationEngine::instance()
    ->enqueueCmd_CheckOtherFieldBusIsActive(this, ui-
>pbn_swch_rtu, 0);
```

```
C++
// ModBusForm::showEvent / hideEvent →
ResourceInterface::setModbusConnectScheduler
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SetModbusConnectScheduler, true);
```

8.2.3 函数实现

```
C++
// modbusinterface.cpp
int ModbusInterface::writeModbusConfig(CobotModbusParaConfig
```

```

modbusConfig)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    ModbusParaConfig rModbusConfig;
    memcpy(&rModbusConfig, &modbusConfig,
sizeof(ModbusParaConfig));
    int iRet = _IFieldBus
        ->getModbusSwitch()-
>writeModbusConfig(rModbusConfig);

    bool ok = false;
    _IRobotArm->saveFileCommond(&ok);
    if (!ok) {
        qDebug() << __FUNCTION__ << "| save result : " << ok;
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet == 0;
}

int ModbusInterface::readModbusConnStatus(CobotModbusConnectSts
&modbusPara)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    ModbusConnectSts rModbusPara;
    int iRet = _IFieldBus
        ->getModbusSwitch()->readModbusConnStatus(rModbusPara);
    memcpy(&modbusPara, &rModbusPara, sizeof(rModbusPara));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet == 0;
}

bool ModbusInterface::checkModbusOperatePermission()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IFieldBus
        ->getModbusSwitch()->checkModbusOperatePermission();
}

bool ModbusInterface::checkSaveModbusPermission()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

```

```

return _IFieldBus
    ->getModbusSwitch()->checkSaveModbusPermission();
}

```

```

C++
// fieldbusinterface.cpp
bool FieldBusInterface::checkOtherFieldBusIsActive(int
currFieldBus)
{
    return _IFieldBus->checkOtherFieldBusIsActive(
        static_cast<InoRobBusiness::FieldBusType>(currFieldBus));
}

```

```

C++
// resourceinterface.cpp
void ResourceInterface::setModbusConnectScheduler(const bool
scheduler)
{
    _IResource->setModbusConnectScheduler(scheduler);
}

```

8.3 EtherNet/IP (writeEthernetIpConfig / readEthernetIpConnStatus/ checkEthernetIpOperatePermission / setEthernetIPScheduler)

8.3.1 说明

接口	功能	返回值	参数	参数说明
int	写入	int	eipConfig	拷贝为

EthernetIPInterface::writeEthernetIpConfig(CobotEthernetIpPara eipConfig)	EtherNet/IP 从站参数并保存工程			EthernetIpPara 后 getEthernetIPSwitch()->writeEthernetIpConfig
int EthernetIPInterface::readEthernetIpConnStatus(CobotEthernetIpSts &eipPara)	读取激活与连接状态	int	eipPara 出参	—
bool EthernetIPInterface::checkEthernetIPOperatePermission()	操作权限	bool	—	—
bool EthernetIPInterface::checkSaveEthernetIPPermission()	保存权限	bool	—	—
void ResourceInterface::setEthernetIPScheduler(const bool scheduler)	连接状态调度	无	scheduler	

8.3.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_ReadEthernetIpConnStatus);
```

C++

```
CommunicationEngine::instance()->enqueueCmd(this,  
AbstractCmd::CmdType_Communication_GetEthDatas);
```

```
C++  
CommunicationEngine::instance()  
    ->enqueueCmd_CheckOtherFieldBusIsActive(this, ui-  
>switch_ethernetip, 1);
```

```
C++  
CobotEthernetIpPara params;  
params.ActiveCmd = ui->switch_ethernetip->isChecked();  
CommunicationEngine::instance()-  
>enqueueCmd_writeEthernetIpConfig(this, params);
```

```
C++  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType_SetEthernetIPConnectScheduler,  
true);
```

8.3.3 函数实现

```
C++  
// ethernetipinterface.cpp  
int EthernetIPInterface::writeEthernetIpConfig(CobotEthernetIpPara  
eipConfig)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
  
    EthernetIpPara param;  
    memcpy(&param, &eipConfig, sizeof(EthernetIpPara));  
    int iRet = _IFieldBus->getEthernetIPSwitch()-  
>writeEthernetIpConfig(param);  
  
    LOG_INFO(QString("write ethernetip ret  
= %1").arg(QString::number(iRet)));  
  
    bool ok = false;  
    _IRobotArm->saveFileCommond(&ok);
```



```

        if (!ok) {
            qDebug() << __FUNCTION__ << "| save result : " << ok;
        }

        FREQ_LOG_PRINT_TIMESTAMP
        return iRet == 0;
    }

int
EthernetIPInterface::readEthernetIpConnStatus(CobotEthernetIpSts
&eipPara)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    EthernetIpSts ipsts;
    int iRet = _IFieldBus->getEthernetIPSwitch()-
>readEthernetIpConnStatus(ipsts);
    memcpy(&eipPara, &ipsts, sizeof(EthernetIpSts));

    LOG_INFO(QString("read eip, ActiveSts = %1, ConnectSts
= %2").arg(QString::number(eipPara.ActiveSts),
QString::number(eipPara.ConnectSts)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet == 0;
}

bool EthernetIPInterface::checkEthernetIPOperatePermission()
{
    return _IFieldBus->getEthernetIPSwitch()-
>checkEthernetIPOperatePermission();
}

bool EthernetIPInterface::checkSaveEthernetIPPermission()
{
    return _IFieldBus->getEthernetIPSwitch()-
>checkSaveEthernetIPPermission();
}

```

```

C++
// resourceinterface.cpp
void ResourceInterface::setEthernetIPScheduler(const bool
scheduler)

```

```
{
    _IResource->setEthernetIPScheduler(scheduler);
}
```

8.4 EtherCAT （writeEthernetCatConfig/ readEthernetCatConnStatus / checkEthernetCatOperatePermission / checkSaveEthernetCatPermission/ReadEtherCATProp erties/WriteEtherCATEnhance）

8.4.1 说明

接口	功能	返回值	参数	参数说明
int EthernetCatInterface::writeEthernetCatConfig(InoEthcatPara ecatConfig)	写入 EtherCAT 从站参数并保存工程	int	ecatConfig	getEtherCAT Switch()->writeEtherCATSlaveConfig
int EthernetCatInterface::readEthernetCatConnStatus(InoEthcatSts &ecatPara)	读取从站连接/别名等状态	int	ecatPara 出参	—
bool EthernetCatInterface::checkEthernetCatOperatePermission	操作权限	bool	—	—

peratePermission()				
bool EthernetCatInterface::checkSaveEthernetCatPermission()	保存权限	bool	—	—
int EthernetCatInterface::ReadEtherCATProperties(INO_ETHCAT_PROP &buf)	读取 ESC/端口统计/增强开关等属性	int	buf 出参	字段逐一从 ETHCAT_PROP 拷贝
int EthernetCatInterface::WriteEtherCATEnhance(quint16 &ARMSetLinkEnhanceSwitch, quint16 &EtherCATXMLReset)	写入链路增强 / XML 复位相关开关	int	两开关引用	getEtherCATSwitch()->WriteEtherCATEnhance

参数

```
Thrift
typedef struct INO_ETHCAT_PROP
{
    INO_ETHCAT_PROP()
    {
        u8LinkState = 0;
        u8EscState = 0;
        u16AppFaultCode = 0;
        u16AppStateCode = 0;
        u8Port0InvFraCount = 0;
        u8Port0AptErrFraCount = 0;
        u8Port1InvFraCount = 0;
        u8Port1AptErrFraCount = 0;
        u8Port0ForErrCount = 0;
```

```

    u8Port1ForErrCount = 0;
    u8DataFrmProcessingErrCount = 0;
    u8PIDErrCount = 0;
    u8Port0LinkLostCount = 0;
    u8Port1LinkLostCount = 0;
    u16MCUUpSlaveLost = 0;
    u16SlaveAdderss = 0;
    u16ESCVerInfor = 0;
    u16MCUUpXMLVerInfor = 0;
    u16SoftVersion = 0;
    u16ARMSetLinkEnhanSwitch = 0;
    u16EtherCATXMLReset = 0;
}

    quint8 u8LinkState;                /* PHY - link 状态(Bit0 为端
    口 0, Bit1 为端口 1, 0:No Link, 1:Link) */
    quint8 u8EscState;                /* EtherCAT 状态机(1: Init
2: PreOP 4: SafeOP 8: OP) */
    quint16 u16AppFaultCode;          /* 通信故障码 */
    quint16 u16AppStateCode;          /* 应用层状态码 */
    quint8 u8Port0InvFraCount;        /* 端口 0 无效帧计数 */
    quint8 u8Port0AptErrFraCount;     /* 端口 0 接收错误帧计数 */
    quint8 u8Port1InvFraCount;        /* 端口 1 无效帧计数 */
    quint8 u8Port1AptErrFraCount;     /* 端口 1 接收错误帧计数 */
    quint8 u8Port0ForErrCount;        /* [7:0]: 端口 0 转发错误计
数 */
    quint8 u8Port1ForErrCount;        /* [15:8]: 端口 1 转发错误计
数 */
    quint8 u8DataFrmProcessingErrCount; /* [7:0]: 数据帧处理单元错误
计数 */
    quint8 u8PIDErrCount;             /* [15:8]: PDI 错误计数 */
    quint8 u8Port0LinkLostCount;      /* [7:0]: 端口 0 链接丢失计
数 */
    quint8 u8Port1LinkLostCount;      /* [15:8]: 端口 1 链接丢失计
数 */
    quint16 u16MCUUpSlaveLost;        /* MCU 上传从站累积丢帧计数;
ARM 读 */
    quint16 u16SlaveAdderss;          /* MCU 上传主站设置的站点正
名; ARM 读 */
    quint16 u16ESCVerInfor;           /* MCU 上传 ESC 的版本信息;
ARM 读 */

```

```

    quint16 u16MCUUpXMLVerInfor;          /* MCU 上传 XML 版本信息; ARM
读 */
    quint16 u16SoftVersion;               /* MCU 上传软件版本号; ARM
读 */
    quint16 u16ARMSetLinkEnhanSwitch;      //ARM 设定链路增强开关,
ARM 可读可写(0-open; 1-close)
    quint16 u16EtherCATXMLReset;          //ARM 设定执行 XML 复位(0-
open; 1-close)
} INO_ETHCAT_PROP;

typedef struct stInoEthcatPara
{
    stInoEthcatPara()
    {
        ActiveCmd = 0;
        SiteAlias = 0;
        MaxFramLossTimes = 0;
    };

    unsigned char ActiveCmd;               // 0 表示关闭, 1 表示激活
    unsigned short SiteAlias;              // 站点别名 (默认值为 0)
    unsigned short MaxFramLossTimes;       // 最大丢站次数 (默认值为 8
次, 最小值为 8, 最大值为 65535)
} InoEthcatPara;

```

8.4.2 调用

```

C++
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_ReadEtherCatStatus);

```

```

C++
CommunicationEngine::instance()
    ->enqueueCmd_writeEthernetCatConfig(this, params,
    ARMSetLinkEnhanSwitch, EtherCATXMLReset);

```

```

C++
CommunicationEngine::instance()
    ->enqueueCmd_CheckOtherFieldBusIsActive(this, ui-

```

```
>switch_ethernetcat, 2);
```

C++

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType_SetEthercatConnectScheduler, true);
```

8.4.3 函数实现

C++

```
// ethernetcatinterface.cpp  
int EthernetCatInterface::writeEthernetCatConfig(InoEthcatPara  
ecatConfig)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
  
    EthcatPara param;  
    memcpy(&param, &ecatConfig, sizeof(EthcatPara));  
    int iRet = _IFieldBus->getEtherCATSwitch()-  
>writeEtherCATSlaveConfig(param);  
  
    bool ok = false;  
    _IRobotArm->saveFileCommond(&ok);  
    if (!ok) {  
        qDebug() << __FUNCTION__ << "| save result : " << ok;  
    }  
  
    FREQ_LOG_PRINT_TIMESTAMP  
    return iRet == 0;  
}  
  
int EthernetCatInterface::readEthernetCatConnStatus(InoEthcatSts  
&ecatPara)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
  
    EthcatSts ipsts;  
    int iRet = _IFieldBus  
        ->getEtherCATSwitch()->readEtherCATConnStatus(ipsts);  
    memcpy(&ecatPara, &ipsts, sizeof(EthcatSts));  
    LOG_INFO(QString("[readEthernetCatConnStatus]open = %1, conn
```

```

= %2").arg(QString::number(ipsts.ActiveSts),

QString::number(ipsts.ConnectSts)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet == 0;
}

bool EthernetCatInterface::checkEthernetCatOperatePermission()
{
    return _IFieldBus
        ->getEtherCATSwitch()->checkEtherCATOperatePermission();
}

bool EthernetCatInterface::checkSaveEthernetCatPermission()
{
    return _IFieldBus
        ->getEtherCATSwitch()->checkSaveEtherCATPermission();
}

int EthernetCatInterface::ReadEtherCATProperties(INO_ETHCAT_PROP
&buf)
{
    int ret = 0;

    ETHCAT_PROP robotEthcatProp;
    ret = _IFieldBus->getEtherCATSwitch()
        ->ReadEtherCATProperties(robotEthcatProp);
    buf.u8LinkState = robotEthcatProp.u8LinkState;
    buf.u8EscState = robotEthcatProp.u8EscState;
    buf.u16AppFaultCode = robotEthcatProp.u16AppFaultCode;
    buf.u16AppStateCode = robotEthcatProp.u16AppStateCode;
    buf.u8Port0InvFraCount = robotEthcatProp.u8Port0InvFraCount;
    buf.u8Port0AptErrFraCount =
robotEthcatProp.u8Port0AptErrFraCount;
    buf.u8Port1InvFraCount = robotEthcatProp.u8Port1InvFraCount;
    buf.u8Port1AptErrFraCount =
robotEthcatProp.u8Port1AptErrFraCount;
    buf.u8Port0ForErrCount = robotEthcatProp.u8Port0ForErrCount;
    buf.u8Port1ForErrCount = robotEthcatProp.u8Port1ForErrCount;
    buf.u8DataFrmProcessingErrCount =
robotEthcatProp.u8DataFrmProcessingErrCount;
    buf.u8PIDErrCount = robotEthcatProp.u8PIDErrCount;
    buf.u8Port0LinkLostCount =
robotEthcatProp.u8Port0LinkLostCount;

```

```

        buf.u8Port1LinkLostCount =
robotEthcatProp.u8Port1LinkLostCount;
        buf.u16MCUUpSlaveLost = robotEthcatProp.u16MCUUpSlaveLost;
        buf.u16SlaveAdderss = robotEthcatProp.u16SlaveAdderss;
        buf.u16ESCVerInfor = robotEthcatProp.u16ESCVerInfor;
        buf.u16MCUUpXMLVerInfor = robotEthcatProp.u16MCUUpXMLVerInfor;
        buf.u16SoftVersion = robotEthcatProp.u16SoftVersion;
        buf.u16ARMSetLinkEnhanSwitch =
robotEthcatProp.u16ARMSetLinkEnhanSwitch;
        buf.u16EtherCATXMLReset = robotEthcatProp.u16EtherCATXMLReset;

LOG_INFO(QString("[ReadEtherCATProperties]ARMSetLinkEnhanSwitch
= %1, EtherCATXMLReset = %2")
        .arg(QString::number(robotEthcatProp.u16ARMSetLin
kEnhanSwitch),
QString::number(robotEthcatProp.u16EtherCATXMLReset)));

        LOG_INFO(QString("[ReadEtherCATProperties]ret
= %1").arg(QString::number(ret)));

        return ret;
    }

int EthernetCatInterface::WriteEtherCATEnhan(
    quint16 &ARMSetLinkEnhanSwitch, quint16 &EtherCATXMLReset)
{
    int ret = 0;

    LOG_INFO(QString("[WriteEtherCATEnhan]ARMSetLinkEnhanSwitch
= %1, EtherCATXMLReset = %2")
        .arg(QString::number(ARMSetLinkEnhanSwitch),
QString::number(EtherCATXMLReset)));
    ret = _IFieldBus->getEtherCATSwitch()
        ->WriteEtherCATEnhan(ARMSetLinkEnhanSwitch,
EtherCATXMLReset);

    LOG_INFO(QString("[WriteEtherCATEnhan]ret
= %1").arg(QString::number(ret)));

    return ret;
}

```



```
C++
// resourceinterface.cpp
void ResourceInterface::setEtherCatScheduler(const bool scheduler)
{
    _IResource->setEtherCatScheduler(scheduler);
}
```

8.5 主站总线 — 配置导入完成通知 (sendMasterConfigFile)

8.5.1 说明

接口	功能	返回值	参数	参数说明
void FieldBusInterface::sendMasterConfigFile(AbstractCmd *cmd)	校验解压后的主站组态文件列表，匹配控制器后下发至控制器并通知导入完成	无	cmd 中带 zipFileList、temDesFilePath (BIND 宏解包)	成功/失败通过 signal_sendMasterConfigZipFile_result 回传提示

8.5.2 调用

```
C++
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SendMasterConfigZipFile,
    zipFileList, temDesFilePath);
```

8.5.3 函数实现

```

C++
// fieldbusinterface.cpp
void FieldBusInterface::sendMasterConfigFile(AbstractCmd *cmd)
{
    auto[zipFileList, temDesFilePath] = BIND(cmd, const
QStringList, const QString);
    QString msg;

    bool bCanOpenFile = false;           // 是否存在 CanOpen 文件
    bool bDeviceNetFile = false;         // 是否存在 DeviceNet 文件
    bool bFollowCraftCaliCfg = false;    // 是否存在视觉一拖多文件
    bool bFBusMasterParaCfg = false;     // 是否存在主站总线配置必须
文件
    bool bEip = false;                   // 是否存在 Eip 文件

    if (zipFileList.size() == 0) {
        return;
    }

    // 检测压缩包所包含的文件
    for (size_t i = 0; i < zipFileList.size(); i++)
    {
        if (zipFileList[i] == CANOPEN_FILENAMME)
        {
            bCanOpenFile = true;
        }

        if (zipFileList[i] == DEVICENET_FILENAME)
        {
            bDeviceNetFile = true;
        }

        if (zipFileList[i] == ETHERNETIP_FILENAME)
        {
            bEip = true;
        }

        if (zipFileList[i] == FOLLOWCRAFT_FILENAME)
        {
            bFollowCraftCaliCfg = true;
        }
    }
}

```

```

        if (zipFileList[i] == FBUSMASTERPARACFG_FILENAME)
        {
            bFBusMasterParaCfg = true;
        }

        // 判断导入的组态文件是否匹配当前控制器
        if (zipFileList[i].contains(MASTER_FILE_NAME_SUFFIX))
        {
            string masterFile = zipFileList[i].toStdString();
            if (_IFieldBus->getEtherCATSwitch()-
>isMatchedCurrentBox(masterFile) != ERR_OK)
            {
                msg = CommTr::tr("The imported configuration file
does not match the current controller.");
                CommunicationEngine::instance()-
>signal_sendMasterConfigZipFile_result(cmd->m_object, false, msg);
                return;
            }
        }

        if (!bFBusMasterParaCfg)
        {
            msg = CommTr::tr("The main station bus configuration file
is missing necessary files, and the import failed.");
            CommunicationEngine::instance()->
                signal_sendMasterConfigZipFile_result(cmd->m_object,
false, msg);
            return;
        }

        if (bDeviceNetFile)
        {
            // 检验文件的 json 格式是否正确
            string deviceFilePath = (temDesFilePath + "/" +
DEVICENET_FILENAME).toStdString();
            if (!_IFieldBus->getEtherCATSwitch()-
>checkDeviceNetFile(deviceFilePath))
            {
                msg = CommTr::tr("The JSON format of the file is
incorrect.");
                CommunicationEngine::instance()->
                    signal_sendMasterConfigZipFile_result(cmd-
>m_object, false, msg);
            }
        }
    }
}

```

```

        return;
    }
}

// 放置文件至控制器
int nRet = _IFieldBus->putFieldBusMasterCfgToController(temDesFilePath.toStdString(),
bDeviceNetFile, bCanOpenFile, bFollowCraftCaliCfg, bEip);
if (nRet != ERR_OK)
{
    msg = CommTr::tr("Failed to put the file to the
controller.");
    CommunicationEngine::instance()->
        signal_sendMasterConfigZipFile_result(cmd->m_object,
false, msg);
    return;
}

// 通知控制器组态配置文件已导入
nRet = _IFieldBus->networkFieldBusImportWrite();
if (nRet != ERR_OK) {
    msg = CommTr::tr("The import of the main station bus
configuration failed. Please try again.");
    CommunicationEngine::instance()->
        signal_sendMasterConfigZipFile_result(cmd->m_object,
false, msg);
}

    msg = CommTr::tr("The main station bus configuration has been
successfully imported, and a restart of the controller will take
effect");
    CommunicationEngine::instance()->
        signal_sendMasterConfigZipFile_result(cmd->m_object, true,
msg);

    // 通知 ethercat 解析文件
}

```

8.6 IO 映射 (refreshFieldbusFromControllerDatas /

getFieldbusIOMapDatas)

8.6.1 说明

接口	功能	返回值	参数	参数说明
bool PeripheralCon figInterface::re freshFieldbus FromControlle rDatas()	通知控制器侧 刷新 IO 映射 缓存	bool	—	_IFieldBus- >getIOMap()- >refreshField busIOMap()
bool PeripheralCon figInterface::g etFieldbusIO MapDatas(QL ist<Ino_Fieldb usIOMapData > &data)	读取总线 IO 映射条目（功 能码、变量 名、映射地址 等）	bool	data 出参	getIOMap()- >getFieldbusI OMapDatas() 转换为 TP 侧 列表

```
C++
#pragma once
#include <QString>

enum Ino_IOType
{
    IOTP_IN = 0,    // input
    IOTP_OUT      // output
};

enum Ino_MemoryLength : unsigned int
{
    ML_BIT = 1,     // Bit
    ML_CHAR = 4,    // Char
    ML_BYTE = 8,    // Byte
    ML_WORD = 16,   // Word
    ML_DWORD = 32  // DWord
};

struct Ino_FieldbusIOMapData
```

```

{
    Ino_IOType ioType = Ino_IOType::IOTP_IN;
// IO 类型
    unsigned int funcId = 0; // funID
    int memAddr = -1; // 内存地址
    Ino_MemoryLength length = Ino_MemoryLength::ML_BIT;
// 内存长度
    QString name;

    ...
};

enum IOMappingCmdType{
    ResetIOMapping = 0,
    ImportIOMapping = 1,
    ExportIOMapping = 2,
    SaveDatas = 3,
};

```

8.6.2 调用

```

C++
// IOMappingTableModel::slot_startRefreshValues(true)
CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_Refresh_IOMapping_Values_FromController);
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Get_IOMapping_Values, &m_datas);

```

8.6.3 函数实现

```

C++
// peripheralconfiginterface.cpp
bool
PeripheralConfigInterface::getFieldbusIOMapDatas(QList<Ino_FieldbusIOMapData> &data)
{

```

```

FREQ_LOG_PRINT_TIMESTAMP_THREAD

std::vector<InoRobBusiness::FieldbusIOMapData> values
    = _IFieldBus
        ->getIOMap()
        ->getFieldbusIOMapDatas();
for (size_t i = 0; i < values.size(); ++i) {

data.push_back(Ino_FieldbusIOMapData((Ino_IOType)values[i].ioType,
                                     values[i].funcId,
                                     values[i].memAddr,

(Ino_MemoryLength)values[i].length,

QString::fromStdString(values[i].name)));
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool
PeripheralConfigInterface::refreshFieldbusFromControllerDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IFieldBus
        ->getIOMap()
        ->refreshFieldbusIOMap();
}

```

8.7 IO 映射 (saveIOMappingData)

8.7.1 说明

接口	功能	返回值	参数	参数说明
bool PeripheralCon	将表格中的映射写入控制器	bool	data 全量条目	转为 std::vector<Fi

figInterface::saveIOMappingData(const QVector<Ino_FieldbusIOMapData> &data)				eldbusIOMapData> 后 saveFieldbusIOMap
---	--	--	--	---

Ino_FieldbusIOMapData 同 8.6.1

8.7.2 调用

```
C++
// IOMappingTableModel::saveDatas (本地校验通过后)
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Save_IOMapping_Values, m_datas);
```

8.7.3 函数实现

```
C++
// peripheralconfiginterface.cpp
bool PeripheralConfigInterface::saveIOMappingData(
    const QVector<Ino_FieldbusIOMapData> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    std::vector<InoRobBusiness::FieldbusIOMapData> values;
    for (int i = 0; i < data.size(); ++i) {
        values.push_back(InoRobBusiness::FieldbusIOMapData(
            (InoRobBusiness::IOType)data[i].ioType,
            data[i].funcId,
            data[i].memAddr,
            (InoRobBusiness::MemoryLength)data[i].length,
            data[i].name.toStdString()));
    }
    return _IFieldBus
        ->getIOMap()
        ->saveFieldbusIOMap(values);
}
```

8.8 IO 映射 (saveIOMappingRemarks)

8.8.1 说明

接口	功能	返回值	参数	参数说明
void PeripheralConfigInterface::saveIOMappingRemarks(AbstractCmd *cmd)	保存映射成功后，可选将变量名同步到当前激活工程的 I/O 说明（标签）	无	cmd 中带 QVector<Ino_FieldbusIOMapData>	依赖已加载工程且后缀非 .pro；经 ILabel 修改描述并保存

8.8.2 调用

```
C++
// IOMappingTableModel::saveRemarks（保存成功弹框选「是」后）
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Save_IOMapping_Remarks, m_datas);
```

8.8.3 函数实现

```
C++
// peripheralconfiginterface.cpp
void PeripheralConfigInterface::saveIOMappingRemarks(AbstractCmd
*cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    auto [datas] = ((CmdDatas<QVector<Ino_FieldbusIOMapData>>
*)cmd)->m_data;
```

```

std::string name;
int re = comm()->project()->getActiveProject(name);
std::string suffix = comm()->project()-
>ProgramFileExtension();
// 函数返回值为 ERR_OK, 及获取到 string 不为空代表有激活的工程
if (re != ERR_OK || name.empty() || suffix == "pro"){
    emit CommunicationEngine::instance()->
        signal_needMainWidgetWarning(QObject::tr(
            "No active project available. Please load a project
and save it before attempting this operation."));
    return;
}

ILabel *lable = comm()->project()->GetLabel();
int size = datas.size();
int modify = InoRobUtil::OK;
for (int i = 0; i < size; i++)
{
    if (datas[i].memAddr >= 0)
    {
        if (datas[i].ioType == Ino_IOTYPE::IOTP_IN)
        {
            if (datas[i].length == Ino_MemoryLength::ML_BIT)
            {
                lable-
>removeDescription(LabelType::IO_INPUT_BIT,
datas[i].name.toStdString());
                modify = lable-
>ModifyItemDescription(LabelType::IO_INPUT_BIT, datas[i].memAddr,
datas[i].name.toStdString());
                if(modify != InoRobUtil::OK){
                    break;
                }
            }
            else if (datas[i].length ==
Ino_MemoryLength::ML_WORD)
            {
                lable-
>removeDescription(LabelType::IO_INPUT_WORD,
datas[i].name.toStdString());
                modify = lable-
>ModifyItemDescription(LabelType::IO_INPUT_WORD, datas[i].memAddr
/ 16, datas[i].name.toStdString());
                if(modify != InoRobUtil::OK){

```

```

        break;
    }
}
else if (datas[i].ioType == Ino_IOType::IOTP_OUT)
{
    if (datas[i].length == Ino_MemoryLength::ML_BIT)
    {
        lable-
>removeDescription(LabelType::IO_OUTPUT_BIT,
datas[i].name.toStdString());
        modify = lable-
>ModifyItemDescription(LabelType::IO_OUTPUT_BIT, datas[i].memAddr,
datas[i].name.toStdString());
        if(modify != InoRobUtil::OK){
            break;
        }
    }
    else if (datas[i].length ==
Ino_MemoryLength::ML_WORD)
    {
        lable-
>removeDescription(LabelType::IO_OUTPUT_WORD,
datas[i].name.toStdString());
        modify = lable-
>ModifyItemDescription(LabelType::IO_OUTPUT_WORD, datas[i].memAddr
/ 16, datas[i].name.toStdString());
        if(modify != InoRobUtil::OK){
            break;
        }
    }
}
}
if(modify == InoRobUtil::OK){
    modify = lable->SaveLabels();
    if(modify == InoRobUtil::OK){
        emit CommunicationEngine::instance()
            ->signal_IOMapping_cmd_result(cmd->m_cmdType,
true);
        emit CommunicationEngine::instance()->
            signal_needMainWidgetInfo(QObject::tr("Description
synchronized to the current project successfully.));
    }else{

```


8.9.2 调用

```
C++
// IOMappingForm::slot_import
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Import_IOMapping_Files, fileName);
```

```
C++
// IOMappingForm::on_pbn_export_clicked
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Export_IOMapping_Files, fileName);
```

8.9.3 函数实现

```
C++
// peripheralconfiginterface.cpp
bool PeripheralConfigInterface::importIOMappingFile(const QString
&fileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IFieldBus
        ->getIOMap()
        ->importFieldbusIoMapFromFile(fileName.toStdString());
}

bool PeripheralConfigInterface::exportIOMappingFile(const QString
&fileName)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IFieldBus
        ->getIOMap()
        ->exportFieldbusIoMapToFile(fileName.toStdString());
}
```

8.10 IO 映射 (resetFieldbusIOMap)

8.10.1 说明

接口	功能	返回值	参数	参数说明
bool PeripheralConfigInterface::resetFieldbusIOMap(Ino_IOMapResetMethod method)	按策略重置 IO 映射	bool	method	转为 InoRobBusiness::IOMapResetMethod 后 resetFieldbusIOMap

```
C++
#pragma once

#include <QString>
#include <QColor>
#include <QVariant>

// 总线
const int STANDARD_IO_START_INDEX = 0;    // 标准 IO 起始下标(以 byte 参考)
const int STANDARD_IO_COUNT = 16;         // 标准 IO 大小(以 byte 参考)
const int TOOL_IO_START_INDEX = 16;       // 协作工具 IO 起始下标(以 byte 参考)
const int TOOL_IO_COUNT = 2;              // 协作工具 IO 大小(以 byte 参考)
const int FIELDBUS_IO_START_INDEX = 64;   // 现场总线 IO 起始下标(以 byte 参考)
const int FIELDBUS_IO_COUNT = 512;        // 现场总线 IO 大小(以 byte 参考)
const int MEMORY_IO_START_INDEX = 1600;   // 内存 IO 起始下标(以 byte 参考)
const int MEMORY_IO_COUNT = 128;         // 内存 IO 大小(以 byte 参考)
```

```

enum InoIOType {
    InoIOType_CommonIO = 0, // 常用 IO(设置了标签或者备注)
    InoIOType_StandardIO, // 标准 IO
    InoIOType_FieldbusIO, // 现场总线 IO
    InoIOType_MemoryIO, // 内存 IO
    InoIOType_ToolIO, // 末端 IO
    InoIOTypeSize
};

enum Ino_IOMapResetMethod {
    UNKNOW_IO = 0, // 未知 IO
    DIGITAL_IO = 1, // 标准 IO
    FIELDBUS_IO = 2, // 从站 IO
    MEMORY_IO = 3, // 内存 IO
    SYSTEM_IO = 4, // 系统 IO
    FIELDBUS_MASTER_IO = 5, // 主站 IO
};

enum BaseType {
    BaseType_Decimal = 0, // 十进制
    BaseType_Binary, // 二进制
    BaseType_Hexadecimal, // 十六进制
};

enum ShowType {
    ShowType_Bit = 0, // bit
    ShowType_Byte = 1, // 字节
    ShowType_Word = 2, // 字
    ShowType_Size = 3
};

#ifdef INOCOBOTTP_MSVC_QT5
Q_DECLARE_METATYPE(ShowType)
#endif

```

8.10.2 调用

```

C++
// IOMappingForm::on_pbn_default_clicked (示例: 现场总线默认)
CommunicationEngine::instance()->enqueueCmd_getData(

```

```
this, AbstractCmd::CmdType_Reset_IOMapping_Values,
FIELDBUS_IO);
```

```
C++
// IOMappingForm::resetWithMethod
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Reset_IOMapping_Values,
    (Ino_IOMapResetMethod)(method));
```

8.10.3 函数实现

```
C++
// peripheralconfiginterface.cpp
bool
PeripheralConfigInterface::resetFieldbusIOMap(Ino_IOMapResetMethod
method)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IFieldBus
        ->getIOMap()
        -
>resetFieldbusIOMap((InoRobBusiness::IOMapResetMethod)method);
}
```

8.11 夹持器 (enableGripperThreePosition)

8.11.1 说明

接口	功能	返回值	参数	参数说明
bool RobotControll nterface::enab leGripperThre ePosition(bool enable)	设置夹持器三 位置使能开关	bool (ret == ERROR_OK)	enable: true 开启 / false 关 闭	调用 _IControl- >SetGripperT hreePositionE nable(enable)

8.11.2 调用

```
C++
// GripperForm::on_pbn_enableGripperThreePosition_clicked (节选; 可选编译条件下会先校验安全监控等)
CommunicationEngine::instance()
    ->enqueueCmd_enableGripperThreePosition(this, checked);
```

8.11.3 函数实现

```
C++
// robotcontrolinterface.cpp
bool RobotControlInterface::enableGripperThreePosition(bool enable)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = _IControl->SetGripperThreePositionEnable(enable);
    FREQ_LOG_PRINT_TIMESTAMP
    return (ret == ERROR_OK);
}
```

8.12 地址分配-总线内存分配 (getAddressAssignMsg/setAddressAssignMsg)

8.12.1 说明

接口	功能	返回值	参数	参数说明
void FieldBusInterface::getAddressAssignMsg(AbstractCmd	读取当前现场总线类型、内存分配配置及Modbus 地址	无	cmd	readMemoryAssignConfig + readModbusConnStatus + GetBusInstall

*cmd)	类型、总线安装标志；向 UI 发 signal_get_fieldBusAddress AssignResult ； 无 EtherCAT 硬件时可能下发 关闭 EtherCAT 从站配置			； 结构体尺寸 不一致时告警
void FieldBusInterface::setAddressAssignMsg(AbstractCmd *cmd)	将 INO_FIELDBUS_MEM_ASSIGN_CONFIG 写入控制器 并保存工程文件	无	cmd 中带配置	writeMemoryAssignConfig + saveFileCommand;

```

C++
// 总线类型
typedef enum
{
    INO_FB_Unknown = 0,
    INO_FB_Modbus = 1,
    INO_FB_EthernetIp = 2,
    INO_FB_EtherCATSlave = 3,
    INO_FB_FINS = 4,
    INO_FB_MC = 5,
    INO_FB_Profinet = 6,
} INO_E_FieldBusType;

// 现场总线从站配置信息
typedef struct INO_MODBUS_ADDRASSIGN_CONFIG
{
    INO_MODBUS_ADDRASSIGN_CONFIG()
    {
        inputSize = 256;
        outputSize = 256;
    }
}

```

```

        modbusInputCoilSize = 20;
        modbusInputRegSize = 236;
        modbusOutputCoilSize = 20;
        modbusOutputRegSize = 236;
    };
    unsigned short inputSize;
    unsigned short outputSize;
    unsigned short modbusInputCoilSize;
    unsigned short modbusInputRegSize;
    unsigned short modbusOutputCoilSize;
    unsigned short modbusOutputRegSize;
} INO_ModbusAddrassignConfig;

```

```

struct INO_EIP_ADDRASSIGN_CONFIG
{
    INO_EIP_ADDRASSIGN_CONFIG()
    {
        inputSize = 256;
        outputSize = 256;
    };
    unsigned short inputSize;
    unsigned short outputSize;
};

```

```

struct INO_ETHERCAT_ADDRASSIGN_CONFIG
{
    INO_ETHERCAT_ADDRASSIGN_CONFIG()
    {
        inputSize = 256;
        outputSize = 256;
    };
    unsigned short inputSize;
    unsigned short outputSize;
};

```

```

struct INO_MC_ADDRASSIGN_CONFIG
{
    INO_MC_ADDRASSIGN_CONFIG()
    {
        inputSize = 256;
        outputSize = 256;
        mcInputStartAddr = 1025;
        mcInputRegSize = 2025;
        mcOutputStartAddr = 256;
    };
};

```

```

        mcOutputRegSize = 256;
    };
    unsigned short inputSize;
    unsigned short outputSize;
    unsigned short mcInputStartAddr;
    unsigned short mcInputRegSize;
    unsigned short mcOutputStartAddr;
    unsigned short mcOutputRegSize;
};

struct INO_PN_ADDRASSIGN_CONFIG
{
    INO_PN_ADDRASSIGN_CONFIG()
    {
        inputSize = 127;
        outputSize = 127;
    }
    unsigned short inputSize;
    unsigned short outputSize;
};

struct INO_FIELDBUS_MEM_ASSIGN_CONFIG
{
    INO_FIELDBUS_MEM_ASSIGN_CONFIG()
    {
        memset(Version, 0, sizeof(char) * 8);
        for (int i = 0; i < 4; i++)
        {
            INO_MC_ADDRASSIGN_CONFIG mc;
            mcConfig[i] = mc;
        }
    };
    char Version[8];
    INO_MODBUS_ADDRASSIGN_CONFIG modbusConfig;
    INO_EIP_ADDRASSIGN_CONFIG eipConfig;
    INO_ETHERCAT_ADDRASSIGN_CONFIG etherCATConfig;
    INO_MC_ADDRASSIGN_CONFIG mcConfig[4];
    INO_PN_ADDRASSIGN_CONFIG pnConfig;
};

```

8.12.2 调用

```

C++
// AddressAssignForm::updateUI / showEvent
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_GetFieldBusAddressAssignConfig);

```

```

C++
// AddressAssignForm::on_pbn_save_clicked (各总线分支完成
DataParsing 后)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SetFieldBusAddressAssignConfig,
    _fieldBusConfig);

```

8.12.3 函数实现

```

C++
// fieldbusinterface.cpp
void FieldBusInterface::getAddressAssignMsg(AbstractCmd *cmd)
{
    int fieldBusType;
    FIELDBUS_MEM_ASSIGN_CONFIG configTemp;
    ModbusConnectSts modbusSts;
    int ret1 = _IFieldBus->getCurFieldBusType(fieldBusType);
    int ret2 = _IFieldBus->readMemoryAssignConfig(configTemp);
    int ret3 = _IFieldBus->getModbusSwitch()-
>readModbusConnStatus(modbusSts);

    char filedBusInstallType[4]; // 总线安装标志
    _IMonitor->GetBusInstall(filedBusInstallType);
    if (ret1 != ERR_OK || ret2 != ERR_OK || ret3 != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("Failed to get peripheral
configuration."));
        return;
    }
    INO_FIELDBUS_MEM_ASSIGN_CONFIG config;
    if (sizeof(INO_FIELDBUS_MEM_ASSIGN_CONFIG) ==
sizeof(FIELDBUS_MEM_ASSIGN_CONFIG)) {
        memcpy(&config, &configTemp,

```

```

sizeof(INO_FIELDBUS_MEM_ASSIGN_CONFIG));
    bool isHasEtherCat = (filedBusInstallType[0] >> 0) & 0x01;
    emit CommunicationEngine::instance()-
>signal_get_fieldBusAddressAssignResult(
        cmd->m_object,
        fieldBusType,
        config,
        modbusSts.ModbusAddrType == 1,
        isHasEtherCat);
    if (!isHasEtherCat) {
        EthcatPara ethcatPara;
        ethcatPara.ActiveCmd = 0;
        ethcatPara.SiteAlias = 1;
        ethcatPara.MaxFramLossTimes = 8;
        // 关闭
        int nRet = _IFieldBus->getEtherCATSwitch()-
>writeEtherCATSlaveConfig(ethcatPara);
        if (nRet != ERR_OK) {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                CommTr::tr("Failed to save EtherCAT
parameters."));
            return;
        }
    } else
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("An error occurred during data copying due
to inconsistent size of the structure."));
}

void FieldBusInterface::setAddressAssignMsg(AbstractCmd *cmd)
{
    auto [config] = ((CmdDatas<INO_FIELDBUS_MEM_ASSIGN_CONFIG>
*)cmd)->m_data;
    if (sizeof(INO_FIELDBUS_MEM_ASSIGN_CONFIG) !=
sizeof(FIELDBUS_MEM_ASSIGN_CONFIG)) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("An error occurred during data copying due
to inconsistent size of the structure."));
        return;
    } else {

```

```

        FIELDBUS_MEM_ASSIGN_CONFIG configTemp;
        memcpy(&configTemp, &config,
sizeof(INO_FIELDBUS_MEM_ASSIGN_CONFIG));

        if (_IRobotArm->checkFileSaveFlag() == false) {
            return;
        }
        int nRet = _IFieldBus-
>writeMemoryAssignConfig(configTemp);
        if (nRet != ERR_OK) {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                CommTr::tr("Failed to save address allocation
configuration."));
            return;
        } else {
            bool ok = false;
            _IRobotArm->saveFileCommond(&ok);
            if (!ok)
                emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                    CommTr::tr("Failed to save file."));
            else{
                emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                    CommTr::tr("The bus configuration has been
modified and takes effect upon system reboot."));
                PRINT_MSG(CommTr::tr("Bus address allocation
config successfully sent to controller."));
            }
        }
    }
}

```

8.13 工程序号配置

(getProjectNoAssign/setProjectNoAssign)

8.13.1 说明

接口	功能	返回值	参数	参数说明
void FieldBusInterface::getProjectNoAssign(AbstractCmd *cmd)	从控制器下载工程序号配置文件，解析后与工作站工程列表合并，发 signal_get_projectNoConfig	无	cmd	getProjectIndexConfigObject(); downloadConfigFile → analysisConfigFile → ReadProjectList
void FieldBusInterface::setProjectNoAssign(AbstractCmd *cmd)	按表格重写IndexConfig并保存至本地再上传控制器	无	cmd 中带 QList<int>、 QStringList	delConfig / addConfig / modifyConfig 后 saveConfigFile

8.13.2 调用

```
C++
// ProjectNoConfigTableModel::operationWhenShow
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType::CmdType_getProjectNoConfig);
```

```
C++
// ProjectNoConfigTableModel::saveConfig (校验通过后)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_setProjectNoConfig,
    m_listIndexConfiged, m_listProjectConfiged);
```

8.13.4 函数实现

```
C++
// fieldbusinterface.cpp
void FieldBusInterface::getProjectNoAssign(AbstractCmd *cmd)
{
    InoRobBusiness::IIndexConfig *config = _IPeripheral-
```



```

>getProjectIndexConfigObject();
    config->initConfigArr();
    bool isExisted = false;
    int ret = config->downloadConfigFile(isExisted, config-
>configFileLocalPath(), config->configFileCtrlPath());
    if (ret != ERROR_OK && ret != -4) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("Failed to get config file."));
        return;
    }
    QStringList projectConfiged;
    QList<int> indexConfiged;
    QStringList projectsAll;

    std::vector<ProjectFolderInfo> prjInofs;
    int nRet = InoRobBusiness::gs_Workstation.GetObject()-
>GetProject()->ReadProjectList(prjInofs, _IConnection->GetIp());
    if (nRet != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("Failed to get project list."));
        return;
    }
    unsigned int size = prjInofs.size();
    for (int i = 0; i < size; ++i) {

projectsAll.push_back(QString::fromStdString(prjInofs[i].projectNa
me));
    }

    if (isExisted) {
        int ret2 = config->analysisConfigFile(config-
>configFileLocalPath());
        if (ret2 == ERROR_OK) {
            std::vector<InoRobBusiness::IndexConfig> configs =
config->getConfigArr();
            unsigned int size = configs.size();
            for (int i = 0; i < size; ++i) {

projectConfiged.push_back(QString::fromStdString(configs[i].name))
;
                indexConfiged.push_back(configs[i].index);
            }
        }
    }

```

```

        } else {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                CommTr::tr("Failed to parse the config file."));
            return;
        }
    }
    if (indexConfiged.size() == projectConfiged.size())
        emit CommunicationEngine::instance()-
>signal_get_projectNoConfig(cmd->m_object,

indexConfiged,

projectConfiged,

projectsAll);
    // qDebug() << indexConfiged << projectConfiged <<
indexAvilable << projectsAll;
}

void FieldBusInterface::setProjectNoAssign(AbstractCmd *cmd)
{
    InoRobBusiness::IIndexConfig *instance = _IPeripheral-
>getProjectIndexConfigObject();
    auto [indexConfiged, projectConfiged] = ((CmdDatas<QList<int>,
QStringList> *)cmd)->m_data;
    std::vector<InoRobBusiness::IndexConfig> configs = instance-
>getConfigArr();
    int oldSize = configs.size();
    int newSize = indexConfiged.size();
    for (int i = 0; i < oldSize; ++i) {
        instance->delConfig(configs[i]);
    }
    int ret = 0;
    for (int i = 0; i < newSize; ++i) {
        instance->addConfig();
        InoRobBusiness::IndexConfig temp;
        temp.index = indexConfiged[i];
        std::string projetTemp = projectConfiged[i].toString();
        temp.name = std::string(projetTemp.c_str());
        configs = instance->getConfigArr();
        if(temp.name != configs[i].name || temp.index !=
configs[i].index)
            ret = instance->modifyConfig(configs[i], temp);
    }
}

```

```

        if(ret != ERR_OK){
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                CommTr::tr("Failed to save the config file."));
            return;
        }
    }
    if (instance->saveConfigFile(instance->configFileLocalPath(),
instance->configFileCtrlPath()) != ERROR_OK)
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("Failed to save the config file."));
}

```

8.14 点文件序号配置 (getPointFileNoAssign/setPointFileNoAssign)

8.14.1 说明

接口	功能	返回值	参数	参数说明
void FieldBusInterface::getPointFileNoAssign(AbstractCmd *cmd)	下载点文件序号配置，解析后与当前工程点文件列表合并	无	cmd	GetProject()->GetInfo().robPointFileArr 提供可选点文件全集
void FieldBusInterface::setPointFileNoAssign(AbstractCmd *cmd)	设置点文件序号配置	无	cmd	

8.14.2 调用

```
C++
// PointNoConfigTableModel::operationWhenShow
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType::CmdType_getPointFileNoConfig);
```

```
C++
// PointNoConfigTableModel::saveConfig (校验通过后)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_setPointFileNoConfig,
    m_listIndexConfiged, m_listProjectConfiged);
```

8.14.4 函数实现

```
C++
// fieldbusinterface.cpp
void FieldBusInterface::getPointFileNoAssign(AbstractCmd *cmd)
{
    InoRobBusiness::IIndexConfig *config = _IPeripheral-
>getRPFileIndexConfigObject();
    config->initConfigArr();
    bool isExisted = false;
    int ret = config->downloadConfigFile(isExisted, config-
>configFileLocalPath(), config->configFileCtrlPath());
    if (ret != ERROR_OK && ret != -4) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
            CommTr::tr("Failed to get config file."));
        return;
    }
    QStringList pointfileConfiged;
    QList<int> indexConfiged;
    QStringList pointsFileAll;

    std::vector<ProjectFolderInfo> prjInofs;
    std::vector<string> files =
    InoRobBusiness::gs_Workstation.GetObject()->GetProject()-
>GetInfo().robPointFileArr;
```

```

        unsigned int size = files.size();
        for (int i = 0; i < size; ++i) {
            pointsFileAll.push_back(QString::fromStdString(files[i]));
        }

        if (isExisted) {
            int ret2 = config->analysisConfigFile(config-
>configFileLocalPath());
            if (ret2 == ERROR_OK) {
                std::vector<InoRobBusiness::IndexConfig> configs =
config->getConfigArr();
                unsigned int size = configs.size();
                for (int i = 0; i < size; ++i) {

pointfileConfiged.push_back(QString::fromStdString(configs[i].name
));

                indexConfiged.push_back(configs[i].index);
            }
        } else {
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                CommTr::tr("Failed to parse the config file."));
            return;
        }
    }
    if (indexConfiged.size() == pointfileConfiged.size())
        emit CommunicationEngine::instance()-
>signal_get_PointNoConfig(cmd->m_object,

indexConfiged,

pointfileConfiged,

pointsFileAll);
    // qDebug() << indexConfiged << indexConfiged <<
pointsFileAll;
}

void FieldBusInterface::setPointFileNoAssign(AbstractCmd *cmd)
{
    InoRobBusiness::IIndexConfig *instance = _IPeripheral-
>getRPFileIndexConfigObject();
    auto [indexConfiged, projectConfiged] = ((CmdDatas<QList<int>,
QStringList> *)cmd)->m_data;

```

```

        std::vector<InoRobBusiness::IndexConfig> configs = instance-
>getConfigArr();
        int oldSize = configs.size();
        int newSize = indexConfiged.size();
        for (int i = 1; i < oldSize; ++i) {
            instance->delConfig(configs[i]);
        }

        int ret = 0;
        for (int i = 1; i < newSize; ++i) {
            instance->addConfig();
            InoRobBusiness::IndexConfig temp;
            temp.index = indexConfiged[i];
            std::string projectTemp =
projectConfiged[i].toStdString();
            temp.name = std::string(projectTemp.c_str());
            configs = instance->getConfigArr();
            if(temp.name != configs[i].name || temp.index !=
configs[i].index) {
                ret = instance->modifyConfig(configs[i], temp);
                if(ret != ERR_OK){
                    emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                        CommTr::tr("Failed to save the config
file."));
                    return;
                }
            }
        }

        if (instance->saveConfigFile(instance->configFileLocalPath(),
instance->configFileCtrlPath()) != ERROR_OK)
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(
                CommTr::tr("Failed to save the config file."));
    }
}

```

9. 机器人姿态

9.1 绝对零点 (setAbsoluteZeroPoint /

getAbsoluteZeroPoint / getAbsoluteZeroCurrentValue/resetAbsoluteZeroPoint ByAxisNo/getResetAbsoluteZeroPointByAxisNo)

9.1.1 说明

接口	功能	返回值	参数	参数说明
void RobotPointInterface::setAbsoluteZeroPoint(AbstractCmd *absCmd)	将各轴绝对零点写入控制器并记日志	无	absCmd	((CmdDatas<QList<int>> *) absCmd)->m_data; 先经 StatusChecks::cobotCheck (示教器、手动低速、下使能等); _IRobotArm->saveAbsZeroValue
bool RobotPointInterface::getAbsoluteZeroPoint(QList<double> &pos, QList<int> &min, QList<int> &max)	读取当前轴绝对零点值及允许范围	bool	pos/min/max 出参	_IRobotArm->getAbsZeroValue; 范围由 IRobotParamRange 读取结构名 AbsZeroPoint、参数名 J1...J6 (整型 min/max, 界面与示教一致为脉冲量级)
void RobotPointInterface::getAbsoluteZeroCurrentValue(AbstractCmd *absCmd)	读当前编码器 / 电机位置并发出 signal_getAbsoluteZeroCurrent_result	无	absCmd	BIND (cmd, int): index == -1 时 UI 更新整表; _IRobotArm->getCurrMotorPos
void RobotPointInterface::re	对指定轴发起伺服零点	无	cmd	BIND (cmd, int) 轴号; _IRobotArm-

setAbsoluteZeroPointByAxisNo(AbstractCmd *cmd)	复位			>resetAbsZeroValue; 结果经 signal_resetAbsoluteZero_result
void RobotPointInterface::getResetAbsoluteZeroPointByAxisNo(AbstractCmd *cmd)	查询伺服零点复位进度 / 结果	无	cmd	BIND (cmd, int); _IRobotArm->getResetAbsZeroValue; 经 signal_getResetAbsoluteZero_result 回传

9.1.2 调用

C++

```
// AbslueteZeroForm::on_pbn_update_clicked（已连接且界面可见时刷新）
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_GetAbsoluteZero);
```

C++

```
// AbslueteZeroForm::on_pbn_save_clicked（本体上电、范围校验通过后）
CommunicationEngine::instance()->enqueueCmd_setData(this,
AbstractCmd::CmdType_SetAbsoluteZero,
m_editGroupManager->getEditIntValueByRound());
```

C++

```
// AbslueteZeroForm::getCurrent（单轴或 on_pbn_getCurrentAll → index = -1）
CommunicationEngine::instance()->enqueueCmd_setData(this,
AbstractCmd::CmdType_GetAbsoluteZeroCurrentMotorValue, index);
```

C++


```

// AbsoluteZeroForm::on_pbn_reset_clicked (下使能、本体上电、选定轴号合法时)
CommunicationEngine::instance()->enqueueCmd_setData(this,
AbstractCmd::CmdType_ResetAbsoluteZero, axisNo);
// 复位回调 result==0 时再排队:
CommunicationEngine::instance()->enqueueCmd_setData(this,
AbstractCmd::CmdType_GetResetAbsoluteZeroResult, axisNo);

```

9.1.3 调用涉及的类型说明

```

C++
// robotpointinterface.cpp – RobotPointInterfacePrivate (同文件, 匿名命名空间 Internal)
void RobotPointInterfacePrivate::setAbsoluteZeroPoint(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [pos] = ((CmdDatas<QList<int>> *)absCmd)->m_data;
    // 控制权是否在示教器、手动低速模式、下使能状态、不在调试模式下
    bool check =
InoRobBusiness::StatusChecks::getInstance().cobotCheck(
    InoRobBusiness::StatusItemGroup::CTRL_AUTHORITY_TEACHPAD
    | InoRobBusiness::StatusItemGroup::DEVICE_MODE_MANUAL_LOW
    | InoRobBusiness::StatusItemGroup::CTRL_STATUS_DISENABLE
    | InoRobBusiness::StatusItemGroup::DEBUG_MODE_EDIT
    | InoRobBusiness::StatusItemGroup::CTRL_STATUS_EMERGENCY);
    if (!check) {
        return;
    }
    double data[AXIS_NUM] = {0};
    QString print = "AbsoluteZeroPoint value is";

    for (int i = 0; i < pos.size(); ++i) {
        data[i] = pos[i];
    }
    for (int i = 0; i < AXIS_NUM; ++i) {
        print += " " + QString::number(data[i]);
    }
    LOG_INFO(print);
    int ret = _IRobotArm->saveAbsZeroValue(data);
    if (ret != ERROR_OK)

```

```

        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to save
data."));
        FREQ_LOG_PRINT_TIMESTAMP
    }

bool
RobotPointInterfacePrivate::getAbsoluteZeroPoint(QList<double>
&pos, QList<int> &min, QList<int> &max)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    double data[AXIS_NUM];
    pos.clear();
    min.clear();
    max.clear();
    int ret = _IRobotArm->getAbsZeroValue(data);
    if (ret == ERROR_OK) {
        for (int i = 0; i < AXIS_NUM; ++i) {
            pos.push_back(data[i]);
        }
    }
    if (ret != ERROR_OK)
        return false;
    std::string paraStructName = "AbsZeroPoint";
    InoRobBusiness::IRobotParamRange *param = q->comm()-
>robotParam()->getRobotParamRange();
    for (int i = 0; i < 6; ++i) {
        std::string paramName = "J" + std::to_string(i + 1);
        min.push_back(param->getIntMinValue(paraStructName,
paramName));
        max.push_back(param->getIntMaxValue(paraStructName,
paramName));
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return ret == ERROR_OK;
}

void
RobotPointInterfacePrivate::getAbsoluteZeroCurrentValue(AbstractCm
d *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [index] = BIND(cmd, int);
#ifdef INOCOBOTTP_MSVC_QT5

```

```

        QList<double> pos;
        for(int i = 0; i < AXIS_NUM; i++) {
            pos.append(0);
        }
    #else
        QList<double> pos(AXIS_NUM, 0);
    #endif
    int ret = _IRobotArm->getCurrMotorPos(&pos[0]);
    if (ret == ERROR_OK)
        emit CommunicationEngine::instance()-
>signal_getAbsoluteZeroCurrent_result(cmd->m_object, index, pos);
    else
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to get current
encoder value."));
    FREQ_LOG_PRINT_TIMESTAMP
}

void
RobotPointInterfacePrivate::resetAbsoluteZeroPointByAxisNo(Abstrac
tCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [axisNo] = BIND(cmd, int);
    int ret = _IRobotArm->resetAbsZeroValue(axisNo);
    emit CommunicationEngine::instance()-
>signal_resetAbsoluteZero_result(cmd->m_object,axisNo,ret);
    FREQ_LOG_PRINT_TIMESTAMP
}

void
RobotPointInterfacePrivate::getResetAbsoluteZeroPointByAxisNo(Abst
ractCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [axisNo] = BIND(cmd, int);
    uint16_t result = 0;
    int ret = _IRobotArm->getResetAbsZeroValue(axisNo,result);
    if(ret == ERROR_OK){
        ret = result;
    }

    emit CommunicationEngine::instance()-
>signal_getResetAbsoluteZero_result(cmd->m_object,axisNo,ret);

```

```
FREQ_LOG_PRINT_TIMESTAMP
}
```

9.2 工作原理

(getWorikOriginData/setWorikOriginPoint/setWorikOriginPointTriggerData/getWorikOriginPoint/getWorikOriginTriggerData)

9.2.1 说明

接口	功能	返回值	参数	参数说明
void RobotPointInterface::getWorikOriginData(AbstractCmd *cmd)	依次组装 5 组位姿与触发数据并发射结果信号；失败时拼接告警	无	cmd	内部循环 i=0..4 调用 getWorikOriginPoint、getWorikOriginTriggerData
void RobotPointInterface::setWorikOriginPoint(const int index, InoJPos &pos)	保存某一组的关节与外轴数据至控制器	无	index、pos	_IRobotArm->saveWorkOriginData(pos.JointData, pos.EPosData, index); index==0/1 与 ≥2 文案分支不同
void RobotPointInterface::setWorikOriginPointTriggerData(AbstractCmd *cmd)	保存回原点触发输出配置	无	cmd	BIND(cmd, int, int, int, InoJPos) → JHomeTriggertOutData; _IPosition->setHomeTriggerOutData; 成功后

md *cmd)				_IRobotArm->saveFileCommand
bool RobotPointInterface::getWorkOriginPoint(const int index, InoJPos &pos, QList<double> &min, QList<double> &max)	读取指定组关节并组装角度上下限列表	bool	index; pos/min/max 出参	ReadWorkOriginPts; 范围来自 WorkOriginParam {index} 与 WorkOriginParam {index+6} 各 6 个 Jk(°) (共 12 维与编辑控件一致)
bool RobotPointInterface::getWorkOriginTriggerData(const int index, InoJPos &pos, int &mode, int &outIndex)	读取指定组触发数据 (供 getWorkOriginData 使用)	bool	index; pos/mode/outIndex 出参	_IPosition->getHomeTriggerOutData; 填回 InoJPos 的 Joint/EPos 偏移字段

```

C++
#pragma once
#include <QString>
#include <QVariant>

const int ARM_TYPE_COUNT = 4;
const int POSE_AXIS_COUNT = 6;
const int EXT_AXIS_COUNT = 6;
const int JOINT_AXIS_COUNT = 8;

typedef struct InoJPos {
    double JointData[JOINT_AXIS_COUNT]; // J1-J8
    double EPosData[EXT_AXIS_COUNT]; // E1-E6

```

```

InoJPos()
{
    for (int i = 0; i < JOINT_AXIS_COUNT; i++) {
        JointData[i] = 0;
    }
    for (int i = 0; i < EXT_AXIS_COUNT; i++) {
        EPosData[i] = 0;
    }
}

InoJPos(double JointData[JOINT_AXIS_COUNT], double
EPosData[EXT_AXIS_COUNT]){
    for (int i = 0; i < JOINT_AXIS_COUNT; i++) {
        this->JointData[i] = JointData[i];
    }
    for (int i = 0; i < EXT_AXIS_COUNT; i++) {
        this->EPosData[i] = EPosData[i];
    }
}

InoJPos &operator=(const InoJPos &other)
{
    if (this != &other) {
        for (int i = 0; i < JOINT_AXIS_COUNT; i++) {
            JointData[i] = other.JointData[i];
        }

        for (int i = 0; i < EXT_AXIS_COUNT; i++) {
            EPosData[i] = other.EPosData[i];
        }
    }

    return *this;
}

bool operator==(const InoJPos &other) const {
    if (this != &other) {
        for (int i = 0; i < JOINT_AXIS_COUNT; i++) {
            if(abs(JointData[i] - other.JointData[i]) >
0.00001){
                return false;
            }
        }
        for (int i = 0; i < EXT_AXIS_COUNT; i++) {
            if(abs(EPosData[i] - other.EPosData[i]) >
0.00001){
                return false;
            }
        }
    }
}

```

```

        }
    }
    }
    return true;
}
} InoJPos;

```

9.2.2 调用

```

C++
// WorkOriginForm::on_pbn_update_clicked (已连接且界面可见)
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_GetWorkOriginData);

```

```

C++
// WorkOriginForm::on_pbn_save_clicked (取当前编辑区为 InoJPos 后)
CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_SetWorkOrigin,
    m_currentIndex,
    pos);

```

```

C++
// WorkOriginForm::on_pbn_saveHomeTrigger_clicked
CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_SetWorkOriginTriggerData,
    m_currentIndex,
    mode,
    outIndex,
    pos);

```

9.2.3 函数实现

```

C++
// robotpointinterface.cpp – RobotPointInterfacePrivate (同文件)
void RobotPointInterfacePrivate::setWorikOriginPoint(const int
index, InoJPos &pos)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = _IRobotArm->saveWorkOriginData(pos.JointData,
pos.EPosData, index);
    if (ret != ERROR_OK) {
        QString warning;
        if (index == 0)
            warning = QObject::tr("Failed to save initial pose.");
        else if (index == 1)
            warning = QObject::tr("Failed to save delivery
pose.");
        else
            warning = QObject::tr("Failed to save work origin
{0}.").
                .replace("{0}", QString::number(index));
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(warning);
    }

    FREQ_LOG_PRINT_TIMESTAMP
}

void
RobotPointInterfacePrivate::setWorikOriginPointTriggerData(Abstrac
tCmd *cmd)
{
    auto [index, mode, outIndex, pos] = BIND(cmd, int, int, int,
InoJPos);
    JHomeTriggertOutData data;
    data.index = index;
    data.mode = mode;
    data.outNum = outIndex;
    for (int i = 0; i < JOINT_AXIS_COUNT; ++i) {
        data.JointOffsetData[i] = pos.JointData[i];
    }
    for (int i = 0; i < EXT_AXIS_COUNT; ++i) {
        data.EPosOffsetData[i] = pos.EPosData[i];
    }
    int ret = _IPosition->setHomeTriggerOutData(data);
    bool ok = false;

```



```

        if(ret == ERROR_OK)
            _IRobotArm->saveFileCommond(&ok);
        if (ret != ERROR_OK || !ok) {
            QString warning;
            if (index == 0)
                warning = QObject::tr("Failed to save initial pose
trigger data.");
            else if (index == 1)
                warning = QObject::tr("Failed to save delivery pose
trigger data.");
            else
                warning = QObject::tr("Failed to save work origin {0}
trigger data.")
                    .replace("{0}", QString::number(index));
            emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(warning);
        }
    }

bool RobotPointInterfacePrivate::getWorikOriginPoint(const int
index,
                                                    InoJPos &source,
                                                    QList<double> &min,
                                                    QList<double> &max)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    JPos pos;
    int ret = _IRobotArm->ReadWorkOriginPts(pos, index);
    source = MetaTypeConversion::tp2InoApi_jPos(pos);
    if (ret != ERROR_OK)
        return false;
    InoRobBusiness::IRobotParamRange *param = q->comm()-
>robotParam()->getRobotParamRange();
    std::string paraStructName = "WorkOriginParam" +
std::to_string(index);
    for (int j = 0; j < 6; ++j) {
        std::string paramName = "J" + std::to_string(j + 1) +
"(" + std::to_string(index) + ")";
        min.push_back(param->getDoubleMinValue(paraStructName,
paramName));
        max.push_back(param->getDoubleMaxValue(paraStructName,
paramName));
    }
    paraStructName = "WorkOriginParam" + std::to_string(index +

```

```

6);
    for (int j = 0; j < 6; ++j) {
        std::string paramName = "J" + std::to_string(j + 1) +
            "(°)";
        min.push_back(param->getDoubleMinValue(paraStructName,
            paramName));
        max.push_back(param->getDoubleMaxValue(paraStructName,
            paramName));
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool RobotPointInterfacePrivate::getWorikOriginTriggerData(const
int index,

                                                                    InoJPos &pos,
                                                                    int &mode,
                                                                    int &outIndex)
{
    // virtual int getHomeTriggerOutData(JHomeTriggertOutData
&data) = 0;
    JHomeTriggertOutData data;
    data.index = index;
    int ret = _IPosition->getHomeTriggerOutData(data);
    if (ret != ERROR_OK)
        return false;
    for (int i = 0; i < EXT_AXIS_COUNT; ++i) {
        pos.EPosData[i] = data.EPosOffsetData[i];
    }
    for (int i = 0; i < JOINT_AXIS_COUNT; ++i) {
        pos.JointData[i] = data.JointOffsetData[i];
    }
    mode = data.mode;
    outIndex = data.outNum;
    return true;
}

void RobotPointInterfacePrivate::getWorikOriginData(AbstractCmd
*cmd)
{
    QString errorMsg;
    for (int i = 0; i < 5; ++i) {
        InoJPos pos, homeTriggerData;
        QList<double> min, max;

```

```

        int mode = 0, outIndex = -1;
        if (getWorikOriginPoint(i, pos, min, max) &&
            getWorikOriginTriggerData(i, homeTriggerData, mode, outIndex))
            emit CommunicationEngine::instance()
                ->signal_getWorkOriginZeroResult(
                    cmd->m_object, i, pos, min, max, mode,
                    outIndex, homeTriggerData);
        else {
            if (i == 0)
                errorMsg += QObject::tr("Failed to read initial
pose home trigger data.") + "\n";
            else if (i == 1)
                errorMsg += QObject::tr("Failed to read delivery
pose home trigger data.") + "\n";
            else
                errorMsg += QObject::tr("Failed to read work
origin {0} home trigger data.")
                    .replace("{0}", QString::number(i
- 2))
                    + "\n";
        }
    }
    if (!errorMsg.isEmpty()) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(errorMsg);
    }
}

```

10.调试

10.1 串口开关（readComState/writeComState）

10.1.1 说明

接口	功能	返回值	参数	参数说明
void RobotDebug:: readComStat e(AbstractCm	从控制器配置 读取串口开/关 并发	无	cmd	_IRCCConfig- >readComSwi tch; ans==0 映射为关，否

d *absCmd)	signal_getComState_result			则开
void RobotDebug::writeComState(AbstractCmd *absCmd)	写入串口开关并提示保存结果	无	cmd	((CmdDatas<bool> *)cmd)->m_data; _IRCCConfig->saveComSwitch (1/0)

10.1.2 调用

```
C++
// COMSwitchForm::on_pbn_update_clicked
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_ReadComState);
```

```
C++
// COMSwitchForm::on_pbn_save_clicked
CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_WriteComState,
    ui->rbn_open->isChecked());
```

10.1.3 函数实现

```
C++
// robotdebug.cpp – RobotDebug 公开封装
void RobotDebug::readComState(AbstractCmd *absCmd)
{
    d->readComState(absCmd);
}

void RobotDebug::writeComState(AbstractCmd *absCmd)
{
    d->writeComState(absCmd);
}
```

```
}
```

```
C++
// robotdebug.cpp – RobotDebugPrivate
void RobotDebugPrivate::readComState(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ans;
    int nRet = _IRCCConfig->readComSwitch(ans);
    if (nRet != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to read COM
status."));
    } else {
        emit CommunicationEngine::instance()-
>signal_getComState_result(absCmd->m_object, ans == 0 ? false :
true);
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

void RobotDebugPrivate::writeComState(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [state] = ((CmdDdatas<bool> *)absCmd)->m_data;
    int nRet = _IRCCConfig->saveComSwitch(state ? 1 : 0);
    if (nRet != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to save COM
status."));
    } else {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("COM status saved
successfully and takes effect after controller restart. "));
    }
    FREQ_LOG_PRINT_TIMESTAMP
}
```

10.2 网络调试 (ping)

10.2.1 说明

接口	功能	返回值	参数	参数说明
void RobotDebug::ping(Abstract Cmd *absCmd)	对给定 IP 执行 Ping，组包统计字符串并回传 UI	无	cmd	((CmdDatas<QString>*)absCmd)->m_data 为 IP; _IConnection->ping(..., 4) 包数固定为 4

10.2.2 调用

```
C++
// NetworkDebugForm::on_pbn_ping_clicked
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_Ping, ui->edit_ip->text());
```

10.2.3 函数实现

```
C++
// robotdebug.cpp - RobotDebug 公开封装
void RobotDebug::ping(AbstractCmd *absCmd)
{
    d->ping(absCmd);
}
```

```
C++
```

```

// robotdebug.cpp – RobotDebugPrivate
void RobotDebugPrivate::ping(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int nSend = 0, nRecv = 0;
    double nLoss = 100.0, minT = 0.0, maxT = 0.0, aveT = 0.0;
    auto [ip] = ((CmdDatas<QString> *)absCmd)->m_data;
    _IConnection->ping(nSend, nRecv, nLoss, minT, maxT, aveT,
ip.toStdString(), 4);
    QString res = QObject::tr("Ping statistics for {0}:
").replace("{0}", ip) + "\r\n"
        + QString(QObject::tr("Packets: Sent = %1,
Accepted = %2, Loss = %3%") + "\r\n")
        .arg(QString::number(nSend),
QString::number(nRecv), QString::number(nLoss, 'f', 2));
    emit CommunicationEngine::instance()-
>signal_getPing_result(absCmd->m_object, res);
    FREQ_LOG_PRINT_TIMESTAMP
}

```

10.3 系统诊断（startDiagnose/ stopDiagnose / readDiagnosePercent/startExportReportToControllerU SB/readErrExportStaPercentToControllerUSB/startEx portReportToLocal/readErrExportStaPercentToLocal ）

10.3.1 说明

接口	功能	返回值	参数	参数说明
void RobotDebug:: startDiagnose (AbstractCmd	按三路勾选启 动系统诊断	无	cmd	CmdDatas<b ool,bool,bool> → _IMaintenanc

*absCmd)				e- >startDiagnose
void RobotDebug:: stopDiagnose (AbstractCmd *absCmd)	停止诊断任务	无	cmd	CmdDatas<Q Variant>, 列 表 0..2 为三件 bool
void RobotDebug:: readDiagnose Percent(AbstractCmd *absCmd)	读取诊断保存 进度, 驱动进 度条信号	无	cmd	_IMaintenanc e- >ReadErrSav eSta 映射 ErrsaveSta → InoTaskState
void RobotDebug:: startExportRe portToControll erUSB(AbstractCmd *absCmd)	发起导出到控 制器 U 盘, 并 上传示教器日 志目录等	无	cmd	CmdDatas<Q Object*,bool,b ool,bool,int> : 对话框指 针、三路勾 选、type (末 次/全部)
void RobotDebug:: readErrExport StaPercentTo ControllerUSB (AbstractCmd *absCmd)	查询导出到 USB 的进度 (含上传 teachpant 日 志阶段)	无	cmd	与 ProcessBarFo rmWithComm 配对
void RobotDebug:: startExportRe portToLocal(A bstractCmd *absCmd)	发起导出到本 地目录	无	cmd	CmdDatas<b ool,bool,bool,i nt,int,QString >: 三路、 type、dst、本 地路径
void RobotDebug:: readErrExport	查询本地导出 进度并触发	无	cmd	见 robotdebug.cp p UDF

StaPercentTo Local(Abstract Cmd *absCmd)	FTP/目录拷贝			downloadDire ctory 等
---	----------	--	--	-------------------------

10.3.2 调用

```
C++
// SystemDiagnosisForm::on_pbn_start_clicked
CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_StartDiagnose,
    ui->checkBox_system->isChecked(),
    ui->checkBox_logic->isChecked(),
    ui->checkBox_trace->isChecked());
```

```
C++
// SystemDiagnosisForm::on_pbn_export_clicked - 导出到控制器 USB
(节选)
CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_StartExportDiagnoseReportToUSB,
    ProcessBarFormWithComm::instance(),
    ui->checkBox_system->isChecked(),
    ui->checkBox_logic->isChecked(),
    ui->checkBox_trace->isChecked(),
    type);
```

```
C++
// SystemDiagnosisForm::on_pbn_export_clicked - 导出到本地 (节选,
path 为新建子目录)
CommunicationEngine::instance()->enqueueCmd_setData(this,
    AbstractCmd::CmdType_StartExportDiagnoseReportToLocal,
    ui->checkBox_system->isChecked(),
    ui->checkBox_logic->isChecked(),
    ui->checkBox_trace->isChecked(),
    type,
    dest,
```

```
dirPath);
```

10.3.3 函数实现

C++

```
// robotdebug.cpp – RobotDebugPrivate (诊断核心逻辑全文)
void RobotDebugPrivate::startDiagnose(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [sys, logic, track] = ((CmdDatas<bool, bool, bool>
*)absCmd)->m_data;
    int nRet = _IMaintenance->startDiagnose(sys, logic, track);
    if (nRet != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to start system
diagnosis."));
    } else {
        emit CommunicationEngine::instance()-
>signal_systemDiagnoseTaskHasStartSucess(absCmd->m_object,
absCmd->m_cmdType);
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

void RobotDebugPrivate::stopDiagnose(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [variant] = ((CmdDatas<QVariant> *)absCmd)->m_data;
    QVariantList list = variant.toList();
    int nRet = _IMaintenance->stopDiagnose(list[0].toBool(),
                                            list[1].toBool(),
                                            list[2].toBool());

    emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_stopResult(absCmd->m_object, nRet
== ERR_OK, "");
    FREQ_LOG_PRINT_TIMESTAMP
}

void RobotDebugPrivate::readDiagnosePercent(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (_IConnection->GetConnectionStatus())
```

```

        != InoRobBusiness::CONTROLLER_CONNECTION_STATUS_CONNECTED)
{
    emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_currentState(
        absCmd->m_object, false, 0, Task_UnknowState, "");
    return;
}
ErrsaveSta sta; //-1-保存失败 0-null 1-保存中 2-保存成功 3-
保存终止
    int nRet = _IMaintenance->ReadErrSaveSta(sta);
    InoTaskState state;
    switch (sta.saveRes) {
    case (-1): {
        state = Task_Falid;
        break;
    }
    case (0): {
        state = Task_UnknowState;
        break;
    }
    case (1): {
        state = Task_InProcess;
        break;
    }
    case (2): {
        state = Task_FinishedSuccess;
        break;
    }
    case (3): {
        state = Task_WasForciblyTerminated;
        break;
    }
    default:
        state = Task_UnknowState;
        break;
    }
    emit CommunicationEngine::instance()-
>signal_processBarFormWithComm_currentState(
        absCmd->m_object, nRet == ERR_OK, sta.saveProgress, state,
        "");
    FREQ_LOG_PRINT_TIMESTAMP
}

```

10.4 通配 (getGeneralMatchOpen / setGeneralMatchOpen/allowTracingGeneralMatch/getGeneralMatchRecord)

10.4.1 说明

接口	功能	返回值	参数	参数说明
bool GeneralMatchInterface::getGeneralMatchOpen()	读取通配功能是否开启	bool	—	robotParam()->getGeneralMatch()->getGeneralMatchOpen()
bool GeneralMatchInterface::setGeneralMatchOpen(bool open)	设置通配开关	bool	open	委托 setGeneralMatchOpen
bool GeneralMatchInterface::allowTracingGeneralMatch()	是否允许获取通配溯源记录	bool	—	供线程在 GetRecords 前校验
int GeneralMatchInterface::getGeneralMatchRecord(const std::string &fileName)	将通配记录拉取到指定路径	int (0 成功意 含以控制器为 准)	fileName	日志见 generalmatchi nterface.cpp

C++

```

constexpr int COBOT_MATCH_DB_SIZE_MAX = 8; // 通配 DB 电阻最大数量

typedef struct stCobotGeneralMatchInfo
{
    std::string bodyRobotName; // 本体名
    std::string controllerRobotName; // 控制器当前
适配本体名
    int matchState = -1; // 0: 检查
中, 1: 匹配, 2: 不匹配, 3: FPGA 软硬件不满足要求,
// 4: 本体与
控制器功率不匹配, 5: 本体与控制器参数版本不一致,
// 6: 本体与
控制器机型不一致, 7: 控制器刷机
    int factoryFlag = -1; // 出厂标志
1: 出厂, 0: 非出厂
    int direction = -1; // 同步方向
0: 本体同步到控制柜, 1: 控制柜同步到本体
    std::string bodyHardwareVersion; // 本体存储器
硬件版本
    std::string bodySoftwareVersion; // 本体存储器
软件版本
    std::string bodySN; // 本体 SN 码
    std::string mainFPGAHardwareVersion; // 核心板
FPGA 硬件版本
    std::string mainFPGASoftwareVersion; // 核心板
FPGA 软件版本
    std::string controllerSN; // 控制器 SN
码
    int bodyDB[COBOT_MATCH_DB_SIZE_MAX] = { -1 }; // 本
体 DB 电阻
    int bodyBrakeType[COBOT_MATCH_DB_SIZE_MAX] = { -1 }; // 本
体励磁抱闸
    int bodyPower[COBOT_MATCH_DB_SIZE_MAX] = { -1 }; // 本
体功率
    std::string bodyParamVersion; // 本体参数版
本
    std::string bodyParamCompatibilityVersion; // 本体参数兼
容版本
    std::string controllerParamCompatibilityVersion; // 控制器软件

```

兼容版本

```
int generalOpen = -1; // 通配开启状
态 0: 关闭, 1: 开启

stCobotGeneralMatchInfo()
{
    for (int i = 0; i < COBOT_MATCH_DB_SIZE_MAX; i++)
    {
        bodyDB[i] = -1;
        bodyBrakeType[i] = -1;
        bodyPower[i] = -1;
    }
}
} CobotGeneralMatchInfo;
Q_DECLARE_METATYPE(CobotGeneralMatchInfo);
```

10.4.2 调用

C++

```
// GeneralMatchForm::updateUI (已连接时)
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_GeneralMatch_GetOpenStatus);
```

C++

```
// GeneralMatchForm::on_pbn_switch_clicked
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
AbstractCmd::CmdType_GeneralMatch_SetOpenStatus, !bSwitch);
```

C++

```
// GeneralMatchForm::on_pbn_traceability_clicked (选择目录并建立
MatchLog 子目录后)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_GeneralMatch_GetRecords,
sExportPath.toStdString());
```

10.4.3 函数实现

```
C++
// generalmatchinterface.cpp - 本章涉及的四个方法全文
bool GeneralMatchInterface::getGeneralMatchOpen()
{
    return robotParam()->getGeneralMatch()->getGeneralMatchOpen();
}

bool GeneralMatchInterface::setGeneralMatchOpen(bool open)
{
    bool bRet = robotParam()->getGeneralMatch()-
>setGeneralMatchOpen(open);
    LOG_INFO(QString("[setGeneralMatchOpen]open = %1, bret = %2")
        .arg(QString::number(open),
        QString::number(bRet)));
    return bRet;
}

bool GeneralMatchInterface::allowTracingGeneralMatch()
{
    return robotParam()->getGeneralMatch()-
>allowTracingGeneralMatch();
}

int GeneralMatchInterface::getGeneralMatchRecord(const std::string
&fileName)
{
    int iRet = robotParam()->getGeneralMatch()-
>getGeneralMatchRecord(fileName);
    LOG_INFO(QString("[getGeneralMatchRecord]filename = %1, iRet
= %2")
        .arg(QString::fromStdString(fileName),
        QString::number(iRet)));
    return iRet;
}
```

11. 安全配置

11.1 安全模块

(GetSafeHardVer/SafeParaReset/ExportFunctionSafeParaFile/ImportFunctionSafeParaFile/GetSafeParaCommon/SetSafeParaCommon/SafetyWriteParams)

11.1.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::GetSafeHardVer(std::string &system, std::string &moni, std::string &commu)	读取安全 MCU 侧系统 / 监控 / 通讯版本号	int (0 成功)	出参三段版本字符串	_ISafeParaSettingMgr->GetSafeHardVer
int SafetyInterface::SafeParaReset()	初始化功能安全参数并轮询结果	int	—	对应 CmdType_SafetyReset ; 结果经 signal_safety_comm_result
int SafetyInterface::ExportFunctionSafeParaFile(const std::string exportRoute)	导出安全参数 JSON	int	文件完整路径	CmdType_SafetyExportConfig
int SafetyInterface::ImportFunctionSafeParaFile(const std::string importRoute)	导入安全参数 JSON	int	文件完整路径	CmdType_SafetyImportConfig
int SafetyInterface::GetSafeParaCommon(int key, int offset, int	从模型层读取安全参数缓冲	int	key 见 SafeParaEnum / Cobot_EEP	与 ReadSafeParamsFromMcuDirect

length, unsigned char *buf, bool forceFlag)			*	线程逻辑配套
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	写入模型层缓冲	int	同上	与 WriteSafeParamsToMcu Direct 配套
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	将已编辑项写入安全MCU	int	—	监控开关写RAM 键时使用

11.1.2 调用

C++

```
// ExpansionCardForm::updateUI (已连接)
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_SafetyGetHardVer);
```

C++

```
// ExpansionCardForm::on_pbn_initialize_clicked
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_SafetyReset);
```

C++

```
// ExpansionCardForm::on_pbn_export_clicked /
on_pbn_import_clicked (路径以 ../safety_params.json 结尾)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetyExportConfig,
    sExportPath.toStdString());
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetyImportConfig,
    sExportPath.toStdString());
```

C++

// ExpansionCardForm::on_pbn_syncstatus_clicked – 写 RAM 激活安全监控

CommunicationEngine::instance()-

```
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_RAM_ACTIVE_SAFETY_MONITOR, 0, 1, sizeof(uint8_t),
    (unsigned char *)&m_activeSafety, false);
```

11.1.3 函数实现

C++

// safetyinterface.cpp (节选本章相关)

```
int SafetyInterface::GetSafeHardVer(std::string &system,
std::string &moni,
                                std::string &commu)
```

```
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _ISafeParaSettingMgr->GetSafeHardVer(system, moni,
commu);
}
```

```
int SafetyInterface::SafeParaReset()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _ISafeParaSettingMgr->SafeParaReset(true);
}
```

```
int SafetyInterface::ExportFunctionSafeParaFile(const std::string
exportRoute)
{
    int iRet = _ISafeParaSettingMgr-
>ExportFunctionSafeParaFile(exportRoute);
    LOG_INFO(QString("safety config export, ret = %1, path
= %2").arg(QString::number(iRet), exportRoute.data()));
    return iRet;
}
```

```
int SafetyInterface::ImportFunctionSafeParaFile(const std::string
importRoute)
{
```

```

    int iRet = _ISafeParaSettingMgr-
>ImportFunctionSafeParaFile(importRoute);
    LOG_INFO(QString("safety config import, ret = %1, path
= %2").arg(QString::number(iRet), importRoute.data()));
    return iRet;
}

```

11.2 I/O

(QuerySafeIORRealTime/SetSafeParaCommon/SafetyWriteParams)

11.2.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface:: QuerySafeIORRealTime(bool switchFlag)	打开 / 关闭 安全 IO 实 时电平轮询 线程	int	true 开启 / false 关闭	UI 显示后 signal_safei o_changed 推送 CobotSafeI O
int SetSafeParaCo mmon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将 IO / 停机 配置写入模 型层	int	key: 功能安全功能特性 枚举值, 详见 SafeParaEnum offset: 该数据相对首个 同类数据的偏移 length: 该数据长度 buf: 将该数据放入 bufforceFlag: 强制写入	由 SafetySnycl O 分支调用
int SafetyInterface:: SafetyWritePara ms(int *range, int num)	按键值数组 批量下发	int	上电: {DI,DO,STOPCONF}; 未上电: {DI,DO}	见 communicati onthread.cp p CmdType_S afetySnyclO

```

C++
typedef struct stCobotSafeIO {
    stCobotSafeIO()
    {
        memset(DI, 0X00, sizeof(DI));
        memset(DO, 0X00, sizeof(DO));
    }
    CobotIO DI[COBOT_DI_NUM];
    CobotIO DO[COBOT_DO_NUM];
} CobotSafeIO;

typedef struct stCobotIO
{
    stCobotIO()
    {
        u8Attr = 0;
        u8AMcuValue = 0;
        u8BMcuValue = 0;
    }

    quint8 u8Attr;                /* 功能属性 */
    quint8 u8AMcuValue;           /* MCUA 电平值 */
    quint8 u8BMcuValue;           /* MCUB 电平值 */
} CobotIO;

typedef struct stCobotStopConfig
{
    unsigned int u32TimeOut0;      /* 0 类停机最
长延时时间 ms */
    unsigned int u32TimeOut1;      /* 1 类停机最
长延时时间 ms */
    unsigned int u32Distance0;     /* 0 类停机最
长距离 */
    unsigned int u32Distance1;     /* 1 类停机最
长距离 */
    unsigned char u8EStop;         /* DI 急停方式
*/
    unsigned char u8SafeStop;      /* DI 安全门停
机方式 */
} CobotStopConfig;

enum SafeParaEnum : int16s

```

```

{
    EEP_CRC = 0,                // CRC
    EEP_PASSWORD = 1,          // 密码
    EEP_STOPCONF = 2,          // 停机配置, stopConf STOPCONF
    EEP_LOW_SPEED = 3,         // 安全低速, safeLowSpeed
SAFELOWSPEED
    EEP_DI = 4,                // 安全 IO DI, safeIO.DI S_IO
    EEP_DO = 5,                // 安全 IO DO, safeIO.DO S_IO
    EEP_APOSWAY = 6,           // 轴位置生效方式,
axisParas.u8APosWay int8u
    EEP_AVELWAY = 7,           // 轴速度生效方式,
axisParas.u8AVelWay int8u
    EEP_APOS_0 = 8,            // 轴位置 0, axisParas.APosGroup
S_APOS
    EEP_APOS_1 = 9,            // 轴位置 1
    EEP_APOS_2 = 10,           // 轴位置 2
    EEP_APOS_3 = 11,           // 轴位置 3
    EEP_APOS_4 = 12,           // 轴位置 4
    EEP_APOS_5 = 13,           // 轴位置 5
    EEP_APOS_6 = 14,           // 轴位置 6
    EEP_APOS_7 = 15,           // 轴位置 7
    EEP_AVEL_0 = 16,           // 轴速度 0, axisParas.AVelGroup
S_AVEL
    EEP_CVELWAY = 24,          // 笛卡尔速度生效方式,
carteParas.u8CVelWay int8u
    EEP_CPOSWAY = 25,          // 笛卡尔位置生效方式,
carteParas.u8CPosWay int8u
    EEP_CVEL_0 = 26,           // 笛卡尔速度 0,
carteParas.CVelGroup S_CVEL
    EEP_IZONE_STATUS_0 = 34,    // 干涉区状态 0,
carteParas.CPosGroup INTERFER_ZONE_STATUS
    EEP_ITOOL_STATUS_0 = 50,    // 末端监测对象状态状态 0,
INTERFER_ZONE_STATUS
    EEP_IZONE_PARA_0 = 66,      // 干涉区配置参数 0, INTERFER_ZONE
    EEP_IZONE_WOBJ_0 = 82,      // 干涉区工件参数 0,
INTERFER_ZONE_WOBJPARAM
    EEP_ITOOL_PARA_0 = 98,      // 末端监测对象配置参数 0,
INTERFER_TOOL
    EEP_ITOOL_MTCPTOOL_0 = 114, // 末端监测对象, MTCP 工具参数 0,
INTERFER_ZONE_MTCPTOOLPARAM
    EEP_ITOOL_MTCPTOOL_F = 129, // 末端监测对象, MTCP 工具参数 15

```

```

EEP_SAFEIO_DITRIGGER = 130, // 安全 IO DI 阈值,
u32DITrigerValue
EEP_TOOLPARA_0 = 131, // 工具参数,
toolWobjMgr.ToolDataGroup ToolData
EEP_TOOLPARA_15 = 146,
EEP_WOBJPARA_0 = 147, // 工件参数,
toolWobjMgr.WobjDataGroup WobjData
EEP_WOBJPARA_15 = 162,
EEP_SAFE TOOLMODE = 163, // 安全工具设置, safeToolMode
SafeToolSetting, 同步保存到 EEPROM
#ifdef COBOT
EEP_CTCPFORCE = 164, // 笛卡尔力
EEP_STATICWAY = 165, // 安全静止生效方式
EEP_STATIC_0 = 166, // 安全静止 0
EEP_STATIC_1 = 167, // 安全静止 1
EEP_STATIC_2 = 168, // 安全静止 2
EEP_STATIC_3 = 169, // 安全静止 3
EEP_STATIC_4 = 170, // 安全静止 4
EEP_STATIC_5 = 171, // 安全静止 5
EEP_STATIC_6 = 172, // 安全静止 6
EEP_STATIC_7 = 173, // 安全静止 7
EEP_HOMING = 174, // 安全原点
EEP_BODYMOMENTUM = 175, // 整机动量
EEP_BODYPOWER = 176, // 整机功率
EEP_TCPDIRECTIONWAY = 177, // TCP 方向生效方式
EEP_TCPDIRECTION_0 = 178, // TCP 方向 0
EEP_TCPDIRECTION_1 = 179, // TCP 方向 1
EEP_TCPDIRECTION_2 = 180, // TCP 方向 2
EEP_TCPDIRECTION_3 = 181, // TCP 方向 3
EEP_TCPDIRECTION_4 = 182, // TCP 方向 4
EEP_TCPDIRECTION_5 = 183, // TCP 方向 5
EEP_TCPDIRECTION_6 = 184, // TCP 方向 6
EEP_TCPDIRECTION_7 = 185, // TCP 方向 7
EEP_STOPDISTANCE = 186, // 停机距离
EEP_STOPTIME = 187, // 停机时间
EEP_STOP_TOLERANCE = 188, // 停机抖动容差
EEP_REDUCEMODE_APOS_0 = 189, // 缩减模式轴位置
EEP_REDUCEMODE_APOS_1 = 190, // 缩减模式轴位置
EEP_REDUCEMODE_APOS_2 = 191, // 缩减模式轴位置
EEP_REDUCEMODE_APOS_3 = 192, // 缩减模式轴位置
EEP_REDUCEMODE_APOS_4 = 193, // 缩减模式轴位置

```

```

EEP_REDUCEMODE_APOS_5 = 194, // 缩减模式轴位置
EEP_REDUCEMODE_APOS_6 = 195, // 缩减模式轴位置
EEP_REDUCEMODE_APOS_7 = 196, // 缩减模式轴位置
EEP_REDUCEMODE_AVEL_0 = 197, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_1 = 198, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_2 = 199, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_3 = 200, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_4 = 201, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_5 = 202, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_6 = 203, // 缩减模式轴速度
EEP_REDUCEMODE_AVEL_7 = 204, // 缩减模式轴速度
EEP_REDUCEMODE_CVEL = 205, // 缩减模式笛卡尔速度
EEP_ELBOW_SWITCH = 206, // 肘部监控开关
EEP_POSITIONDISMATHWAY = 207, // 位置不匹配生效方式
EEP_POSITIONDISMATH_0 = 208, // 位置不匹配 0
EEP_POSITIONDISMATH_1 = 209, // 位置不匹配 1
EEP_POSITIONDISMATH_2 = 210, // 位置不匹配 2
EEP_POSITIONDISMATH_3 = 211, // 位置不匹配 3
EEP_POSITIONDISMATH_4 = 212, // 位置不匹配 4
EEP_POSITIONDISMATH_5 = 213, // 位置不匹配 5
EEP_POSITIONDISMATH_6 = 214, // 位置不匹配 6
EEP_POSITIONDISMATH_7 = 215, // 位置不匹配 7
EEP_DRAGTEACH = 216, // 拖动示教
EEP_CHECKYES = 217, // 数据同步到 EEPROM
RAM_TCP_DIRECT_GET_VECTOR_0 = 218, // 获取组末端向量 0
RAM_TCP_DIRECT_GET_VECTOR_1 = 219, // 获取组末端向量 1
RAM_TCP_DIRECT_GET_VECTOR_2 = 220, // 获取组末端向量 2
RAM_TCP_DIRECT_GET_VECTOR_3 = 221, // 获取组末端向量 3
RAM_TCP_DIRECT_GET_VECTOR_4 = 222, // 获取组末端向量 4
RAM_TCP_DIRECT_GET_VECTOR_5 = 223, // 获取组末端向量 5
RAM_TCP_DIRECT_GET_VECTOR_6 = 224, // 获取组末端向量 6
RAM_TCP_DIRECT_GET_VECTOR_7 = 225, // 获取组末端向量 7
RAM_ACTIVE_SAFETY_MONITOR = 226, // 激活安全监控
RAM_CHECKYES = 227, // 数据同步 RAM
EEP_SYSTEM_READY = 228, // EEP 握手成功
RAM_RESEST = 229,
RAM_RESEST_STATUS = 230,
SPARA_MEMBER_NUM = 231, // 数据地图长度(内存变量个数)
#else
EEP_CHECKYES = 164, // 数据同步到 EEPROM

```

```

    RAM_CHECKYES = 165,      // 数据同步 RAM
    EEP_SYSTEM_READY = 166,  // EEP 握手成功
    RAM_RESEST = 167,
    RAM_RESEST_STATUS = 168,
    SPARA_MEMBER_NUM = 169,  //数据地图长度(内存变量个数)
#endif
};

```

11.2.2 调用

```

C++
// SafetyIOConfigForm::showEvent / hideEvent - 实时 IO
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetyQuerySafeIORealTime, true);
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetyQuerySafeIORealTime, false);

```

```

C++
// SafetyIOConfigForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySnycIO, m_io,
    m_diTiggerLimit, m_stopconfigdata, bPowerOn);

```

11.2.3 函数实现

```

C++
// safetyinterface.cpp
int SafetyInterface::QuerySafeIORealTime(bool switchFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr-
>QuerySafeIORealTime(switchFlag);
    qDebug() << __FUNCTION__ << ", iRet = " << iRet << ",
switchFlag = " << switchFlag;
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

```



```

int SafetyInterface::SetSafeParaCommon(int key, int offset, int
length,
                                unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->SetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::SafetyWriteParams(int *range, int num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Write(range, num);
    LOG_INFO(QString("[%1]ret = %2")
                .arg(__FUNCTION__, QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

```

11.3 工具模式

(SafetyReadParams/GetSafeParaCommon/SetSafeParaCommon/SafetyWriteParams)

11.3.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SafetyReadParams(int key, int offset, int num)	从安全 MCU 读入模型层	int		与 GetSafeParaCommon 串联
int SafetyInterface::GetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	读 CobotSafe ToolSetting 到界面缓冲	int	key: 功能安全功能特性枚举值, 详见 SafeParaEnum offset: 该数据相对首个同类数据的偏移 length: 该数据长度 buf: 将该数据放入 buf forceFlag: 若该数据部位 SYNC 状态, 则从 SafeMCU 获取, 该逻辑耗时较长	—
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将界面数据下发模型层, 并将数据 status 设置为 SPS_EDIT ED	int	key: 功能安全功能特性枚举值, 详见 SafeParaEnum offset: 该数据相对首个同类数据的偏移 length: 该数据长度 buf: 将该数据放入 buf	—

			forceFlag: 强制写入	
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	下达 MCU	int	key: 功能安全功能特性枚举值, 详见 SafeParaEnum offset: 该数据相对首个同类数据的偏移 num: 基于 key + offset 位置, 获取 num 个数据	—

```

C++
typedef struct stCobotSafeToolSetting
{
    unsigned short SafeToolMode;           // 0 - FollowSys, 1 - Fixed, 2 - Manual
    unsigned short FixedSafeToolNo;       // Tool 1 - 16
    short DoNo[COBOT_DO_NUM];
} CobotSafeToolSetting;

```

11.3.2 调用

```

C++
// SafetyToolModeConfigForm::updateUI
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_SAFETOOLMODE, 0, 1,
    sizeof(CobotSafeToolSetting),
    (unsigned char *)&m_toolSettings, false);

```

```

C++
// SafetyToolModeConfigForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(

```

```
    this, Cobot_EEP_SAFETOOLMODE, 0, 1,
    sizeof(CobotSafeToolSetting),
    (unsigned char *)&m_toolSettings, false);
```

11.3.3 函数实现

SQL

```
int SafetyInterface::SafetyReadParams(int key, int offset, int
num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Read(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(num),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::GetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->GetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}
```

```

int SafetyInterface::SetSafeParaCommon(int key, int offset, int
length,
                                unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->SetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::SafetyWriteParams(int key, int offset, int
num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Write(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(num),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

```

11.4 缩减模式 (ReadSpeedReducing/WriteSpeedReducing)

11.4.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::ReadSpeedReducing(SpeedReducingConf &conf)	读取缩减等级 百分比	int	出参 speedLevel1 / speedLevel2	内部 S_SPEEDREDUCINGCONF 与 TP 结构 memcpy
int SafetyInterface::WriteSpeedReducing(const SpeedReducingConf &conf)	写入缩减等级	int	—	同上

```
C++
typedef struct stSpeedReducingConf
{
    quint32 speedLevel1;    ///< 缩减等级 1: 速度缩减百分比
    quint32 speedLevel2;    ///< 缩减等级 2: 速度缩减百分比
} SpeedReducingConf;
```

11.4.2 调用

```
C++
// SpeedReducingForm::updateUI
CommunicationEngine::instance()->enqueueCmd_getData(
    this, AbstractCmd::CmdType_SafetyReadSpeedReducing, config);
```

```
C++
// SpeedReducingForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetyWriteSpeedReducing, config);
```

11.4.3 函数实现

```
C++
// safetyinterface.cpp
int SafetyInterface::WriteSpeedReducing(const SpeedReducingConf
&conf)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    S_SPEEDREDUCINGCONF speedReducing;
    memcpy(&speedReducing, &conf, sizeof(speedReducing));
    int iRet = _ISafeParaSettingMgr-
>WriteSpeedReducing(speedReducing);
    LOG_INFO(QString("[%1]iRet = %2, status
= %3.").arg(__FUNCTION__, QString::number(iRet),
QString::number(iRet)));
    qDebug() << __FUNCTION__ << "| ret = " << iRet << ", l1 = " <<
conf.speedLevel1
        << ", l2 = " << conf.speedLevel2;
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::ReadSpeedReducing(SpeedReducingConf &conf)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    S_SPEEDREDUCINGCONF speedReducing;
    int iRet = _ISafeParaSettingMgr-
>ReadSpeedReducing(speedReducing);
    memcpy(&conf, &speedReducing, sizeof(speedReducing));
    LOG_INFO(QString("[%1]iRet = %2, status
= %3.").arg(__FUNCTION__, QString::number(iRet),
QString::number(iRet)));
    qDebug() << __FUNCTION__ << "| ret = " << iRet << ", l1 = " <<
conf.speedLevel1
        << ", l2 = " << conf.speedLevel2;
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}
```

11.5 急停与启动策略

(readEmgTrigMethod/saveEmgTrigMethod/readStartReleaseMode/writeStartReleaseMode)

11.5.1 说明

接口	功能	返回值	参数	参数说明
int ResourceInterface::readEmgTrigMethod(quint8 &iMethod)	读急停触发策略（示教器 + 三位置 / 仅示教器等）	int	出参 iMethod	_IRCCConfig->readEmgTrigMethod
int ResourceInterface::saveEmgTrigMethod(quint16 iMethod)	保存急停触发策略	int	先 checkPermissionBeforeSaveEmgTrigMethod	CmdType_SaveEmgTrigMethod
int DebuggerInterface::readStartReleaseMode(uint8_t &mode)	读启动按钮释放后是否停机策略	int	TP 对象类型	_IDebugger->readStartReleaseMode
int DebuggerInterface::writeStartReleaseMode(uint8_t mode)	写启动按钮策略	int	枚举 StartReleaseMode	成功后 setCurStartReleaseStrategy

11.5.2 调用

C++

```
CommunicationEngine::instance()->enqueueCmd(this,  
AbstractCmd::CmdType_ReadEmgTrigMethod);  
CommunicationEngine::instance()->enqueueCmd(this,  
AbstractCmd::CmdType_Debugger_StartReleaseRead);
```

C++

```
CommunicationEngine::instance()->enqueueCmd_setData(this,  
AbstractCmd::CmdType_SaveEmgTrigMethod, method);  
CommunicationEngine::instance()->enqueueCmd_setData(this,  
AbstractCmd::CmdType_Debugger_StartReleaseWrite, startrelease);
```

11.5.3 函数实现

C++

```
// resourceinterface.cpp  
int ResourceInterface::readEmgTrigMethod(quint8 &iMethod)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    return _IRCCConfig->readEmgTrigMethod(iMethod);  
}  
  
int ResourceInterface::saveEmgTrigMethod(quint16 iMethod)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    if (!_IRCCConfig->checkPermissionBeforeSaveEmgTrigMethod()) {  
        return -1;  
    }  
  
    return _IRCCConfig->saveEmgTrigMethod(iMethod);  
}
```

C++

```
// debuginterface.cpp  
int DebuggerInterface::writeStartReleaseMode(uint8_t mode)  
{
```

```

FREQ_LOG_PRINT_TIMESTAMP_THREAD

int iRet = _IDebuger->writeStartReleaseMode(
    InoRobBusiness::StartReleaseObjType::TP,
    static_cast<InoRobBusiness::StartReleaseMode>(mode));

LOG_INFO(QString("[writeStartReleaseMode]iRet
= %1").arg(QString::number(iRet)));

if (iRet != 0) {
    return iRet;
}

_IDebuger->setCurStartReleaseMode(
    InoRobBusiness::StartReleaseObjType::TP,
    static_cast<InoRobBusiness::StartReleaseMode>(mode));

return 0;
}

int DebuggerInterface::readStartReleaseMode(uint8_t &mode)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoRobBusiness::StartReleaseMode retMode;
    int iRet = _IDebuger->readStartReleaseMode(
        InoRobBusiness::StartReleaseObjType::TP, retMode);

    LOG_INFO(QString("[readStartReleaseMode]retMode = %1, ret
= %2").arg(
        QString::number(static_cast<uint8_t>(retMode)),
        QString::number(iRet)));

    mode = static_cast<uint8_t>(retMode);
    return iRet;
}

```

12.安全监控

12.1 低速监控

(GetSafeParaCommon/SetSafeParaCommon/SafetyReadParams/SafetyWriteParams)

12.1.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::GetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	从模型层读取安全参数缓冲（供 UI 显示）	int	key, offset, length, buf, forceFlag	key: 安全参数键（本节为 Cobot_EEP_LOW_SPEED） offset: 条目索引（单条为 0） length: 单条数据长度（本节为 sizeof(CobotSafeLowSpeed)） buf: 输出缓冲区（拷贝至 UI） forceFlag: 是否强制覆盖（一般为 false）
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将 UI 缓冲写入模型层（待下发）	int	key, offset, length, buf, forceFlag	含义同上；buf 为输入缓冲区（从 UI 拷贝至模型层）
int SafetyInterface::SafetyRead	从安全 MCU 读取到模型层	int	key, offset, num	key: 安全参数键（Cobot_EEP

Params(int key, int offset, int num)				_LOW_SPEE D) offset: 条 目索引 (0) num: 条目数 量 (1)
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	将模型层指定 键写入安全 MCU	int	key, offset, num	含义同上; 写 入 MCU

```

C++
typedef struct stCobotSafeLowSpeed
{
    stCobotSafeLowSpeed()
    {
        u8Active = 0;    /* 生效 */
        u8StopCat = 0;   /* 停机 */
        f64Speed = 0;    /* 安全低速 */
    }

    quint8 u8Active;           /* 生效 */
    quint8 u8StopCat;          /* 停机 */
    double f64Speed;           /* 安全低速 */
} CobotSafeLowSpeed;

```

12.1.2 调用

```

C++
// LowSpeedMonitorForm::updateUI (刷新/显示)
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_LOW_SPEED, 0, 1, sizeof(CobotSafeLowSpeed),
    (unsigned char *)&m_lowspeed, false);

```

```

C++
// LowSpeedMonitorForm::on_pbn_synchronize_clicked (同步)
CommunicationEngine::instance()-

```

```
>enqueueCmd_WriteSafetyParamsToMcuDirect(  
    this, Cobot_EEP_LOW_SPEED, 0, 1, sizeof(CobotSafeLowSpeed),  
    (unsigned char *)&m_lowspeed, false);
```

12.1.3 函数实现

```
C++  
// communicationthread.cpp (void  
CommunicationThread::processTasks(AbstractCmd *absCmd) 节选:  
read/write direct)  
case AbstractCmd::CmdType_ReadSafeParamsFromMcuDirect: {  
    CmdReadSafeParamFromMcuDirect *cmd  
        = static_cast<CmdReadSafeParamFromMcuDirect *>(absCmd);  
  
    int iRet = m_commInstance->SafetyReadParams(cmd->key, cmd-  
>offset,  
                                                cmd->num);  
  
    if (iRet != 0) {  
        emit CommunicationEngine::instance()  
            ->signal_safety_readparamsfrommcudirect(  
                absCmd->m_object, false, cmd->key);  
        break;  
    }  
  
    // 将模型层数据读取到 GUI  
    bool bRet = true;  
    for (int i = 0; i < cmd->num; ++i) {  
        iRet = m_commInstance->GetSafeParaCommon(  
            cmd->key, cmd->offset + i,  
            cmd->len, cmd->buf + cmd->len * i,  
            cmd->forceFlag);  
        bRet &= (iRet == 0);  
    }  
  
    if (!bRet) {  
        emit CommunicationEngine::instance()  
            ->signal_safety_readparamsfrommcudirect(  
                absCmd->m_object, false, cmd->key);  
        break;  
    }  
}
```

```

        emit CommunicationEngine::instance()
            ->signal_safety_readparamsfrommcudirect(
                absCmd->m_object, iRet == 0, cmd->key);
        break;
    }
case AbstractCmd::CmdType_WriteSafeParamsToMcuDirect: {
    CmdWriteSafeParamToMcuDirect *cmd
        = static_cast<CmdWriteSafeParamToMcuDirect *>(absCmd);

    bool bRet = true;
    for (int i = 0; i < cmd->num; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
            cmd->key, cmd->offset + i,
            cmd->len, cmd->buf + cmd->len * i,
            cmd->forceFlag);

        bRet &= (iRet == 0);
    }

    if (!bRet) {
        emit CommunicationEngine::instance()
            ->signal_safety_writeparamstomcudirect(
                absCmd->m_object, false, cmd->key);
        break;
    }

    int iRet = m_commInstance->SafetyWriteParams(cmd->key, cmd->offset,
                                                    cmd->num);

    emit CommunicationEngine::instance()
        ->signal_safety_writeparamstomcudirect(
            absCmd->m_object, iRet == 0, cmd->key);
    break;
}

```

```

C++
// safetyinterface.cpp
int SafetyInterface::SafetyReadParams(int key, int offset, int
num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Read(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret

```

```

= %5")
        .arg(__FUNCTION__,
              QString::number(key),
              QString::number(offset),
              QString::number(num),
              QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::GetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->GetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
              .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::SetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->SetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
              .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));

```

```

        FREQ_LOG_PRINT_TIMESTAMP
        return iRet;
    }

    int SafetyInterface::SafetyWriteParams(int key, int offset, int
num)
    {
        FREQ_LOG_PRINT_TIMESTAMP_THREAD
        int iRet = _ISafeParaSettingMgr->Write(key, offset, num);
        LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(num),
                    QString::number(iRet)));
        FREQ_LOG_PRINT_TIMESTAMP
        return iRet;
    }

```

12.2 碰撞检测

**(readColProtectionParams/writeColProtectionParam
s/readColProtectionRecommendLevel/resetColProtect
ionRecommendLevel)**

12.2.1 说明

接口	功能	返回值	参数	参数说明
int CrashDetectIn terface::readC olProtectionP arams(Collisio nProtectionP arams ¶ms)	读取碰撞检测 参数	int (0 成功)	params	params: 出 参, 碰撞参数 结构体 (level/strateg y/collidist)

int CrashDetectInterface::writeColProtectionParams(const CollisionProtectionParams ¶ms)	写入碰撞检测参数并保存机型参数	int	params	params: 入参; 写入成功后 _IRCCConfig->SavePf() 持久化
int CrashDetectInterface::readColProtectionRecommendLevel(qint16 &level)	读取推荐碰撞等级	int	level	level: 出参, 推荐等级 (界面用于标记推荐值)
int CrashDetectInterface::resetColProtectionRecommendLevel()	重置推荐碰撞等级	int	—	无参数; 重置后可再次读取推荐等级

```

C++
typedef struct stCollisionProtectionParams
{
    stCollisionProtectionParams()
    {
        level = 0;
        strategy = 0;
        colldist = 0;
    }
    qint16 level;
    qint16 strategy;
    qint32 colldist;
} CollisionProtectionParams;

```

12.2.2 调用

```

C++
// CollisionProtectionForm::updateUI (刷新/显示)

```

```
CommunicationEngine::instance()->enqueueCmd(this,
AbstractCmd::CmdType_CrashDetect_Read);
```

```
C++
// CollisionProtectionForm::on_pbn_synchronize_clicked (同步)
CollisionProtectionParams params;
params.level = ui->slider_level->value();
// strategy: 3 规划停止 / 4 立即停止 / 7 立即回退 (携带 colldist)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_CrashDetect_Write, params);
```

```
C++
// CollisionProtectionForm::on_pbn_resetvalue_clicked (重置推荐值)
CommunicationEngine::instance()->enqueueCmd(
    this, AbstractCmd::CmdType_CrashDetect_ResetRecommandValue);
```

12.2.3 函数实现

```
C++
// communicationthread.cpp (void
CommunicationThread::processTasks(AbstractCmd *absCmd) 节选：碰撞检测)
case AbstractCmd::CmdType_CrashDetect_Read: {
    CollisionProtectionParams params;
    int iRet = Communication::instance()-
>readColProtectionParams(params);
    emit CommunicationEngine::instance()
        ->signal_readColProtectionParams_result(absCmd->m_object,
iRet == 0, params);
    break;
}
case AbstractCmd::CmdType_CrashDetect_Write: {
    auto [params] = ((CmdDatas<CollisionProtectionParams>
*)absCmd)->m_data;
    int iret = Communication::instance()-
>writeColProtectionParams(params);
    emit CommunicationEngine::instance()
        ->signal_writeColProtectionParams_result(absCmd->m_object,
iret == 0);
```

```

        break;
    }
    case AbstractCmd::CmdType_CrashDetect_ReadRecommandValue: {
        qint16 level = 0;
        int iRet = Communication::instance()-
>readColProtectionRecommandLevel(level);
        emit CommunicationEngine::instance()
            ->signal_readColProtectionRecommandLevel_result(absCmd-
>m_object, iRet == 0, level);
        break;
    }
    case AbstractCmd::CmdType_CrashDetect_ResetRecommandValue: {
        int iRet = Communication::instance()-
>resetColProtectionRecommandLevel();

        if (iRet != 0) {
            emit CommunicationEngine::instance()
                -
>signal_resetColProtectionRecommandLevel_result(absCmd->m_object,
iRet == 0, 0);
            break;
        }

        qint16 level = 0;
        iRet = Communication::instance()-
>readColProtectionRecommandLevel(level);
        emit CommunicationEngine::instance()
            ->signal_resetColProtectionRecommandLevel_result(absCmd-
>m_object, iRet == 0, level);
        break;
    }
}

```

```

C++
//
src/libs/communication/interfaceconversionlayer/crashdetectinterfa
ce.cpp
int
CrashDetectInterface::readColProtectionParams(CollisionProtectiont
Params &params)
{
    InoRobBusiness::CobotColDetectPara colParams;
    int iret = _IRobotArm->getCrashDetectionObject()-
>readColDetectPara(colParams);
}

```

```

        memcpy(&params, &colParams, sizeof(colParams));
        return iret;
    }

    int CrashDetectInterface::writeColProtectionParams(const
    CollisionProtectionParams &params)
    {
        InoRobBusiness::CobotColDetectPara colParams;
        memcpy(&colParams, &params, sizeof(params));
        int iret = _IRobotArm->getCrashDetectionObject()-
        >writeColDetectPara(colParams);
        if (iret != 0) {
            return iret;
        }
        _IRCCConfig->SavePf();
        return iret;
    }

    int CrashDetectInterface::readColProtectionRecommendLevel(qint16
    &level)
    {
        int iret = _IRobotArm->getCrashDetectionObject()-
        >readColDetectRecommadLevel(level);
        return iret;
    }

    int CrashDetectInterface::resetColProtectionRecommendLevel()
    {
        int iret = _IRobotArm->getCrashDetectionObject()-
        >resetColDetectRecommadLevel();
        return iret;
    }

```

12.3 安全原点监控

(SafetyReadParams/SafetyWriteParams)

12.3.1 说明

接口	功能	返回值	参数	参数说明
----	----	-----	----	------

int SafetyInterface::SafetyReadParams(int key, int offset, int num)	从安全 MCU 读取安全原点参数到模型层	int	key, offset, num	key: Cobot_EEP_HOMINGoffset: 0num: 1
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	写入安全原点参数到安全 MCU	int	key, offset, num	含义同上

```

C++
typedef struct stCobotSafetyHoming
{
    uint8_t u8SafePointActiveWay;          /* 生效方式 */
    /*
    uint8_t u8SafePointActiveMode;        /* 生效模式
    0-手动 1-自动 2-手动+自动 */
    float f32SafeErrorThreshold[U_COBOT_AXIS_NUM]; /* 轴误差阈值 */
    /*
    float f32SafePointValue[U_COBOT_AXIS_NUM]; /* 轴安全原点
    设置值 */
} CobotSafetyHoming;

```

12.3.2 调用

```

C++
// HomeMonitorForm::updateUI
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_HOMING, 0, 1, sizeof(CobotSafetyHoming),
    (unsigned char *)&m_homing, false);

```

```

C++
// HomeMonitorForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_HOMING, 0, 1, sizeof(CobotSafetyHoming),

```

```
(unsigned char *)&m_homing, false);
```

C++

```
// HomeMonitorForm::on_pbn_movehere_pressed (移动到安全原点)
CommunicationEngine::instance()-
>enqueueCmd_robotMoveJointTeach(this, pt);
```

12.3.3 函数实现

安全参数 read/write direct 的线程分发与接口实现见 **12.1.3** (已给出完整代码)。补充「移动到安全原点」所用的关节示教完整实现如下。

C++

```
// communicationengine.cpp
void CommunicationEngine::enqueueCmd_robotMoveJointTeach(
    QObject *object, const RoadPoint &roadPoint)
{
    if (!m_isQuit) {
        CmdRobotMoveJoint *cmd = new CmdRobotMoveJoint;
        cmd->m_object = object;
        cmd->m_cmdType
            = AbstractCmd::CmdType_RobotMoveJointTeach;
        cmd->m_roadPoint = roadPoint;

        appendCmdInfo(cmd);
    }
}
```

C++

```
// communicationthread.cpp (void
CommunicationThread::processTasks(AbstractCmd *absCmd) 节选)
case AbstractCmd::CmdType_RobotMoveJointTeach: {
    qDebug() << "CmdType_RobotMoveJointTeach";
    if (!Communication::instance()->IsEnable()) {
        PRINT_MSG(tr("Please enable the robot first.));
        break;
    }

    emit CommunicationEngine::instance()->signal_contimotion();
```

```

CmdRobotMoveJoint *cmd
    = static_cast<CmdRobotMoveJoint *>(absCmd);
bool isSuccess = m_commInstance->robotMoveJointTeachInterface(
    cmd->m_roadPoint);
emit CommunicationEngine::instance()
    ->signal_axisMoveInterface_result(absCmd->m_object,
isSuccess);
    break;
}

```

```

C++
// robotcontrolinterface.cpp
bool RobotControlInterface::robotMoveJointTeachInterface(
    const RoadPoint &roadPoint)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    bool ret = true;

    ret = _IMotion->JointMoveToPointStart(
        MetaTypeConversion::tp2InoApi_roadPoint(roadPoint));

    if (ret != 0) {
        LOG_INFO(QString("robotMoveJointTeachInterface
ret : %1").arg(QString::number(ret)));
    }

    FREQ_LOG_PRINT_TIMESTAMP
    return ret;
}

```

12.4 位置不匹配监控

(SetSafeParaCommon/SafetyWriteParams)

12.4.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将多组位置不匹配阈值写入模型层	int	key, offset, length, buf, forceFlag	key: Cobot_EEP_POSITIONDISMATCH_0 (组参数) 或 Cobot_EEP_POSITIONDISMATCHWAY (生效方式) offset: 组索引 (0 ~ 7) length: 单组长度 (sizeof(CobotPosDismatchUnit)) 或 sizeof(uint8) buf: 指向 CobotPosDismatch 内部数据块/生效方式值 forceFlag: 是否强制覆盖 (函数实现 12.1)
int SafetyInterface::SafetyWriteParams(int *range, int num)	批量将多个键写入安全 MCU	int	range, num	range: 键值数组 (8 组 + 1 个 way, 共 9 个 key) num: 数组长度 (9)

```

C++
typedef struct stCobotPosDismatchUnit
{
    uint8_t u8PmUnitSwitch;

```



```

uint8_t u8PmUnitStopWay;
float f32PmUnitMax[COBOT_POSDISMATCH_LIMIT_NUM];
float f32ReduceModePmUnitMax[COBOT_POSDISMATCH_LIMIT_NUM];
} CobotPosDismatchUnit;

typedef struct stCobotPosDismatch
{
    uint8_t u8PdmActiveWay;
    CobotPosDismatchUnit
stPosMatchGrop[COBOT_POSDISMATCH_GROUP_NUM];
} CobotPosDismatch;

```

12.4.2 调用

```

C++
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_POSITIONDISMATCH_0, 0,
COBOT_POSDISMATCH_GROUP_NUM,
    sizeof(CobotPosDismatchUnit),
    (unsigned char *)&m_posdismatch.stPosMatchGrop[0], false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_POSITIONDISMATCHWAY, 0, 1, sizeof(quint8),
    (unsigned char *)&m_posdismatch.u8PdmActiveWay, false);

```

```

C++
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySnycPositionDismatch,
m_posdismatch);

```

12.4.3 函数实现

```

C++
// src/libs/communication/threadengine/communicationthread.cpp
case AbstractCmd::CmdType_SafetySnycPositionDismatch: {
    auto [params] = ((CmdDatas<CobotPosDismatch> *)absCmd)-
>m_data;
    bool bRet = true;

```

```

// 将轴速度参数写入模型层
for (int i = 0; i < COBOT_POSDISMATCH_GROUP_NUM; ++i) {
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_POSITIONDISMATCH_0, i,
sizeof(CobotPosDismatchUnit),
        (unsigned char *)&params.stPosMatchGrop[0] +
sizeof(CobotPosDismatchUnit) * i,
        false);

    bRet &= (iRet == 0);
}

// 将生效方式值写入模型层
int iRet = m_commInstance->SetSafeParaCommon(
    Cobot_EEP_POSITIONDISMATCHWAY, 0, sizeof(quint8),
    (unsigned char *)&params.u8PdmActiveWay, false);

bRet &= (iRet == 0);

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_syncPositionDismatch_result(absCmd-
>m_object, false);
    break;
}

// 将多条模型层的数据写入到 mcu
int keyArr[9]
    = {Cobot_EEP_POSITIONDISMATCH_0,
Cobot_EEP_POSITIONDISMATCH_0 + 1, Cobot_EEP_POSITIONDISMATCH_0 +
2,
        Cobot_EEP_POSITIONDISMATCH_0 + 3,
Cobot_EEP_POSITIONDISMATCH_0 + 4, Cobot_EEP_POSITIONDISMATCH_0 +
5,
        Cobot_EEP_POSITIONDISMATCH_0 + 6,
Cobot_EEP_POSITIONDISMATCH_0 + 7,
        Cobot_EEP_POSITIONDISMATCHWAY};
iRet = m_commInstance->SafetyWriteParams(keyArr, 9);

emit CommunicationEngine::instance()
    ->signal_safety_syncPositionDismatch_result(absCmd-
>m_object, iRet == 0);
break;
}

```

12.5 拖动示教 TCP 速度监控 (SafetyReadParams/SafetyWriteParams)

12.5.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SafetyReadParams(int key, int offset, int num)	读取拖动示教 监控参数	int	key, offset, num	key: Cobot_EEP_ DRAGTEACH offset: 0num: 1
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	写入拖动示教 监控参数	int	key, offset, num	含义同上

```
C++  
typedef struct stCobotDragTeachInterface  
{  
    uint8_t u8active;  
    uint8_t u8StopCat;  
    float tcpVel;  
} CobotDragTeachInterface;
```

12.5.2 调用

```
C++  
// DragTeachMonitorForm::updateUI
```

```
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_DRAGTEACH, 0, 1,
    sizeof(CobotDragTeachInterface),
    (unsigned char *)&m_dragTeach, false);
```

```
C++
// DragTeachMonitorForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_DRAGTEACH, 0, 1,
    sizeof(CobotDragTeachInterface),
    (unsigned char *)&m_dragTeach, false);
```

12.5.3 函数实现

同 12.1.3（通用安全参数 read/write direct 流程，已给出完整代码）。

12.6 安全静止监控
(SafetySnycSafetyStatic/SafetyWriteParams)

12.6.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将静止监控各组参数写入模型层	int	key, offset, length, buf, forceFlag	key: Cobot_EEP_STATIC_0 （组参数）或 Cobot_EEP_STATICWAY （生效方式） offset: 组索

				引 (0 ~ 7) length: 单组 长度 (sizeof(CobotStaticMonitorUnit)) 或 sizeof(quint8) buf: 指向 CobotStaticMonitor 内部数 据块/生效方式 值 forceFlag: 是否强制覆盖
int SafetyInterface::SafetyWriteParams(int *range, int num)	批量将多个键 写入安全 MCU	int	range, num	range: 键值 数组 (8 组 + 1 个 way, 共 9 个 key) num: 数组长 度 (9)

```

C++
typedef struct stCobotStaticMonitorUnit
{
    uint8_t u8BsSwitch;
    uint8_t u8BsStopWay;
    float f32ToleranceThreshold[COBOT_STATIC_LIMIT_NUM];
    uint8_t u8AposSwitch[COBOT_STATIC_LIMIT_NUM];
} CobotStaticMonitorUnit;

typedef struct stCobotStaticMonitor
{
    uint8_t u8ActiveWay;
    CobotStaticMonitorUnit
    stSafeBodyStaticGroup[COBOT_STATIC_GROUP_NUM];
} CobotStaticMonitor;

```

12.6.2 调用

C++

```

// StopMonitorStaticForm::updateUI
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_STATIC_0, 0, COBOT_STATIC_GROUP_NUM,
    sizeof(CobotStaticMonitorUnit),
    (unsigned char *)&m_staticParams.stSafeBodyStaticGroup[0],
    false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_STATICWAY, 0, 1, sizeof(quint8),
    (unsigned char *)&m_staticParams.u8ActiveWay, false);

```

```

C++
// StopMonitorStaticForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySnycSafetyStatic,
    m_staticParams);

```

12.6.3 函数实现

```

C++
// src/libs/communication/threadengine/communicationthread.cpp
case AbstractCmd::CmdType_SafetySnycSafetyStatic: {
    auto [params] = ((CmdDatas<CobotStaticMonitor> *)absCmd)-
>m_data;
    bool bRet = true;
    // 将轴速度参数写入模型层
    for (int i = 0; i < COBOT_STATIC_GROUP_NUM; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
            Cobot_EEP_STATIC_0, i, sizeof(CobotStaticMonitorUnit),
            (unsigned char *)&params.stSafeBodyStaticGroup[0] +
            sizeof(CobotStaticMonitorUnit) * i,
            false);

        bRet &= (iRet == 0);
    }

    // 将生效方式值写入模型层
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_STATICWAY, 0, sizeof(quint8),

```

```

        (unsigned char *)&params.u8ActiveWay, false);

bRet &= (iRet == 0);

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_syncSafetyStatic_result(absCmd-
>m_object, false);
    break;
}

// 将多条模型层的数据写入到 mcu
int keyArr[9]
    = {Cobot_EEP_STATIC_0, Cobot_EEP_STATIC_0 + 1,
Cobot_EEP_STATIC_0 + 2,
        Cobot_EEP_STATIC_0 + 3, Cobot_EEP_STATIC_0 + 4,
Cobot_EEP_STATIC_0 + 5,
        Cobot_EEP_STATIC_0 + 6, Cobot_EEP_STATIC_0 + 7,
Cobot_EEP_STATICWAY};
iRet = m_commInstance->SafetyWriteParams(keyArr, 9);

emit CommunicationEngine::instance()
    ->signal_safety_syncSafetyStatic_result(absCmd->m_object,
iRet == 0);
break;
}

```

13.关节监控

13.1 速度 (SetSafeParaCommon/SafetyWriteParams)

13.1.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length,	将 UI 编辑后的参数写入模型层	int	key, offset, length, buf, forceFlag	key: 写入的安全参数键 (本节: Cobot_EEP_AVEL_0 / Cobot_EEP_

unsigned char *buf, bool forceFlag)				REDUCEMO DE_AVEL_0 / Cobot_EEP_ AVELWAY) offset: 条目 索引 (组数 组: 0~7; way: 0) length: 单条 长度 (sizeof(Cobo tAVel) / sizeof(CobotR educeModeA Vel) / sizeof(uint8)) buf: 指向对 应数据块 (按 offset 偏移) forceFlag: 是 否强制覆盖 (常用 false)
int SafetyInterfac e::SafetyWrite Params(int *range, int num)	批量写入多个 key 到安全 MCU	int	range, num	range: key 数 组 (本节共 17 个: AVEL 8 + ReduceAVel 8 + AVELWay 1) num: 数 组长度 (17)

src/libs/utils/metatype/safety/safetydefines.h (节选) :

```
C++
typedef struct stCobotAVel {
    stCobotAVel()
    {
        u8active = 0;
        u8StopCat = 0;
```



```

        memset(f64AVelMax, 0x00, sizeof(f64AVelMax));
    }

    quint8 u8active;                /* 生效 */
    quint8 u8StopCat;               /* 停机 */
    double f64AVelMax[COBOT_AVEL_LIMIT_NUM]; /* 最大速度限制 */
} CobotAVel;

typedef struct stCobotReduceModeAVel
{
    double f64AVelReduceModeMax[U_COBOT_AXIS_NUM]; /*
缩减模式下最大速度限制(轴速度) */
} CobotReduceModeAVel;

typedef struct stCobotSAxis {
    quint8 u8APosWay;                /* 轴位置生效方式 */
    quint8 u8AVelWay;               /* 轴速度生效方式 */
    CobotAPos APosGroup[COBOT_APOS_GROUP_NUM];
    CobotAVel AVelGroup[COBOT_AVEL_GROUP_NUM];
} CobotSAxis;

typedef struct stCobotReduceMode
{
    CobotReduceModeAPos stReduceModeApos[COBOT_APOS_GROUP_NUM];
    CobotReduceModeAVel stReduceModeAvel[COBOT_AVEL_GROUP_NUM];
    CobotReduceModeCVel stReduceModeCvel;
} CobotReduceMode;

```

13.1.2 调用

```

C++
// AxisMonitorSpeedForm::updateUI (刷新: 读 AVEL、AVELWAY、
REDUCEMODE_AVEL)
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_AVEL_0, 0, COBOT_APOS_GROUP_NUM,
    sizeof(CobotAVel),
    (unsigned char *)&m_axis.AVelGroup[0], false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(

```

```

        this, Cobot_EEP_AVELWAY, 0, 1, sizeof(quint8),
        (unsigned char *)&m_axis.u8AVelWay, false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_REDUCEMODE_AVEL_0, 0, COBOT_APOS_GROUP_NUM,
    sizeof(CobotReduceModeAVel),
    (unsigned char *)&m_reduceAVel.stReduceModeAVel[0], false);

```

```

C++
// AxisMonitorSpeedForm::on_pbn_synchronize_clicked (同步: 线程聚合
写)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySncAxisSpeed, m_axis,
    m_reduceAVel);

```

13.1.3 函数实现

```

C++
// communicationthread.cpp (void
CommunicationThread::processTasks(AbstractCmd *absCmd))
case AbstractCmd::CmdType_SafetySncAxisSpeed: {
    auto [params, reducemodeParams] = ((CmdDatas<CobotSAxis,
CobotReduceMode> *)absCmd)->m_data;

    bool bRet = true;
    // 将轴速度参数写入模型层
    for (int i = 0; i < COBOT_AVEL_GROUP_NUM; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
            Cobot_EEP_AVEL_0, i, sizeof(CobotAVel),
            (unsigned char *)&params.AVelGroup[0] +
sizeof(CobotAVel) * i,
            false);

        bRet &= (iRet == 0);
    }

    // 将缩减模式轴速度
    for (int i = 0; i < COBOT_AVEL_GROUP_NUM; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(

```

```

        Cobot_EEP_REDUCEMODE_AVEL_0, i,
sizeof(CobotReduceModeAVel),
        (unsigned char *)&reducemodeParams.stReduceModeAVel[0]
+ sizeof(CobotReduceModeAVel) * i,
        false);

    bRet &= (iRet == 0);
}

// 将生效方式值写入模型层
int iRet = m_commInstance->SetSafeParaCommon(
    Cobot_EEP_AVELWAY, 0, sizeof(quint8),
    (unsigned char *)&params.u8AVelWay, false);

bRet &= (iRet == 0);

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_syncAxisSpeed_result(absCmd->m_object,
false);
    break;
}

// 将多条模型层的数据写入到 mcu
int keyArr[17]
    = {Cobot_EEP_AVEL_0, Cobot_EEP_AVEL_0 + 1,
Cobot_EEP_AVEL_0 + 2,
        Cobot_EEP_AVEL_0 + 3, Cobot_EEP_AVEL_0 + 4,
Cobot_EEP_AVEL_0 + 5,
        Cobot_EEP_AVEL_0 + 6, Cobot_EEP_AVEL_0 + 7,
        Cobot_EEP_REDUCEMODE_AVEL_0,
Cobot_EEP_REDUCEMODE_AVEL_0 + 1,
        Cobot_EEP_REDUCEMODE_AVEL_0 + 2,
Cobot_EEP_REDUCEMODE_AVEL_0 + 3,
        Cobot_EEP_REDUCEMODE_AVEL_0 + 4,
Cobot_EEP_REDUCEMODE_AVEL_0 + 5,
        Cobot_EEP_REDUCEMODE_AVEL_0 + 6,
Cobot_EEP_REDUCEMODE_AVEL_0 + 7,
        Cobot_EEP_AVELWAY};
iRet = m_commInstance->SafetyWriteParams(keyArr, 17);

emit CommunicationEngine::instance()
    ->signal_safety_syncAxisSpeed_result(absCmd->m_object,
iRet == 0);

```

```
break;
}
```

13.2 位置 (SetSafeParaCommon/SafetyWriteParams)

13.2.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将 UI 编辑后的参数写入模型层	int	key, offset, length, buf, forceFlag	key: Cobot_EEP_APOS_0 / Cobot_EEP_REDUCEMODE_APOS_0 / Cobot_EEP_APOSWAYoffset: 组索引 (0 ~ 7; way 为 0) length: sizeof(CobotAPos) / sizeof(CobotReduceModeAPos) / sizeof(uint8) buf: 指向对应数据块 (按 offset 偏移) forceFlag: 是否强制覆盖
int SafetyInterface::SafetyWriteParams(int *range, int	批量写入多个 key 到安全 MCU	int	range, num	range: key 数组 (APos 8 + ReduceAPos 8 + APosWay 1) num: 17

num)				
------	--	--	--	--

```
C++
typedef struct stCobotAPos {
    stCobotAPos()
    {
        u8Active = 0;
        u8StopCat = 0;
        memset(f64APosMax, 0x00, sizeof(f64APosMax));
        memset(f64APosMin, 0x00, sizeof(f64APosMin));
    }

    quint8 u8Active;
    quint8 u8StopCat;
    double f64APosMax[COBOT_APOS_LIMIT_NUM];
    double f64APosMin[COBOT_APOS_LIMIT_NUM];
} CobotAPos;

typedef struct stCobotReduceModeAPos
{
    double f64APosReduceModeMax[U_COBOT_AXIS_NUM];
    double f64APosReduceModeMin[U_COBOT_AXIS_NUM];
} CobotReduceModeAPos;
```

13.2.2 调用

```
C++
// AxisMonitorPositionForm::updateUI (刷新: 读 APOS、APOSWAY、
REDUCEMODE_APOS)
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_APOS_0, 0, COBOT_APOS_GROUP_NUM,
    sizeof(CobotAPos),
    (unsigned char *)&m_axis.APosGroup[0], false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_APOSWAY, 0, 1, sizeof(quint8),
    (unsigned char *)&m_axis.u8APosWay, false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
```

```

this, Cobot_EEP_REDUCEMODE_APOS_0, 0, COBOT_APOS_GROUP_NUM,
sizeof(CobotReduceModeApos),
(unsigned char *)&m_reduceApos.stReduceModeApos[0], false);

```

C++

// AxisMonitorPositionForm::on_pbn_synchronize_clicked (同步: 线程聚合写)

```

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySsyncAxisPosition, m_axis,
    m_reduceApos);

```

13.2.3 函数实现

C++

// communicationthread.cpp (void

CommunicationThread::processTasks(AbstractCmd *absCmd))

case AbstractCmd::CmdType_SafetySsyncAxisPosition: {

```

    auto [params, reduceParams] = ((CmdDatas<CobotSAxis,
CobotReduceMode> *)absCmd)->m_data;

```

```

    bool bRet = true;

```

// 将轴位置参数写入模型层

```

for (int i = 0; i < COBOT_AVEL_GROUP_NUM; ++i) {
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_APOS_0, i, sizeof(CobotApos),
        (unsigned char *)&params.AposGroup[0] +
sizeof(CobotApos) * i,
        false);

```

```

    bRet &= (iRet == 0);

```

```

}

```

// 将轴位置缩减模式参数写入模型层

```

for (int i = 0; i < COBOT_AVEL_GROUP_NUM; ++i) {
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_REDUCEMODE_APOS_0, i,
sizeof(CobotReduceModeApos),
        (unsigned char *)&reduceParams.stReduceModeApos[0] +
sizeof(CobotReduceModeApos) * i,
        false);

```

```

        bRet &= (iRet == 0);
    }

    // 将生效方式值写入模型层
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_APOSWAY, 0, sizeof(quint8),
        (unsigned char *)&params.u8APosWay, false);

    bRet &= (iRet == 0);

    if (!bRet) {
        emit CommunicationEngine::instance()
            ->signal_safety_syncAxisPosition_result(
                absCmd->m_object, false);
        break;
    }

    // 将多条模型层的数据写入到 mcu
    int keyArr[17]
        = {Cobot_EEP_APOS_0, Cobot_EEP_APOS_1, Cobot_EEP_APOS_2,
            Cobot_EEP_APOS_3, Cobot_EEP_APOS_4, Cobot_EEP_APOS_4 +
1,
            Cobot_EEP_APOS_6, Cobot_EEP_APOS_7,
            Cobot_EEP_REDUCEMODE_APOS_0,
Cobot_EEP_REDUCEMODE_APOS_0 + 1,
            Cobot_EEP_REDUCEMODE_APOS_0 + 2,
Cobot_EEP_REDUCEMODE_APOS_0 + 3,
            Cobot_EEP_REDUCEMODE_APOS_0 + 4,
Cobot_EEP_REDUCEMODE_APOS_0 + 5,
            Cobot_EEP_REDUCEMODE_APOS_0 + 6,
Cobot_EEP_REDUCEMODE_APOS_0 + 7,
            Cobot_EEP_APOSWAY};
    iRet = m_commInstance->SafetyWriteParams(keyArr, 17);

    emit CommunicationEngine::instance()
        ->signal_safety_syncAxisPosition_result(
            absCmd->m_object, iRet == 0);
    break;
}

```

14.TCP 监控

14.1 速度 (SetSafeParaCommon/SafetyWriteParams)

14.1.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将笛卡尔速度/生效方式写入模型层	int	key, offset, length, buf, forceFlag	key: Cobot_EEP_CVEL_0 / Cobot_EEP_CVELWAY / Cobot_EEP_REDUCEMODE_CVELoffset: 数组型 key 的组索引 (0~7) ; 单条 key 为 0length: sizeof(CobotCVel) / sizeof(quint8) / sizeof(CobotReduceModeCVel)buf: 指向数据块 (按 offset 偏移) forceFlag: 是否强制覆盖 (常用 false)
int SafetyInterface::SafetyWriteParams(int *range, int num)	批量写入多个 key 到安全 MCU	int	range, num	range: 本节共 10 个 key (CVEL 8 + CVELWAY + REDUCEMODE_CVEL)

				num: 数组长度 (10)
--	--	--	--	----------------

```

C++
typedef struct stCobotReduceModeCVel
{
    double f64CVelReduceModeMax[COBOT_CVEL_GROUP_NUM];
} CobotReduceModeCVel;

typedef struct stCobotCVel {
    stCobotCVel()
    {
        u8Active = 0;
        u8StopCat = 0;
        f64CVelMax = 0;
    }
    quint8 u8Active;
    quint8 u8StopCat;
    double f64CVelMax;
} CobotCVel;

typedef struct stCobotCcartesian {
    quint8 u8CPosWay;
    quint8 u8CVelWay;
    // ...
    CobotCVel CVelGroup[COBOT_IZONE_CVEL_ARRAY_LENGTH];
} CobotCcartesian;

```

14.1.2 调用

```

C++
// CartesianMonitoSpeedForm::updateUI (刷新)
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_CVEL_0, 0, COBOT_IZONE_CVEL_ARRAY_LENGTH,
    sizeof(CobotCVel),
    (unsigned char *)&m_carteParas.CVelGroup[0], false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_CVELWAY, 0, 1, sizeof(quint8),
    (unsigned char *)&m_carteParas.u8CVelWay, false);

```

```

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_REDUCEMODE_CVEL, 0, 1,
    sizeof(CobotReduceModeCVel),
    (unsigned char *)&m_reduceCVel, false);

```

```

C++
// CartesianMonitoSpeedForm::on_pbn_synchronize_clicked (同步: 线程
聚合写)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySyncCartesianSpeed,
    m_carteParas, m_reduceCVel);

```

14.1.3 函数实现

```

C++
// communicationthread.cpp (void
CommunicationThread::processTasks(AbstractCmd *absCmd))
case AbstractCmd::CmdType_SafetySyncCartesianSpeed: {
    auto [params, reduceParams] = ((CmdDatas<CobotCcartesian,
CobotReduceModeCVel> *)absCmd)->m_data;

    bool bRet = true;
    // 将笛卡尔速度限制写入模型层
    for (int i = 0; i < COBOT_IZONE_CVEL_ARRAY_LENGTH; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
            Cobot_EEP_CVEL_0, i, sizeof(CobotCVel),
            (unsigned char *)&params.CVelGroup[0] +
sizeof(CobotCVel) * i,
            false);

        bRet &= (iRet == 0);
    }

    // 将生效方式值写入模型层
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_CVELWAY, 0, sizeof(quint8),

```

```

        (unsigned char *)&params.u8CVelWay, false);

bRet &= (iRet == 0);

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_syncCartesianSpeed_result(
            absCmd->m_object, false);
    break;
}

// 将缩减模式值写入模型层
iRet = m_commInstance->SetSafeParaCommon(
    Cobot_EEP_REDUCEMODE_CVEL, 0, sizeof(CobotReduceModeCVel),
    (unsigned char *)&reduceParams, false);

bRet &= (iRet == 0);

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_syncCartesianSpeed_result(
            absCmd->m_object, false);
    break;
}

// 将多条模型层的数据写入到 mcu
int keyArr[10]
    = {Cobot_EEP_CVEL_0, Cobot_EEP_CVEL_0 + 1,
Cobot_EEP_CVEL_0 + 2,
    Cobot_EEP_CVEL_0 + 3, Cobot_EEP_CVEL_0 + 4,
Cobot_EEP_CVEL_0 + 5,
    Cobot_EEP_CVEL_0 + 6, Cobot_EEP_CVEL_0 + 7,
Cobot_EEP_CVELWAY,
    Cobot_EEP_REDUCEMODE_CVEL};
iRet = m_commInstance->SafetyWriteParams(keyArr, 10);

emit CommunicationEngine::instance()
    ->signal_safety_syncCartesianSpeed_result(
        absCmd->m_object, iRet == 0);
break;
}

```

14.2 位置 (SetSafeParaCommon/SafetyWriteParams)

14.2.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	写入干涉区/末端对象概览与生效方式到模型层	int	key, offset, length, buf, forceFlag	key: Cobot_EEP_I ZONE_STAT US_0 / Cobot_EEP_I TOOL_STAT US_0 / Cobot_EEP_ CPOSWAYoff set: 数组索引 (0 ~ 15) 或 0 (way) length: sizeof(Colnter ZoneStatus) 或 sizeof(quint8) buf: 数据块指针 forceFlag: 是否强制覆盖
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	按 idx 写单条 key 到安全 MCU	int	key, offset, num	用于只写变化的 IZONE_STAT US_0+i、 ITOOL_STAT US_0+i; num=1

14.2.2 调用

```

C++
// CartesianMonitorPositionForm::updateUI (刷新)
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_IZONE_STATUS_0, 0,
    CO_IZONE_INTERFERZONESSTATUSLENGTH,
    sizeof(CoInterZoneStatus),
    (unsigned char *)m_cartesian.CPosGroup.IZONES_STATUS, false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_ITOOL_STATUS_0, 0,
    CO_IZONE_INTERFERZONESSTATUSLENGTH,
    sizeof(CoInterZoneStatus),
    (unsigned char *)m_cartesian.CPosGroup.ITOOLS_STATUS, false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_CPOSWAY, 0, 1, sizeof(quint8),
    (unsigned char *)&m_cartesian.u8CPosWay, false);

```

```

C++
// CartesianMonitorPositionForm::on_pbn_synchronize_clicked (同步)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySetInterZoneStatus, 0);

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySyncCartesianPosition,
    m_cartesian, vecZone, vecObject);

```

14.2.3 函数实现

```

C++
// communicationthread.cpp (void
CommunicationThread::processTasks(AbstractCmd *absCmd))
case AbstractCmd::CmdType_SafetySyncCartesianPosition: {
    auto [params, vecZone, vecObject] =
    ((CmdDatas<CobotCcartesian, QVector<int>, QVector<int> >
    *)absCmd)->m_data;

```

```

    bool bRet = true;
    // 将干涉区概览信息写入模型层
    for (int i = 0; i < CO_IZONE_INTERFERZONESSTATUSLENGTH; ++i) {
        if (!vecZone.contains(i)) continue;
        int iRet = m_commInstance->SetSafeParaCommon(
            Cobot_EEP_IZONE_STATUS_0, i,
sizeof(CoInterZoneStatus),
            (unsigned char *)&params.CPosGroup.IZONES_STATUS[0]
            + sizeof(CoInterZoneStatus) * i,
            false);

        bRet &= (iRet == 0);
    }

    // 将末端监测对象概览写入模型层
    for (int i = 0; i < CO_IZONE_INTERFERZONESSTATUSLENGTH; ++i) {
        if (!vecObject.contains(i)) continue;
        int iRet = m_commInstance->SetSafeParaCommon(
            Cobot_EEP_ITOOL_STATUS_0, i,
sizeof(CoInterZoneStatus),
            (unsigned char *)&params.CPosGroup.ITOOLS_STATUS[0]
            + sizeof(CoInterZoneStatus) * i,
            false);

        bRet &= (iRet == 0);
    }

    // 将笛卡尔位置生效写入模型层
    int iRet = m_commInstance->SetSafeParaCommon(
        Cobot_EEP_CPOSWAY, 0, sizeof(quint8),
        (unsigned char *)&params.u8CPosWay, false);

    bRet &= (iRet == 0);

    if (!bRet) {
        emit CommunicationEngine::instance()
            ->signal_safety_syncCartesianPosition_result(
                absCmd->m_object, false);
        break;
    }

    for (size_t i = 0; i < vecZone.size(); ++i) {
        iRet = m_commInstance->

```

```

>SafetyWriteParams(Cobot_EEP_IZONE_STATUS_0, vecZone[i], 1);
    bRet &= (iRet == 0);
}
for (size_t i = 0; i < vecObject.size(); ++i) {
    iRet = m_commInstance-
>SafetyWriteParams(Cobot_EEP_ITOOL_STATUS_0, vecObject[i], 1);
    bRet &= (iRet == 0);
}
iRet = m_commInstance->SafetyWriteParams(Cobot_EEP_CPOSWAY, 0,
1);
bRet &= (iRet == 0);

emit CommunicationEngine::instance()
->signal_safety_syncCartesianPosition_result(
    absCmd->m_object, bRet);
break;
}

```

14.3 力

(SafetyReadParams/GetSafeParaCommon/SetSafeParaCommon/SafetyWriteParams)

14.3.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SafetyReadParams(int key, int offset, int num)	从安全 MCU 读取到模型层	int	key, offset, num	key: Cobot_EEP_CTCPFORCE offset: 0num: 1
int SafetyInterface::GetSafeParaCommon(int	从模型层拷回 UI 缓冲	int	key, offset, length, buf, forceFlag	length=sizeof(CobotTcpForce), buf=&m_tcpfo

key, int offset, int length, unsigned char *buf, bool forceFlag)				rceParams
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将 UI 缓冲写入模型层	int	同上	buf=&m_tcpforceParams
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	写入安全 MCU	int	key, offset, num	同上

C++

```
typedef struct stCobotTcpForceUnit
{
    uint8_t u8TcpUnitSwitch;
    uint8_t u8TcpUnitStopWay;
    float f32TcpForceMax;
    float f32ReduceModeTcpForceMax;
} CobotTcpForceUnit;

typedef struct stCobotTcpForce
{
    uint8_t u8TcpActiveWay;
    CobotTcpForceUnit stTcpForceGrop[COBOT_TCP_FORCE_GROUP_NUM];
} CobotTcpForce;
```

14.3.2 调用


```
C++
// CartesianMonitorStrengthForm::updateUI
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_CTCPFORCE, 0, 1, sizeof(CobotTcpForce),
    (unsigned char *)&m_tcpforceParams, false);
```

```
C++
// CartesianMonitorStrengthForm::on_pbn_synchronize_clicked
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_CTCPFORCE, 0, 1, sizeof(CobotTcpForce),
    (unsigned char *)&m_tcpforceParams, false);
```

14.3.3 函数实现

同 12.1.3

14.4 方向 (SetSafeParaCommon/SafetyWriteParams)

14.4.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将方向监控组 参数/生效方式 写入模型层	int	key, offset, length, buf, forceFlag	key: Cobot_EEP_T CPDIRECTION_0 / Cobot_EEP_T CPDIRECTION_1 offset: 组索引 (0 ~ 7) 或 0

				(way) length: sizeof(CobotTcpDirectionUnit) / sizeof(uint8) buf: 数据块指针 (按 offset 偏移) forceFlag: 常用 false
int SafetyInterface::SafetyWriteParams(int *range, int num)	批量写入多个 key 到安全 MCU	int	range, num	range: 方向组 8 + way 1, 共 9 个 keynum: 9

```

C++
typedef struct stCobotTcpVector
{
    uint8_t u8Switch;
    float f32VectorX;
    float f32VectorY;
    float f32VectorZ;
    float f32Deg;
} CobotTcpVector;

typedef struct stCobotTcpDirectionUnit
{
    uint8_t u8TdSwitch;
    uint8_t u8TdStopWay;
    CobotTcpVector stVectorGroupX;
    CobotTcpVector stVectorGroupZ;
} CobotTcpDirectionUnit;

typedef struct stCobotTcpDirection
{
    uint8_t u8ActiveWay;
    CobotTcpDirectionUnit
stTcpDeGroup[COBOT_TCP_DIRECT_GROUP_NUM];

```

```
} CobotTcpDirection;
```

14.4.2 调用

```
C++
// CartesianMonitorDirectForm::updateUI
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_TCPDIRECTION_0, 0, COBOT_TCP_DIRECT_GROUP_NUM,
    sizeof(CobotTcpDirectionUnit),
    (unsigned char *)&m_tcpDirectionParas.stTcpDeGroup[0], false);

CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_TCPDIRECTIONWAY, 0, 1, sizeof(quint8),
    (unsigned char *)&m_tcpDirectionParas.u8ActiveWay, false);
```

```
C++
// CartesianMonitorDirectForm::on_pbn_synchronize_clicked (线程聚合
写)
CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType_SafetySyncCartesianDirection,
    m_tcpDirectionParas);
```

14.4.3 函数实现

```
C++
// communicationthread.cpp (void
CommunicationThread::processTasks(AbstractCmd *absCmd))
case AbstractCmd::CmdType_SafetySyncCartesianDirection: {
    auto [params] = ((CmdDatas<CobotTcpDirection> *)absCmd)-
>m_data;

    bool bRet = true;
    for (int i = 0; i < COBOT_TCP_DIRECT_GROUP_NUM; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
```

```

        Cobot_EEP_TCPDIRECTION_0, i,
sizeof(CobotTcpDirectionUnit),
        (unsigned char *)&params.stTcpDeGroup[0] +
sizeof(CobotTcpDirectionUnit) * i,
        false);

    bRet &= (iRet == 0);
}

int iRet = m_commInstance->SetSafeParaCommon(
    Cobot_EEP_TCPDIRECTIONWAY, 0, sizeof(quint8),
    (unsigned char *)&params.u8ActiveWay, false);

bRet &= (iRet == 0);

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_syncCartesianDirection_result(
            absCmd->m_object, false);
    break;
}

int keyArr[9]
    = {Cobot_EEP_TCPDIRECTION_0, Cobot_EEP_TCPDIRECTION_0 + 1,
Cobot_EEP_TCPDIRECTION_0 + 2,
        Cobot_EEP_TCPDIRECTION_0 + 3, Cobot_EEP_TCPDIRECTION_0
+ 4, Cobot_EEP_TCPDIRECTION_0 + 5,
        Cobot_EEP_TCPDIRECTION_0 + 6, Cobot_EEP_TCPDIRECTION_0
+ 7, Cobot_EEP_TCPDIRECTIONWAY};
iRet = m_commInstance->SafetyWriteParams(keyArr, 9);

emit CommunicationEngine::instance()
    ->signal_safety_syncCartesianDirection_result(
        absCmd->m_object, iRet == 0);
break;
}

```

15.停机监控

15.1 时间

(GetSafeParaCommon/SetSafeParaCommon/SafetyReadParams/SafetyWriteParams)

15.1.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::GetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	从模型层读取停机时间参数（供UI 显示）	int	key, offset, length, buf, forceFlag	key: 本节为Cobot_EEP_STOPTIME（187） offset: 条目索引（本节为0） length: 单条数据长度（本节为sizeof(CobotStopTime)) buf: 输出缓冲区（拷贝至m_stopTime） forceFlag: 是否强制覆盖（常用false）
int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将UI 侧修改写入模型层（待下发）	int	key, offset, length, buf, forceFlag	含义同上；buf 为输入缓冲（从m_stopTime 写入模型层） m_stopTime.u8StActiveWay: 生效方式（界面“触发方式”下拉框索引） m_stopTime.u32TimeThresholdMax: 最大时间阈值（ms）
int SafetyInterface::SafetyReadParams	从安全MCU 读取	int	key, offset, num	key: Cobot_EEP_STO

(int key, int offset, int num)	停机时间参数到模型层			PTIMEoffset: 0num: 1
int SafetyInterface::SafetyWriteParams (int key, int offset, int num)	将模型层停机时间参数写入安全MCU	int	key, offset, num	含义同上; 写入MCU

```

C++
/* 停机时间监控 */
typedef struct stCobotStopTime
{
    uint8_t u8StActiveWay;          /* 功能生效方式 */
    uint32_t u32TimeThresholdMax;   /* 最大时间阈值 */
} CobotStopTime;

```

```

C++
Cobot_EEP_STOPTIME = 187,          // 停机时间

```

15.1.2 调用

```

C++
// 读取（刷新）
CommunicationEngine::instance()->enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_STOPTIME, 0, 1, sizeof(CobotStopTime),
    (unsigned char *)&m_stopTime, false);

// 写入（同步）
CommunicationEngine::instance()->enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_STOPTIME, 0, 1, sizeof(CobotStopTime),
    (unsigned char *)&m_stopTime, false);

```

15.1.3 函数实现

```
C++
int SafetyInterface::SafetyReadParams(int key, int offset, int
num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Read(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(num),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::GetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->GetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::SetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
```

```

        int iRet = _ISafeParaSettingMgr->SetSafeParaCommon(key,
offset, length, buf, forceFlag);
        LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
        FREQ_LOG_PRINT_TIMESTAMP
        return iRet;
    }

int SafetyInterface::SafetyWriteParams(int key, int offset, int
num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Write(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
            .arg(__FUNCTION__,
                QString::number(key),
                QString::number(offset),
                QString::number(num),
                QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

```

C++

```

case AbstractCmd::CmdType_ReadSafeParamsFromMcuDirect: {
    CmdReadSafeParamFromMcuDirect *cmd
        = static_cast<CmdReadSafeParamFromMcuDirect *>(absCmd);

    int iRet = m_commInstance->SafetyReadParams(cmd->key, cmd-
>offset,
                                                cmd->num);

    if (iRet != 0) {
        emit CommunicationEngine::instance()
            ->signal_safety_readparamsfrommcudirect(
                absCmd->m_object, false, cmd->key);
        break;
    }
}

```



```

// 将模型层数据读取到 GUI
bool bRet = true;
for (int i = 0; i < cmd->num; ++i) {
    iRet = m_commInstance->GetSafeParaCommon(
        cmd->key, cmd->offset + i,
        cmd->len, cmd->buf + cmd->len * i,
        cmd->forceFlag);
    bRet &= (iRet == 0);
}

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_readparamsfrommcudirect(
            absCmd->m_object, false, cmd->key);
    break;
}

emit CommunicationEngine::instance()
    ->signal_safety_readparamsfrommcudirect(
        absCmd->m_object, iRet == 0, cmd->key);
break;
}

case AbstractCmd::CmdType_WriteSafeParamsToMcuDirect: {
    CmdWriteSafeParamToMcuDirect *cmd
        = static_cast<CmdWriteSafeParamToMcuDirect *>(absCmd);

    bool bRet = true;
    for (int i = 0; i < cmd->num; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
            cmd->key, cmd->offset + i,
            cmd->len, cmd->buf + cmd->len * i,
            cmd->forceFlag);

        bRet &= (iRet == 0);
    }

    if (!bRet) {
        emit CommunicationEngine::instance()
            ->signal_safety_writeparamstomcudirect(
                absCmd->m_object, false, cmd->key);
        break;
    }
}

```

```
int iRet = m_commInstance->SafetyWriteParams(cmd->key, cmd->offset,
                                              cmd->num);

emit CommunicationEngine::instance()
    ->signal_safety_writeparamstomcudirect(
        absCmd->m_object, iRet == 0, cmd->key);
break;
}
```

15.2 距离

(GetSafeParaCommon/SetSafeParaCommon/SafetyReadParams/SafetyWriteParams)

15.2.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::GetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	从模型层 读取停机 距离参数 (供 UI/ 表格显示)	int	key, offset, length, buf, forceFlag	key: 本节为 Cobot_EEP_STOPDISTANCE (186) offset: 条目索引 (本节为 0) length: 单条数据长度 (本节为 sizeof(CobotStopDistance)) buf: 输出缓冲 (拷贝至 m_stopDistance) forceFlag: 是否强制覆盖 (常用 false)
int	将 UI 侧	int	key, offset,	含义同上; buf

SafetyInterface::SetSafeParaComm (int key, int offset, int length, unsigned char *buf, bool forceFlag)	修改写入模型层 (待下发)		length, buf, forceFlag	为输入缓冲（从 m_stopDistance 写入模型层） m_stopDistance.u8StActiveWay：生效方式（界面“触发方式”下拉框索引） m_stopDistance.f32DistanceMax：最大距离阈值（界面输入框，代码以 float 保存） m_stopDistance.u8TcpStopDistanceSwitch：TCP 停机距离监控开关 m_stopDistance.u8AposDistanceSwitch：轴停机距离监控开关 m_stopDistance.f32AposDistanceDeg[i]：第 \i\ 轴阈值（deg），由表格每行阈值上限写入
int SafetyInterface::SafetyReadParams(int key, int offset, int num)	从安全 MCU 读取停机距离参数到模型层	int	key, offset, num	key: Cobot_EEP_STOPDISTANCEoffset: 0num: 1
int SafetyInterface::SafetyWriteParams(int key, int offset,	将模型层停机距离参数写入	int	key, offset, num	含义同上；写入 MCU

int num)	安全 MCU			
----------	-----------	--	--	--

```

C++
/* 停机距离监控 */
typedef struct stCobotStopDistance
{
    uint8_t u8StActiveWay;          /* 功能生效方式 */
    float f32DistanceMax;          /* 最大距离阈值 */
    uint8_t u8TcpStopDistanceSwitch; /* TCP 停机距离监控开关 */
    uint8_t u8AposDistanceSwitch;   /* 轴停机距离监控开关 */
    float f32AposDistanceDeg[U_COBOT_AXIS_NUM]; /* 轴距离阈值 deg */
} CobotStopDistance;

```

```

C++
Cobot_EEP_STOPDISTANCE = 186,      // 停机距离

```

15.2.2 调用

```

C++
// 读取（刷新）
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_STOPDISTANCE, 0, 1, sizeof(CobotStopDistance),
    (unsigned char *)&m_stopDistance, false);

// 写入（同步）
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_STOPDISTANCE, 0, 1, sizeof(CobotStopDistance),
    (unsigned char *)&m_stopDistance, false);

```

15.2.3 函数实现

与 15.1.3 完全一致（仅 key/len/buf 等参数不同），本节不重复粘贴。

15.3 位置波动

(GetSafeParaCommon/SetSafeParaCommon/SafetyReadParams/SafetyWriteParams)

15.3.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface: :GetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	从模型层读取 停机抖动（位置波动）容差 （供 UI/表格 显示）	int	key, offset, length, buf, forceFlag	key: 本节为 Cobot_EEP_STOP_TOLERANCE (188) offset: 条目 索引（本节为 0）length: 单条数据长度 （本节为 sizeof(CobotTolerance)) buf: 输出缓冲 （拷贝至 m_stopTolerance) forceFlag: 是否强制覆盖 （常用 false)
int SafetyInterface: :SetSafeParaCommon(int key, int offset, int length, unsigned char	将 UI 侧修改 写入模型层 （待下发）	int	key, offset, length, buf, forceFlag	含义同上; buf 为输入缓冲（从 m_stopTolerance 写入模型 层）

*buf, bool forceFlag)				m_stopTolerance.f32StopDisTolerance[i] : 第 \i\ 轴停机容差阈值 (界面范围 [0.01, 10], 表格每行阈值上限写入)
int SafetyInterface: :SafetyReadParams(int key, int offset, int num)	从安全 MCU 读取停机抖动容差到模型层	int	key, offset, num	key: Cobot_EEP_STOP_TOLERANCEoffset : 0num: 1
int SafetyInterface: :SafetyWriteParams(int key, int offset, int num)	将模型层停机抖动容差写入安全 MCU	int	key, offset, num	含义同上; 写入 MCU

```

C++
typedef struct stCobotTolerance
{
    float f32StopDisTolerance[U_COBOT_AXIS_NUM];    /* 停机容差阈值
    停机距离界面中设置 */
} CobotTolerance;

```

```

C++
Cobot_EEP_STOP_TOLERANCE = 188,    // 停机抖动容差

```

15.3.2 调用

```

C++
// 读取（刷新）
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(

```

```
        this, Cobot_EEP_STOP_TOLERANCE, 0, 1, sizeof(CobotTolerance),
        (unsigned char *)&m_stopTolerance, false);

// 写入（同步）
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_STOP_TOLERANCE, 0, 1, sizeof(CobotTolerance),
    (unsigned char *)&m_stopTolerance, false);
```

15.3.3 函数实现

与 15.1.3 完全一致（仅 key/len/buf 等参数不同），本节不重复粘贴。

16.整机监控

16.1 功率
(ReadSafeParamsFromMcuDirect/WriteSafeParamsToMcuDirect)

16.1.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface::GetSafeParamCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	从模型层读取整机功率监控参数（供 UI 显示）	int	key, offset, length, buf, forceFlag	key: 本节为 Cobot_EEP_BODYPOWER (176) offset: 条目索引（本节为 0） length: 单条数据长度（本节为 sizeof(CobotBodyPower)） buf: 输出缓冲（拷贝至 m_bodyPower） forceFlag: 是否强制覆盖（常用 false）

int SafetyInterface::SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	将 UI 侧修改写入模型层（待下发）	int	key, offset, length, buf, forceFlag	含义同上；buf 为输入缓冲（从 m_bodyPower 写入模型层） m_bodyPower.u8BpActiveWay：功率监控生效方式（界面“触发方式/生效方式”下拉框索引） m_bodyPower.stBodyPowerGrop[i].u8BpUnitSwitch：第 \i\ 组激活开关（界面“激活”） m_bodyPower.stBodyPowerGrop[i].u8BpUnitStopWay：第 \i\ 组停机类型（界面“停机类型”） m_bodyPower.stBodyPowerGrop[i].f32BpUintMax：第 \i\ 组正常模式功率阈值（W） m_bodyPower.stBodyPowerGrop[i].f32ReduceModeBpUintMax：第 \i\ 组缩减模式功率阈值（W）
int SafetyInterface::SafetyReadParams(int key, int offset, int num)	从安全 MCU 读取整机功率参数到模型层	int	key, offset, num	key： Cobot_EEP_BODYPOWERoffset ： 0num： 1
int SafetyInterface::SafetyWriteParams(int key, int offset, int num)	将模型层整机功率参数写入安全 MCU	int	key, offset, num	含义同上；写入 MCU

C++

Cobot_EEP_BODYPOWER = 176, // 整机功率

C++


```

/* 整机功率监控 */
typedef struct stCobotBodyPowerUnit
{
    uint8_t u8BpUnitSwitch;           /* 组激活开关 */
    uint8_t u8BpUnitStopWay;          /* 组停机类型 */
    float f32BpUnitMax;                /* 组整机功率最大阈值 */
    float f32ReduceModeBpUnitMax;     /* 缩减模式下组整机功率
最大阈值 */
} CobotBodyPowerUnit;

typedef struct stCobotBodyPower
{
    uint8_t u8BpActiveWay;
    CobotBodyPowerUnit
stBodyPowerGrop[COBOT_BODY_POWER_GROUP_NUM];
} CobotBodyPower;

```

16.1.2 调用

```

C++
// 读取（刷新）
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_BODYPOWER, 0, 1, sizeof(CobotBodyPower),
    (unsigned char *)&m_bodyPower, false);

// 写入（同步）
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_BODYPOWER, 0, 1, sizeof(CobotBodyPower),
    (unsigned char *)&m_bodyPower, false);

```

16.1.3 函数实现

```

C++
int SafetyInterface::SafetyReadParams(int key, int offset, int
num)

```

```

{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Read(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(num),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::GetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->GetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,
                    QString::number(key),
                    QString::number(offset),
                    QString::number(length),
                    QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::SetSafeParaCommon(int key, int offset, int
length,
                                     unsigned char *buf, bool
forceFlag)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->SetSafeParaCommon(key,
offset, length, buf, forceFlag);
    LOG_INFO(QString("[%1]key = %2, offset = %3, length = %4, ret
= %5")
                .arg(__FUNCTION__,

```

```

        QString::number(key),
        QString::number(offset),
        QString::number(length),
        QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

int SafetyInterface::SafetyWriteParams(int key, int offset, int
num)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _ISafeParaSettingMgr->Write(key, offset, num);
    LOG_INFO(QString("[%1]key = %2, offset = %3, num = %4, ret
= %5")
        .arg(__FUNCTION__,
        QString::number(key),
        QString::number(offset),
        QString::number(num),
        QString::number(iRet)));
    FREQ_LOG_PRINT_TIMESTAMP
    return iRet;
}

```

```

C++
case AbstractCmd::CmdType_ReadSafeParamsFromMcuDirect: {
    CmdReadSafeParamFromMcuDirect *cmd
        = static_cast<CmdReadSafeParamFromMcuDirect *>(absCmd);

    int iRet = m_commInstance->SafetyReadParams(cmd->key, cmd-
>offset,
                                                cmd->num);

    if (iRet != 0) {
        emit CommunicationEngine::instance()
            ->signal_safety_readparamsfrommcudirect(
                absCmd->m_object, false, cmd->key);
        break;
    }

    // 将模型层数据读取到 GUI
    bool bRet = true;
    for (int i = 0; i < cmd->num; ++i) {
        iRet = m_commInstance->GetSafeParaCommon(

```

```

        cmd->key, cmd->offset + i,
        cmd->len, cmd->buf + cmd->len * i,
        cmd->forceFlag);
    bRet &= (iRet == 0);
}

if (!bRet) {
    emit CommunicationEngine::instance()
        ->signal_safety_readparamsfrommcudirect(
            absCmd->m_object, false, cmd->key);
    break;
}

emit CommunicationEngine::instance()
    ->signal_safety_readparamsfrommcudirect(
        absCmd->m_object, iRet == 0, cmd->key);
break;
}
case AbstractCmd::CmdType_WriteSafeParamsToMcuDirect: {
    CmdWriteSafeParamToMcuDirect *cmd
        = static_cast<CmdWriteSafeParamToMcuDirect *>(absCmd);

    bool bRet = true;
    for (int i = 0; i < cmd->num; ++i) {
        int iRet = m_commInstance->SetSafeParaCommon(
            cmd->key, cmd->offset + i,
            cmd->len, cmd->buf + cmd->len * i,
            cmd->forceFlag);

        bRet &= (iRet == 0);
    }

    if (!bRet) {
        emit CommunicationEngine::instance()
            ->signal_safety_writeparamstomcudirect(
                absCmd->m_object, false, cmd->key);
        break;
    }

    int iRet = m_commInstance->SafetyWriteParams(cmd->key, cmd->offset,
                                                    cmd->num);

    emit CommunicationEngine::instance()
        ->signal_safety_writeparamstomcudirect(

```

```
absCmd->m_object, iRet == 0, cmd->key);  
break;  
}
```

16.2 动量
(GetSafeParaCommon/SetSafeParaCommon/SafetyReadParams/SafetyWriteParams)

16.2.1 说明

接口	功能	返回值	参数	参数说明
int SafetyInterface: :GetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool forceFlag)	从模型层读取 整机动量监控 参数 (供 UI 显示)	int	key, offset, length, buf, forceFlag	key: 本节为 Cobot_EEP_BODY MOMENTUM (175) offset: 条目 索引 (本节为 0) length: 单条数据长 度 (本节为 sizeof(CobotBodyM omentum)) buf: 输 出缓冲 (拷贝至 m_bodyMomentum) forceFlag: 是否强 制覆盖 (常用 false)
int SafetyInterface: :SetSafeParaCommon(int key, int offset, int length, unsigned char *buf, bool	将 UI 侧修改 写入模型层 (待下发)	int	key, offset, length, buf, forceFlag	含义同上; buf 为输 入缓冲 (从 m_bodyMomentum 写入模型层) m_bodyMomentum. u8BmActiveWay: 动量监控生效方式 (界面下拉框索引)

forceFlag)				m_bodyMomentum. stBodyMomGrop[i].u 8BpUnitSwitch: 第 i 组激活开关 (界面 “激活”) m_bodyMomentum. stBodyMomGrop[i].u 8BpUnitStopWay: 第 i 组停机类型 (界 面 “停机类型”) m_bodyMomentum. stBodyMomGrop[i].f 32BpUnitMax: 第 i 组正常模式动量阈值 (kgm/s) m_bodyMomentum. stBodyMomGrop[i].f 32ReduceModeBpUi ntMax: 第 i 组缩减 模式动量阈值 (kgm/s)
int SafetyInterface: :SafetyReadPa rams(int key, int offset, int num)	从安全 MCU 读取整机动量 参数到模型层	int	key, offset, num	key: Cobot_EEP_BODY MOMENTUMOffset : 0num: 1
int SafetyInterface: :SafetyWritePar ams(int key, int offset, int num)	将模型层整机 动量参数写入 安全 MCU	int	key, offset, num	含义同上; 写入 MCU

```
C++  
Cobot_EEP_BODYMOMENTUM = 175,          // 整机动量
```

```
C++  
/* 整机动量监控 */  
typedef struct stCobotBodyMomentumUnit
```

```

{
    uint8_t u8BpUnitSwitch;           /* 组激活开关 */
    uint8_t u8BpUnitStopWay;          /* 组停机类型 */
    float f32BpUnitMax;               /* 组整机动量最大阈值 */
    float f32ReduceModeBpUnitMax;     /* 缩减模式下组整机动量
最大阈值 */
} CobotBodyMomentumUnit;

typedef struct stBodyMomentum
{
    uint8_t u8BmActiveWay;
    CobotBodyMomentumUnit
stBodyMomGrop[COBOT_BODY_MOMENTUM_GROUP_NUM];
} CobotBodyMomentum;

```

16.2.2 调用

```

C++
// 读取（刷新）
CommunicationEngine::instance()-
>enqueueCmd_ReadSafetyParamsFromMcuDirect(
    this, Cobot_EEP_BODYMOMENTUM, 0, 1, sizeof(CobotBodyMomentum),
    (unsigned char *)&m_bodyMomentum, false);

// 写入（同步）
CommunicationEngine::instance()-
>enqueueCmd_WriteSafetyParamsToMcuDirect(
    this, Cobot_EEP_BODYMOMENTUM, 0, 1, sizeof(CobotBodyMomentum),
    (unsigned char *)&m_bodyMomentum, false);

```

16.2.3 函数实现

与 16.1.3 完全一致（仅 key/len/buf 不同），本节不重复粘贴。

17.全局变量

17.1 B / R / D / Str / P / JP

17.1.1 说明

接口	功能	返回值	参数	参数说明
void GlobalVarInterface ::setGlobalVarScheduler_B(bool flag)	打开 / 关闭 B 类全局变量调度 (轮询)	void	flag	true/false; 当前界面侧相关 CmdType_SetGlobalVar_Scheduler_B 多为注释未启用。
void GlobalVarInterface ::setGlobalVarScheduler_R(bool flag)	打开 / 关闭 R 类全局变量调度	void	flag	同上。
void GlobalVarInterface ::setGlobalVarScheduler_D(bool flag)	打开 / 关闭 D 类全局变量调度	void	flag	同上。
void GlobalVarInterface ::setGlobalVarScheduler_PR(bool flag)	打开 / 关闭 PR 类全局变量调度	void	flag	声明在接口中; 监控 B/R/D/Str/P/J P 本节主链路未用。
void GlobalVarInterface ::setGlobalVarScheduler_String(bool flag)	打开 / 关闭 Str 类全局变量调度	void	flag	同上。
bool GlobalVarInterface ::getGlobalVar_B_everyCol(QVector<QVector<QVariant>> &)	刷新并读取 B 变量各列到 QVariant 矩阵	bool	data	出参 data: 多列 (名 / 值 / 标签 / 描述 / 收藏等由上层模型约定); 失败表示 _IResource-

				>updateGlobalVarBDatas()等未成功。
bool GlobalVarInterface::getGlobalVar_R_everyCol(QVector<QVector<QVariant>> &)	刷新并读取 R 变量各列	bool	data	同 B，数据源为 updateGlobalVarRDatas / getGlobalVarRDatas。
bool GlobalVarInterface::getGlobalVar_D_everyCol(QVector<QVector<QVariant>> &)	刷新并读取 D 变量各列	bool	data	同 B，双精度列。
bool GlobalVarInterface::getGlobalVar_String_everyCol(QVector<QVariant>> &)	刷新并读取 Str 各列	bool	data	同 B，字符串列。
bool GlobalVarInterface::getGlobalVar_P_everyColForCurrentShow(int ¤tRow, QVector<QVector<QVariant>> &)	按当前滚动位置分页读取 P 各列	bool	currentRow, data	currentRow: 列表行号；内部换算页范围后填充 data。
bool GlobalVarInterface::getGlobalVar_JP_everyColForCurrentShow(int ¤tRow, QVector<QVector<QVariant>> &)	按当前滚动位置分页读取 JP 各列	bool	currentRow, data	同 P，数据源为 getGlobalJPDatas。

bool GlobalVarInterface ::getGlobalVar_B_ valuesCol(QVector <QVariant> &)	仅读 B 值列	bool	data	非 MSVC 路 径。
bool GlobalVarInterface ::getGlobalVar_R_ valuesCol(QVector <QVariant> &)	仅读 R 值列	bool	data	同上。
bool GlobalVarInterface ::getGlobalVar_D_ valuesCol(QVector <QVariant> &)	仅读 D 值列	bool	data	同上。
bool GlobalVarInterface ::getGlobalVar_Stri ng_valuesCol(QVe ctor<QVariant> &)	仅读 Str 值列	bool	data	同上。
bool GlobalVarInterface ::getGlobalVar_P_ valuesCol(QVector <QVector<QVari ant>> &)	P 值列批量读 (占位)	bool	data	当前实现直接 return true, 未拉数。
bool GlobalVarInterface ::getGlobalVar_JP_ _valuesCol(QVect or<QVector<QVari ant>> &)	JP 值列批量 读 (占位)	bool	data	同上。
bool GlobalVarInterface ::setGlobalVar_B_ values(const int index, const unsigned char	写 B 单点值	bool	index, data	index: B 下 标; data: unsigned char。

data)				
bool GlobalVarInterface ::setGlobalVar_R_ values(const int index, const int data)	写 R 单点值	bool	index, data	data: int。
bool GlobalVarInterface ::setGlobalVar_D_ values(const int index, const double data)	写 D 单点值	bool	index, data	data: double。
bool GlobalVarInterface ::setGlobalVar_Stri ng_values(const int index, const QString &data)	写 Str 单点值	bool	index, data	data: 字符 串。
bool GlobalVarInterface ::setGlobalVar_B_f avorite(const int index, const bool isFavorite)	写 B 收藏	bool	index, isFavorite	成功 / 失败由 _IResource 决 定。
bool GlobalVarInterface ::setGlobalVar_R_f avorite(const int index, const bool isFavorite)	写 R 收藏	bool	index, isFavorite	同上。
bool GlobalVarInterface ::setGlobalVar_D_f avorite(const int index, const bool isFavorite)	写 D 收藏	bool	index, isFavorite	同上。

void GlobalVarInterface ::ModifyItemLabel(InoLabelType type, int index, const QString &label)	修改工程标签 (标签列)	void	type, index, label	经 Label 服务 写回工程; 与 CmdType_Get /SetGlobalVar _* 线程命令无 直接对应, 属 标签编辑链 路。
void GlobalVarInterface ::ModifyItemDescription(InoLabelType type, int index, const QString &des)	修改工程标签 描述 (描述 列)	void	type, index, des	同上。

索引	子类	刷新 CmdTy pe	写入 CmdTy pe	收藏 CmdTy pe	结果信号
0	GlobalVarFormB	CmdTy pe_Get Global Var_B_ EveryC ol	CmdTy pe_Set GlobalV ar_B_V alues	CmdTy pe_Set GlobalV ar_B_F avorite	signal_globalVar _B_GetDatas_re sult
1	GlobalVarFormR	CmdTy pe_Get Global Var_R_ EveryC ol	CmdTy pe_Set GlobalV ar_R_V alues	CmdTy pe_Set GlobalV ar_R_F avorite	signal_globalVar _R_GetDatas_re sult
2	GlobalVarFormD	CmdTy pe_Get Global Var_D_ EveryC ol	CmdTy pe_Set GlobalV ar_D_V alues	CmdTy pe_Set GlobalV ar_D_F avorite	signal_globalVar _D_GetDatas_re sult

4	GlobalVarFormString	CmdType_GetGlobalVar_String_EveryCol	CmdType_SetGlobalVar_String_Values	-	signal_globalVar_String_GetData_result
5	GlobalVarFormP	CmdType_GetGlobalVar_P_EveryColForCurrentShow	-	-	signal_globalVar_P_GetData_result
6	GlobalVarFormJP	CmdType_GetGlobalVar_JP_EveryColForCurrentShow	-	-	signal_globalVar_JP_GetData_result

17.1.2 调用

B

```

C++
CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_GetGlobalVar_B_EveryCol,
    m_allItemModel->getAllValuesAddress());

CommunicationEngine::instance()->enqueueCmd_setData(
    this, AbstractCmd::CmdType::CmdType_SetGlobalVar_B_Values,
    row,
    (unsigned char)(data.toInt()));

CommunicationEngine::instance()->enqueueCmd_setData(

```

```
this, AbstractCmd::CmdType::CmdType_SetGlobalVar_B_Favorite,  
row,  
data.toBool());
```

R

C++

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_GetGlobalVar_R_EveryCol,  
    m_allItemModel->getAllValuesAddress());
```

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType::CmdType_SetGlobalVar_R_Values,  
    row,  
    data.toInt());
```

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType::CmdType_SetGlobalVar_R_Favorite,  
    row,  
    data.toBool());
```

D

C++

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_GetGlobalVar_D_EveryCol,  
    m_allItemModel->getAllValuesAddress());
```

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType::CmdType_SetGlobalVar_D_Values,  
    row,  
    data.toDouble());
```

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this, AbstractCmd::CmdType::CmdType_SetGlobalVar_D_Favorite,  
    row,  
    data.toBool());
```

Str

C++

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_GetGlobalVar_String_EveryCol,  
    m_allItemModel->getAllValuesAddress());  
  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_SetGlobalVar_String_Values,  
    row,  
    data.toString());
```

P

C++

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
  
    AbstractCmd::CmdType::CmdType_GetGlobalVar_P_EveryColForCurrentShow,  
    currentShowRowStart,  
    m_allItemModel->getAllValuesAddress());
```

JP

C++

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
  
    AbstractCmd::CmdType::CmdType_GetGlobalVar_JP_EveryColForCurrentShow,  
    currentShowRowStart,  
    m_allItemModel->getAllValuesAddress());
```


17.1.3 函数实现

```
C++
bool
GlobalVarInterface::getGlobalVar_B_everyCol(QVector<QVector<QVariant>> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!_IResource->updateGlobalVarBDatas())
        return false;
    data.clear();
    data = MetaTypeConversion::tp2InoApi_globalVarBVariant(
        _IResource->getGlobalVarBDatas());
    return true;
}

bool GlobalVarInterface::setGlobalVar_B_values(const int index,
const unsigned char data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->setGlobalVarBValue(data, index);
}

bool GlobalVarInterface::setGlobalVar_B_favorite(const int index,
const bool isFavorite)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->setGlobalVarBFavorite(isFavorite, index);
}

bool GlobalVarInterface::getGlobalVar_P_everyColForCurrentShow(int
&currentRow, QVector<QVector<QVariant>> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    //每页 5 个点 读前 4 页 当前页 加后 5 页 一共 10 页
    int currentPage = currentRow / 5;
    int startPage = currentPage - 4;
    int endPage = currentPage + 5;
    int pageNum = startPage - endPage + 1;
    if(startPage < 0)
```



```

        startPage = 0;
        if(endPage > GLOBAL_P_COUNT / 5)
            endPage = GLOBAL_P_COUNT / 5;
        pageNum = endPage - startPage + 1;
        std::vector<InoRobBusiness::GlobalPData> source(pageNum * 5);
        bool ret = _IResource->getGlobalPDatas(source, startPage,
        pageNum, false);
        if (!ret) {
            return false;
        }
        data =
        MetaTypeConversion::tp2InoApi_globalVarPVariant(source);
        return true;
    }

    bool
    GlobalVarInterface::getGlobalVar_JP_everyColForCurrentShow(int
    &currentRow, QVector<QVector<QVariant>> &data)
    {
        FREQ_LOG_PRINT_TIMESTAMP_THREAD

        //每页 5 个点
        int currentPage = currentRow / 5;
        int startPage = currentPage - 4;
        int endPage = currentPage + 5;
        int pageNum = startPage - endPage + 1;
        if(startPage < 0)
            startPage = 0;
        if(endPage > GLOBAL_P_COUNT / 5)
            endPage = GLOBAL_P_COUNT / 5;
        pageNum = endPage - startPage + 1;
        std::vector<InoRobBusiness::GlobalJPData> source(pageNum * 5);
        bool ret = _IResource->getGlobalJPDatas(source, startPage,
        pageNum, false);
        if (!ret) {
            return false;
        }
        data =
        MetaTypeConversion::tp2InoApi_globalVarJPVariant(source);
        return true;
    }

```

(getGlobalVar_R_everyCol / getGlobalVar_D_everyCol /
 getGlobalVar_String_everyCol 与 setGlobalVar_R_values /

setGlobalVar_D_values / setGlobalVar_String_values /
setGlobalVar_*_favorite 在同文件，模式与 B 一致，仅资源接口与类型不同。)

```
C++
case AbstractCmd::CmdType_GetGlobalVar_B_EveryCol: {
#if defined(INOCOBOTTP_MSVC_QT5)
    auto [address]
        = ((CmdDatas<QVector<QVector<QVariant>> *> *)absCmd)->m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;

    if (m_commInstance->getGlobalVar_B_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_B_GetDatas_result(
                        i, index, different);
            }
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_B_GetDatas_result(-1, index,
different);
    }
#else
    auto [address]
        = ((CmdDatas<QList<QList<QVariant>> *> *)absCmd)->m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;

    if (m_commInstance->getGlobalVar_B_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
```

```

        emit CommunicationEngine::instance()
            ->signal_globalVar_B_GetDatas_result(
                i, index, different);
    }
}
} else {
    emit CommunicationEngine::instance()
        ->signal_globalVar_B_GetDatas_result(-1, index,
different);
}
#endif
break;
}
case AbstractCmd::CmdType_GetGlobalVar_R_EveryCol: {
#ifdef INOCOBOTTP_MSVC_QT5
    auto [address]
        = ((CmdDatas<QVector<QVector<QVariant>> *> *)absCmd)-
>m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_R_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_R_GetDatas_result(
                        i, index, different);
            }
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_R_GetDatas_result(-1, index,
different);
    }
} else
    auto [address]
        = ((CmdDatas<QList<QList<QVariant>> *> *)absCmd)->m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;

```

```

    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_R_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_R_GetDatas_result(
                        i, index, different);
            }
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_R_GetDatas_result(-1, index,
different);
    }
#endif
    break;
}

case AbstractCmd::CmdType_GetGlobalVar_D_EveryCol: {
#ifdef INOCOBOTTP_MSVC_QT5
    auto [address]
        = ((CmdDatas<QVector<QVector<QVariant>> *> *)absCmd)-
>m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_D_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_D_GetDatas_result(
                        i, index, different);
            }
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_D_GetDatas_result(-1, index,
different);
    }
}

```

```

    }
#else
    auto [address]
        = ((CmdDatas<QList<QList<QVariant>> *> *)absCmd)->m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_D_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_D_GetDatas_result(
                        i, index, different);
            }
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_D_GetDatas_result(-1, index,
different);
    }
#endif
    break;
}
case AbstractCmd::CmdType_GetGlobalVar_String_EveryCol: {
#ifdef INOCOBOTTP_MSVC_QT5
    auto [address]
        = ((CmdDatas<QVector<QVector<QVariant>> *> *)absCmd)-
>m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_String_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_String_GetDatas_result(

```

```

        i, index, different);
    }
}
} else {
    emit CommunicationEngine::instance()
        ->signal_globalVar_String_GetDatas_result(
            -1, index, different);
}
#else
    auto [address]
        = ((CmdDatas<QList<QList<QVariant>> *> *)absCmd)->m_data;

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_String_everyCol(data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            if (compareAndObtainDifferentDatas(
                (*address)[i], data[i], index, different)) {
                emit CommunicationEngine::instance()
                    ->signal_globalVar_String_GetDatas_result(
                        i, index, different);
            }
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_String_GetDatas_result(
                -1, index, different);
    }
}
#endif
    break;
}
case AbstractCmd::CmdType_GetGlobalVar_P_EveryColForCurrentShow: {
#ifdef INOCOBOTTP_MSVC_QT5
    auto [currentRow, address]
        = ((CmdDatas<int, QVector<QVector<QVariant>> *> *)absCmd)-
>m_data;
#else
    auto [currentRow, address]
        = ((CmdDatas<int, QList<QList<QVariant>> *> *)absCmd)-
>m_data;
#endif
}

```

```

    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_P_everyColForCurrentShow(
        currentRow, data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();
            different.clear();
            for (int j = 0; j < data[0].size(); ++j) {
                int row = data[0][j].toInt();
                if (data[i][j] != (*address)[i][row]) {
                    index.push_back(row);
                    different.push_back(data[i][j]);
                }
            }
            emit CommunicationEngine::instance()
                ->signal_globalVar_P_GetDatas_result(i, index,
different);
        }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_P_GetDatas_result(-1, index,
different);
    }
    break;
}
case AbstractCmd::CmdType_GetGlobalVar_JP_EveryColForCurrentShow:
{
#ifdef INOCOBTTP_MSVC_QT5
    auto [currentRow, address]
        = ((CmdDatas<int, QVector<QVector<QVariant>> *> *)absCmd)-
>m_data;
#else
    auto [currentRow, address]
        = ((CmdDatas<int, QList<QList<QVariant>> *> *)absCmd)-
>m_data;
#endif
    QVector<QVector<QVariant>> data;
    QVector<int> index;
    QVector<QVariant> different;
    if (m_commInstance->getGlobalVar_JP_everyColForCurrentShow(
        currentRow, data)) {
        for (int i = 1; i < data.size(); ++i) {
            index.clear();

```

```

        different.clear();
        for (int j = 0; j < data[0].size(); ++j) {
            int row = data[0][j].toInt();
            if (data[i][j] != (*address)[i][row]) {
                index.push_back(row);
                different.push_back(data[i][j]);
            }
        }
        emit CommunicationEngine::instance()
            ->signal_globalVar_JP_GetDatas_result(i, index,
different);
    }
    } else {
        emit CommunicationEngine::instance()
            ->signal_globalVar_JP_GetDatas_result(-1, index,
different);
    }
    break;
}

case AbstractCmd::CmdType_SetGlobalVar_B_Values: {
    auto [row, data] = ((CmdDatas<int, unsigned char> *)absCmd)->m_data;
    m_commInstance->setGlobalVar_B_values(row, data);
    break;
}

case AbstractCmd::CmdType_SetGlobalVar_R_Values: {
    auto [row, data] = ((CmdDatas<int, int> *)absCmd)->m_data;
    m_commInstance->setGlobalVar_R_values(row, data);
    break;
}

case AbstractCmd::CmdType_SetGlobalVar_D_Values: {
    auto [row, data] = ((CmdDatas<int, double> *)absCmd)->m_data;
    m_commInstance->setGlobalVar_D_values(row, data);
    break;
}

case AbstractCmd::CmdType_SetGlobalVar_String_Values: {
    auto [row, data] = ((CmdDatas<int, QString> *)absCmd)->m_data;
    m_commInstance->setGlobalVar_String_values(row, data);
    break;
}

case AbstractCmd::CmdType_SetGlobalVar_B_Favorite: {
    auto [row, data] = ((CmdDatas<int, bool> *)absCmd)->m_data;
    if (!m_commInstance->setGlobalVar_B_favorite(row, data)) {
        emit CommunicationEngine::instance()

```



```

        ->signal_needMainWidgetWarning(
            tr("Failed to save favorites."));
    }
    break;
}
case AbstractCmd::CmdType_SetGlobalVar_R_Favorite: {
    auto [row, data] = ((CmdDatas<int, bool> *)absCmd)->m_data;
    if (!m_commInstance->setGlobalVar_R_favorite(row, data)) {
        emit CommunicationEngine::instance()
            ->signal_needMainWidgetWarning(
                tr("Failed to save favorites."));
    }
    break;
}
case AbstractCmd::CmdType_SetGlobalVar_D_Favorite: {
    auto [row, data] = ((CmdDatas<int, bool> *)absCmd)->m_data;
    if (!m_commInstance->setGlobalVar_D_favorite(row, data)) {
        emit CommunicationEngine::instance()
            ->signal_needMainWidgetWarning(
                tr("Failed to save favorites."));
    }
    break;
}
case AbstractCmd::CmdType_SetGlobalVar_Scheduler_B: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setGlobalVarScheduler_B(state);
    break;
}
case AbstractCmd::CmdType_SetGlobalVar_Scheduler_R: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setGlobalVarScheduler_R(state);
    break;
}
case AbstractCmd::CmdType_SetGlobalVar_Scheduler_D: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setGlobalVarScheduler_D(state);
    break;
}
case AbstractCmd::CmdType_SetGlobalVar_Scheduler_String: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setGlobalVarScheduler_String(state);
    break;
}
}

```

18.I/O

18.1 数字输入

(setSchedulerInput/getInputControlAuthority/setInputShowType/getInputLabelAndRemarkForCurrentShow/getInputValuesStandard/getInputValuesTool/getInputValuesFieldbus/getInputValuesMemory/writeInputStatusForce/writeInputStatusByBit)

18.1.1 说明

接口	功能	返回值	参数	参数说明
void IOInterface::setSchedulerInput(bool scheduler)	打开/关闭 DI 调度（轮询）	void	scheduler	由 CmdType_Set_Input_Schedule 触发（当前实现为注释占位）。
void IOInterface::getInputControlAuthority(AbstructCmd *cmd)	获取 DI 控制权限/可控范围（标准 DI 数、总线 DI 数）	void	cmd	由 CmdType_Get_Input_ControlAuthority 触发；结果通过 signal_getDigitalInputControlAuthority(obj, standardSize, fieldbusSizeBits) 回传。
void IOInterface::setInputShowT	设置 DI 显示单位	void	showType	由 CmdType_Get_Input_Label

ype(const ShowType &showType)	(Bit/Byte/Word)			sAndRemark 前置调用 setInputShowType((ShowType)type)。
bool IOInterface::getInputLabelAndRemarkForCurrentShow(QVector<InLabelItem> &data)	读取 DI 标签/描述（当前显示范围）	bool	data	data: 输出标签数组（含index/label/description/originalName）。
bool IOInterface::getInputValuesStandard(QList<quint8> &data)	读取 Standard DI 值（附带强制高/低数组）	bool	data	data: 按字节存储；布局为：[DI 值 STANDARD_IO_COUNT] + [ForceHigh STANDARD_IO_COUNT] + [ForceLow STANDARD_IO_COUNT]。
void IOInterface::getInputValuesTool(AbstractCmd *cmd)	读取 Tool DI 值（附带强制高/低数组）	void	cmd	由 CmdType_GetInputValuesTool 触发，内部读 TOOL_IO_SIZE*3 并做差量回传。
bool IOInterface::getInputValuesFieldbus(QList<quint8> &data)	读取 Fieldbus DI 值	bool	data	data: FIELDBUS_IO_COUNT 字节。

bool IOInterface::getInputValuesMemory(QList<quint8>&data)	读取 Memory DI 值	bool	data	data: MEMORY_IO_COUNT 字节。
bool IOInterface::writeInputStatusForce(AbstractCmd *cmd)	切换 DI 强制 (强制高/低 ↔ 不强制)	bool	cmd	由 CmdType_Set_Input_Values Force 触发; 载荷含 bitIndex,value ,forceHighByte,forceLowByte (见调用小节)。
bool IOInterface::writeInputStatusByBit(AbstractCmd *cmd)	写入 DI 值 (在强制条件下写强制高/低) 或 Memory DI	bool	cmd	由 CmdType_Set_Input_Values 触发; 载荷含 bitIndex,value Byte,forceHighByte,forceLowByte。

18.1.2 调用

C++

// 显示页面: 开启调度 + 获取控制权限

```
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_Input_Schedule,
    true);
CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType::CmdType_Get_Input_ControlAuthority);
```

C++

// 隐藏页面：关闭调度

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_Set_Input_Schedule,  
    false);
```

C++

// 周期刷新：标签/描述（按 Bit/Byte/Word）

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Input_LabelsAndRemark,  
    m_type_bit_byte_word);
```

C++

// 周期刷新：读取不同 IOType 的 DI 值

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Input_ValuesStandard,  
    &m_valueStandard);
```

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Input_ValuesTool,  
    &m_valueTool);
```

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Input_ValuesFieldBus,  
    &m_valueFieldbus);
```

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Input_ValuesMemory,  
    &m_valueMemory);
```

C++

// 写入：强制开关（Standard/Tool）

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_Set_Input_ValuesForce,
```

```

m_startIndex + row,
m_valueStandard[row / 8],
m_valueStandard[row / 8 + STANDARD_IO_COUNT],
m_valueStandard[row / 8 + STANDARD_IO_COUNT * 2]);

```

C++

// 写入: DI 状态 (Standard/Tool/Memory)

```

CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_Input_Values,
    m_startIndex + row,
    m_valueStandard[row / 8],
    m_valueStandard[row / 8 + STANDARD_IO_COUNT],
    m_valueStandard[row / 8 + STANDARD_IO_COUNT * 2]);

```

```

CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_Input_Values,
    m_startIndex + row,
    m_valueMemory[row / 8],
    temp1,
    temp2);

```

18.1.3 函数实现

DI 相关

C++

```

case AbstractCmd::CmdType_Set_Input_Schedule: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setSchedulerInput(state);
    break;
}
case AbstractCmd::CmdType_Get_Input_ControlAuthority: {
    m_commInstance->getInputControlAuthority(absCmd);
    break;
}
case AbstractCmd::CmdType_Get_Input_ValuesStandard: {
    auto [address] = ((CmdDatas<QList<quint8> *> *)absCmd)-
>m_data;
    getListDatas(QVariant(InoIOType_StandardIO),

```

```

        *address,
        &Communication::getInputValuesStandard,

&CommunicationEngine::signal_input_getDatas_result);
    break;
}
case AbstractCmd::CmdType_Get_Input_ValuesTool: {
    m_commInstance->getInputValuesTool(absCmd);
    break;
}
case AbstractCmd::CmdType_Get_Input_ValuesFieldBus: {
    auto [address] = ((CmdDdatas<QList<quint8> *> *)absCmd)-
>m_data;
    getListDdatas(QVariant(InoIOType_FieldbusIO),
        *address,
        &Communication::getInputValuesFieldbus,

&CommunicationEngine::signal_input_getDatas_result);
    break;
}
case AbstractCmd::CmdType_Get_Input_ValuesMemory: {
    auto [address] = ((CmdDdatas<QList<quint8> *> *)absCmd)-
>m_data;
    getListDdatas(QVariant(InoIOType_MemoryIO),
        *address,
        &Communication::getInputValuesMemory,

&CommunicationEngine::signal_input_getDdatas_result);
    break;
}
case AbstractCmd::CmdType_Get_Input_LabelsAndRemark: {
    auto [type] = ((CmdDdatas<int> *)absCmd)->m_data;
    m_commInstance->setInputShowType((ShowType)(type));

#if defined(INOCOBOTTP_MSVC_QT5)
    QList<InoLabelItem> data;
    if (m_commInstance-
>getInputLabelAndRemarkForCurrentShow(data.toVector())) {
        emit CommunicationEngine::instance()
            ->signal_input_getLabelAndRemark_result(
                absCmd->m_object, type, data);
    }
#else
    QVector<InoLabelItem> data;

```

```

        if (m_commInstance->getInputLabelAndRemarkForCurrentShow(data)) {
            emit CommunicationEngine::instance()
                ->signal_input_getLabelAndRemark_result(
                    absCmd->m_object, type, data);
        }
    #endif

    break;
}
case AbstractCmd::CmdType_Set_Input_Values: {
    m_commInstance->writeInputStatusByBit(absCmd);
    break;
}
case AbstractCmd::CmdType_Set_Input_ValuesForce: {
    m_commInstance->writeInputStatusForce(absCmd);
    break;
}
}

```

```

C++
static InoRobBusiness::IResource::IOType
convertTypeToInoRobtbussiness(InoIOType ioType)
{
    switch (ioType) {
        case InoIOType_CommonIO: return
InoRobBusiness::IResource::IOType::COMMON_IO;
        case InoIOType_StandardIO: return
InoRobBusiness::IResource::IOType::STANDARD_IO;
        case InoIOType_FieldbusIO: return
InoRobBusiness::IResource::IOType::SLAVE_FIELDBUS_IO;
        case InoIOType_MemoryIO: return
InoRobBusiness::IResource::IOType::MEMORY_IO;
        case InoIOType_ToolIO: return
InoRobBusiness::IResource::IOType::TOOL_IO;
        default: return InoRobBusiness::IResource::IOType::COMMON_IO;
    }
    return InoRobBusiness::IResource::IOType::COMMON_IO;
}

void IOInterface::getInputControlAuthority(AbstractCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

```



```

    int standardSize = 0, fieldBusSize = 256; // standardSize bit
fieldBusSize byte
    standardSize = _IMonitor->GetInputNum();
    FIELDBUS_MEM_ASSIGN_CONFIG memAsgnCfg;
    int fieldbusType = 0;
    InoRobBusiness::IFieldbus *fieldBus = _IFieldBus;
    int ret = fieldBus->readMemoryAssignConfig(memAsgnCfg);
    if (ret != ERR_OK) {
        emit MWARNING(QObject::tr("Failed to get I/O control
configuration."));
    } else {
        if (fieldBus->getCurFieldBusType(fieldbusType) != ERR_OK)
            emit MWARNING(QObject::tr("Failed to get fieldbus
I/O."));
        else {
            fieldbusType = (E_FieldBusType)fieldbusType;
            // 根据当前激活的总线类型，执行对应的赋值
            switch (fieldbusType) {
                case E_FieldBusType::FB_Unknow:
                    break;
                case E_FieldBusType::FB_Modbus:
                    fieldBusSize = memAsgnCfg.modbusConfig.inputSize;
                    break;
                case E_FieldBusType::FB_EthernetIp:
                    fieldBusSize = memAsgnCfg.eipConfig.inputSize;
                    break;
                case E_FieldBusType::FB_EtherCATSlave:
                    fieldBusSize =
memAsgnCfg.etherCATConfig.inputSize;
                    break;
                case E_FieldBusType::FB_FINS:
                    break;
                case E_FieldBusType::FB_MC: {
                    McConnectPara mcPara[InoRobBusiness::MAX_MC_NUM];
                    InoRobBusiness::IMCSwitch *pMCSwitch = fieldBus-
>getMCSwitch();
                    if (pMCSwitch == nullptr)
                        break;
                    // 读取所有 MC 的连接状态
                    int ret = pMCSwitch->readAllMcConnectSts(mcPara);
                    if (ret != ERR_OK) {
                        emit MWARNING(QObject::tr("Failed to get MC
configuration."));
                    }
                }
            }
        }
    }

```

```

        break;
    }
    // 单位为字，转换为 Byte，需要乘以 2。先默认赋值第一个
MC 的
        fieldBusSize =
memAsgnCfg.mcConfig[0].mcInputRegSize * 2;
    // 遍历哪个激活了，则对应进行赋值
    for (int i = 0; i < InoRobBusiness::MAX_MC_NUM;
i++) {
        if (mcPara[i].ConnectFlag == 1) {
            fieldBusSize =
memAsgnCfg.mcConfig[i].mcInputRegSize * 2;
        }
    }
    break;
}
default:
    break;
}
    emit CommunicationEngine::instance()-
>signal_getDigitalInputControlAuthority(cmd->m_object,
standardSize, fieldBusSize * 8);
    }
}
    FREQ_LOG_PRINT_TIMESTAMP
}

// in fieldbus
bool IOInterface::getInputValuesFieldbus(QList<quint8> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!updateFieldbusInputDatas())
        return false;
    std::array<int8u, FIELDBUS_IO_COUNT> fieldbusInputValues
        = _IResource->getSlaveFieldbusInputValues();
    data = QList<quint8>(fieldbusInputValues.begin(),
fieldbusInputValues.end());
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

// in 设置当前显示 IO 类型（常用 标准 总线 内存）通知模型层刷新对应类型
数据

```

```

void IOInterface::setInputIOType(InoIOType ioType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->setInputIOType(
        convertTypeToInoRobtbussiness(ioType));
    FREQ_LOG_PRINT_TIMESTAMP
}

// in 设置当前显示 IO 单位 (bit bytes word)
void IOInterface::setInputShowType(const ShowType &showType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->setInputShowType(

static_cast<InoRobBusiness::IResource::ShowType>(showType));
    FREQ_LOG_PRINT_TIMESTAMP
}

// in standard
bool IOInterface::getInputValuesStandard(QList<quint8> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!updateStandardInputDatas())
        return false;
    std::array<int8u, STANDARD_IO_COUNT> standardInputValues
        = _IResource
            ->getStandardInputValues();
    std::array<int8u, STANDARD_IO_COUNT> fhDatas
        = _IResource
            ->getStandardInputForceHighValues();
    std::array<int8u, STANDARD_IO_COUNT> flDatas
        = _IResource
            ->getStandardInputForceLowValues();
    data = QList<quint8>(standardInputValues.begin(),
standardInputValues.end());
    data.append(QList<quint8>(fhDatas.begin(), fhDatas.end()));
    data.append(QList<quint8>(flDatas.begin(), flDatas.end()));

    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

```

```

// in Memory
bool IOInterface::getInputValuesMemory(QList<quint8> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!updateMemoryInputDatas())
        return false;
    std::array<int8u, MEMORY_IO_COUNT> memoryInputValues
        = _IResource
            ->getMemoryInputValues();
    data = QList<quint8>(memoryInputValues.begin(),
memoryInputValues.end());
    return true;
}

bool
IOInterface::getInputLabelAndRemarkForCurrentShow(QVector<InoLabel
Item> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    bool ret = _IResource
        ->updateInputLabelAndRemark();
    if (!ret)
        return false;
    IoLabelItems items = _IResource
        ->getInputLabelAndRemark();
    data.clear();
    for (int i = 0; i < items.LabelsArray.size(); ++i) {
        InoLabelItem item;
        item.nLabelId = items.LabelsArray[i].nLabelId;
        item.nIndex = items.LabelsArray[i].nIndex;
        item.sLabel =
QString::fromStdString(items.LabelsArray[i].sLabel);
        item.sDescription =
QString::fromStdString(items.LabelsArray[i].sDescription);
        item.sOriginalName =
QString::fromStdString(items.LabelsArray[i].sOriginalName);
        data.push_back(item);
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

```

```

bool IOInterface::writeInputStatusByBit(AbstractCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    InoRobBusiness::IResource *resource = _IResource;
    auto [bitIndex, value_, high, low] = BIND(cmd, int, quint8,
quint8, quint8);
    quint16 byteIndex = bitIndex / 8;
    quint8 bitIndexInByte = bitIndex % 8;
    quint8 temp = 0x01 << bitIndexInByte;
    bool bitValue = value_ & temp;
    if (bitIndex < (STANDARD_IO_START_INDEX + STANDARD_IO_COUNT +
TOOL_IO_COUNT) * 8) {
        bool fh = high & temp;
        bool fl = low & temp;
        if ((fh && !fl) || (!fh && fl)) // 强制高、低才写
        {
            if (fh) {
                high &= (~temp);
                low |= temp;
            } else {
                low &= (~temp); // 将 bitIndex 位置 0, 其他位不变
                high |= temp;   // 将 bitIndex 位置 1, 其他位不变
            }
        } else {
            return false;
        }
        std::vector<unsigned char> arraryH, arraryL;
        arraryH.push_back(high);
        arraryL.push_back(low);
        int ret1 = resource->writeInputForceHighStatus(byteIndex,
1, arraryH);
        int ret2 = resource->writeInputForceLowStatus(byteIndex,
1, arraryL);
    } else {
        bool ret = _IResource->setMemoryInputStatus(bitIndex -
MEMORY_IO_START_INDEX * 8, !bitValue);
        return false;
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return false;
}

bool IOInterface::writeInputStatusForce(AbstractCmd *cmd)

```

```

{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    InoRobBusiness::IResource *resource = _IResource;
    auto [bitIndex, value_, high_, low_] = BIND(cmd, int, quint8,
quint8, quint8);
    quint8 high = high_, low = low_;
    if (bitIndex < (STANDARD_IO_START_INDEX + STANDARD_IO_COUNT +
TOOL_IO_COUNT) * 8) {
        quint16 byteIndex = bitIndex / 8;
        quint8 bitIndexInByte = bitIndex % 8;
        quint8 temp = 0x01 << bitIndexInByte;
        bool bitState = value_ & (0x01 << bitIndexInByte);
        bool fh = high & temp;
        bool fl = low & temp;
        if (!fh && !fl) {
            if (bitState) {
                high |= temp;
                low &= (~temp);
            } else {
                high &= (~temp);
                low |= temp;
            }
        } else if ((fh && !fl) || (!fh && fl)) // 强制 => 不强制
        {
            // 强制高、强制低都置 0、io 位置 off
            high &= (~temp);
            low &= (~temp);
        } else {
            return false;
        }
        std::vector<int8u> arraryH, arraryL;
        arraryH.push_back(high);
        arraryL.push_back(low);
        int ret1 = resource->writeInputForceHighStatus(byteIndex,
1, arraryH);
        int ret2 = resource->writeInputForceLowStatus(byteIndex,
1, arraryL);
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

void IOInterface::getInputValuesTool(AbstractCmd *cmd)

```

```

{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

#ifdef INOCOBOTTP_MSVC_QT5
    QList<quint8> data;
    for (int i = 0; i < TOOL_IO_SIZE * 3; ++i) {
        data.append(0);
    }
#else
    QList<quint8> data(TOOL_IO_SIZE * 3);
#endif
    int ret1 = _IResource->readInputStatus(TOOL_IO_STARTINDEX,
TOOL_IO_SIZE, &data[0]);
    int ret2 = _IResource-
>readInputForceHighStatus(TOOL_IO_STARTINDEX, TOOL_IO_SIZE,
&data[TOOL_IO_SIZE]);
    int ret3 = _IResource-
>readInputForceLowStatus(TOOL_IO_STARTINDEX, TOOL_IO_SIZE,
&data[TOOL_IO_SIZE * 2]);
    if (ret1 == ERROR_OK && ret2 == ERROR_OK && ret3 == ERROR_OK)
    {
        auto [address] = ((CmdDatas<QList<quint8> *> *)cmd)-
>m_data;
        QList<int> index;
        QList<quint8> different;
        if (compareAndObtainDifferentDatas(*address, data, index,
different)) {
            emit CommunicationEngine::instance()-
>signal_input_getDatas_result(InoIOType_ToolIO, index, different);
        }
        } else {
            PRINT_ERROR(QObject::tr("Failed to read tool DI
status."));
        }
        FREQ_LOG_PRINT_TIMESTAMP
    }

// in
void IOInterface::setSchedulerInput(bool scheduler)
{
    // _IResource->setInputScheduler(scheduler);
}

// in Fieldbus 刷新

```

```

bool IOInterface::updateFieldbusInputDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->updateSlaveFieldbusInputDatas();
}

// in Standard 刷新
bool IOInterface::updateStandardInputDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->updateStandardInputDatas();
}

// in Memory 刷新
bool IOInterface::updateMemoryInputDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->updateMemoryInputDatas();
}

```

18.2 数字输出

(setSchedulerOutput/getOutputControlAuthority/setOutputShowType/getOutputLabelAndRemarkForCurrentShow/getOutputValuesStandard/getOutputValuesTool/getOutputValuesFieldbus/getOutputValuesMemory/writeOutputStatusByBit)

18.2.1 说明

接口	功能	返回值	参数	参数说明
void	打开/关闭 DO	void	scheduler	由

IOInterface::setSchedulerOutput(bool scheduler)	调度（轮询）			CmdType_Set_Output_Schedule 触发（当前实现为注释占位）。
void IOInterface::getOutputControlAuthority(AbstractCmd *cmd)	获取 DO 控制权限/可控范围（以及控制掩码）	void	cmd	由 CmdType_Get_Output_ControlAuthority 触发；通过 signal_getDigitalOutputControlAuthority(...) 回传标准/总线/内存控制数组。
void IOInterface::setOutputShowType(const ShowType &showType)	设置 DO 显示单位（Bit/Byte/Word）	void	showType	由 CmdType_Get_Output_LabelsAndRemark 前置调用 setOutputShowType((ShowType)type)。
bool IOInterface::getOutputLabelAndRemarkForCurrentShow(QVector<InoLabelItem> &data)	读取 DO 标签/描述（当前显示范围）	bool	data	输出标签数组。
bool IOInterface::getOutputValuesStandard(QList<quint8> &src)	读取 Standard DO 值	bool	src	src: STANDARD_IO_COUNT 字节。

void IOInterface::getOutputValue sTool(Abstrac tCmd *cmd)	读取 Tool DO 值	void	cmd	由 CmdType_Get _Output_Valu esTool 触发, 内部读 TOOL_IO_SI ZE 并差量回 传。
bool IOInterface::g etOutputValue sFieldbus(QLi st<quint8> &src)	读取 Fieldbus DO 值	bool	src	src: FIELDBUS_I O_COUNT 字 节。
bool IOInterface::g etOutputValue sMemory(QLi st<quint8> &src)	读取 Memory DO 值	bool	src	src: MEMORY_IO _COUNT 字 节。
void IOInterface::w riteOutputStat usByBit(Abstr actCmd *cmd)	写 DO 单点状 态 (按 realIndex 自动 选择 IOType)	void	cmd	由 CmdType_Set _Output_Valu es 触发; 载荷 为 uiRow, realIndex, bitValue。

18.2.2 调用

C++

// 显示页面: 开启调度 + 获取控制权限

```
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_Output_Schedule,
    true);
```

```
CommunicationEngine::instance()->enqueueCmd(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Output_ControlAuthority);
```

C++

// 隐藏页面：关闭调度

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_Set_Output_Schedule,  
    false);
```

C++

// 周期刷新：标签/描述（按 Bit/Byte/Word）

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Output_LabelsAndRemark,  
    m_type_bit_byte_word);
```

C++

// 周期刷新：读取不同 IOType 的 DO 值

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Output_ValuesStandard,  
    &m_valueStandard);
```

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Output_ValuesTool,  
    &m_valueTool);
```

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Output_ValuesFieldBus,  
    &m_valueFieldbus);
```

```
CommunicationEngine::instance()->enqueueCmd_getData(  
    this,  
    AbstractCmd::CmdType::CmdType_Get_Output_ValuesMemory,
```

```
&m_valueMemory);
```

```
C++  
// 写入: DO 单点  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_Set_Output_Values,  
    row,  
    m_startIndex + row,  
    state);
```

18.2.3 函数实现

```
C++  
case AbstractCmd::CmdType_Set_Output_Schedule: {  
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;  
    m_commInstance->setSchedulerOutput(state);  
    break;  
}  
case AbstractCmd::CmdType_Get_Output_ControlAuthority: {  
    m_commInstance->getOutputControlAuthority(absCmd);  
    break;  
}  
case AbstractCmd::CmdType_Get_Output_ValuesStandard: {  
    auto [address] = ((CmdDatas<QList<quint8> *> *)absCmd)-  
>m_data;  
    getListDatas(QVariant(InoIOType_StandardIO),  
        *address,  
        &Communication::getOutputValuesStandard,  
        &CommunicationEngine::signal_output_getDatas_result);  
    break;  
}  
case AbstractCmd::CmdType_Get_Output_ValuesTool: {  
    m_commInstance->getOutputValuesTool(absCmd);  
    break;  
}  
case AbstractCmd::CmdType_Get_Output_ValuesFieldBus: {  
    auto [address] = ((CmdDatas<QList<quint8> *> *)absCmd)-  
>m_data;
```

```

        getListDatas(QVariant(InoIOType_FieldbusIO),
                      *address,
                      &Communication::getOutputValuesFieldbus,

&CommunicationEngine::signal_output_getDatas_result);
        break;
    }
    case AbstractCmd::CmdType_Get_Output_LabelsAndRemark: {
        auto [type] = ((CmdDatas<int> *)absCmd)->m_data;
        m_commInstance->setOutputShowType((ShowType)type);
        #if defined(INOCOBOTTP_MSVC_QT5)
            QList<InoLabelItem> data;
            if (m_commInstance-
>getOutputLabelAndRemarkForCurrentShow(data.toVector())) {
                emit CommunicationEngine::instance()
                    ->signal_output_getLabelAndRemark_result(
                        absCmd->m_object, type, data);
            }
        #else
            QVector<InoLabelItem> data;
            if (m_commInstance-
>getOutputLabelAndRemarkForCurrentShow(data)) {
                emit CommunicationEngine::instance()
                    ->signal_output_getLabelAndRemark_result(
                        absCmd->m_object, type, data);
            }
        #endif
        break;
    }
    case AbstractCmd::CmdType_Get_Output_ValuesMemory: {
        auto [address] = ((CmdDatas<QList<quint8> *> *)absCmd)-
>m_data;
        getListDatas(QVariant(InoIOType_MemoryIO),
                      *address,
                      &Communication::getOutputValuesMemory,

&CommunicationEngine::signal_output_getDatas_result);
        break;
    }
    case AbstractCmd::CmdType_Set_Output_Values: {
        m_commInstance->writeOutputStatusByBit(absCmd);
        break;
    }
}

```

```

C++
void IOInterface::getOutputControlAuthority(AbstractCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    int startByteId, byteSize;
    std::vector<unsigned char> standardControl(STANDARD_IO_SIZE,
0);
    startByteId = STANDARD_IO_STARTINDEX;
    byteSize = STANDARD_IO_SIZE;
    InoRobBusiness::IResource *resource = _IResource;
    int ret = resource->readOutputControls(startByteId, byteSize,
standardControl);
    if (ret != ERROR_OK)
        return;

    std::vector<unsigned char>
fiedlBusControl(SLAVE_FIELDBUS_IO_SIZE, 0);
    startByteId = SLAVE_FIELDBUS_IO_STARTINDEX;
    byteSize = SLAVE_FIELDBUS_IO_SIZE;
    ret = resource->readOutputControls(startByteId, byteSize,
fiedlBusControl);
    if (ret != ERROR_OK)
        return;

    std::vector<unsigned char> memoryControl(MEMORY_IO_SIZE, 0);
    startByteId = MEMORY_IO_STARTINDEX;
    byteSize = MEMORY_IO_SIZE;
    ret = resource->readOutputControls(startByteId, byteSize,
memoryControl);
    if (ret != ERROR_OK)
        return;

    int standardSize = _IMonitor->GetOutputNum();
    int fieldBusSize = 256;
    FIELDBUS_MEM_ASSIGN_CONFIG memAsgnCfg;
    InoRobBusiness::IFieldbus *fieldBus = _IFieldBus;
    ret = fieldBus->readMemoryAssignConfig(memAsgnCfg);
    if (ret != ERROR_OK)
        return;
    int fieldbusType = 0; // 单位是 byte
    if (fieldBus->getCurFieldBusType(fieldbusType) == ERR_OK)
        fieldbusType = (E_FieldBusType)fieldbusType;

```

```

switch (fieldbusType) {
case E_FieldBusType::FB_Unknow:
    break;
case E_FieldBusType::FB_Modbus:
    fieldBusSize = memAsgnCfg.modbusConfig.outputSize;
    break;
case E_FieldBusType::FB_EthernetIp:
    fieldBusSize = memAsgnCfg.eipConfig.outputSize;
    break;
case E_FieldBusType::FB_EtherCATSlave:
    fieldBusSize = memAsgnCfg.etherCATConfig.outputSize;
    break;
case E_FieldBusType::FB_FINS:
    break;
case E_FieldBusType::FB_MC: {
    McConnectPara mcPara[InoRobBusiness::MAX_MC_NUM];
    InoRobBusiness::IMCSwitch *pMCSwitch = fieldBus-
>getMCSwitch();
    if (pMCSwitch == nullptr)
        break;
    // 读取所有 MC 的连接状态
    int ret = pMCSwitch->readAllMcConnectSts(mcPara);
    if (ret != ERR_OK) {
        LOG_ERROR(QObject::tr("Failed to get MC connection
status."));
        return;
    }
    // 单位为字，转换为 Byte，需要乘以 2。先默认赋值第一个 MC 的
    fieldBusSize = memAsgnCfg.mcConfig[0].mcOutputRegSize * 2;
    // 遍历哪个激活了，则对应进行赋值
    for (int i = 0; i < InoRobBusiness::MAX_MC_NUM; i++) {
        if (mcPara[i].ConnectFlag == 1) {
            fieldBusSize =
memAsgnCfg.mcConfig[i].mcOutputRegSize * 2;
        }
    }
    break;
}
default:
    break;
}
emit CommunicationEngine::instance()-
>signal_getDigitalOutputControlAuthority(
    cmd->m_object,

```

```

        standardSize,
        fieldBusSize * 8,
        QList<quint8>(standardControl.begin(),
standardControl.end()),
        QList<quint8>(fiedlBusControl.begin(),
fiedlBusControl.end()),
        QList<quint8>(memoryControl.begin(),
memoryControl.end())));
    FREQ_LOG_PRINT_TIMESTAMP
}

// out fieldbus
bool IOInterface::getOutputValuesFieldbus(QList<quint8> &src)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!updateFieldbusOutputDatas())
        return false;
    std::array<int8u, FIELDBUS_IO_COUNT> fieldbusOutputValues
        = _IResource
            ->getSlaveFieldbusOutputValues();
    src = QList<quint8>(fieldbusOutputValues.begin(),
                        fieldbusOutputValues.end());
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool IOInterface::getOutputValuesMemory(QList<quint8> &src)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!updateMemoryOutputDatas())
        return false;
    std::array<int8u, MEMORY_IO_COUNT> memoryOutputValues
        = _IResource
            ->getMemoryOutputValues();
    src = QList<quint8>(memoryOutputValues.begin(),
memoryOutputValues.end());
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

bool
IOInterface::getOutputLabelAndRemarkForCurrentShow(QVector<InoLabe

```



```

lItem> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    bool ret = _IResource
                ->updateOutputLabelAndRemark();
    if (!ret)
        return false;
    IoLabelItems items = _IResource
                        ->getOutputLabelAndRemark();
    data.clear();
    for (int i = 0; i < items.LabelsArray.size(); ++i) {
        InoLabelItem item;
        item.nLabelId = items.LabelsArray[i].nLabelId;
        item.nIndex = items.LabelsArray[i].nIndex;
        item.sLabel =
QString::fromStdString(items.LabelsArray[i].sLabel);
        item.sDescription =
QString::fromStdString(items.LabelsArray[i].sDescription);
        item.sOriginalName =
QString::fromStdString(items.LabelsArray[i].sOriginalName);
        data.push_back(item);
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

void IOInterface::setOutputIOType(InoIOType ioType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->setOutputIOType(
        convertTypeToInoRobtbussiness(ioType));
    FREQ_LOG_PRINT_TIMESTAMP
}

void IOInterface::setOutputShowType(const ShowType &showType)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->setOutputShowType(
        static_cast<InoRobBusiness::IResource::ShowType>(showType));
    FREQ_LOG_PRINT_TIMESTAMP
}

```

```

}

bool IOInterface::getOutputValuesStandard(QList<quint8> &src)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    if (!updateStandardOutputDatas())
        return false;
    src.clear();

#ifdef INOCOBOTTP_MSVC_QT5
    for (int i = 0; i < STANDARD_IO_COUNT; ++i) {
        src.append(0);
    }
#else
    src.resize(STANDARD_IO_COUNT);
#endif

    std::array<int8u, STANDARD_IO_COUNT> standardOutputValues
        = _IResource
            ->getStandardOutputValues();
    src = QList<quint8>(standardOutputValues.begin(),
                        standardOutputValues.end());
    FREQ_LOG_PRINT_TIMESTAMP
    // 该通讯码无错误处理
    return true;
}

void IOInterface::writeOutputStatusByBit(AbstractCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    auto [uiRow, realIndex, bitValue] = BIND(cmd, int, int, bool);
    if (realIndex < STANDARD_IO_STARTINDEX + STANDARD_IO_SIZE * 8)
        setOutputIOType(InoIOType_StandardIO);
    else if (realIndex < (TOOL_IO_STARTINDEX + TOOL_IO_SIZE) * 8)
    {
        _IResource->writeOutputStatusByBit(realIndex, bitValue);
        return;
    } else if (realIndex < (SLAVE_FIELDBUS_IO_STARTINDEX +
SLAVE_FIELDBUS_IO_SIZE) * 8)
        setOutputIOType(InoIOType_FieldbusIO);
    else
        setOutputIOType(InoIOType_MemoryIO);
}

```

```

        qDebug() << uiRow << realIndex << bitValue <<
"writeOutputStatusByBit";
        _IResource->setOutputStatus(uiRow, realIndex, bitValue);
        FREQ_LOG_PRINT_TIMESTAMP
    }

void IOInterface::getOutputValuesTool(AbstractCmd *cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

#ifdef INOCOBOTTP_MSVC_QT5
    QList<quint8> data;
    for (int i = 0; i < TOOL_IO_SIZE; ++i) {
        data.append(0);
    }
#else
    QList<quint8> data(TOOL_IO_SIZE);
#endif
    int ret = _IResource->readOutputStatus(TOOL_IO_STARTINDEX,
TOOL_IO_SIZE, &data[0]);
    if (ret == ERROR_OK) {
        auto [address] = ((CmdDatas<QList<quint8> *> *)cmd)-
>m_data;
        QList<int> index;
        QList<quint8> different;
        if (compareAndObtainDifferentDatas(*address, data, index,
different)) {
            emit CommunicationEngine::instance()-
>signal_output_getDatas_result(InoIOType_ToolIO, index,
different);
        }
    } else {
        PRINT_ERROR(QObject::tr("Failed to read tool DO
status."));
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

void IOInterface::setSchedulerOutput(bool scheduler)
{
    // _IResource->setOutputScheduler(scheduler);
}

// out  Fieldbus 刷新

```

```

bool IOInterface::updateFieldbusOutputDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->updateSlaveFieldbusOutputValues();
}

// out Standard 刷新
bool IOInterface::updateStandardOutputDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->updateStandardOutputValues();
}

// out Memory 刷新
bool IOInterface::updateMemoryOutputDatas()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    return _IResource->updateMemoryOutputValues();
}

```

18.3 模拟输入

(getDatasAD/setSchedulerAD/getCobotDatasADDA/setCobotDatasADDA)

18.3.1 说明

接口	功能	返回值	参数	参数说明
void IOInterface::setSchedulerAD(const bool scheduler)	设置 AD 刷新 调度开关	void	scheduler	true 开启 AD 定时刷新调 度；false 关 闭。

bool IOInterface::getDatasAD(QList<Ino_AD_DA_Data> &data)	读取 (IRLink) AD 数据列表	bool	data	输出参数：返回的 AD 列表（每行包含 status/labs/remarks）。
void IOInterface::getCobotDdatasADDA(AbstractCmd *cmd)	读取“协作电柜+末端”的 AD/DA 数据（按 type 区分 AD/DA）	void	cmd	由 CmdType_GetCobot_AD_DA_Values 触发；载荷为 state(AD/DA), data(QList<Ino_Cobot_AD_DA_Data>*)。当 state==Ino_Cobot_ADDA_type_AD 时读取 AD。
void IOInterface::setCobotDdatasADDA(AbstractCmd *cmd)	写入“协作电柜+末端”的 AD/DA 配置信息（电流/电压模式等）	void	cmd	由 CmdType_SetCobot_AD_DA_Values 触发；载荷为 data(Ino_Cobot_ADDA_Data)。通常用于设置 AD 的 kind(电流/电压)。

C++

// AD、DA 状态

```
struct Ino_AD_DA_Status {
```

```
    int favorite = 0;           // 收藏
```

```
    int kind = 0;              // 0 电流 1 电压
```

```
    double minValue = 0.0;     // 最小值
```

```

    double maxValue = 20.0;    // 最大值 默认电流 0-20mA
    double actualValue = 0.0;  // 实际的值
    int out = 0;                // 输出开关 0 不输出 1 输出
    ...
};

// AD、DA 数据
struct Ino_AD_DA_Data {
    Ino_AD_DA_Status status; // 状态
    QString labs;            // 标签
    QString remarks;         // 备注
    ...
};

enum Ino_Cobot_ADDA_type : unsigned char{
    Ino_Cobot_ADDA_type_AD = 1,
    Ino_Cobot_ADDA_type_DA = 2,
};

enum Ino_Cobot_ADDA_Signal_type : unsigned char{
    ADDA_Type_Current = 1,
    ADDA_Type_Voltage = 2,
};

// AD、DA 状态 协作电柜自带的 AD DA
struct Ino_Cobot_ADDA_Status {
    unsigned short type = 0;          // 1AD 2DA
    unsigned short setModelEnable = 0; // 是否可以配置电流/电压模式
    unsigned short index = 0;         // 协作 电柜 101 102 末端
103 104
    unsigned short currentConfigState = 0; // 0 已配置 配置中
    unsigned short kind = 0;              // 1 电流 2 电压
    unsigned short out = 0;               // 输出开关 0 不输出
1 输出
    unsigned short deviceType = 0;        // 类别 0IRLink
1EtherCat 2 末端 3 电柜
    double minValue = 0.0;               // 最小值
    double maxValue = 20.0;              // 最大值 默认电流 0-20mA
    double actualValue = 0.0;            // 实际的值
    ...
};

```

```
};

// AD、DA 数据 协作电柜自带的 AD DA
struct Ino_Cobot_ADDA_Data {
    Ino_Cobot_ADDA_Status status; // 状态
    QString labs;                // 标签
    QString remarks;             // 备注

    ...
};
```

18.3.2 调用

```
C++
// 读取 AD（备用页面：backup_adform）
CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_AD_Values,
    &m_datas);
```

```
C++
// 读取“协作电柜+末端”AD（当前页面：adform）
CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_Cobot_AD_DA_Values,
    Ino_Cobot_ADDA_type_AD,
    &m_cobotDatas);
```

```
C++
// 写入 AD 模式（电流/电压）（当前页面：adform）
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_Cobot_AD_DA_Values,
    temp);
```

18.3.3 函数实现

AD

```
C++
// AD
case AbstractCmd::CmdType_Get_AD_Values: {
    auto [address]
        = ((CmdDatas<QList<Ino_AD_DA_Data> *> *)absCmd)->m_data;
    getListDatas(QVariant(),
                 *address,
                 &Communication::getDatasAD,
                 &CommunicationEngine::signal_AD_getDatas_result);
    break;
}
case AbstractCmd::CmdType_Set_AD_Schedule: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setSchedulerAD(state);
    break;
}

case AbstractCmd::CmdType_Get_Cobot_AD_DA_Values: {
    m_commInstance->getCobotDatasADDA(absCmd);
    break;
}
case AbstractCmd::CmdType_Set_Cobot_AD_DA_Values: {
    m_commInstance->setCobotDatasADDA(absCmd);
    break;
}
}
```

```
C++
void IOInterface::setSchedulerAD(const bool scheduler)
{
    // _IResource->setADScheduler(scheduler);
    FREQ_LOG_PRINT_TIMESTAMP
}

bool IOInterface::getDatasAD(QList<Ino_AD_DA_Data> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    bool ret = _IResource->updateIRLinkADData();
    if (!ret)
        return false;
    std::vector<InoRobBusiness::IRLinkADData> value =
        _IResource->getIRLinkADData();
}
```



```

    for (size_t i = 0; i < value.size(); ++i) {
        Ino_AD_DA_Data temp;
        temp.labs = QString::fromStdString(value[i].labs);
        temp.remarks = QString::fromStdString(value[i].remarks);
        temp.status.favorite = value[i].status.favorite;
        temp.status.kind = value[i].status.kind;
        temp.status.minValue = value[i].status.minValue;
        temp.status.maxValue = value[i].status.maxValue;
        temp.status.actualValue = value[i].status.actualValue;
        temp.status.out = value[i].status.out;
        data.push_back(temp);
    }
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

Ino_Cobot_ADDA_Data
ConvertBussinessCobot_ADDA_DataToCommunication(const
InoRobBusiness::Cobot_ADDA_Data &data)
{
    Ino_Cobot_ADDA_Data temp;
    temp.labs = QString::fromStdString(data.labs);
    temp.remarks = QString::fromStdString(data.remarks);
    temp.status.type = data.status.type;
    temp.status.index = data.status.index;
    temp.status.setModelEnable = data.status.setModelEnable;
    temp.status.currentConfigState =
data.status.currentConfigState;
    temp.status.deviceType = data.status.deviceType;
    temp.status.kind = data.status.kind;
    temp.status.minValue = data.status.minValue;
    temp.status.maxValue = data.status.maxValue;
    temp.status.actualValue = data.status.actualValue;
    temp.status.out = data.status.out;
    return temp;
}

void IOInterface::getCobotDatasADDA(AbstractCmd *cmd)
{
    // readCobotAnalogIODatas
    auto [state, data] = BIND(cmd, Ino_Cobot_ADDA_type,
    QList<Ino_Cobot_ADDA_Data> *);
    std::vector<InoRobBusiness::Cobot_ADDA_Data> src;
    int ret = _IResource->readCobotAnalogIODatas(state ==

```

```

Ino_Cobot_ADDA_type_DA, src);
    if (ret != ERROR_OK)
        return;
    QList<int> index;
    QList<Ino_Cobot_ADDA_Data> all;
    QList<Ino_Cobot_ADDA_Data> different;
    int size = src.size();
    for (int i = 0; i < size; ++i) {

all.push_back(ConvertBussinessCobot_ADDA_DataToCommunication(src[i
]));
        // qDebug()<<i<<all[i].status.kind;
    }
    if (data->size() != size) {
        for (int i = 0; i < size; ++i) {
            index.push_back(i);
        }
        emit CommunicationEngine::instance()-
>signal_Cobot_ADDA_getDatas_result(cmd->m_object, true, index,
all);
        return;
    }

    if (compareAndObtainDifferentDatas((*data), all, index,
different)) {
        emit CommunicationEngine::instance()-
>signal_Cobot_ADDA_getDatas_result(cmd->m_object, false, index,
different);
        return;
    }
    return;
}

void IOInterface::setCobotDatasADDA(AbstractCmd *cmd)
{
    auto [data] = ((CmdDatas<Ino_Cobot_ADDA_Data> *)cmd)->m_data;
    InoRobBusiness::Cobot_ADDA_Status temp;
    temp.actualValue = data.status.actualValue;
    temp.index = data.status.index;
    temp.kind = data.status.kind;
    temp.type = data.status.type;
    temp.out = data.status.out;
    int ret = _IResource->writeCobotAnalogIODatas(temp);
    if (ret != ERROR_OK) {

```

```

        emit MWARNING("Set AD/DA datas falid!");
    } else {
        PRINT_MSG(CommTr::tr("AD/DA config information sent."));
    }
    return;
}

```

18.4 模拟输出

(getDatasDA/setSchedulerDA/setDataDA/getCobotDat asADDA/setCobotDatasADDA)

18.4.1 说明

接口	功能	返回值	参数	参数说明
void IOInterface::s etSchedulerD A(const bool scheduler)	设置 DA 刷新 调度开关	void	scheduler	true 开启 DA 定时刷新调 度; false 关 闭。
bool IOInterface::g etDatasDA(Q List<Ino_AD_ DA_Data> &data)	读取 (IRLink) DA 数据列表	bool	data	输出参数: 返 回的 DA 列表 (每行包含 status/labs/re marks)。
bool IOInterface::s etDataDA(con st int index, const int flag, const Ino_AD_DA_ Data &value)	写入 (IRLink) DA 输出 (分两次 写入: flag=0/1)	bool	index, flag, value	index: DA 通 道索引; flag: 写入阶 段标记 (调用 方会连续写两 次 0/1); value: 包含 actualValue/o ut/kind 等配置 的 DA 数据。

void IOInterface::getCobotDatas ADDA(AbstractCmd *cmd)	读取“协作电柜 +末端”的 AD/DA 数据 (按 type 区 分 AD/DA)	void	cmd	由 CmdType_Get _Cobot_AD_ _DA_Values 触 发; 当 state==Ino_C obot_ADDA_t ype_DA 时读 取 DA。
void IOInterface::setCobotDatas ADDA(AbstractCmd *cmd)	写入“协作电柜 +末端”的 DA 输出与模式	void	cmd	由 CmdType_Set _Cobot_AD_ _DA_Values 触 发; 载荷为 data(Ino_Cob ot_ADDA_Dat a), 常用字 段: kind/out/actua lValue。

18.4.2 调用

C++

```
// 读取 DA (备用页面: backup_daform)
CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_DA_Values,
    &m_datas);
```

C++

```
// 写入 DA (备用页面: backup_daform)
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_DA_Values,
    m_datas[row],
    row);
```

C++

```
// 读取“协作电柜+末端”DA（当前页面：daform）
CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_Cobot_AD_DA_Values,
    Ino_Cobot_ADDA_type_DA,
    &m_cobotDatas);
```

C++

```
// 写入“协作电柜+末端”DA（当前页面：daform）
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_Cobot_AD_DA_Values,
    temp);
```

18.4.3 函数实现

DA

C++

```
// DA
case AbstractCmd::CmdType_Get_DA_Values: {
    auto [address]
        = ((CmdDatas<QList<Ino_AD_DA_Data> *> *)absCmd)->m_data;
    getListDatas(QVariant(),
        *address,
        &Communication::getDatasDA,
        &CommunicationEngine::signal_DA_getDatas_result);
    break;
}
case AbstractCmd::CmdType_Set_DA_Schedule: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setSchedulerAD(state);
    break;
}
case AbstractCmd::CmdType_Set_DA_Values: {
    auto [data, index]
        = ((CmdDatas<Ino_AD_DA_Data, int> *)absCmd)->m_data;
    bool first = m_commInstance->setDataDA(index, 0, data);
    bool second = m_commInstance->setDataDA(index, 1, data);
    emit CommunicationEngine::instance()
```

```

        ->signal_DA_setDatas_result(first && second);
    break;
}

case AbstractCmd::CmdType_Get_Cobot_AD_DA_Values: {
    m_commInstance->getCobotDatasADDA(absCmd);
    break;
}

case AbstractCmd::CmdType_Set_Cobot_AD_DA_Values: {
    m_commInstance->setCobotDatasADDA(absCmd);
    break;
}
}

```

```

C++
void IOInterface::setSchedulerDA(const bool scheduler)
{
    // _IResource->setDAScheduler(scheduler);
    FREQ_LOG_PRINT_TIMESTAMP
}

bool IOInterface::getDatasDA(QList<Ino_AD_DA_Data> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    // bool ret = _IResource->updateIRLinkDADatas();
    // if (!ret)
    //     return false;
    // std::vector<InoRobBusiness::IRLinkADDADData> value =
    _IResource->getIRLinkDADatas();
    // for (size_t i = 0; i < value.size(); ++i) {
    //     Ino_AD_DA_Data temp;
    //     temp.labs = QString::fromStdString(value[i].labs);
    //     temp.remarks =
    QString::fromStdString(value[i].remarks);
    //     temp.status.favorite = value[i].status.favorite;
    //     temp.status.kind = value[i].status.kind;
    //     temp.status.minValue = value[i].status.minValue;
    //     temp.status.maxValue = value[i].status.maxValue;
    //     temp.status.actualValue = value[i].status.actualValue;
    //     temp.status.out = value[i].status.out;
    //     data.push_back(temp);
    // }
    FREQ_LOG_PRINT_TIMESTAMP
}

```

```

        return true;
    }

    bool IOInterface::setDataDA(const int index,
                                const int flag,
                                const Ino_AD_DA_Data &value)
    {
        FREQ_LOG_PRINT_TIMESTAMP_THREAD

        // InoRobBusiness::IRLinkADDADData temp;
        // temp.favorite = value.status.favorite;
        // temp.kind = value.status.kind;
        // temp.minValue = value.status.minValue;
        // temp.maxValue = value.status.maxValue;
        // temp.actualValue = value.status.actualValue;
        // temp.out = value.status.out;
        // int ret = _IResource->writeDAValue(index, flag, temp);
        // FREQ_LOG_PRINT_TIMESTAMP
        // if (ret == ERR_OK)
        //     return true;
        return false;
    }

```

18.5 系统数字输入

(getDatasSystemIn/getSysInputRemark/setScheduler SysInput)

18.5.1 说明

接口	功能	返回值	参数	参数说明
void IOInterface::setSchedulerSysInput(const bool scheduler)	设置系统数字输入 (SysDI) 刷新调度开关	void	scheduler	true 开启 SysDI 定时刷新调度; false 关闭。
bool IOInterface::g	读取系统数字	bool	data	输出参数:

etDataSystemIn(QList<bool> &data)	输入 (SysDI) 当前值			SysDI 值列表 (按索引 0..N) ; true 表示 ON, false 表示 OFF。
bool IOInterface::getSysInputRemark(QStringList &data)	读取系统数字输入 (SysDI) 备注/描述	bool	data	输出参数: SysDI 备注列表 (每个索引对应一条说明文本)。
bool IOInterface::getSysInputRemarkList(QList<QString> &data)	读取系统数字输入 (SysDI) 备注/描述 (Qt5 分支)	bool	data	Qt5 下使用 QList<QString> 承接备注列表; 语义同 getSysInputRemark。

18.5.2 调用

C++

```
// 显示页面: 开启调度 + 读取备注 + 周期刷新 SysDI 值
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_SystemIn_Schedule,
    true);

CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_SystemIn_Reamarks,
    &m_remarks);

CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_SystemIn_Values,
    &m_values);
```

C++


```
// 隐藏页面：关闭调度
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_SystemIn_Schedule,
    false);
```

18.5.3 函数实现

```
C++
// SYSTEM IN
case AbstractCmd::CmdType_Get_SystemIn_Values: {
    auto [address] = ((CmdDatas<QList<bool> *> *)absCmd)->m_data;
    getListDatas(QVariant(),
        *address,
        &Communication::getDatasSystemIn,

&CommunicationEngine::signal_SystemIn_getDatas_result);
    break;
}
case AbstractCmd::CmdType_Get_SystemIn_Reamarks: {
    auto [address] = ((CmdDatas<QList<QString> *> *)absCmd)-
>m_data;

#if defined(INOCOBOTTP_MSVC_QT5)
    getListDatas(QVariant(),
        *address,
        &Communication::getSysInputRemarkList,

&CommunicationEngine::signal_SystemIn_getRemarks_result);
#else
    getListDatas(QVariant(),
        *address,
        &Communication::getSysInputRemark,

&CommunicationEngine::signal_SystemIn_getRemarks_result);
#endif
    break;
}
case AbstractCmd::CmdType_Set_SystemIn_Schedule: {
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setSchedulerSysInput(state);
    break;
}
```

```
}
```

```
C++
bool IOInterface::getDatasSystemIn(QList<bool> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->updateSysInputDatas();
    std::array<bool, SYSIO_SIZE> value = _IResource-
>getSysInputStatus();
    data = QList<bool>(value.begin(), value.end());
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

void IOInterface::setSchedulerSysInput(const bool scheduler)
{
    // _IResource->setSysInputScheduler(scheduler);
}

bool IOInterface::getSysInputRemark(QStringList &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->updateSysInputDatas();
    std::array<std::string, SYSIO_SIZE> remark = _IResource-
>getSysInputRemarks();
    foreach (auto &str, remark) {
        data.push_back(QString::fromStdString(str));
    };
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

#if defined(INOCOBOTTP_MSVC_QT5)
bool IOInterface::getSysInputRemarkList(QList<QString> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->updateSysInputDatas();
    std::array<std::string, SYSIO_SIZE> remark = _IResource-
>getSysInputRemarks();
    foreach (auto &str, remark) {
```

```
        data.push_back(QString::fromStdString(str));
    };
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}
#endif
```

18.6 系统数字输出

(getDataSystemOut/getSysOutputRemark/setSchedulerSysOutput)

18.6.1 说明

接口	功能	返回值	参数	参数说明
void IOInterface::setSchedulerSysOutput(const bool scheduler)	设置系统数字输出 (SysDO) 刷新调度开关	void	scheduler	true 开启 SysDO 定时刷新调度; false 关闭。
bool IOInterface::getDataSystemOut(QList<bool> &data)	读取系统数字输出 (SysDO) 当前值	bool	data	输出参数: SysDO 值列表 (按索引 0..N) ; true 表示 ON, false 表示 OFF。
bool IOInterface::getSysOutputRemark(QStringList &data)	读取系统数字输出 (SysDO) 备注/描述	bool	data	输出参数: SysDO 备注列表 (每个索引对应一条说明文本)。

bool IOInterface::getSysOutputRemarkList(QList<QString> &data)	读取系统数字输出 (SysDO) 备注/描述 (Qt5 分支)	bool	data	Qt5 下使用 QList<QString> 承接备注列表; 语义同 getSysOutputRemark。
--	---------------------------------	------	------	--

18.6.2 调用

```
C++
// 显示页面: 开启调度 + 读取备注 + 周期刷新 SysDO 值
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_SystemOut_Schedule,
    true);

CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_SystemOut_Reamarks,
    &m_remarks);

CommunicationEngine::instance()->enqueueCmd_getData(
    this,
    AbstractCmd::CmdType::CmdType_Get_SystemOut_Values,
    &m_values);
```

```
C++
// 隐藏页面: 关闭调度
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Set_SystemOut_Schedule,
    false);
```

18.6.3 函数实现

```
C++
// SYSTEM OUT
case AbstractCmd::CmdType_Get_SystemOut_Values: {
    auto [address] = ((CmdDatas<QList<bool> *> *)absCmd)->m_data;
```

```

        getListDatas(QVariant(),
                      *address,
                      &Communication::getDatasSystemOut,

&CommunicationEngine::signal_SystemOut_getDatas_result);
        break;
    }
    case AbstractCmd::CmdType_Get_SystemOut_Reamarks: {
        auto [address] = ((CmdDatas<QList<QString> *> *)absCmd)->m_data;
        #if defined(INOCOBOTTP_MSVC_QT5)
            getListDatas(QVariant(),
                          *address,
                          &Communication::getSysOutputRemarkList,
                          &CommunicationEngine::
                            signal_SystemOut_getRemarks_result);
        #else
            getListDatas(QVariant(),
                          *address,
                          &Communication::getSysOutputRemark,
                          &CommunicationEngine::
                            signal_SystemOut_getRemarks_result);
        #endif

        break;
    }
    case AbstractCmd::CmdType_Set_SystemOut_Schedule: {
        auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
        m_commInstance->setSchedulerSysOutput(state);
        break;
    }
}

```

```

C++
bool IOInterface::getDatasSystemOut(QList<bool> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->updateSysOutputDatas();
    std::array<bool, SYSIO_SIZE> value = _IResource-
>getSysOutputStatus();
    data = QList<bool>(value.begin(), value.end());
    FREQ_LOG_PRINT_TIMESTAMP

```

```

        return true;
    }

void IOInterface::setSchedulerSysOutput(const bool scheduler)
{
    // _IResource->setSysOutputScheduler(scheduler);
}

bool IOInterface::getSysOutputRemark(QStringList &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->updateSysOutputDatas();
    std::array<std::string, SYSIO_SIZE> remark = _IResource-
>getSysOutputRemarks();
    foreach (auto &str, remark) {
        data.push_back(QString::fromStdString(str));
    };
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}

#if defined(INOCOBOTTP_MSVC_QT5)
bool IOInterface::getSysOutputRemarkList(QList<QString> &data)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    _IResource->updateSysOutputDatas();
    std::array<std::string, SYSIO_SIZE> remark = _IResource-
>getSysOutputRemarks();
    foreach (auto &str, remark) {
        data.push_back(QString::fromStdString(str));
    };
    FREQ_LOG_PRINT_TIMESTAMP
    return true;
}
#endif

```

19.通讯状态

19.1 设备连接 (getDeviceConnectionState)

19.1.1 说明

接口	功能	返回值	参数	参数说明
void ConnectionInterface::getDeviceConnectionState(AbstractCmd *cmd)	获取设备连接状态 (EtherNet1/ EtherNet2/控制器 USB/控制器 SD 卡/ EtherCAT1/I R-Link1)	void	cmd	由 CmdType_getDeviceConnectionState 触发； cmd->m_object 用于回传信号目标对象； 输出通过 signal_getDeviceConnectionResult(obj, labelState) 返回 6 条状态文本。

19.1.2 调用

```
C++
// 设备连接页面：周期刷新（500ms）
CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_getDeviceConnectionState);
```

19.1.3 函数实现

```
C++
void ConnectionInterface::getDeviceConnectionState(AbstractCmd
*cmd)
{
    QStringList value;
```

```

ushort eth0State = 0, eth1State = 0;
int16u moduleCommState[MODULE_NUM];
_IMonitor->GetModuleCommState(moduleCommState);
eth0State = moduleCommState[0];
eth1State = moduleCommState[1];

if (eth0State == 0 && eth1State == 0) {
    for (int i = 0; i < 6; ++i)
        value << ConnectionInterfaceTr::tr("Unknown state",
"No punctuation required");
    emit CommunicationEngine::instance()-
>signal_getDeviceConnectionResult(cmd->m_object, value);
    return;
}

ControllerEthCfg cfg;
int ret = _IRCCfg->getEthConfig(cfg);
if (ret != ERROR_OK) {
    MWARNING(ConnectionInterfaceTr::tr("Failed to get controll
IP address."));
    for (int i = 0; i < 6; ++i)
        value << ConnectionInterfaceTr::tr("Unknown state",
"No punctuation required");
    emit CommunicationEngine::instance()-
>signal_getDeviceConnectionResult(cmd->m_object, value);
    return;
}

switch (moduleCommState[0]) {
case 0:
    value << ConnectionInterfaceTr::tr("Disconnected");
    break;
case 1:
    value << ((cfg.strcEth0.strDHCP == "1" ?
                ConnectionInterfaceTr::tr("Dynamic IP:") :
                ConnectionInterfaceTr::tr("Static IP:"))
        + " " + cfg.strcEth0.strIP.c_str());
    break;
case 2:
    value << ConnectionInterfaceTr::tr("Disabled");
    break;
case 3:
    value << ConnectionInterfaceTr::tr("Unknown state");
    break;
}

```



```

}
switch (moduleCommState[1]) {
case 0:
    value << ConnectionInterfaceTr::tr("Disconnected");
    break;
case 1:
    value << ((cfg.strcEth1.strDHCP == "1" ?
                ConnectionInterfaceTr::tr("Dynamic IP:") :
                ConnectionInterfaceTr::tr("Static IP:"))
              + " " + cfg.strcEth1.strIP.c_str());
    break;
case 2:
    value << ConnectionInterfaceTr::tr("Disabled");
    break;
case 3:
    value << ConnectionInterfaceTr::tr("Unknown state");
    break;
}

switch (moduleCommState[2]) {
case 0:
    value << ConnectionInterfaceTr::tr("Device not connected",
"No punctuation required");
    break;
case 1:
    value << ConnectionInterfaceTr::tr("Connected and
successfully mounted", "No punctuation required");
    break;
case 2:
    value << ConnectionInterfaceTr::tr("Connected but failed
to mount", "No punctuation required");
    break;
default:
    value << ConnectionInterfaceTr::tr("Unknown state", "No
punctuation required");
    break;
}
switch (moduleCommState[3]) {
case 0:
    value << ConnectionInterfaceTr::tr("Device not connected",
"No punctuation required");
    break;
case 1:
    value << ConnectionInterfaceTr::tr("Connected and

```

```

successfully mounted", "No punctuation required");
    break;
    case 2:
        value << ConnectionInterfaceTr::tr("Connected but failed
to mount", "No punctuation required");
        break;
    case 3:
        value << ConnectionInterfaceTr::tr("Storage card format
error", "No punctuation required");
        break;
    default:
        value << ConnectionInterfaceTr::tr("Unknown state", "No
punctuation required");
        break;
}
switch (moduleCommState[4]) {
case 0:
    value << ConnectionInterfaceTr::tr("Communication is
normal", "No punctuation required");
    break;
case 1:
    value << ConnectionInterfaceTr::tr("Slave disconnected",
"No punctuation required");
    break;
case 2:
    value << ConnectionInterfaceTr::tr("Device not mounted",
"No punctuation required");
    break;
case 3:
    value << ConnectionInterfaceTr::tr("Non-EtherCAT device
connected", "No punctuation required");
    break;
case 4:
    value << ConnectionInterfaceTr::tr("Disabled", "No
punctuation required");
    break;
default:
    value << ConnectionInterfaceTr::tr("Unknown state", "No
punctuation required");
    break;
}
switch (moduleCommState[5]) {
case 0:
    value << ConnectionInterfaceTr::tr("Communication is

```

```

normal", "No punctuation required");
    break;
    case 1:
        value << ConnectionInterfaceTr::tr("Slave disconnected",
"No punctuation required");
        break;
    case 2:
        value << ConnectionInterfaceTr::tr("Device not mounted",
"No punctuation required");
        break;
    case 3:
        value << ConnectionInterfaceTr::tr("Non-IR-Link device
connected", "No punctuation required");
        break;
    case 4:
        value << ConnectionInterfaceTr::tr("Disabled", "No
punctuation required");
        break;
    case 5:
        value << ConnectionInterfaceTr::tr("Device not
configured", "No punctuation required");
        break;
    default:
        value << ConnectionInterfaceTr::tr("Unknown state");
        break;
    }
    emit CommunicationEngine::instance()-
>signal_getDeviceConnectionResult(cmd->m_object, value);
}

```

```

C++
case AbstractCmd::CmdType_getDeviceConnectionState: {
    m_commInstance->getDeviceConnectionState(absCmd);
    break;
}

```

19.2 总线状态

(getMbusConnectionState/getEtherNetIpConnectionState/getEtherCatIpConnectionState/getMCConnecti

onState/getProfinetConnectionState)

19.2.1 说明

接口	功能	返回值	参数	参数说明
void ConnectionInterface::getModbusConnectionState(AbstractCmd *cmd)	获取 Modbus RTU/TCP 激活状态、端口、连接表，并决定“EtherNet/IP/EtherCAT”页签是否可用	void	cmd	由 CmdType_getModbusConnectionState 触发；读取 _IResource 的 Modbus 连接状态并通过 signal_getModbusConnectionResult(...) 回传（包含 rtuState/tcpState/isInEtherNet/isInEtherCat/labelStates/tables）。
void ConnectionInterface::getEtherNetIpConnectionState(AbstractCmd *cmd)	获取 EtherNet/IP 激活与连接信息	void	cmd	由 CmdType_getEtherNetIpConnectionState 触发；通过 signal_getEtherNetIpConnectionResult(...) 回传（含 active、slavePort、connect、masterConfig 等）。
void ConnectionInterface::getEtherCatIpConnectionState(AbstractCmd *cmd)	获取 EtherCAT 激活与连接信息	void	cmd	由 CmdType_getEtherCatIpConnectionState 触发；通

ractCmd *cmd)				过 signal_getEtherC atConnectionRes ult(...) 回传（含 active/connect/lab elStates）。
void ConnectionInterfac e::getMCConnectio nState(AbstractCm d *cmd)	获取 MC 连接状 态表	void	cmd	由 CmdType_getMC ConnectionState 触发；通过 signal_getMCCon nectionResult(...) 回传（含是否激活 /状态文案/表格数 据）。
void ConnectionInterfac e::getProfinetConn ectionState(Abstra ctCmd *cmd)	获取 Profinet 激 活/连接/IP 等信 息	void	cmd	由 CmdType_getPro finetConnectionSt ate 触发；通过 signal_getProfinet ConnectionResult (...) 回传（含 active/connect/lab elStates）。

19.2.2 调用

```
C++
// 总线状态页面：周期刷新（500ms），根据当前 tab 切换 CmdType
CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_getMobusConnectionState);

CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_getEtherNetIpConnectionState);

CommunicationEngine::instance()->enqueueCmd(
    this,
```

```

        AbstractCmd::CmdType_getEtherCatIpConnectionState);

CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_getMCConnectionState);

CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_getProfinetConnectionState);

```

19.2.3 函数实现

```

C++
case AbstractCmd::CmdType_getModbusConnectionState: {
    m_commInstance->getModbusConnectionState(absCmd);
    break;
}
case AbstractCmd::CmdType_getEtherNetIpConnectionState: {
    m_commInstance->getEtherNetIpConnectionState(absCmd);
    break;
}
case AbstractCmd::CmdType_getEtherCatIpConnectionState: {
    m_commInstance->getEtherCatIpConnectionState(absCmd);
    break;
}
case AbstractCmd::CmdType_getMCConnectionState: {
    m_commInstance->getMCConnectionState(absCmd);
    break;
}
case AbstractCmd::CmdType_getProfinetConnectionState: {
    m_commInstance->getProfinetConnectionState(absCmd);
    break;
}
}

```

```

C++
void ConnectionInterface::getModbusConnectionState(AbstractCmd
*cmd)
{
    bool ok = _IResource->updateModbusConnectStatus();
}

```

```

    if (!ok)
        return;

    InoRobBusiness::ModbusConnectStatus status = _IResource-
>getModbusConnectStatus();
    QStringList labelAns;
    switch (status.modbusRtuCS.status) {
    case 1:
        labelAns << ConnectionInterfaceTr::tr("Slave activated");
        break;
    case 2:
        labelAns << ConnectionInterfaceTr::tr("Master activated");
        break;
    default:
        labelAns << ConnectionInterfaceTr::tr("Not activated");
        break;
    }
    switch (status.modbusTcpCS.status) {
    case 1:
        labelAns << ConnectionInterfaceTr::tr("Slave activated");
        break;
    case 2:
        labelAns << ConnectionInterfaceTr::tr("Master activated");
        break;
    default:
        labelAns << ConnectionInterfaceTr::tr("Not activated");
        break;
    }
    bool activeRtu = (status.modbusRtuCS.status == 1 ||
status.modbusRtuCS.status == 2);
    bool activeTcp = (status.modbusTcpCS.status == 1 ||
status.modbusTcpCS.status == 2);

    labelAns << QString::number(status.modbusTcpCS.port);
    QList<QStringList> tableAns;
    tableAns << QStringList() << QStringList() << QStringList();
    for (int i = 0; i < TCP_CLIENT_NUM; i++) {
        if (status.modbusTcpConnectFlag[i] == 1) {
            tableAns[0] << (char *)(status.modbusTcpClientIP[i]);
            tableAns[1] <<
QString::number(status.modbusTcpClientPort[i]);
            tableAns[2] << ConnectionInterfaceTr::tr("Connected");
        } else {
            tableAns[0] << "----";

```

```

        tableAns[1] << "----";
        tableAns[2] <<
ConnectionInterfaceTr::tr("Disconnected");
    }
}
char busInstall[4];
_IMonitor->GetBusInstall(busInstall);
bool isHasEthercat = (busInstall[0]>>0) & 0x01;
emit CommunicationEngine::instance()-
>signal_getModbusConnectionResult(cmd->m_object,

activeRtu,

activeTcp,

status.modbusAddrType ==1,

status.modbusAddrType ==1 && isHasEthercat ,

labelAns,

tableAns);
}

void ConnectionInterface::getMCConnectionState(AbstractCmd *cmd)
{
    bool ok = _IResource->updateMCStatus();
    if (!ok)
        return;
    bool _mcActiveStatus = _IResource->getMCActiveStatus();
    std::array<InoRobBusiness::MCConnectStatus, MC_CONNECT_NUM>
        _mcConnectStatus = _IResource->getMCConnectStatus();
    QList<QStringList> ans;
    ans << QStringList() << QStringList() << QStringList();
    for (int i = 0; i < MC_CONNECT_NUM; i++) {
        QString ip((char *)_mcConnectStatus[i].serverIP);
        if (!ip.isEmpty())
            ans[0] << ip + " : " +
QString::number(_mcConnectStatus[i].port);
        else
            ans[0] << "----";
        ans[1] << (_mcConnectStatus[i].connect == 1 ?
ConnectionInterfaceTr::tr("Connected") :
ConnectionInterfaceTr::tr("Disconnected"));

```



```

        ans[2] << (_mcConnectStatus[i].autoReconnect == 1 ?
ConnectionInterfaceTr::tr("Yes") :
ConnectionInterfaceTr::tr("No"));
    }

    emit CommunicationEngine::instance()-
>signal_getMCConnectionResult(
        cmd->m_object,
        _mcActiveStatus,
        _mcActiveStatus ? ConnectionInterfaceTr::tr("Active") :
ConnectionInterfaceTr::tr("Inactive"),
        ans);
}

void ConnectionInterface::getProfinetConnectionState(AbstractCmd
*cmd)
{
    bool ok = _IResource->updateProfinetConnectStatus();
    if (!ok)
        return;
    QStringList ans;
    InoRobBusiness::ProfinetConnectStatus status = _IResource-
>getProfinetConnectStatus();
    ans << (status.activeStatus ?
ConnectionInterfaceTr::tr("Active") :
ConnectionInterfaceTr::tr("Inactive"));
    if (status.activeStatus) {
        ans << QString((char *)status.ip);
        ans << QString((char *)status.defaultGateWay);
        ans << QString((char *)status.macAddress);
        ans << QString::number(status.version);
    } else {
        ans << "----";
        ans << "----";
        ans << "----";
        ans << "----";
    }
    ans << (status.connectStatus ?
ConnectionInterfaceTr::tr("Connected") :
ConnectionInterfaceTr::tr("Disconnected"));
    emit CommunicationEngine::instance()-
>signal_getProfinetConnectionResult(
        cmd->m_object,
        status.activeStatus,

```

```

        status.connectStatus,
        ans);
    }

void ConnectionInterface::getEtherNetIpConnectionState(AbstractCmd
*cmd)
{
    _IResource->updateEthernetIPStatus();
    InoRobBusiness::EthernetIPStatus status = _IResource-
>getEthernetIPStatus();
    emit CommunicationEngine::instance()-
>signal_getEtherNetIpConnectionResult(
        cmd->m_object,
        status.active,
        status.active?ConnectionInterfaceTr::tr("Active") :
ConnectionInterfaceTr::tr("Inactive"),
        QString::number(status.slavePort),
        status.connect,
        status.connect ?
QString("%1 : %2").arg(QString((char*)status.masterIP),
QString::number(status.masterPort))
        :
ConnectionInterfaceTr::tr("Disconnected"));
}

void ConnectionInterface::getEtherCatIpConnectionState(AbstractCmd
*cmd)
{
    _IResource->updateEtherCatStatus();
    InoRobBusiness::EtherCATStatus status = _IResource-
>getEtherCatStatus();
    QStringList ans;
    ans<<(status.active?ConnectionInterfaceTr::tr("Active") :
ConnectionInterfaceTr::tr("Inactive"));
    ans<<(status.connect?ConnectionInterfaceTr::tr("Connected") :
ConnectionInterfaceTr::tr("Disconnected"));
    emit CommunicationEngine::instance()-
>signal_getEtherCatConnectionResult(
        cmd->m_object,
        status.active,
        status.connect,
        ans);
}

```

20. 工况

20.1 电流

(resetHisMaxCurrent/readActCurrent/readHisMaxCurrent/setElectricScheduler)

20.1.1 说明

接口	功能	返回值	参数	参数说明
int ResourceInterface::resetHisMaxCurrent()	重置“历史最大电流”	int	无	返回错误码： ERR_OK(0) 成功；其他为失败。
int ResourceInterface::readActCurrent(std::vector<double> &actualCurrent)	读取实时电流（百分比）	int	actualCurrent	输出参数：按关节顺序的实时电流百分比数组 (J1..J6/7) 。
int ResourceInterface::readHisMaxCurrent(std::vector<double> &maxCurrent)	读取历史最大电流（百分比）	int	maxCurrent	输出参数：按关节顺序的历史最大电流百分比数组 (J1..J6/7) 。
void ResourceInterface::setElectricScheduler(bool scheduler)	设置电流监控调度开关	void	scheduler	true：打开调度并先执行一次 updateElectric() 刷新； false：关闭调

				度。
--	--	--	--	----

20.1.2 调用

C++

// 重置历史最大电流

```
CommunicationEngine::instance()->enqueueCmd(  
    this,  
    AbstractCmd::CmdType_WorkCond_ResetHisMaxCurrent);
```

C++

// 刷新：实时电流 + 历史最大电流

```
CommunicationEngine::instance()->enqueueCmd(  
    this,  
    AbstractCmd::CmdType_WorkCond_ReadActCurrent);  
CommunicationEngine::instance()->enqueueCmd(  
    this,  
    AbstractCmd::CmdType_WorkCond_ReadHisMaxCurrent);
```

C++

// 显示/隐藏：开启/关闭电流调度

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_WorkCond_setElectricScheduler,  
    true);  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_WorkCond_setElectricScheduler,  
    false);
```

20.1.3 函数实现

C++

```
void ResourceInterface::setElectricScheduler(bool scheduler)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
```

```

        if (scheduler) {
            _IResource->updateElectric();
        }
        _IResource->setElectricScheduler(scheduler);
    }

int ResourceInterface::readActCurrent(std::vector<double>
&actualCurrent)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _IResource->readCurrentElectric(actualCurrent);
    return iRet;
}

int ResourceInterface::readHisMaxCurrent(std::vector<double>
&maxCurrent)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int iRet = _IResource->readMaxElectric(maxCurrent);
    return iRet;
}

int ResourceInterface::resetHisMaxCurrent()
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _IResource->resetMaxElectric();
}

```

```

C++
case AbstractCmd::CmdType_WorkCond_ResetHisMaxCurrent: {
    int iRet = m_commInstance->resetHisMaxCurrent();
    emit CommunicationEngine::instance()
        ->signal_resetHisMaxCurrent_result(iRet);
    break;
}
case AbstractCmd::CmdType_WorkCond_ReadActCurrent: {
    std::vector<double> retArr(6, 0.0);
    int iRet = m_commInstance->readActCurrent(retArr);
    emit CommunicationEngine::instance()
        ->signal_readActCurrent_result(iRet, retArr);
    break;
}
case AbstractCmd::CmdType_WorkCond_ReadHisMaxCurrent: {

```

```

std::vector<double> retArr(6, 0.0);
int iRet = m_commInstance->readHisMaxCurrent(retArr);
emit CommunicationEngine::instance()
    ->signal_readHisMaxCurrent_result(iRet, retArr);
break;
}
case AbstractCmd::CmdType_WorkCond_setElectricScheduler: {
    auto [scheduler] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setElectricScheduler(scheduler);
    break;
}
}

```

20.2 负载率 (readActLoadrate/setAvgLoadRateScheduler)

20.2.1 说明

接口	功能	返回值	参数	参数说明
int ResourceInterface::readActLoadrate(std::vector<double> &actualLoad)	读取实时平均负载率（百分比）	int	actualLoad	输出参数：按关节顺序的平均负载率百分比数组（J1..J6/7）。
void ResourceInterface::setAvgLoadRateScheduler(bool scheduler)	设置平均负载率监控调度开关	void	scheduler	true：打开调度并先执行一次 updateAvgLoadRate() 刷新；false：关闭调度。

20.2.2 调用

C++

// 刷新：平均负载率

```
CommunicationEngine::instance()->enqueueCmd(  
    this,  
    AbstractCmd::CmdType_WorkCond_ReadActLoadrate);
```

C++

// 显示/隐藏：开启/关闭平均负载率调度

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
  
    AbstractCmd::CmdType::CmdType_WorkCond_setAvgLoadRateScheduler,  
    true);  
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
  
    AbstractCmd::CmdType::CmdType_WorkCond_setAvgLoadRateScheduler,  
    false);
```

20.2.3 函数实现

C++

```
void ResourceInterface::setAvgLoadRateScheduler(bool scheduler)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    if (scheduler) {  
        _IResource->updateAvgLoadRate();  
    }  
    _IResource->setAvgLoadRateScheduler(scheduler);  
}  
  
int ResourceInterface::readActLoadrate(std::vector<double>  
&actualLoad)  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    return _IResource->readCurrentAvgLoadRate(actualLoad);  
}
```

C++

```
case AbstractCmd::CmdType_WorkCond_ReadActLoadrate: {
    std::vector<double> retArr(6, 0.0);
    int iRet = m_commInstance->readActLoadrate(retArr);
    emit CommunicationEngine::instance()
        ->signal_readActLoadrate_result(iRet, retArr);
    break;
}
case AbstractCmd::CmdType_WorkCond_setAvgLoadRateScheduler: {
    auto [scheduler] = ((CmdDatas<bool> *)absCmd)->m_data;
    m_commInstance->setAvgLoadRateScheduler(scheduler);
    break;
}
```

20.3 ABZ (getCobotABZData)

20.3.1 说明

接口	功能	返回值	参数	参数说明
void IOInterface::g etCobotABZD ata(AbstractC md *cmd)	读取编码器 ABZ 数据并回 传到界面	void	cmd	由 CmdType_Ge tABZ_Data 触 发；从 _IMonitor- >readCobotA BZDatas(data) 读取 JSON，并发 出 signal_get_A BZ_Data(cmd ->m_object, ans)。

20.3.2 调用

C++


```
// ABZ 页面：周期刷新（500ms）
CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_GetABZ_Data);
```

20.3.3 函数实现

```
C++
void IOInterface::getCobotABZData(AbstractCmd *cmd)
{
    std::string data;
    int ret = _IMonitor->readCobotABZData(data);
    if (ret != ERROR_OK) {
        PRINT_ERROR(QObject::tr("Failed to read encoder data."));
        return;
    }
    QJsonObject obj =
QJsonDocument::fromJson(data.c_str()).object();
    int ans = obj["encoder"].toInt();
    emit CommunicationEngine::instance()->signal_get_ABZ_Data(cmd-
>m_object, ans);
}
```

```
C++
case AbstractCmd::CmdType_GetABZ_Data: {
    m_commInstance->getCobotABZData(absCmd);
    break;
}
```

21.关于

21.1 版本（getVersionInfo）

21.1.1 说明

接口	功能	返回值	参数	参数说明
----	----	-----	----	------

void ControllerSystemConfig::getVersionInfo(AbstractCmd *cmd)	获取示教器版本、控制器版本及通配/ToolIO 等版本信息列表	void	cmd	由 CmdType_GetVersionInfo 触发；cmd->m_object 用于信号回传目标对象；输出通过 signal_getVersionInfoResult(obj, isSuccess, info) 返回 QStringList info（第 0 位固定为示教器版本，其余为控制器/通配/ToolIO 等扩展项）。
--	---------------------------------	------	-----	---

21.1.2 调用

```
C++
// 关于-版本页：显示时获取版本信息
CommunicationEngine::instance()->enqueueCmd(
    this,
    AbstractCmd::CmdType_GetVersionInfo);
```

21.1.3 函数实现

```
C++
void ControllerSystemConfigPrivate::getVersionInfo(AbstractCmd
*cmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD

    QStringList res;
```

```

    QString errorStr;
    InoRobBusiness::ControllerConnectionStatus state =
_IConnection->GetConnectionStatus();

    // 示教器版本 0.1
    auto teachPadVersion = _IMonitor->getTeachPadVersion();
    res.push_back(S2Q(teachPadVersion));
    if (state !=
InoRobBusiness::ControllerConnectionStatus::CONTROLLER_CONNECTION_
STATUS_CONNECTED) {
        emit CommunicationEngine::instance()-
>signal_getVersionInfoResult(cmd->m_object, true, res);
        return;
    }
    std::vector<std::string> version;
    // int ret = _IMonitor->getControllerVersionInfo(version);
    //失败重读
    int ret = 0;
    int nTime = 0;
    while (true) {
        ret = _IMonitor->getControllerVersionInfo(version);
        nTime++;
        if (ret == ERR_OK || nTime >= 10) {
            break;
        }
    }

    if (ret != ERR_OK || version.size() < 8) {
        errorStr += QObject::tr("Failed to get controller
version.");
        LOG_ERROR(QString("ret is %1, size
is %2").arg(QString::number(ret),
QString::number(version.size())));
        for (int i = 0; i < 9; ++i)
            res.push_back("----");
    } else {
        // 控制器版本 0.2
        res.push_back(S2Q(version[4]));
        // 控制器类型 1
        res.push_back(S2Q(version[0]));
        // 系统软件版本 3
        res.push_back(S2Q(version[1]));
        // 软件掉电保存类型 4
        bool isPowerDownSave = _IMonitor->GetIsPowerDownSave();

```

```

        res.push_back(isPowerDownSave ? QObject::tr("ON") :
QObject::tr("OFF"));
        // 内核版本 5
        res.push_back(S2Q(version[2]));
        // 示教器系统版本 6
        res.push_back("----");
        // 引导软件版本 7
        res.push_back(S2Q(version[3]));
        // 控制器硬件版本 8
        int32u hardWareVer = _IMonitor->GetHardWareVer();
        res.push_back(QString::number(hardWareVer));
        // 控制系统固件版本 9
        res.push_back(S2Q(version[5]));
    }
    // 机器人型号 2
    char robotStructName[128] = {0};
    if (_IMonitor->GetRobotName(robotStructName)) {
        res.insert(res.begin() + 3, robotStructName);
    } else {
        errorStr += (errorStr.isEmpty() ? "" : "\r\n") +
QObject::tr("Failed to get robot name.");
        res.push_back("----");
    }
    // 通配信息
    InoRobBusiness::IGeneralMatch::GeneralMatchInfo matchInfo;
    if (!q->comm()
        ->robotParam()
        ->getGeneralMatch()
        ->getGeneralMatchInfo(matchInfo)) {
        for (int i = 0; i < 7; ++i)
            res.push_back("----");
        errorStr += ((errorStr.isEmpty() ? "" : "\r\n") +
QObject::tr("Failed to get universality info.));
    } else {

res.push_back(S2Q(matchInfo.controllerParamCompatibilityVersion));
// 10
        res.push_back(S2Q(matchInfo.mainFPGAHardwareVersion));
// 11
        res.push_back(S2Q(matchInfo.mainFPGASoftwareVersion));
// 12
        res.push_back(S2Q(matchInfo.bodyHardwareVersion));
// 13
        res.push_back(S2Q(matchInfo.bodySoftwareVersion));

```

```

// 14
    res.push_back(S2Q(matchInfo.bodyParamVersion));
// 15

res.push_back(S2Q(matchInfo.bodyParamCompatibilityVersion));
// 16
    }

    std::string toolIOVersion;
    ret = _IMonitor->readCobotToolIOVersion(toolIOVersion);
    if (ret != ERROR_OK) {
        toolIOVersion.clear();
        errorStr += (errorStr.isEmpty() ? "" : "\r\n") +
QObject::tr("Failed to get tool I/O version info.");
        for (int i = 0; i < 4; ++i)
            res.push_back("----");
    } else {
        QJsonObject json =
QJsonDocument::fromJson(toolIOVersion.c_str()).object();

res.push_back(QString::number(json["ComVersion"].toInt())); //
17

res.push_back(QString::number(json["BootVersion"].toInt())); //
18

res.push_back(QString::number(json["XmlVersion"].toInt())); //
19

res.push_back(QString::number(json["TioVersion"].toInt())); //
20
    }
    if (version.size() >= 10)
        res.push_back(S2Q(version[9]));
    if (!errorStr.isEmpty())
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(errorStr);
    for (auto &i : res)
        if (i.isEmpty() || i == "\r\n" || i == "\n")
            i = "----";
        emit CommunicationEngine::instance()-
>signal_getVersionInfoResult(cmd->m_object, true, res);

    FREQ_LOG_PRINT_TIMESTAMP

```

```
}
```

```
C++
case AbstractCmd::CmdType_GetVersionInfo: {
    m_commInstance->getVersionInfo(absCmd);
    break;
}
```

22.其他

22.1 使能 (enableRobot)

22.1.1 说明

接口	功能	返回值	参数	参数说明
bool RobotControll interface::enab leRobot(const bool &enable)	调用控制器接 口设置使能	bool	enable	true 上使能、 false 下使能; 内部调用 _IControl- >SetEnable(e nable), 返回 ret==0 为成 功。

22.1.2 调用

```
C++
// 主界面：点击使能按钮
CommunicationEngine::instance()->enqueueCmd_enableRobot(this,
checked);
```

22.1.3 函数实现

```

C++
bool RobotControlInterface::enableRobot(const bool &enable)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = _IControl->SetEnable(enable);
    qDebug() << "RobotControlInterface::enableRobot ret : " <<
ret;
    FREQ_LOG_PRINT_TIMESTAMP
    return 0 == ret;
}

```

```

C++
case AbstractCmd::CmdType_EnableRobot: {
    bool isSuccess = m_commInstance->enableRobot(
        static_cast<CmdEnableRobot *>(absCmd)->m_enableRobot);
    emit CommunicationEngine::instance()
        ->signal_enableRobotInterface_result(absCmd->m_object,
isSuccess);
    break;
}

```

22.2 断连 (disconnectController/connectVirtualController)

22.2.1 说明

接口	功能	返回值	参数	参数说明
bool RobotControll n terface::disc onnectControl ler()	主动断开与控 制器连接	bool	无	内部调用 _IConnection- >Close(true) ; 返回 true 表 示断开成功。
bool RobotControll	连接控制器	bool	ip 端口	

```
nterface::connectVirtualController(AbstractCmd *cmd){
```

22.2.2 调用

C++

// 主界面：点击连接状态按钮 -> 选择“断开”

```
CommunicationEngine::instance()->enqueueCmd(  
    this,  
    AbstractCmd::CmdType_DisconnectController);
```

```
CommunicationEngine::instance()->enqueueCmd_setData(  
    this,  
    AbstractCmd::CmdType::CmdType_ConnectVirtualController,  
    ip,  
    port,  
    virtualPath);
```

22.2.3 函数实现

C++

```
bool RobotControlInterface::disconnectController()  
{  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    bool ret = _IConnection->Close(true);  
    if (!ret) {  
        InoRobLog::error("disconnect controller {}:{} failed");  
    }  
  
    FREQ_LOG_PRINT_TIMESTAMP  
    return ret;  
}
```

```
int RobotControlInterface::connectVirtualController(AbstractCmd  
*cmd){  
    FREQ_LOG_PRINT_TIMESTAMP_THREAD  
    LOG_INFO("connectVirtualController start");  
  
    bool isSuccess = false;
```



```

QString msg;

try {

    auto [ip, port, virtualpath] = BIND(cmd, QString, QString,
    QString);

    LOG_INFO(QString("connectVirtualController params: ip=%1,
port=%2, virtualpath=%3")
        .arg(ip).arg(port).arg(virtualpath));

    if (_IConnection == nullptr) {
        msg = QObject::tr("connection object is null");
        LOG_ERROR("connectVirtualController: " + msg);
        return false;
    }

    // bool ret = _IConnection->Close(true);
    // if (!ret) {
    //     InoRobLog::error("disconnect controller {}:{}
failed");
    //     return false;
    // }

    comm()->setIP(ip);
    comm()->setPort(port.toUInt());

    _IConnection->SetIp(ip.toStdString());
    _IConnection->setPort(port.toUInt());

    // 执行连接
    bool rt = _IConnection-
>Connect(InoRobBusiness::CONNECT_TP_TYPE);
    if (!rt) {
        msg = QObject::tr("connect to controller failed");
        LOG_ERROR("connectVirtualController: " + msg);
        return false;
    }

    // 设置控制器类型
    if (ip == "127.0.0.1") {
        // 虚拟控制器

```

```

        if (!virtualpath.isEmpty()) {
            FileCast filecast;

filecast.restoreOriginalLocation(virtualpath.toStdString());
        }
        _IConnection->setControllerType(1);

        if (_IProject) {
            _IProject->GetName(); // 原代码中有这行，可能用于调
试
            // _IProject-
>SetControllerType(TPPlatform::ControllerType::CTRL_TYPE_VIRTUAL
);
        }

        InoRobLog::debug("Connected to virtual controller");
        msg = QObject::tr("Connected to virtual controller");
    }
    else {
        // 真实控制器
        _IConnection->setControllerType(0);
        if (_IProject) {
            // _IProject-
>SetControllerType(TPPlatform::ControllerType::CTRL_TYPE_REAL);
        }
        msg = QObject::tr("Connected to real controller");
    }

    isSuccess = true;
    LOG_INFO("connectVirtualController: " + msg);

} catch (const std::exception& ex) {
    LOG_ERROR("connectVirtualController Exception: " +
QString::fromStdString(std::string(ex.what())));
    isSuccess = false;
}

    emit CommunicationEngine::instance()-
>signal_connectVirtualController_result(cmd->m_object, isSuccess,
msg);

    LOG_INFO(QString("connectVirtualController end.
success=%1").arg(isSuccess));

```

```

        FREQ_LOG_PRINT_TIMESTAMP

        return isSuccess;
    }

```

```

C++
case AbstractCmd::CmdType_DisconnectController: {
    m_commInstance->setAvgLoadRateScheduler(false);
    m_commInstance->setElectricScheduler(false);
    m_commInstance->setModbusConnectScheduler(false);
    m_commInstance->setEthernetIPScheduler(false);
    m_commInstance->setEtherCatScheduler(false);

    bool isSuccess = m_commInstance->disconnectController();
    emit CommunicationEngine::instance()
        ->signal_disconnectControllerInterface_result(
            absCmd->m_object, isSuccess);
    break;
}

case AbstractCmd::CmdType_ConnectVirtualController: {
    m_commInstance->connectVirtualController(absCmd);
    break;
}

```

22.3 模式切换（SetDeviceMode/GetDeviceMode）

22.3.1 说明

接口	功能	返回值	参数	参数说明
Int RobotControll nterface::SetD eviceMode(M	设置控制器运 行模式（自动/ 手动）	int	mode	mode: 目标 模式（如 RobotDevice Mode_Auto/R

etaType::RobotDeviceMode)				obotDeviceMode_Manual_Low 等)；返回 0 成功。
Int RobotControlInterface::GetDeviceMode(MetaType::RobotDeviceMode &mode)	获取当前运行模式	int	moed	

22.3.2 调用

```
C++
// 主界面：点击模式按钮（自动/手动切换）
CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType::CmdType_Control_SetDeviceMode,
    mode);
```

22.3.3 函数实现

```
C++
case AbstractCmd::CmdType_Control_SetDeviceMode: {
    auto [deviceMode]
        = ((CmdDatas<MetaType::RobotDeviceMode> *)absCmd)->m_data;
    int iRet = Communication::instance()-
>SetDeviceMode(deviceMode);
    emit CommunicationEngine::instance()
        ->signal_setdevicemode_result(iRet == 0, deviceMode);
    break;
}

int RobotControlInterface::SetDeviceMode(MetaType::RobotDeviceMode mode)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    return _IControl->SetDeviceMode(
        static_cast<InoRobBusiness::DeviceMode>(mode));
}
```

```

}

int RobotControlInterface::GetDeviceMode(MetaType::RobotDeviceMode
&mode)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    InoRobBusiness::RunMode runmode;
    runmode = _IMonitor->GetRunMode();
    if (runmode == InoRobBusiness::RUN_MODE_REAPPEAR) {
        mode = MetaType::RobotDeviceMode_Auto;
    } else {
        mode = MetaType::RobotDeviceMode_Manual_Low;
    }

    // mode = static_cast<MetaType::RobotDeviceMode>(deviceMode);
    // qDebug() << "Current Device Mode = " <<
static_cast<int>(mode)
    // << " | iRet = " << iRet;
    FREQ_LOG_PRINT_TIMESTAMP
    return 0;
}

```

22.4 急停 (SetEmergency)

22.4.1 说明

接口	功能	返回值	参数	参数说明
void CommunicationEngine::enqueueCmd_setEmergency(QObject *object, const bool &status)	发送急停/解除急停命令	void	object, status	object: 回传信号目标对象; status: true 触发急停、false 解除急停。触发 CmdType_SetEmergency。

bool RobotControlInterface::SetEmergency(bool status)	调用控制器接口设置急停状态	bool	status	内部调用 _IControl->SetEmergency(status), 返回 ret==0 为成功。
--	---------------	------	--------	---

22.4.2 调用

```
C++
// 主界面：点击急停按钮
CommunicationEngine::instance()->enqueueCmd_setEmergency(this,
checked);
```

22.4.3 函数实现

```
C++
bool RobotControlInterface::SetEmergency(bool status)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ret = _IControl->SetEmergency(status);
    qDebug() << "RobotControlInterface::SetEmergency ret : " <<
ret;
    FREQ_LOG_PRINT_TIMESTAMP
    return 0 == ret;
}
```

```
C++
case AbstractCmd::CmdType_SetEmergency: {
    bool isSuccess = m_commInstance->SetEmergency(
        static_cast<CmdSetEmergency *>(absCmd)->m_bStatus);
    emit CommunicationEngine::instance()
        ->signal_setEmergencyInterface_result(absCmd->m_object,
isSuccess);
    break;
}
```

22.5 断连标识 (ReadComState/WriteComState)

22.5.1 说明

接口	功能	返回值	参数	参数说明
<code>void RobotDebugPrivate::readComState(AbstractCmd *absCmd)</code>	读取 COM 开关状态 (用于显示断链/通信开关状态)	void	absCmd	由 CmdType_ReadComState 触发; 读取 _IRCCConfig->readComSwitch(ans); 通过 signal_getComState_result(obj, bool) 回传。
<code>void RobotDebugPrivate::writeComState(AbstractCmd *absCmd)</code>	保存 COM 开关状态 (需控制器重启生效)	void	absCmd	由 CmdType_WriteComState 触发; 载荷 state(bool); 调用 _IRCCConfig->saveComSwitch(state?1:0)。

22.5.2 调用

C++

// 读取/写入 COM 状态

```
CommunicationEngine::instance()->enqueueCmd(
    this,
```

```

        AbstractCmd::CmdType_ReadComState);

CommunicationEngine::instance()->enqueueCmd_setData(
    this,
    AbstractCmd::CmdType_WriteComState,
    state);

```

22.5.3 函数实现

```

C++
void RobotDebugPrivate::readComState(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    int ans;
    int nRet = _IRCCConfig->readComSwitch(ans);
    if (nRet != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to read COM
status."));
    } else {
        emit CommunicationEngine::instance()-
>signal_getComState_result(absCmd->m_object, ans == 0 ? false :
true);
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

void RobotDebugPrivate::writeComState(AbstractCmd *absCmd)
{
    FREQ_LOG_PRINT_TIMESTAMP_THREAD
    auto [state] = ((CmdDatas<bool> *)absCmd)->m_data;
    int nRet = _IRCCConfig->saveComSwitch(state ? 1 : 0);
    if (nRet != ERR_OK) {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("Failed to save COM
status."));
    } else {
        emit CommunicationEngine::instance()-
>signal_needMainWidgetWarning(QObject::tr("COM status saved
successfully and takes effect after controller restart. "));
    }
    FREQ_LOG_PRINT_TIMESTAMP
}

```



```
}
```

```
C++
```

```
case AbstractCmd::CmdType_ReadComState: {  
    m_commInstance->readComState(absCmd);  
    break;  
}  
case AbstractCmd::CmdType_WriteComState: {  
    m_commInstance->writeComState(absCmd);  
    break;  
}
```